ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO

ĐỒ ÁN TỐT NGHIỆP

CHATBOT TƯ VẤN KHÁCH HÀNG

Ngành: Khoa học Máy tính

HỘI ĐỒNG: Hội đồng 17 Khoa học Máy tính

GVHD: TS. Trương Tuấn Anh

GVPB: ThS. Trần Trương Tuấn Phát

—o0o—

SVTH1: Nguyễn Đức An (2112737)

SVTH2: Lê Đình Huy (2113481)

SVTH3: Phạm Đức Thắng (2112336)

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 05/2025

Chữ ký giảng viên hướng dẫn

_____ Ngày: _____

TS. Trương Tuấn Anh (Giảng viên Hướng dẫn)
Tiến sĩ, trưởng bộ môn Công nghệ phần mềm
Khoa Khoa học và Kỹ thuật Máy tính

Lời cảm ơn

Qua 4 năm học tập và rèn luyện tại trường Đại học Bách khoa - ĐH Quốc gia TP.HCM, được sự chỉ bảo và giảng dạy nhiệt tình từ quý thầy cô, đặc biệt là quý thầy cô khoa Khoa học và Kỹ thuật máy tính đã truyền đạt cho chúng em những kiến thức quý giá về lý thuyết cũng như thực hành trong suốt thời gian học ở trường. Cùng với sự nỗ lực của bản thân, chúng em đã hoàn thành tốt môn học Đồ án chuyên ngành của mình.

Từ những kết quả đạt được này, chúng em xin chân thành cảm ơn quý thầy cô trong Khoa đã truyền đạt cho chúng em những kiến thức bổ ích trong thời gian qua. Đặc biệt, chúng em xin chân thành gửi lời cảm ơn đến TS Trương Tuấn Anh, người hướng dẫn đồng hành cùng chúng em suốt học kỳ vừa qua, nhờ sự chỉ bảo nhiệt tình, tâm huyết của thầy mà chúng em có thể hoàn thành tốt môn học đồ án này.

Cuối cùng, chúng em xin gửi lời cảm ơn đến gia đình, bạn bè và các anh chị đi trước đã luôn bên cạnh, kề vai sát cánh, động viên chúng em lúc khó khăn, giúp đỡ chúng em lúc chúng em cần. Bên cạnh đó, nhờ những kinh nghiệm mà các anh chị đi trước đã chia sẻ đã giúp chúng em rất nhiều trong việc hoàn thành môn đồ án này.

Chúng em xin chân thành cảm ơn.

Lời cam đoan

Đồ án của nhóm có tham khảo các tài liệu, bài báo, trang web như được trình bày ở mục tài liệu tham khảo và ở mỗi tham khảo tôi đều trích dẫn nguồn gốc. Nhóm xin cam đoan rằng ngoài những trích dẫn từ các tham khảo trên, toàn bộ nội dung trong báo cáo là do nhóm tự soạn thảo từ những kết quả nghiên cứu của nhóm, không sao chép từ bất kì tài liệu nào khác.

Nhóm sẽ hoàn toàn chịu xử lý theo quy định nếu có bất cứ sai phạm nào so với lời cam đoan.

Nhóm sinh viên thực hiện

Nguyễn Đức An

Lê Đình Huy

Phạm Đức Thắng

Tóm tắt

Sự phát triển vượt bậc của trí tuệ nhân tạo và xử lý ngôn ngữ tự nhiên đã mở đường cho sự ra đời giải pháp chatbot thông minh, hỗ trợ doanh nghiệp tự động hóa dịch vụ chăm sóc khách hàng. Đa số doanh nghiệp, từ trước đến nay, xây dựng chatbot dựa trên các mô hình học sâu được huấn luyện và triển khai cục bộ. Điều này dẫn đến khó khăn trong quá trình tái huấn luyện khi tri thức thay đổi.

Chính vì vậy, đồ án chuyên ngành này tập trung vào việc thiết kế một nền tảng web để cung cấp dịch vụ chatbot dựa trên đám mây. Nền tảng "Chatbot as a service" này cho phép doanh nghiệp dễ dàng tích hợp chatbot tùy chỉnh vào website mà không cần chuyên môn kỹ thuật sâu rộng. Hệ thống đề xuất sẽ tận dụng sức mạnh của các mô hình ngôn ngữ lớn, cơ sở dữ liệu vector đám mây, và khả năng truy xuất tri thức hiệu quả từ kỹ thuật tăng cường truy xuất nâng cao (Advanced RAG) để cung cấp phản hồi chất lượng cao.

Kết quả đồ án cho thấy, nền tảng được thiết kế thể hiện được các tính năng nổi trội bao gồm giao diện thân thiện để cấu hình chatbot, chỉnh sửa tri thức, đào tạo chatbot và tích hợp vào trang web doanh nghiệp. Ngoài ra, hệ thống còn hỗ trợ cả trò chuyện tự động và nhân viên tư vấn trực tiếp, cùng với quản lý lịch hẹn (cho nhân viên) và đặt lịch hẹn (cho khách hàng) trên một giao diện thống nhất.

Thông qua quá trình phân tích đánh giá về mặt lý thuyết, các công nghệ và mô hình sau đã được lựa chọn để thực hiện dự án: Frontend (ReactJS, Vite, NextUI, TailwindCSS), Backend (Django, Postgresql), AI (Gemini 2.5 Flash model, LangChain, Pinecone). Hệ thống được đánh giá cao về tính linh hoạt, khả năng mở rộng dữ liệu và là cơ sở cho các cải tiến trong tương lai, hướng tới một giải pháp chatbot thương mại hóa.

Từ khóa: Artificial Intelligent, Natural Language Processing, Deep Neural Network, On-premise chatbot, Web-based platform, Cloud-based chatbot, Chatbot as a service, Large Language Models, Cloud-based Vector Database, Advanced RAG.

Tóm tắt chương

***Chương 1**: Giới thiệu*

Giới thiệu về đề tài bao gồm vấn đề cần nghiên cứu, các khía cạnh thực tế có liên quan và các hướng giải quyết cho vấn đề đó.

***Chương 2**: Nghiên cứu thị trường*

Xem xét và phân tích các hệ thống tương tự đã và đang vận hành trên thị trường, phân tích giao diện, tính năng và SWOT của chúng.

***Chương 3**: Phân tích hệ thống*

Phân tích nghiệp vụ, các stakeholder, các user story, xác định các yêu cầu chức năng và phi chức năng của hệ thống. Thực hiện vẽ các lược đồ use-case và kịch bản use-case, vẽ các lược đồ activity, lược đồ sequence và lược đồ class nhằm cung cấp một cái nhìn phân tích tổng thể cho giai đoạn thiết kế sau này.

***Chương 4**: Thiết kế hệ thống*

Trình bày kiến trúc của hệ thống về tổng quan và chi tiết, thiết kế giao diện cho hệ thống đồng thời phân tích lược đồ cơ sở dữ liệu (ERD) đã thiết kế.

***Chương 5**: Hiện thực*

Trình bày về những công nghệ được lựa chọn để hiện thực hệ thống, đồng thời giải thích và làm rõ lý do tại sao lại sử dụng các công nghệ đó. Sau đó, sẽ trình bày về cách quản lý mã nguồn, cấu trúc mã nguồn và các giao diện của hệ thống được hiện thực.

***Chương 6**: Kiểm thử và đánh giá*

Trình bày quá trình kiểm thử hệ thống với các phương pháp kiểm thử phù hợp. Các phương pháp kiểm thử được áp dụng nhằm đảm bảo tính đúng đắn, ổn định và hiệu quả của hệ thống. Kết quả kiểm thử cho thấy hệ thống đáp ứng đầy đủ các yêu cầu chức năng, đạt được độ tin cậy cao và sẵn sàng triển khai thực tế.

***Chương 7**: Triển khai và vận hành*

Mô tả các bước triển khai hệ thống sau kiểm thử thành công, bao gồm quá trình chuẩn bị môi trường, cấu hình, đóng gói và đưa hệ thống vào vận hành thực tế. Hệ thống hoạt động ổn định, hỗ trợ doanh nghiệp tạo chatbot tự động chăm sóc khách hàng hiệu quả.

***Chương 8**: Tổng kết*

Nhận xét các kết quả đã đạt được trong suốt quá trình phát triển hệ thống. Từ đó rút ra các ưu điểm, hạn chế của hệ thống và đưa ra hướng phát triển của hệ thống trong tương lai.

Mục lục

1 Giới thiệu **1**
1.1 Đặt vấn đề . . . 1
1.2 Các hướng giải quyết liên quan . . . 2

2 Nghiên cứu thị trường **3**
2.1 AhaChat . . . 3
2.1.1 Giới thiệu . . . 3
2.1.2 UI/UX . . . 4
2.1.3 Các tính năng chính . . . 5
2.1.4 Phân tích SWOT . . . 7
2.2 Fchat . . . 7
2.2.1 Giới thiệu . . . 7
2.2.2 UI/UX . . . 8
2.2.3 Các tính năng chính . . . 10
2.2.4 Phân tích SWOT . . . 11
2.3 TuDongChat . . . 11
2.3.1 Giới thiệu . . . 11
2.3.2 UI/UX . . . 12
2.3.3 Các tính năng chính . . . 13
2.3.4 Phân tích SWOT . . . 14

3 Phân tích hệ thống **14**
3.1 Stakeholders . . . 14
3.1.1 Đội phát triển phần mềm . . . 14
3.1.2 Chủ doanh nghiệp . . . 15
3.1.3 Quản trị viên doanh nghiệp . . . 15
3.1.4 Nhóm hỗ trợ khách hàng cho doanh nghiệp . . . 15
3.1.5 Người dùng cuối (Khách hàng của các doanh nghiệp khách hàng) . . . 15
3.1.6 Nhóm đào tạo và hỗ trợ . . . 16
3.2 User Stories . . . 16
3.2.1 Chủ doanh nghiệp . . . 16
3.2.2 Quản trị viên doanh nghiệp . . . 16
3.2.3 Nhóm hỗ trợ khách hàng của doanh nghiệp . . . 17
3.2.4 Người dùng cuối (Khách hàng của các doanh nghiệp khách hàng) . . . 17
3.3 Yêu cầu chức năng . . . 17
3.3.1 Đăng ký và xác thực . . . 17
3.3.2 Quản lý tài khoản tổ chức . . . 17
3.3.3 Tùy chỉnh giao diện và hành vi chatbot . . . 18
3.3.4 Tạo mã nhúng chatbot . . . 18
3.3.5 Quản lý cơ sở tri thức . . . 18
3.3.6 Trò chuyện với nhân viên hỗ trợ trực tiếp . . . 18
3.3.7 Báo cáo và thống kê . . . 19
3.3.8 Quản lý Gói dịch vụ và Thanh toán . . . 19
3.4 Yêu cầu phi chức năng . . . 19
3.4.1 Hiệu suất (Performance) . . . 19
3.4.2 Tính khả dụng (Availability) . . . 19
3.4.3 Bảo mật (Security) . . . 19
3.4.4 Khả năng sử dụng (Usability) . . . 20
3.4.5 Độ tin cậy (Reliability) . . . 20
3.4.6 Khả năng bảo trì (Maintainability) . . . 20
3.4.7 Tính di động (Portability) . . . 20
3.4.8 Yêu cầu Pháp lý và Đạo đức (Legal & Ethical Requirements) . . . 20
3.5 Biểu đồ Use case . . . 21
3.5.1 Biểu đồ use case tổng . . . 22
3.5.2 Quản lý tài khoản . . . 23
3.5.2.1 Biểu đồ use case . . . 23
3.5.2.2 Use case scenario . . . 23
3.5.3 Đào tạo và tùy chỉnh chatbot . . . 26
3.5.3.1 Biểu đồ use case . . . 26

3.5.3.2 Use case scenario . . . 26
3.5.4 Tích hợp . . . 29
3.5.4.1 Biểu đồ usecase . . . 29
3.5.4.2 Usecase scenerio . . . 29
3.5.5 Quản lý chatbot . . . 30
3.5.5.1 Biểu đồ use case . . . 30
3.5.5.2 Use case scenario . . . 30
3.5.6 Quản lý thanh toán . . . 32
3.5.6.1 Biểu đồ usecase . . . 32
3.5.6.2 Usecase scenerio . . . 32
3.5.7 Tương tác với chatbot . . . 33
3.5.7.1 Biểu đồ use case . . . 33
3.5.7.2 Use case scenario . . . 33
3.6 Biểu đồ Activity . . . 36
3.6.1 Quản lý tài khoản . . . 36
3.6.1.1 Đăng ký . . . 36
3.6.1.2 Đăng nhập và quên mật khẩu . . . 37
3.6.1.3 Chỉnh sửa hồ sơ cá nhân . . . 38
3.6.2 Đào tạo và tùy chỉnh chatbot . . . 40
3.6.2.1 Đào tạo chatbot . . . 40
3.6.2.2 Tùy chỉnh chatbot . . . 41
3.6.3 Tích hợp . . . 42
3.6.4 Quản lý chatbot . . . 43
3.6.4.1 Báo cáo . . . 43
3.6.4.2 Lịch sử hội thoại . . . 44
3.6.5 Quản lý thanh toán . . . 45
3.6.6 Tương tác với chatbot . . . 46
3.6.6.1 Phản hồi tin nhắn khách hàng . . . 46
3.6.6.2 Gửi tin nhắn . . . 47
3.6.6.3 Yêu cầu hỗ trợ . . . 48
3.7 Biểu đồ Sequence . . . 49
3.7.1 Quản lý tài khoản . . . 49
3.7.1.1 Đăng ký . . . 49
3.7.1.2 Đăng nhập . . . 50
3.7.1.3 Chỉnh sửa hồ sơ cá nhân . . . 51
3.7.2 Đào tạo và tùy chỉnh chatbot . . . 52
3.7.2.1 Đào tạo chatbot . . . 52
3.7.2.2 Tùy chỉnh chatbot . . . 55
3.7.3 Quản lý chatbot . . . 57
3.7.3.1 Báo cáo . . . 57
3.7.3.2 Lịch sử hội thoại . . . 58
3.7.4 Quản lý thanh toán . . . 59
3.7.5 Tương tác với chatbot . . . 60
3.7.5.1 Phản hồi tin nhắn khách hàng . . . 60
3.7.5.2 Gửi tin nhắn . . . 62
3.7.5.3 Yêu cầu hỗ trợ . . . 63
3.8 Biểu đồ Class . . . 64

4 Thiết kế hệ thống **68**
4.1 Kiến trúc hệ thống . . . 68
4.1.1 Tổng quan kiến trúc . . . 69
4.1.2 Chi tiết các thành phần chính . . . 69
4.1.2.1 Lớp ứng dụng (Application Layer) . . . 69
4.1.2.2 Lớp giao tiếp (Interface Layer) . . . 69
4.1.2.3 Xử lý tác vụ nền (Background Processing) . . . 70
4.1.2.4 Lớp dữ liệu (Data Layer) . . . 70
4.1.2.5 Dịch vụ AI . . . 70
4.1.2.6 Tích hợp dịch vụ bên ngoài . . . 70
4.2 Giao diện . . . 71
4.2.1 Lược đồ Screen-flow . . . 71

4.2.2 Logo ứng dụng . . . 71
4.2.3 Thiết kế giao diện với Figma . . . 72
4.3 Cơ sở dữ liệu . . . 72
4.3.1 Quản lý Người dùng . . . 72
4.3.1.1 Bảng `users_user`: . . . 72
4.3.2 Quản lý Tổ chức . . . 74
4.3.2.1 Bảng `organization_organization`: . . . 74
4.3.2.2 Bảng `organization_organizationmember`: . . . 75
4.3.2.3 Bảng `organization_organizationinvite`: . . . 75
4.3.3 Quản lý Trò chuyện . . . 76
4.3.3.1 Bảng `chat_conversation`: . . . 76
4.3.3.2 Bảng `chat_message`: . . . 76
4.3.4 Quản lý Chatbot . . . 77
4.3.4.1 Bảng `chatbot_chatbotconfig`: . . . 77
4.3.5 Quản lý Tài liệu Huấn luyện . . . 78
4.3.5.1 Bảng `document_document`: . . . 78
4.3.5.2 Bảng `document_organizationtrainingstatus`: . . . 78
4.3.6 Quản lý Gói đăng ký và Thanh toán . . . 79
4.3.6.1 Bảng `subscription_subscriptiontier`: . . . 79
4.3.6.2 Bảng `subscription_organizationsubscription`: . . . 79
4.3.6.3 Bảng `subscription_additionalusagecharge`: . . . 80
4.3.6.4 Bảng `subscription_payment`: . . . 80

5 Hiện thực **81**
5.1 Công nghệ . . . 81
5.1.1 Lược đồ tổng quan . . . 81
5.1.2 Frontend . . . 81
5.1.2.1 ReactJS . . . 81
5.1.2.2 Vite . . . 83
5.1.2.3 NextUI . . . 84
5.1.2.4 TailwindCSS . . . 84
5.1.3 Backend . . . 84
5.1.3.1 Django và Django REST Framework (DRF) . . . 84
5.1.3.2 Hệ Quản trị Cơ sở dữ liệu PostgreSQL . . . 86
5.1.3.3 Redis (Remote Dictionary Server) . . . 88
5.1.3.4 Dịch vụ Lưu trữ Amazon S3 (Simple Storage Service) . . . 89
5.1.3.5 PayOS . . . 90
5.1.4 AI . . . 91
5.1.4.1 Mô hình Gemini-2.5-Flash . . . 91
5.1.4.2 LangChain . . . 92
5.1.4.3 Pinecone Vector Database . . . 94
5.2 Mã nguồn . . . 95
5.2.1 Quản lý mã nguồn . . . 95
5.2.2 Cấu trúc mã nguồn . . . 96
5.2.2.1 Frontend . . . 96
5.2.2.2 Backend . . . 97
5.2.2.3 AI . . . 98
5.3 Giao diện . . . 99
5.3.1 Trang Đích . . . 99
5.3.2 Trang Xác thực . . . 100
5.3.3 Trang Đào tạo chatbot . . . 102
5.3.4 Trang Tùy chỉnh chatbot . . . 104
5.3.5 Trang Hội thoại . . . 106
5.3.6 Trang Báo cáo . . . 108
5.3.7 Trang Thanh toán . . . 109
5.3.8 Trang Tổ chức . . . 112
5.3.9 Trang Cài đặt . . . 114
5.3.10 Giao diện hộp chat dành cho khách hàng . . . 115

6 Kiểm thử và đánh giá **115**

6.1 Kiểm thử hệ thống . . . 115
6.1.1 Kiểm thử API . . . 115
6.1.2 Tổng quan về công cụ và cấu trúc kiểm thử . . . 115
6.1.3 Các phương pháp kiểm thử phần mềm . . . 116
6.1.4 Kết quả Kiểm thử và Ví dụ . . . 117
6.1.5 Kiểm thử yêu cầu chức năng . . . 120
6.2 Kiểm thử API với Postman . . . 130
6.3 Kiểm thử Backend . . . 130
6.3.1 Tổng quan về công cụ và cấu trúc kiểm thử . . . 130
6.3.2 Các phương pháp kiểm thử phần mềm . . . 131
6.3.3 Kết quả Kiểm thử và Ví dụ . . . 132
6.4 Kiểm thử phản hồi của chatbot . . . 135
6.4.1 Cấu trúc testcase . . . 135
6.4.2 Bias . . . 136
6.4.3 Toxicity . . . 136
6.4.4 Retrieval . . . 137
6.4.5 Generation . . . 137
6.5 Kiểm thử chấp nhận người dùng (User Acceptance Testing - UAT) . . . 138

7 Triển khai và vận hành **142**
7.1 Triển khai Frontend . . . 142
7.2 Triển khai Backend . . . 145
7.3 Triển khai AI . . . 146
7.3.1 Đóng gói . . . 146
7.3.2 Triển khai . . . 147

8 Tổng kết **149**
8.1 Kết quả đạt được . . . 149
8.1.1 Đối với tìm hiểu và phân tích nghiệp vụ . . . 149
8.1.2 Đối với cơ sở lý thuyết và công nghệ . . . 149
8.1.3 Đối với phân tích và thiết kế hệ thống . . . 149
8.1.4 Đối với việc hiện thực và triển khai hệ thống . . . 149
8.1.5 Đối với quá trình thực hiện đồ án . . . 150
8.2 Hướng phát triển và kế hoạch trong tương lai . . . 150
8.2.1 Hướng phát triển . . . 150
8.2.2 Kế hoạch thực hiện . . . 150

Tài liệu tham khảo **153**

Phụ lục **154**

Danh sách hình vẽ

1 Thị trường chatbot ở Bắc Mỹ dự đoán đến năm 2027 (triệu USD). Nguồn: [2] . . . 1
2 Thị phần chatbot năm 2023 phân theo nơi triển khai. Nguồn: [1] . . . 1
3 Thị phần các nền tảng để gắn chatbot năm 2019. Nguồn: [2] . . . 2
4 Giao diện trang Landing page của AhaChat . . . 4
5 Giao diện trang chủ của AhaChat . . . 4
6 Giao diện quản lý fanpage Facebook của AhaChat . . . 5
7 Giao diện quản lý kịch bản của AhaChat . . . 5
8 Giao diện quản lý kịch bản của AhaChat . . . 6
9 Giao diện trang Landing page của Fchat . . . 8
10 Giao diện trang Dashboard của Fchat . . . 8
11 Giao diện quản lý từ khóa tin nhắn của Fchat . . . 9
12 Giao diện khung chat của Fchat . . . 9
13 Các tính năng cơ bản của Fchat . . . 10
14 Giao diện trang Landing page của TuDongChat . . . 12
15 Giao diện Trang chủ của TuDongChat . . . 12
16 Giao diện khung chat của TuDongChat . . . 13
17 Các tính năng cơ bản của TuDongChat . . . 14
18 Biểu đồ use case tổng . . . 22
19 Biểu đồ use case cho Quản lý tài khoản . . . 23
20 Biểu đồ use case cho Đào tạo và tùy chỉnh chatbot . . . 26
21 Biểu đồ use case cho tích hợp . . . 29
22 Biểu đồ use case cho Quản lý chatbot . . . 30
23 Biểu đồ use case cho Quản lý thanh toán . . . 32
24 Biểu đồ use case cho Tương tác với chatbot . . . 33
25 Biểu đồ activity cho Đăng ký . . . 36
26 Biểu đồ activity cho Đăng nhập và quên mật khẩu . . . 37
27 Biểu đồ activity cho Chỉnh sửa hồ sơ cá nhân . . . 38
28 Biểu đồ activity cho Đào tạo chatbot . . . 40
29 Biểu đồ activity cho Tùy chỉnh chatbot . . . 41
30 Biểu đồ activity cho Tích hợp . . . 42
31 Biểu đồ activity cho Báo cáo . . . 43
32 Biểu đồ activity cho Lịch sử hội thoại . . . 44
33 Biểu đồ activity cho Quản lý thanh toán . . . 45
34 Biểu đồ activity cho Phản hồi tin nhắn khách hàng . . . 46
35 Biểu đồ activity cho Gửi tin nhắn . . . 47
36 Biểu đồ activity cho Yêu cầu hỗ trợ . . . 48
37 Biểu đồ sequence cho Đăng ký . . . 49
38 Biểu đồ sequence cho Đăng nhập . . . 50
39 Biểu đồ sequence cho Chỉnh sửa hồ sơ cá nhân . . . 51
40 Biểu đồ sequence cho Đào tạo chatbot . . . 52
41 Biểu đồ sequence cho Xóa tri thức . . . 53
42 Biểu đồ sequence cho Thêm tri thức . . . 54
43 Biểu đồ sequence cho Tùy chỉnh chatbot . . . 55
44 Biểu đồ sequence cho Báo cáo . . . 57
45 Biểu đồ sequence cho Lịch sử hội thoại . . . 58
46 Biểu đồ sequence cho Quản lý thanh toán . . . 59
47 Biểu đồ sequence cho Phản hồi tin nhắn khách hàng . . . 60
48 Biểu đồ sequence cho Gửi tin nhắn . . . 62
49 Biểu đồ sequence cho Yêu cầu hỗ trợ . . . 63
50 Biểu đồ class các thực thể cho tầng dữ liệu . . . 65
51 Biểu đồ kiến trúc hệ thống . . . 68
52 Lược đồ Screen-flow . . . 71
53 Logo của ứng dụng CoolChat . . . 71
54 Công cụ Figma . . . 72
55 Các màn hình được thiết kế trên Figma . . . 72
56 Biểu đồ ERD hệ thống . . . 73
57 Lược đồ tổng quan về mối quan hệ giữa các công nghệ của hệ thống . . . 81
58 Logo React - Nguồn: Internet . . . 82

59 Logo Vite - Nguồn Internet . . . 83
60 Logo NextUI - Nguồn: Internet . . . 84
61 Logo TailwindCSS - Nguồn: Internet . . . 84
62 Logo chính thức của Django Framework . . . 84
63 Logo chính thức của PostgreSQL . . . 86
64 Mô hình ngôn ngữ lớn Gemini-2.5-flash preview. Nguồn: Internet . . . 91
65 Framework LangChain - Nguồn: Internet . . . 92
66 Hệ cơ sở dữ liệu vector Pinecone - Nguồn: Internet . . . 94
67 CoolChat Organization . . . 96
68 Cấu trúc mã nguồn Frontend . . . 97
69 Cấu trúc mã nguồn AI Service . . . 99
70 Giao diện trang đích chính . . . 100
71 Giao diện trang giới thiệu . . . 100
72 Giao diện đăng nhập . . . 101
73 Giao diện đăng ký . . . 101
74 Email tham gia tổ chức được gửi bởi hệ thống . . . 101
75 Giao diện mục Gán nhãn tri thức . . . 102
76 Bản xem trước khi tải xuống tri thức . . . 102
77 Giao diện thêm tri thức dạng tập tin . . . 103
78 Giao diện thêm tri thức dạng văn bản . . . 103
79 Giao diện thêm tri thức dạng website . . . 104
80 Giao diện hộp chat kiểm thử . . . 104
81 Giao diện tùy chỉnh giao diện chatbot . . . 105
82 Giao diện tùy chỉnh phong cách chatbot . . . 105
83 Giao diện cài đặt giới hạn truy vấn . . . 106
84 Giao diện tích hợp . . . 106
85 Giao diện hội thoại . . . 107
86 Giao diện hội thoại cần hỗ trợ . . . 107
87 Giao diện chi tiết hội thoại cho chủ sở hữu và quản trị viên . . . 108
88 Giao diện chi tiết hội thoại cho chủ sở hữu và quản trị viên . . . 108
89 Giao diện báo cáo . . . 109
90 Giao diện xuất báo cáo . . . 109
91 Giao diện thông tin gói đăng ký hiện tại . . . 110
92 Các gói đăng ký . . . 110
93 Các gói bổ sung . . . 111
94 Giao diện thanh toán với PayOS . . . 111
95 Giao diện thanh toán được hiện thực . . . 112
96 Giao diện thông tin tổ chức . . . 112
97 Giao diện danh sách thành viên tổ chức . . . 113
98 Giao diện danh sách lời mời tham gia tổ chức . . . 113
99 Giao diện mời thêm thành viên vào tổ chức . . . 114
100 Giao diện chỉnh sửa thông tin cá nhân . . . 114
101 Giao diện đổi mật khẩu . . . 115
102 Giao diện chatbot phía khách hàng . . . 115
103 Một testcase trong bộ test kiểm tra phản hồi của chatbot . . . 136
104 Thống kê độ tuổi, nghề nghiệp và nơi công tác . . . 139
105 Cách đánh giá chức năng . . . 139
106 Thống kê đánh giá theo thang điểm của người dùng . . . 140
107 Những nhận xét, góp ý từ người dùng . . . 140
108 Thống kê đánh giá mức độ dễ sử dụng . . . 141
109 Thống kê đánh giá mức độ dễ tìm thấy chức năng . . . 141
110 Thống kê đánh giá tốc độ phản hồi . . . 141
111 Thống kê đánh giá khả năng đáp ứng . . . 142
112 Thống kê đánh giá khả năng giới thiệu . . . 142
113 Tạo dự án mới trên Vercel . . . 143
114 Nhập repository trên Vercel . . . 143
115 Cấu hình cài đặt dự án trên Vercel . . . 144
116 Kết quả triển khai trên Vercel . . . 144
117 Vào URL của dự án đã triển khai bằng Vercel . . . 145
118 Quản lý các phiên bản trên Vercel . . . 145

119 Biểu đồ Gantt kế hoạch triển khai dự án . . . 150
120 So sánh giữa hai mô hình RAG. (Bên trái) RAG truyền thống chủ yếu bao gồm ba phần: lập index, truy xuất và tạo. (Bên phải) RAG nâng cao đề xuất nhiều chiến lược tối ưu hóa xung quanh quá trình tiền và hậu truy xuất, với quy trình tương tự như RAG truyền thống, vẫn tuân theo cấu trúc dạng chuỗi. Nguồn: [3] . . . 155

Danh sách bảng

1 Bảng phân tích SWOT cho hệ thống AhaChat . . . 7
2 Bảng phân tích SWOT cho hệ thống Fchat . . . 11
3 Bảng phân tích SWOT cho hệ thống TuDongChat . . . 14
4 Use case scenario cho Đăng ký tài khoản mới . . . 23
5 Use case scenario cho Đăng nhập . . . 24
6 Use case scenario cho Quên mật khẩu . . . 24
7 Use case scenario cho Chỉnh sửa hồ sơ cá nhân . . . 25
8 Use case scenario cho Đổi mật khẩu . . . 25
9 Use case scenario cho Thêm tri thức . . . 26
10 Use case scenario cho Xóa tri thức . . . 27
11 Use case scenario cho Sửa độ ưu tiên . . . 27
12 Use case scenario cho Đào tạo . . . 28
13 Use case scenario cho Tùy chỉnh giao diện chatbot. . . . 28
14 Use case scenario cho Tùy chỉnh phong cách chatbot. . . . 28
15 Use case scenario cho Tạo mã nhúng . . . 29
16 Use case scenario cho Xem báo cáo . . . 30
17 Use case scenario cho Xem báo cáo . . . 31
18 Use case scenario cho Xem báo cáo . . . 31
19 Use case scenario cho Chọn gói thanh toán . . . 32
20 Use case scenario cho Theo dõi sử dụng . . . 33
21 Use case scenario cho Phản hồi tin nhắn khách hàng . . . 34
22 Use case scenario cho Gửi tin nhắn . . . 34
23 Use case scenario cho Yêu cầu nhân viên hỗ trợ . . . 35
26 So sánh Django/DRF với các framework backend phổ biến . . . 85
27 So sánh PostgreSQL với các hệ quản trị CSDL khác . . . 87
28 So sánh Redis với Memcached . . . 88
29 So sánh Amazon S3 với các dịch vụ lưu trữ đối tượng khác . . . 89
33 Thống kê kết quả kiểm thử theo ứng dụng . . . 117
34 Bảng testcase cho màn hình Đào tạo chatbot . . . 123
35 Bảng testcase cho màn hình Tùy chỉnh chatbot . . . 124
36 Bảng testcase cho màn hình Hội thoại . . . 126
37 Bảng testcase cho màn hình Báo cáo . . . 126
38 Bảng testcase cho màn hình Thanh toán . . . 128
39 Bảng testcase cho màn hình Tổ chức . . . 129
40 Bảng testcase cho màn hình Cài đặt . . . 129
41 Bảng testcase cho chức năng Khách hàng . . . 130
42 Thống kê kết quả kiểm thử theo ứng dụng . . . 132
43 Kết quả kiểm tra thiên kiến (Bias) . . . 136
44 Kết quả kiểm tra tính độc hại (Toxicity) . . . 136
45 Kết quả đánh giá chất lượng truy xuất (Precision và Recall) . . . 137
46 Kết quả đánh giá chất lượng câu trả lời (AnswerRelevancy và Faithfulness) . . . 137

1 Giới thiệu

1.1 Đặt vấn đề

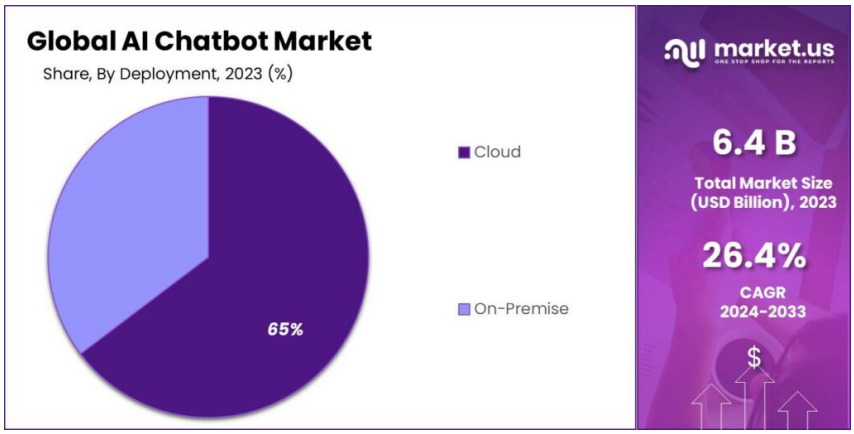
Trong vòng nhiều thế kỷ qua, chúng ta đã chứng kiến sự tăng vọt của lượng dữ liệu người dùng khổng lồ, tạo nền tảng tài nguyên để vận hành các thuật toán học máy và học sâu nhằm xây dựng các hệ thống hướng dữ liệu. Và nổi bật trong các hệ thống như vậy đó chính là các mô hình ngôn ngữ lớn – Large Language model (LLM), có khả năng tạo sinh dữ liệu đáng kinh ngạc đến mức mà các nhà khoa học đương thời gọi chúng là các SOTA (state of the art – tạm dịch Đỉnh cao của công nghệ). Các tác tử hoạt động dựa trên các mô hình đó cũng dần xuất hiện, trong đó, các tác tử đối thoại (conversational agents) hay gọi một cách quen thuộc hơn là các AI chatbot đã dần khẳng định được vị thế của mình trong cộng đồng ứng dụng trí tuệ nhân tạo tạo sinh.

Theo một thống kê thực tế từ đầu năm 2018 của Hubspot, số lượng hàng hóa bán ra cho người dùng trên toàn thế giới thông qua chatbot chiếm tới hơn 47% [4] và con số này cho đến nay chắc chắn đã lớn hơn rất nhiều. Một báo cáo khác, thực hiện bởi trang Fortune Business Insights, dự đoán thị trường chatbot sẽ tăng từ 396,2 triệu USD (năm 2019) đến 1953,3 triệu USD (năm 2027), tương ứng với tốc độ tăng trưởng CAGR đạt 22.5% [2]. Con số này đã phần nào chứng minh chatbot đang ngày càng được ứng dụng rộng rãi trong cuộc sống, đặc biệt là trong lĩnh vực kinh doanh. Rất nhiều doanh nghiệp kinh doanh trong lĩnh vực dịch vụ trên toàn cầu đã ưu tiên triển khai chatbot để xử lý hiệu quả tình huống khi nhu cầu khách hàng tăng cao nhưng đội ngũ nhân viên ít ỏi không đủ sức đảm đương.



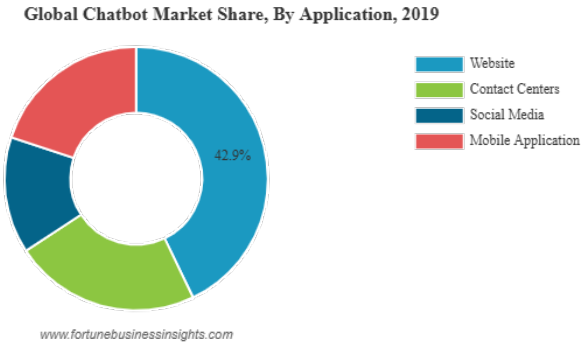
Hình 1: *Thị trường chatbot ở Bắc Mỹ dự đoán đến năm 2027 (triệu USD). Nguồn: [2]*

Một phân tích khác trên trang web market.us cho biết, phân khúc đám mây đang chiếm giữ thị phần áp đảo so với các hệ thống chatbot on-premise (doanh nghiệp tự triển khai) – chiếm khoảng 64.7% vào năm 2023 [1]. Sự thống lĩnh này phần lớn là nhờ tính linh hoạt, khả năng mở rộng và hiệu quả về chi phí mà các nhà cung cấp giải pháp đám mây đem lại, điều mà các doanh nghiệp rất ưa chuộng, bởi họ có thể dễ dàng mở rộng quy mô các giải pháp chatbot của mình theo nhu cầu khách hàng hiện tại mà không tốn công sức đầu tư cho các hạ tầng máy móc cần thiết. Hơn nữa, chatbot AI dựa trên đám mây được hưởng lợi từ các bản cập nhật và cải tiến liên tục do công nghệ điện toán đám mây tạo ra. Các nhà cung cấp có thể triển khai các bản cập nhật trực tiếp vào cơ sở hạ tầng đám mây, đảm bảo rằng tất cả người dùng đều được hưởng lợi từ những tiến bộ mới nhất trong AI và máy học mà không phải trả thêm chi phí hoặc nỗ lực nào. Vị thế dẫn đầu của phân khúc Đám mây cũng được củng cố bởi sự tin tưởng ngày càng tăng vào các biện pháp bảo mật đám mây và việc áp dụng ngày càng nhiều các môi trường làm việc linh động và từ xa, đòi hỏi các giải pháp linh hoạt và dễ tiếp cận.



Hình 2: *Thị phần chatbot năm 2023 phân theo nơi triển khai. Nguồn: [1]*

Về khía cạnh chọn nền tảng để gắn chatbot vào, Website luôn là ưu tiên hàng đầu bởi chúng chính là mặt tiền kỹ thuật số của một doanh nghiệp và hình ảnh một chatbot với lô gô của doanh nghiệp ở góc dưới cùng bên phải màn hình đã trở thành một điều quen thuộc trong tiềm thức của khách hàng. Ngày nay chatbot hỗ trợ đã trở thành một người bạn đồng hành phổ biến trong hành trình trải nghiệm của người dùng khi truy cập một website thương mại điện tử của doanh nghiệp. Xu hướng này dự kiến sẽ tiếp tục khi ngày càng nhiều các công ty nhận ra tầm quan trọng của việc tăng cường sự tương tác của khách hàng trên các kênh kỹ thuật số chính của họ, khiến các tác tử đối thoại này trở thành một yêu cầu chức năng không thể thiếu khi xây dựng các trang web kinh doanh.



Hình 3: *Thị phần các nền tảng để gắn chatbot năm 2019. Nguồn: [2]*

Các lợi ích của một chatbot hỗ trợ khách hàng:

- Tăng tương tác với khách hàng: Với khả năng hoạt động liên tục 24/7, chatbots giúp các doanh nghiệp tương tác hiệu quả với khách hàng mọi lúc mọi nơi. Chúng có thể hỗ trợ đồng thời nhiều khách hàng mà không làm giảm chất lượng dịch vụ và cung cấp phản hồi nhanh chóng đảm bảo khách hàng không phải chờ đợi lâu.
- Tự động hóa và tiết kiệm chi phí: Bằng cách tự động hóa các quy trình hỗ trợ, như trả lời các câu hỏi thường gặp, xử lý các yêu cầu đơn giản, doanh nghiệp có thể tiết kiệm nguồn lực đáng kể. Điều này giúp cắt giảm chi phí vận hành, giảm sự phụ thuộc vào lực lượng nhân viên lớn.
- Hỗ trợ đa ngôn ngữ: Chatbots có thể được lập trình để hỗ trợ nhiều ngôn ngữ khác nhau, giúp mở rộng phạm vi tương tác với khách hàng từ khắp nơi trên thế giới. Điều này giúp tối ưu hóa trải nghiệm khách hàng toàn cầu và vượt qua rào cản ngôn ngữ, mở rộng thị trường.
- Cá nhân hóa trải nghiệm: Với khả năng thu thập và phân tích dữ liệu, chatbot có thể cung cấp các trải nghiệm tương tác được cá nhân hóa cho từng khách hàng, từ đó cải thiện sự hài lòng và lòng trung thành của khách hàng đối với thương hiệu.
- Nâng khả năng nhận diện thương hiệu: Chatbots không chỉ là công cụ hỗ trợ, mà còn có thể được thiết kế để phản ánh phong cách và giá trị của thương hiệu. Việc sử dụng chatbot giúp tăng cường nhận diện thương hiệu, làm cho doanh nghiệp trở nên chuyên nghiệp và hiện đại hơn trong mắt khách hàng.

Tóm lại, các công nghệ chatbot và mô hình ngôn ngữ lớn đang trên đà phát triển chóng mặt và việc ứng dụng chúng vào một lĩnh vực cụ thể đó là chăm sóc khách hàng giúp gia tăng lợi ích kinh tế, thăng hạng trang web trong mắt người dùng và tăng cường uy tín của doanh nghiệp trong mắt những khách hàng mới. Nhu cầu phát triển một nền tảng xây dựng chatbot đám mây để phục vụ nhu cầu của doanh nghiệp trở nên bức thiết hơn bao giờ hết, giúp giảm tải khối lượng công việc cho các phòng ban IT của các công ty cũng như chi phí đầu tư cho các hệ thống on-premise.

1.2 Các hướng giải quyết liên quan

Với sự gia tăng nhu cầu về dịch vụ hỗ trợ 24/7 và yêu cầu về trải nghiệm cá nhân hóa, các doanh nghiệp cần bổ sung các giải pháp về các chatbot trí tuệ nhân tạo để nâng cao chất lượng dịch vụ khách hàng mà không tốn kém quá nhiều nguồn lực. Chính vì thế, đề tài của nhóm tác giả mong muốn cung cấp dịch vụ tạo chatbot nhằm giúp các doanh nghiệp:

- Tiết kiệm thời gian và chi phí: Chatbots tự động hóa các quy trình cơ bản, giúp giảm thiểu chi phí nhân sự và thời gian xử lý yêu cầu.
- Cải thiện trải nghiệm khách hàng: Chatbots phản hồi gần như ngay lập tức và cá nhân hóa các tương tác, tạo ấn tượng tốt hơn với khách hàng, qua đó nâng cao tỷ lệ giữ chân khách hàng (Customer Retention Rate – CRR).

- Tăng cường tính cạnh tranh: Trong thị trường cạnh tranh cao, việc áp dụng công nghệ tiên tiến như chatbot giúp doanh nghiệp nổi bật và dễ dàng tiếp cận với khách hàng hơn. Bên cạnh đó, chatbot cũng mang trong mình nhận diện thương hiệu, giúp ghi điểm trong mắt khách hàng qua đó tăng lợi thế cạnh tranh của công ty.
- Đón đầu xu hướng: Như đã đề cập, chatbots đang dần trở thành xu hướng toàn cầu, giúp doanh nghiệp không chỉ tối ưu hóa dịch vụ mà còn đi trước đối thủ trong việc áp dụng công nghệ vào quy trình kinh doanh. Công ty hòa nhập trong xu hướng chuyển đổi số của nhà nước, qua đó được tạo điều kiện thuận lợi hơn trên thương trường, mở rộng phạm vi tiếp cận khách hàng.

Cụ thể, đề tài hướng đến xây dựng một trang web cung cấp dịch vụ có đăng ký (subscription), cùng giải pháp xây dựng chatbot đám mây hỗ trợ doanh nghiệp nhiều tính năng phổ biến mà không tốn quá nhiều công sức của đội ngũ công nghệ thông tin của công ty hoặc bỏ ra chi phí để xây dựng hạ tầng phần cứng rắc rối. Khi sử dụng ứng dụng này, doanh nghiệp có thể:

- Lựa chọn tài liệu phù hợp để huấn luyện chatbot, định dạng text, docs, hoặc pdf đều khả dụng;
- Tùy chỉnh logo, ảnh đại diện, màu sắc, định dạng khung chat nhằm tăng độ nhận diện thương hiệu;
- Đội ngũ công nghệ của công ty có thể dễ dàng tích hợp vào app, website của công ty thông qua CDN;
- Chatbot sẽ được prompting và fine-tuning để tránh việc đưa ra các câu trả lời không phù hợp với tiêu chuẩn đạo đức;
- Không cần trả thêm bất kỳ khoản chi phí nào khác.

Đề tài sẽ tập trung vào khả năng tự động hóa các tác vụ hỗ trợ khách hàng, chẳng hạn như trả lời các câu hỏi thường gặp (FAQs), hỗ trợ mua hàng, và xử lý các vấn đề cơ bản và cuối cùng là phân tích sơ lược và thử nghiệm trong một số lĩnh vực thực tế như bán lẻ, dịch vụ để kiểm định tính hiệu quả trong các bối cảnh kinh doanh khác nhau hoặc trong giáo dục, nghiên cứu nhằm kiểm tra tính khả thi về mặt liên ngành. Nhóm tác giả mong muốn kết quả của đề tài này sẽ đóng góp vào việc nghiên cứu và phát triển các ứng dụng liên quan đến AI tạo sinh (Generative AI) cũng như liên quan đến xử lý ngôn ngữ tự nhiên (Natural language processing), từ đó mở rộng hiểu biết của chúng ta về cách thức áp dụng các mô hình ngôn ngữ lớn như một công cụ hiệu quả để tối ưu các tác vụ hỗ trợ người dùng mà cụ thể ở đây là chăm sóc khách hàng.

2 Nghiên cứu thị trường

Hiện nay, trên thị trường quốc tế xuất hiện rất nhiều hệ thống chatbot và các nền tảng hỗ trợ chatbot tư vấn khách hàng. Tuy nhiên, trong phạm vi nghiên cứu này, nhóm tác giả sẽ tập trung phân tích một số hệ thống nổi bật tại thị trường Việt Nam. Mỗi hệ thống đều có những ưu điểm và hạn chế riêng, đi kèm với giao diện thân thiện và bộ tính năng đa dạng, phục vụ cho các nhu cầu khác nhau của doanh nghiệp.

2.1 AhaChat

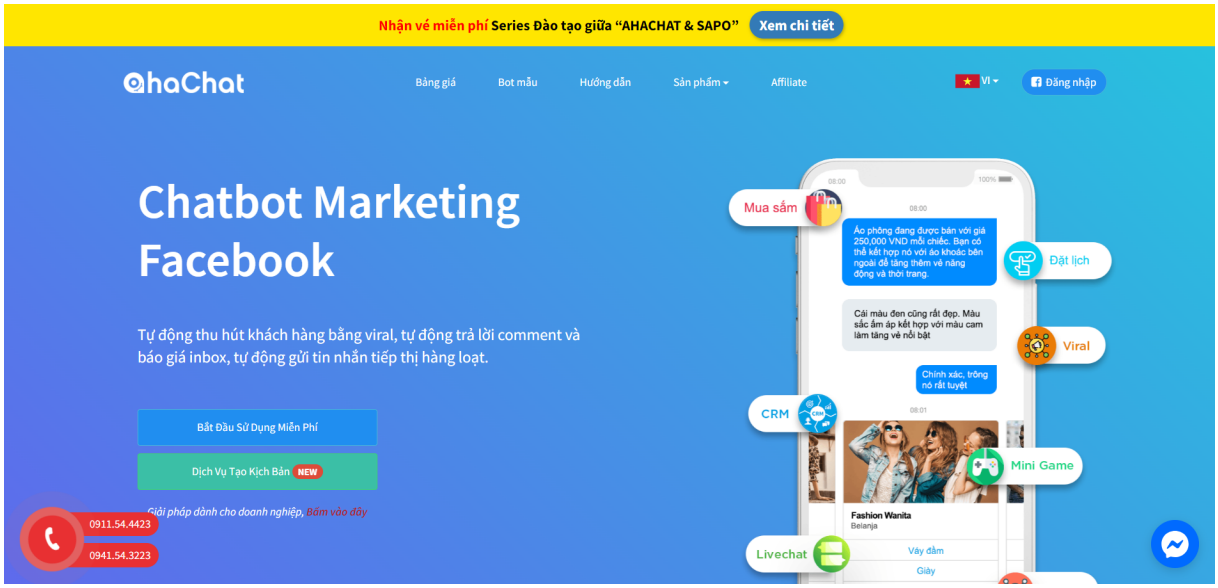
2.1.1 Giới thiệu

AhaChat là một nền tảng tạo chatbot phổ biến tại Việt Nam, được thiết kế để hỗ trợ doanh nghiệp tự động hóa quy trình chăm sóc khách hàng và bán hàng qua các kênh như Facebook Messenger, Zalo và Instagram. Với giao diện trực quan và khả năng tạo chatbot không cần lập trình, AhaChat giúp doanh nghiệp dễ dàng thiết lập các kịch bản trò chuyện tự động, từ việc tư vấn sản phẩm, xử lý đơn hàng, đến chăm sóc khách hàng sau bán.

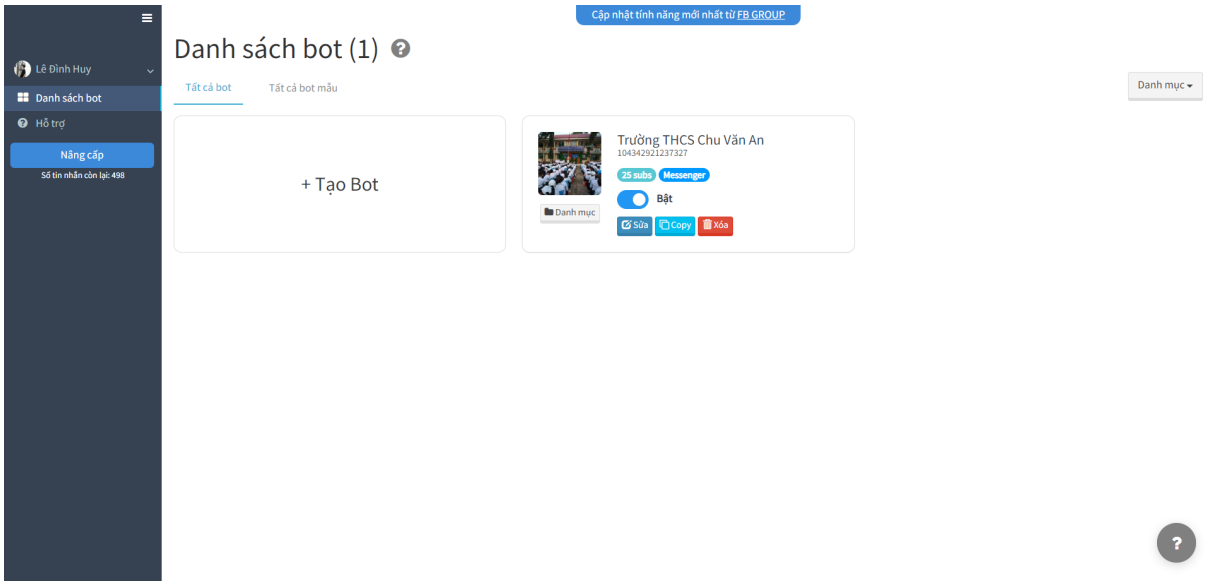
AhaChat được phát triển để giải quyết những thách thức mà nhiều doanh nghiệp gặp phải trong việc duy trì kết nối nhanh chóng và liên tục với khách hàng, đặc biệt qua các nền tảng mạng xã hội như Facebook và Zalo. Nhu cầu tự động hóa các tác vụ như trả lời tin nhắn, xử lý đơn hàng và chăm sóc khách hàng ngày càng trở nên cấp thiết khi số lượng người dùng trực tuyến tăng mạnh. AhaChat ra đời với sứ mệnh hỗ trợ doanh nghiệp giải quyết những vấn đề này thông qua chatbot, giúp tiết kiệm thời gian, chi phí, và tăng cường hiệu quả trong giao tiếp.

Link truy cập

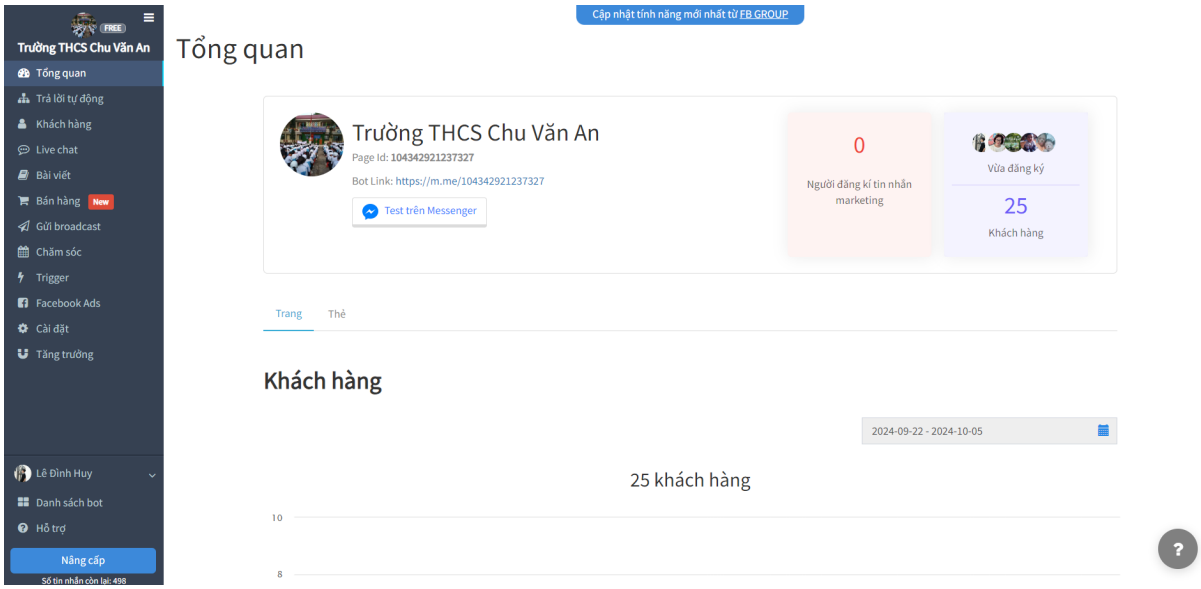
2.1.2 UI/UX



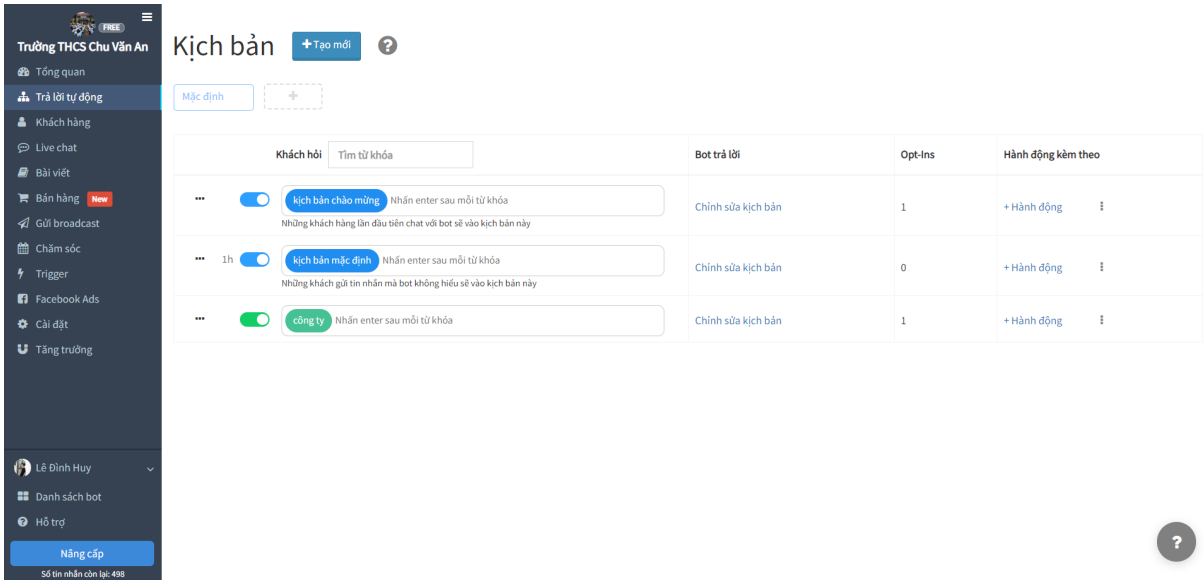
Hình 4: *Giao diện trang Landing page của AhaChat*



Hình 5: *Giao diện trang chủ của AhaChat*



Hình 6: *Giao diện quản lý fanpage Facebook của AhaChat*



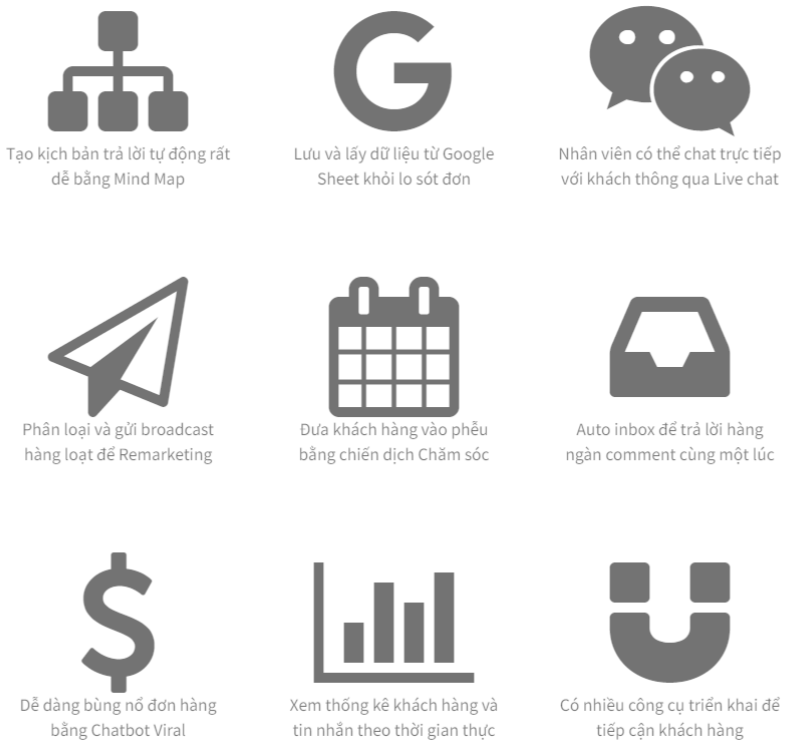
Hình 7: *Giao diện quản lý kịch bản của AhaChat*

2.1.3 Các tính năng chính

AhaChat là một trong những nền tảng chatbot nổi bật tại Việt Nam, giúp doanh nghiệp tự động hóa giao tiếp và chăm sóc khách hàng hiệu quả. Dưới đây là các tính năng nổi bật giúp tối ưu quy trình bán hàng và tương tác khách hàng trên nhiều kênh trực tuyến.

- **Tạo kịch bản trả lời tự động rất dễ bằng Mind Map:** Người dùng có thể xây dựng kịch bản chatbot một cách trực quan bằng sơ đồ tư duy, giúp dễ dàng tạo ra các cuộc hội thoại logic và hiệu quả.
- **Lưu và lấy dữ liệu từ Google Sheets khỏi lo sốt đơn:** Tích hợp Google Sheets giúp doanh nghiệp quản lý và theo dõi đơn hàng dễ dàng, không bỏ sót bất kỳ yêu cầu nào từ khách hàng.
- **Nhân viên có thể chat trực tiếp với khách thông qua Live Chat:** Tính năng Live Chat cho phép nhân viên chăm sóc khách hàng giao tiếp trực tiếp với khách qua chatbot, tăng cường trải nghiệm khách hàng.
- **Phân loại và gửi broadcast hàng loạt để Remarketing:** Doanh nghiệp có thể phân loại khách hàng và gửi tin nhắn hàng loạt cho các chiến dịch tiếp thị lại, giúp thu hút khách hàng tiềm năng quay trở lại.
- **Đưa khách hàng vào phễu bằng chiến dịch Chăm sóc:** Quản lý khách hàng theo phễu tiếp thị và chăm sóc tự động, giúp duy trì mối quan hệ lâu dài với khách hàng.

- **Auto Inbox để trả lời hàng ngàn comment cùng một lúc:** Chức năng tự động gửi tin nhắn trả lời vào hộp thư của hàng ngàn khách hàng cùng lúc, tiết kiệm thời gian quản lý tương tác.
- **Dễ dàng bùng nổ đơn hàng bằng Chatbot Viral:** Sử dụng tính năng chatbot để lan truyền nhanh chóng các thông tin về sản phẩm/dịch vụ, thúc đẩy sự tăng trưởng doanh số.
- **Xem thống kê khách hàng và tin nhắn theo thời gian thực:** Công cụ phân tích cho phép theo dõi và báo cáo khách hàng, tình hình tương tác và hiệu suất của chatbot trong thời gian thực.
- **Có nhiều công cụ triển khai để tiếp cận khách hàng:** AhaChat cung cấp các công cụ đa dạng giúp doanh nghiệp dễ dàng triển khai các chiến lược tiếp cận và thu hút khách hàng hiệu quả hơn.



Hình 8: *Giao diện quản lý kịch bản của AhaChat*

2.1.4 Phân tích SWOT

Strengths	Weaknesses
• Giao diện dễ sử dụng với tính năng kéo-thả Mind Map. • Hỗ trợ tích hợp đa kênh (Facebook, Messenger, Zalo, Instagram). • Tích hợp công cụ CRM giúp quản lý thông tin khách hàng hiệu quả. • Tính năng tự động hóa quy trình bán hàng và chăm sóc khách hàng mạnh mẽ. • Nhiều công cụ marketing như broadcast, remarketing, và chăm sóc tự động.	• Chỉ tập trung chủ yếu vào thị trường Việt Nam và chưa hỗ trợ chatbot trên website. • Khả năng tùy chỉnh nâng cao có thể không đa dạng so với các nền tảng chatbot cao cấp hơn.
Opportunities	**Threats**
• Nhu cầu ngày càng tăng về tự động hóa trong chăm sóc khách hàng và bán hàng trực tuyến tại Việt Nam. • Tiềm năng mở rộng tích hợp với các hệ thống thanh toán, vận chuyển quốc tế.	• Cạnh tranh gay gắt từ các nền tảng mạnh khác ở Việt Nam cũng như quốc tế. • Tốc độ phát triển của công nghệ AI nhanh chóng, yêu cầu cải tiến liên tục để duy trì sự cạnh tranh.

Bảng 1: *Bảng phân tích SWOT cho hệ thống AhaChat*

2.2 Fchat

2.2.1 Giới thiệu

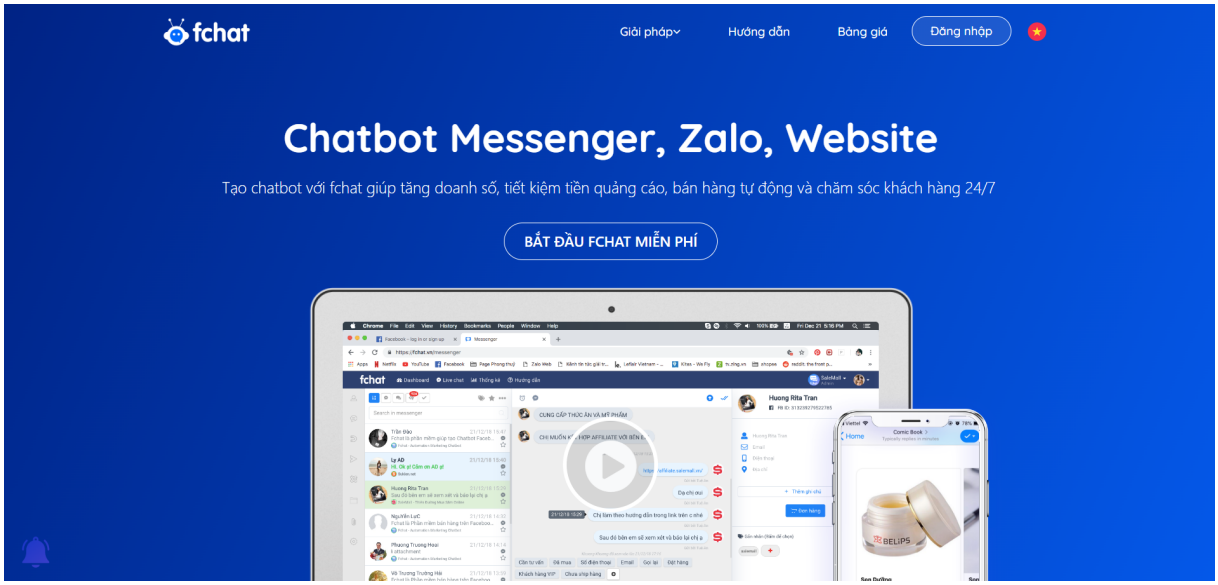
Fchat phát triển bởi Công ty Cổ phần SaleMall (SaleMall JSC) - đơn vị chuyên về các phần mềm quản lý bán hàng, chăm sóc khách hàng và hệ thống marketing. SaleMall nằm trong hệ sinh thái của Inet Group - tập đoàn hơn 18 năm hoạt động với hệ sinh thái đa dạng về công nghệ thông tin từ tên miền, hosting, VAT, website, đào tạo trực tuyến...

Theo đại diện SaleMall, cùng với sự phát triển mạnh mẽ của thương mại điện tử Việt Nam trong những năm gần đây, doanh nghiệp bán hàng luôn tìm kiếm giải pháp giao tiếp hiệu quả trên các nền tảng mạng xã hội, sàn thương mại điện tử, website... Công nghệ mang lại những cơ hội mới nhưng cũng ra tạo thách thức với chủ kinh doanh nếu không cập nhật kịp thời.

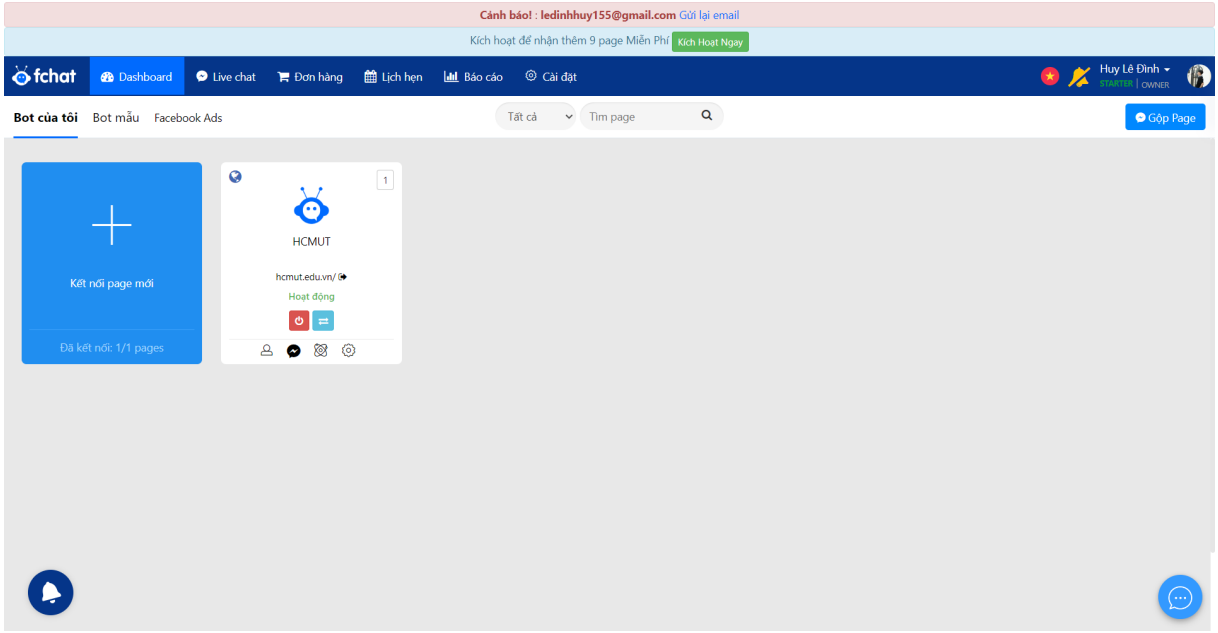
Để giải quyết khó khăn của những nhà bán hàng, Fchat mang đến giải pháp chăm sóc khách hàng hiệu quả hơn. Phần mềm chatbot có khả năng tự động trả lời câu hỏi 24/7, hỗ trợ trả lời tin nhắn và chăm sóc hàng nghìn người cùng một lúc mà không bị gián đoạn. Điều này giúp Fchat trở thành công cụ hỗ trợ hiệu quả cho doanh nghiệp kinh doanh online.

Link truy cập

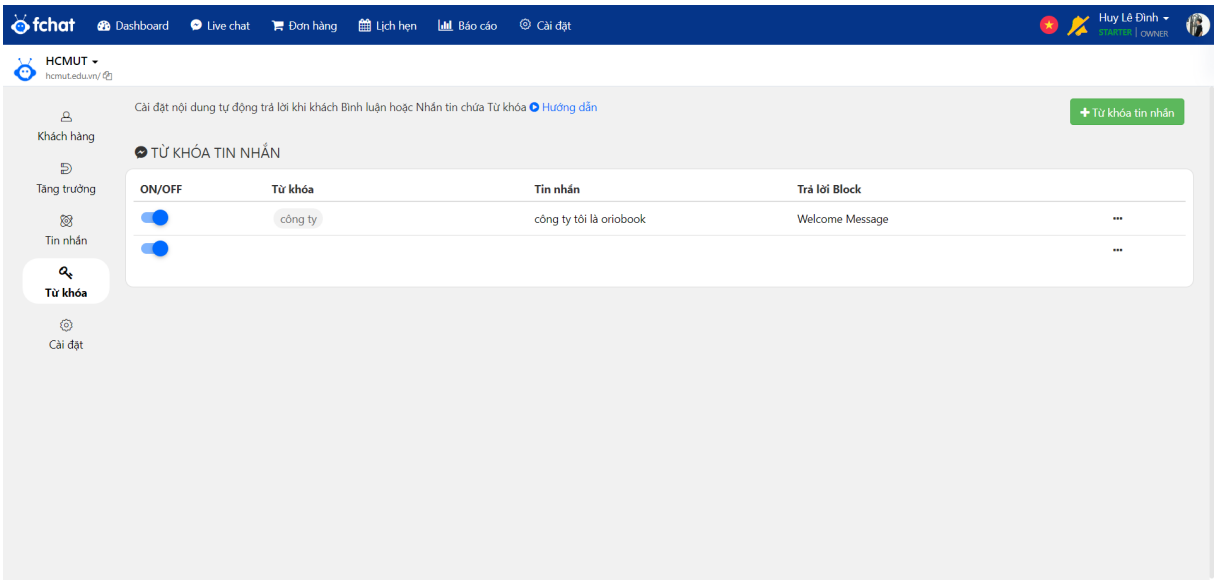
2.2.2 UI/UX



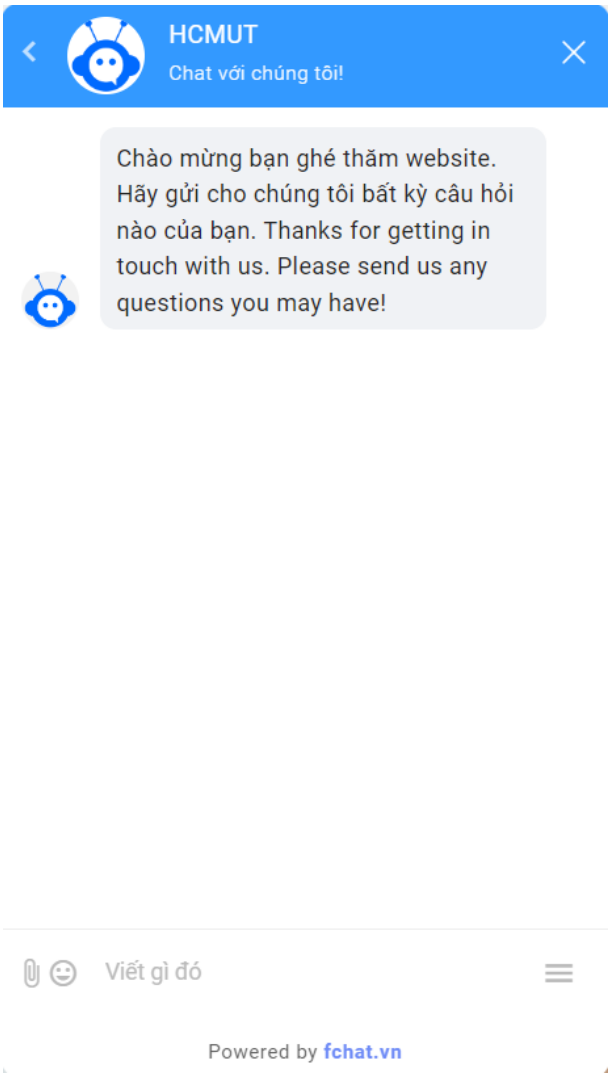
Hình 9: *Giao diện trang Landing page của Fchat*



Hình 10: *Giao diện trang Dashboard của Fchat*



Hình 11: *Giao diện quản lý từ khóa tin nhắn của Fchat*

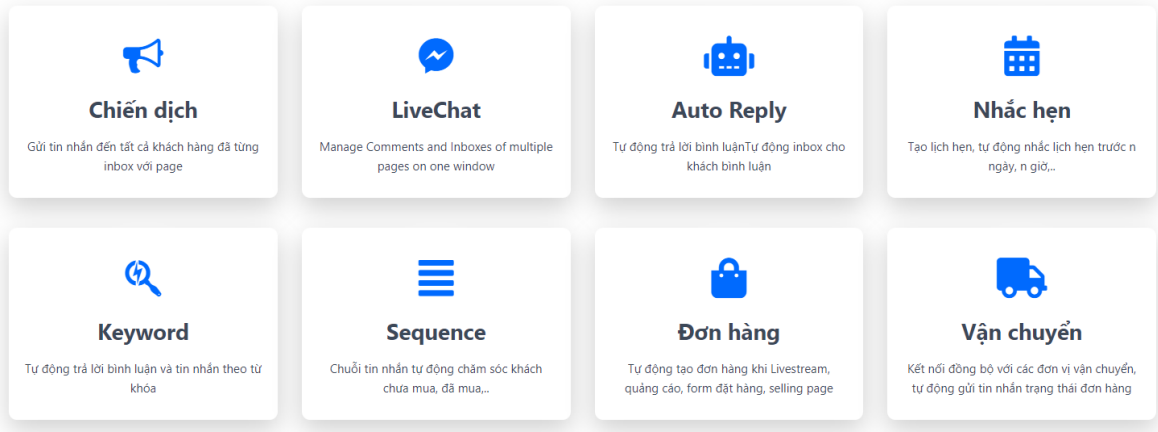


Hình 12: *Giao diện khung chat của Fchat*

2.2.3 Các tính năng chính

Fchat là một nền tảng chatbot dành cho doanh nghiệp, tập trung vào việc tự động hóa giao tiếp với khách hàng thông qua Facebook Messenger và các kênh khác. Dưới đây là các tính năng chính và mô tả về Fchat:

- **Tự động trả lời tin nhắn:** Fchat cho phép thiết lập chatbot tự động trả lời các tin nhắn từ khách hàng trên Facebook Messenger, giúp doanh nghiệp phản hồi nhanh chóng và không bị gián đoạn.
- **Tự động bình luận và trả lời bình luận trên bài viết:** Chatbot Fchat có thể tự động trả lời bình luận của khách hàng trên các bài viết Facebook, đồng thời gửi tin nhắn riêng (inbox) để tiếp tục tư vấn hoặc quảng bá sản phẩm.
- **Gửi tin nhắn hàng loạt (Broadcasting):** Fchat hỗ trợ tính năng gửi tin nhắn hàng loạt đến nhiều khách hàng cùng lúc, giúp doanh nghiệp tiếp thị và quảng bá sản phẩm hiệu quả.
- **Tạo lịch hẹn và tự động nhắc nhở:** Fchat hỗ trợ chức năng tạo lịch hẹn với khách hàng ngay trong hội thoại, đồng thời gửi thông báo nhắc nhở tự động trước số ngày, số giờ tùy chỉnh.
- **Tạo đơn hàng tự động:** Fchat hỗ trợ doanh nghiệp tự động tạo đơn hàng khi khách hàng tương tác qua các kênh như Livestream, quảng cáo, form đặt hàng hoặc trang bán hàng (selling page). Điều này giúp quy trình bán hàng trở nên nhanh chóng và tiện lợi.
- **Kết nối và đồng bộ với các đơn vị vận chuyển:** Fchat có thể kết nối với các đơn vị vận chuyển như Giao Hàng Nhanh, Giao Hàng Tiết Kiệm, Viettel Post, v.v., giúp tự động đồng bộ và quản lý đơn hàng vận chuyển.
- **Thống kê và báo cáo chi tiết:** Fchat cung cấp các công cụ thống kê chi tiết về hiệu quả tương tác của chatbot, giúp doanh nghiệp nắm bắt được mức độ hiệu quả của từng chiến dịch và kịch bản.



Hình 13: *Các tính năng cơ bản của Fchat*

2.2.4 Phân tích SWOT

Strengths	Weaknesses
• Tích hợp tính năng hỗ trợ tự động hóa bán hàng qua Livestream. • Hỗ trợ tạo lịch hẹn và tự động nhắc nhở khách hàng. • Kết nối với các đơn vị vận chuyển và tự động cập nhật trạng thái đơn hàng. • Nền tảng thân thiện với người dùng, dễ dàng tích hợp vào các hệ thống kinh doanh.	• Tập trung vào Facebook Messenger, website, thiếu đa dạng kênh hỗ trợ như Instagram hoặc Zalo. • Quá nhiều tính năng gây phức tạp, khó thể thành thạo trong khoảng thời gian ngắn.
Opportunities	**Threats**
• Tiềm năng lớn trong thị trường livestream bán hàng, đặc biệt tại Việt Nam. • Khả năng phát triển các tính năng tích hợp cho doanh nghiệp trong lĩnh vực thương mại điện tử.	• Các nền tảng cạnh tranh khác ngày càng hoàn thiện và cung cấp các tính năng tương tự. • Yêu cầu phát triển tính năng mới liên tục để đáp ứng nhu cầu người dùng.

Bảng 2: *Bảng phân tích SWOT cho hệ thống Fchat*

2.3 TuDongChat

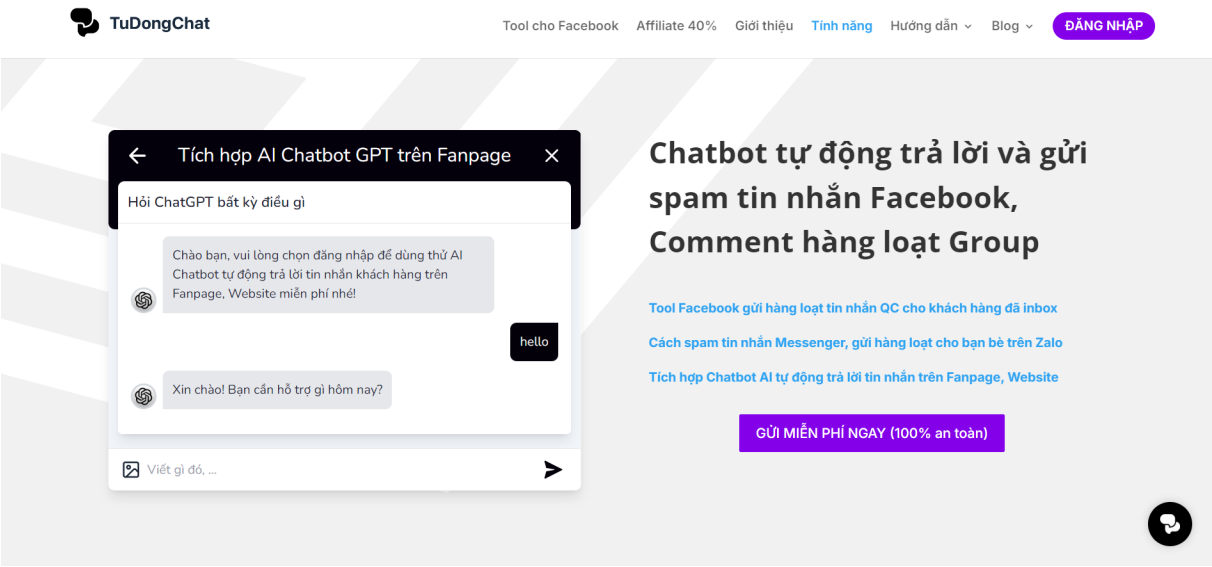
2.3.1 Giới thiệu

TuDongChat là công cụ AI Chatbot sử dụng sức mạnh của trí tuệ nhân tạo nhằm xây dựng những tính năng tự động hàng loạt, thay thế đến 99 % sức mạnh con người. Với công cụ này, người dùng có thể tích hợp vào bất cứ nền tảng nào như website, Facebook, Zalo một cách dễ dàng mà không cần biết về kiến thức lập trình.

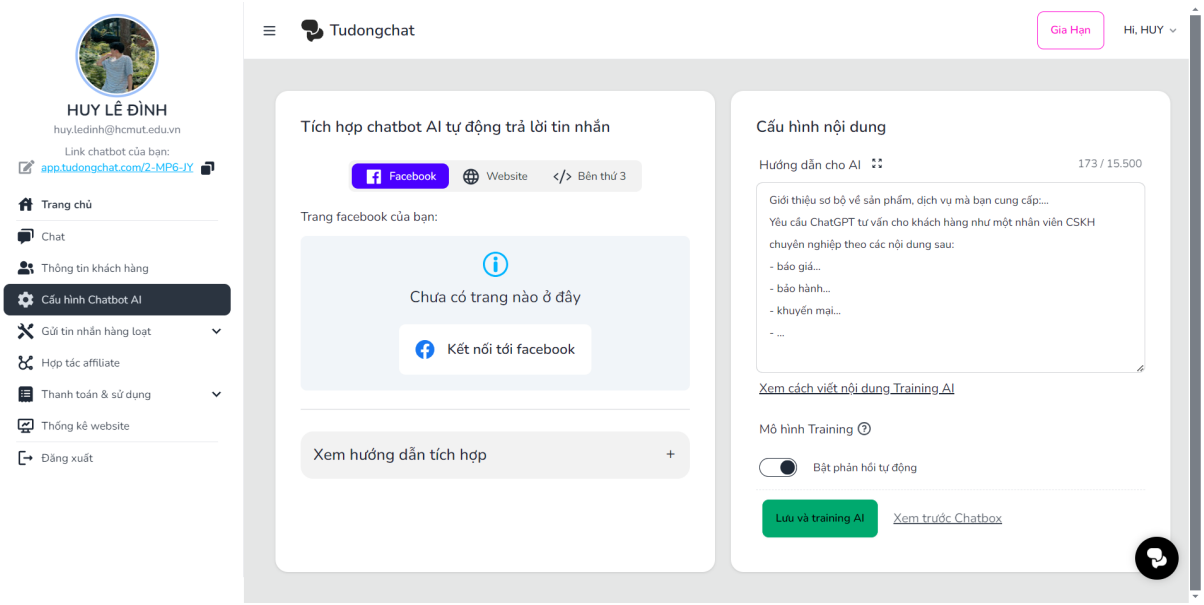
Được phát triển bởi công ty Anthropic, một trong những công ty dẫn đầu về công nghệ trí tuệ nhân tạo. TuDongChat ra mắt vào tháng 6/2022 với sứ mệnh cung cấp giải pháp Chatbot tiện lợi, hiệu quả cho các doanh nghiệp. Mặc dù chỉ mới hoạt động trên thị trường được 2 năm, song phần mềm này nhanh chóng nhận được sự quan tâm của đông đảo khách hàng.

Link truy cập

2.3.2 UI/UX



Hình 14: *Giao diện trang Landing page của TuDongChat*



Hình 15: *Giao diện Trang chủ của TuDongChat*

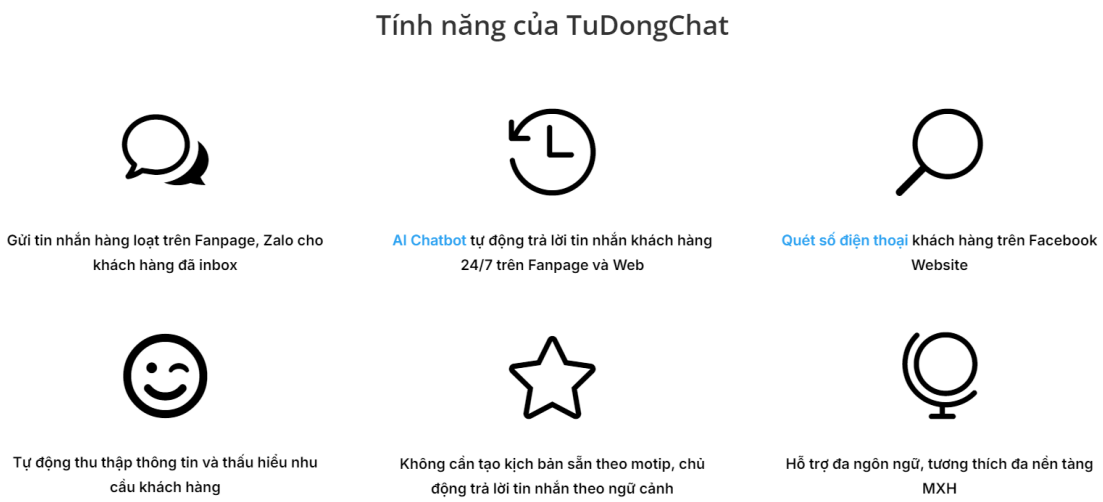


Hình 16: *Giao diện khung chat của TuDongChat*

2.3.3 Các tính năng chính

Với sức mạnh của trí tuệ nhân tạo, TuDongChat cung cấp các tính năng cơ bản sau:

- **Tự động gửi tin hàng loạt:** Giúp doanh nghiệp gửi thông tin tiếp thị cho khách hàng về sản phẩm mới, chương trình khuyến mãi hoặc sự kiện sắp diễn ra. Bằng việc sử dụng trí tuệ nhân tạo để tùy biến thông điệp một cách linh hoạt, không spam, đảm bảo an toàn tuyệt đối cho tài khoản mạng xã hội Facebook & Zalo của bạn.
- **Tự động tìm kiếm khách hàng tiềm năng:** AI Chatbot TuDongChat không dừng lại ở việc chờ đợi khách hàng liên hệ, mà còn tự động tham gia vào các hội nhóm Facebook, tìm kiếm các bài viết liên quan và tương tác với người dùng tiềm năng, giúp bạn mở rộng mạng lưới khách hàng một cách tự nhiên và linh hoạt.
- **Tự động spam comment Facebook:** AI Chatbot TuDongChat với khả năng đọc hiểu và sáng tạo nội dung theo ngữ cảnh, sẽ giúp bạn spam comment trên các hội nhóm Facebook nhưng vẫn đảm bảo an toàn 100% cho nick Facebook của bạn.
- **Tự động tương tác và trả lời khách hàng một cách tự nhiên:** Cuối cùng nhưng không kém phần quan trọng, chatbot của chúng tôi không chỉ là một trợ lý cấp cao. Chúng tôi sử dụng trí tuệ nhân tạo ChatGPT để tương tác với khách hàng trên mọi nền tảng – từ website đến fanpage Facebook của bạn. Từ việc trả lời các câu hỏi cơ bản đến việc hỗ trợ mua hàng, chúng tôi đảm bảo rằng mỗi cuộc trò chuyện đều diễn ra một cách tự nhiên và đáp ứng nhanh chóng.



Hình 17: *Các tính năng cơ bản của TuDongChat*

2.3.4 Phân tích SWOT

Strengths	Weaknesses
• Hỗ trợ tạo chatbot tự động cho doanh nghiệp mà không yêu cầu kỹ năng lập trình. • Tính năng tích hợp đa kênh từ Facebook Messenger, Zalo đến website. • Giao diện quản lý đơn giản và tập trung vào tự động hóa các tác vụ liên quan đến chăm sóc khách hàng.	• Tính năng chưa đa dạng bằng các nền tảng lớn khác, hạn chế trong việc tùy chỉnh các kịch bản phức tạp. • Thiếu các công cụ phân tích sâu sắc để theo dõi hành vi khách hàng và hiệu quả chatbot.
Opportunities	**Threats**
• Tăng trưởng của thị trường chatbot tại Việt Nam, đặc biệt trong các ngành bán lẻ và dịch vụ. • Cơ hội phát triển và mở rộng thêm tính năng tự động hóa nâng cao và tích hợp với nhiều nền tảng khác.	• Sự xuất hiện của các đối thủ cạnh tranh lớn trong nước và quốc tế với nhiều tính năng hơn. • Công nghệ phát triển nhanh chóng đòi hỏi liên tục đổi mới để theo kịp xu hướng.

Bảng 3: *Bảng phân tích SWOT cho hệ thống TuDongChat*

3 Phân tích hệ thống

3.1 Stakeholders

3.1.1 Đội phát triển phần mềm

Nhóm phát triển phần mềm chịu trách nhiệm xây dựng, thử nghiệm và triển khai sản phẩm theo các yêu cầu được cung cấp.

- **Nhu cầu:**
  - Nhóm cần có phạm vi dự án và các yêu cầu được xác định rõ ràng để đảm bảo lập kế hoạch hiệu quả và đồng nhất với các mục tiêu được đặt ra.

  - Nhóm cần được tạo điều kiện tiếp cận các công cụ và công nghệ cần thiết để thực hiện nhiệm vụ của mình một cách hiệu quả và sáng tạo.

- **Ảnh hưởng:** Nhóm phát triển có ảnh hưởng lớn do nhóm tham gia trực tiếp vào quá trình thực hiện dự án. Công việc của nhóm tác động trực tiếp đến chất lượng và chức năng của sản phẩm cuối cùng, khiến cho sự đóng góp và sự hài lòng của nhóm phát triển quan trọng đối với sự thành công của dự án.

3.1.2 Chủ doanh nghiệp

Đây là những chủ doanh nghiệp sử dụng hệ thống để tạo và triển khai chatbot AI trên trang web của họ.

- **Nhu cầu:**
  - Người dùng cần một giao diện trực quan và tích hợp liền mạch để tăng hiệu quả và giảm thiểu thời gian triển khai.
  - Chatbot cần có khả năng tùy chỉnh giao diện để phù hợp với nhận diện thương hiệu của doanh nghiệp.
  - Chức năng của chatbot cần đáng tin cậy và hiệu suất cao để duy trì sự hài lòng của người dùng.
  - Hệ thống cần đảm bảo bảo vệ dữ liệu và tuân thủ các quy định liên quan.
  - Cần có kênh hỗ trợ nhanh chóng và tài liệu hướng dẫn chi tiết giúp người dùng sử dụng nền tảng một cách hiệu quả.
  - Giá cả rõ ràng và hợp lý giúp người dùng quản lý ngân sách hợp lý.
  - Quản lý tổ chức và các thành viên của tổ chức
- **Ảnh hưởng:** Các công ty khách hàng có ảnh hưởng lớn vì các yêu cầu của họ định hình mạnh mẽ các tính năng và chức năng của sản phẩm. Phản hồi và nhu cầu của họ định hình nên hướng phát triển của hệ thống.

3.1.3 Quản trị viên doanh nghiệp

Quản trị viên hỗ trợ chủ sở hữu doanh nghiệp trong việc quản lý chatbot của doanh nghiệp mình.

- **Nhu cầu:**
  - Nhu cầu tương tự chủ doanh nghiệp, không đòi hỏi việc quản lý tổ chức
- **Ảnh hưởng:** Quản trị viên hệ thống có ảnh hưởng cao do vai trò quan trọng của họ gần như tương đương với chủ sở hữu doanh nghiệp

3.1.4 Nhóm hỗ trợ khách hàng cho doanh nghiệp

Các nhóm này xử lý các yêu cầu phức tạp của khách hàng và quản lý các tương tác được chuyển tiếp từ chatbot.

- **Nhu cầu:**
  - Hệ thống có khả năng chuyển đổi liền mạch giữa chatbot và các tác nhân con người để duy trì chất lượng dịch vụ.
  - Cho phép truy cập vào lịch sử hội thoại và phân tích, từ đó hiểu các vấn đề của khách hàng và cải thiện dịch vụ.
  - Việc nhắn tin với khách hàng cần đáp ứng phản hồi theo thời gian thực nhằm hỗ trợ kịp thời nhu cầu của khách hàng
- **Ảnh hưởng:** Các nhóm hỗ trợ khách hàng có ảnh hưởng trung bình vì phản hồi và kinh nghiệm của họ ảnh hưởng đến khả năng sử dụng và hiệu quả của hệ thống.

3.1.5 Người dùng cuối (Khách hàng của các doanh nghiệp khách hàng)

Đây là những cá nhân tương tác với chatbot trên trang web của doanh nghiệp nhằm tìm kiếm thông tin hoặc có nhu cầu hỗ trợ.

- **Nhu cầu:**
  - Chatbot cần có khả năng phản hồi nhanh chóng và chính xác cho các truy vấn.
  - Tương tác với chatbot được cá nhân hóa, cải thiện mức độ tương tác và trải nghiệm của người dùng.

  - Hệ thống cần đảm bảo quyền riêng tư và bảo mật dữ liệu của người dùng.
  - Có khả năng chuyển đổi tượng giao tiếp từ AI qua con người để đảm bảo hỗ trợ toàn diện.
- **Ảnh hưởng:** Người dùng cuối có ảnh hưởng trực tiếp thấp nhưng tác động gián tiếp đến dự án thông qua hành vi sử dụng và phản hồi của họ, thông tin này cũng sẽ định hình cho các cải tiến của hệ thống.

3.1.6 Nhóm đào tạo và hỗ trợ

Các nhóm này chịu trách nhiệm cung cấp dịch vụ hướng dẫn, đào tạo và hỗ trợ liên tục cho các công ty khách hàng.

- **Nhu cầu:**
  - Phát triển tài liệu hướng dẫn toàn diện, chi tiết giúp khách hàng hiểu và sử dụng nền tảng hiệu quả.
  - Giải quyết hiệu quả các vấn đề và thắc mắc: Hỗ trợ nhanh chóng và hiệu quả là điều cần thiết để duy trì lòng tin của khách hàng.
  - Thu thập phản hồi để cải tiến liên tục: Thu thập phản hồi giúp tinh chỉnh các quy trình đào tạo và hỗ trợ.
- **Ảnh hưởng:** Các nhóm đào tạo và hỗ trợ có ảnh hưởng từ thấp đến trung bình, chủ yếu thông qua tác động của họ đến tỷ lệ áp dụng và sự hài lòng của khách hàng thông qua hiệu quả đào tạo.

3.2 User Stories

3.2.1 Chủ doanh nghiệp

- **Là** chủ doanh nghiệp, **tôi muốn** quy trình đăng ký tài khoản mới an toàn với xác minh email và đăng nhập bảo mật, **để** bảo vệ thông tin công ty và truy cập hệ thống một cách an toàn.
- **Là** chủ doanh nghiệp, **tôi muốn** quản lý và cập nhật hồ sơ công ty, bao gồm tên, logo và thông tin liên hệ, **để** duy trì thông tin chính xác và phù hợp với thương hiệu.
- **Là** chủ doanh nghiệp, **tôi muốn** quản lý các thành viên của công ty, **để** các thành viên sẽ hỗ trợ công ty trong việc quản lý chatbot cũng như chặm sóc khách hàng.
- **Là** chủ doanh nghiệp, **tôi muốn** tải lên, quản lý và phân loại dữ liệu tri thức đa dạng, **để** xây dựng và duy trì cơ sở tri thức cho chatbot một cách hiệu quả.
- **Là** chủ doanh nghiệp, **tôi muốn** tùy chỉnh giao diện và hành vi của chatbot bao gồm giọng điệu, lời chào và màu sắc, **để** phù hợp với nhận diện thương hiệu và tạo trải nghiệm người dùng tốt hơn.
- **Là** chủ doanh nghiệp, **tôi muốn** đào tạo mô hình AI, **để** có thể cập nhật cập nhật tri thức mới lên chatbot và chatbot hoạt động hiệu quả khi đào tạo xong.
- **Là** chủ doanh nghiệp, **tôi muốn** tích hợp mã nhúng JavaScript vào trang web doanh nghiệp với hướng dẫn chi tiết và tối ưu hóa hiệu suất, **để** triển khai chatbot dễ dàng và không ảnh hưởng đến trải nghiệm người dùng.
- **Là** chủ doanh nghiệp, **tôi muốn** bảng điều khiển quản lý chatbot hiển thị dữ liệu thời gian thực và tạo báo cáo phân tích chi tiết, **để** theo dõi số liệu để có thể đánh giá và cải thiện chất lượng dịch vụ.
- **Là** chủ doanh nghiệp, **tôi muốn** quản lý thanh toán với các gói giá linh hoạt, xử lý thanh toán an toàn và theo dõi việc sử dụng, **để** quản lý ngân sách và chi phí hiệu quả.
- **Là** chủ doanh nghiệp, **tôi muốn** thiết lập giới hạn truy vấn của khách hàng, **để** tránh các trường hợp phá hoại không mong muốn, hao hụt tài nguyên của doanh nghiệp.

3.2.2 Quản trị viên doanh nghiệp

- **Là** quản trị viên doanh nghiệp, **tôi muốn** quy trình đăng ký tài khoản mới an toàn với xác minh email và đăng nhập bảo mật, **để** bảo vệ thông tin công ty và truy cập hệ thống một cách an toàn.
- **Là** quản trị viên doanh nghiệp, **tôi muốn** tải lên, quản lý và phân loại dữ liệu tri thức đa dạng, **để** xây dựng và duy trì cơ sở tri thức cho chatbot một cách hiệu quả.

- **Là** quản trị viên doanh nghiệp, **tôi muốn** tùy chỉnh giao diện và hành vi của chatbot bao gồm giọng điệu, lời chào và màu sắc, **để** phù hợp với nhận diện thương hiệu và tạo trải nghiệm người dùng tốt hơn.
- **Là** quản trị viên doanh nghiệp, **tôi muốn** đào tạo mô hình AI, **để** có thể cập nhật cập nhật tri thức mới lên chatbot và chatbot hoạt động hiệu quả khi đào tạo xong.
- **Là** quản trị viên doanh nghiệp, **tôi muốn** tích hợp mã nhúng JavaScript vào trang web doanh nghiệp với hướng dẫn chi tiết và tối ưu hóa hiệu suất, **để** triển khai chatbot dễ dàng và không ảnh hưởng đến trải nghiệm người dùng.
- **Là** quản trị viên doanh nghiệp, **tôi muốn** bảng điều khiển quản lý chatbot hiển thị dữ liệu thời gian thực và tạo báo cáo phân tích chi tiết, **để** theo dõi số liệu để có thể đánh giá và cải thiện chất lượng dịch vụ.
- **Là** quản trị viên doanh nghiệp, **tôi muốn** quản lý thanh toán với các gói giá linh hoạt, xử lý thanh toán an toàn và theo dõi việc sử dụng, **để** quản lý ngân sách và chi phí hiệu quả.
- **Là** quản trị viên doanh nghiệp, **tôi muốn** thiết lập giới hạn truy vấn của khách hàng, **để** tránh các trường hợp phá hoại không mong muốn, hao hụt tài nguyên của doanh nghiệp.

3.2.3 Nhóm hỗ trợ khách hàng của doanh nghiệp

- **Là** nhóm hỗ trợ khách hàng, **tôi muốn** chuyển giao liền mạch các cuộc trò chuyện từ chatbot sang nhân viên hỗ trợ với thông báo thời gian thực và truy cập lịch sử cuộc trò chuyện, **để** giải quyết các vấn đề phức tạp một cách hiệu quả và duy trì ngữ cảnh.
- **Là** nhóm hỗ trợ khách hàng, **tôi muốn** hệ thống lưu trữ an toàn lịch sử hội thoại và truy xuất dữ liệu khi cần, **để** phân tích và cải thiện chất lượng hỗ trợ.

3.2.4 Người dùng cuối (Khách hàng của các doanh nghiệp khách hàng)

- **Là** người dùng cuối, **tôi muốn** chatbot hỗ trợ tiếng Việt, duy trì ngữ cảnh trong cuộc trò chuyện và hiểu rõ ý định người dùng, **để** giao tiếp dễ dàng và nhận được trợ giúp mạch lạc.
- **Là** người dùng cuối, **tôi muốn** chatbot phản hồi nhanh chóng và chính xác, cũng như có thể đề nghị đặt lại truy vấn hoặc cung cấp thêm thông tin khi cần, **để** nhận được trợ giúp kịp thời và phù hợp.
- **Là** người dùng cuối, **tôi muốn** được chuyển giao cuộc trò chuyện sang nhân viên hỗ trợ khi cần thiết **để** giải quyết vấn đề một cách hiệu quả và lên kế hoạch tương tác hợp lý.

3.3 Yêu cầu chức năng

3.3.1 Đăng ký và xác thực

- Hệ thống phải cung cấp chức năng đăng ký tài khoản mới cho tổ chức, bao gồm bước xác thực địa chỉ email.
- Hệ thống phải cho phép người dùng đăng nhập bằng email và mật khẩu đã đăng ký.
- Hệ thống phải cung cấp chức năng đặt lại mật khẩu, trong đó người dùng nhận được một liên kết qua email có giới hạn thời gian hiệu lực.

3.3.2 Quản lý tài khoản tổ chức

- Hệ thống phải cho phép quản trị viên tổ chức cập nhật thông tin hồ sơ của tổ chức, bao gồm: tên tổ chức, logo, thông tin liên hệ và địa chỉ.
- Hệ thống phải cho phép quản trị viên tổ chức gửi lời mời tham gia tổ chức cho thành viên mới qua email.
- Hệ thống phải hỗ trợ phân quyền truy cập cho thành viên tổ chức dựa trên các vai trò được định nghĩa, bao gồm 3 vai trò: Chủ sở hữu, Quản trị viên, và Nhân viên chăm sóc khách hàng.
- Hệ thống phải cho phép quản trị viên tổ chức quản lý danh sách thành viên, bao gồm chức năng thêm và xóa thành viên.

3.3.3 Tùy chỉnh giao diện và hành vi chatbot

- Hệ thống phải cho phép người dùng quản trị tùy chỉnh các yếu tố giao diện hiển thị của chatbot (ví dụ: bảng màu, logo, hình nền, phông chữ, kích thước) để đáp ứng nhận diện thương hiệu của tổ chức.
- Hệ thống phải cung cấp cho người dùng quản trị lựa chọn ít nhất 4 phong cách hội thoại được định nghĩa trước cho chatbot.
- Hệ thống phải cho phép người dùng quản trị tùy chỉnh các tin nhắn mặc định của chatbot, bao gồm lời chào và các phản hồi mặc định.
- Hệ thống phải cung cấp chức năng xem trước cho phép người dùng quản trị đánh giá các thay đổi về giao diện và cấu hình trước khi áp dụng.
- Hệ thống phải cho phép người dùng quản trị cấu hình giới hạn tần suất yêu cầu (rate limiting) đối với các tương tác của người dùng cuối với chatbot.

3.3.4 Tạo mã nhúng chatbot

- Hệ thống phải cung cấp chức năng tạo mã nhúng JavaScript để tích hợp chatbot, đảm bảo kích thước tải về ban đầu của script không vượt quá 100KB.
- Hệ thống phải cho phép quản trị viên tổ chức cấu hình danh sách các tên miền được phép nhúng và hiển thị chatbot.
- Giao diện chatbot nhúng phải đảm bảo hiển thị và hoạt động phù hợp trên các kích thước viewport khác nhau, tối thiểu từ 320px (thiết bị di động) đến 2560px (màn hình desktop lớn).

3.3.5 Quản lý cơ sở tri thức

- Hệ thống phải cho phép người dùng quản trị tải lên các tệp tài liệu với giới hạn kích thước tối đa 20MB cho mỗi tệp.
- Chức năng tải lên tài liệu phải hỗ trợ các định dạng tệp phổ biến, bao gồm: PDF, DOC, DOCX, và TXT.
- Hệ thống phải cung cấp các phương thức nhập liệu bổ sung cho cơ sở tri thức, bao gồm nhập văn bản trực tiếp qua giao diện và trích xuất nội dung từ địa chỉ URL được cung cấp.
- Hệ thống phải cung cấp giao diện quản lý cho phép người dùng quản trị thực hiện các thao tác trên cơ sở tri thức, bao gồm: thêm, chỉnh sửa và xóa tri thức.
- Giao diện quản lý cơ sở tri thức phải hỗ trợ chức năng tìm kiếm và lọc các đơn vị tri thức dựa trên các tiêu chí như: tên, loại hình, và mức độ ưu tiên.
- Hệ thống phải cho phép người dùng quản trị tải về các tệp tài liệu đã được tải lên hệ thống trước đó.
- Hệ thống phải đảm bảo rằng chỉ có một tiến trình huấn luyện được thực thi đồng thời cho mỗi tổ chức tại một thời điểm.

3.3.6 Trò chuyện với nhân viên hỗ trợ trực tiếp

- Hệ thống phải hỗ trợ chức năng chuyển giao liền mạch cuộc trò chuyện từ chatbot sang nhân viên hỗ trợ trực tuyến khi có yêu cầu từ người dùng.
- Hệ thống phải cung cấp giao diện cho phép nhân viên hỗ trợ tiếp nhận, theo dõi và phản hồi các cuộc trò chuyện được phân công cho họ.
- Hệ thống phải cung cấp cho quản trị viên tổ chức quyền truy cập để xem lịch sử hoặc nội dung các cuộc hội thoại, bao gồm cả các tương tác với chatbot và các phiên hỗ trợ trực tiếp bởi nhân viên.
- Hệ thống phải phân phối các yêu cầu hỗ trợ trực tiếp tới các nhân viên hỗ trợ một cách cân bằng.
- Hệ thống phải cho phép nhân viên hỗ trợ hoặc quản trị viên tổ chức kết thúc các phiên trò chuyện, ví dụ như khi vấn đề đã được giải quyết hoặc theo yêu cầu quản trị.

3.3.7 Báo cáo và thống kê

- Hệ thống phải cung cấp một giao diện hiển thị các chỉ số hiệu suất và sử dụng chính, bao gồm: tổng số tin nhắn đã xử lý, tổng số phiên hội thoại (phân loại theo chatbot và nhân viên hỗ trợ), và tổng chi phí sử dụng dịch vụ. Giao diện này cần hỗ trợ lọc dữ liệu theo khoảng thời gian.
- Hệ thống phải cho phép quản trị viên tổ chức xuất dữ liệu thống kê chi tiết hoặc các báo cáo tổng hợp từ giao diện báo cáo.

3.3.8 Quản lý Gói dịch vụ và Thanh toán

- Hệ thống phải cung cấp ít nhất 2 gói dịch vụ trả phí, mỗi gói có các tính năng và giới hạn sử dụng riêng biệt được mô tả chi tiết và rõ ràng.
- Ngoài các gói dịch vụ trả phí định kỳ, hệ thống cần hỗ trợ mô hình định giá theo mức sử dụng thực tế cho một số tài nguyên nhất định (ví dụ: dung lượng lưu trữ tài liệu, số lượng yêu cầu AI, số lượng tài khoản nhân viên hỗ trợ).
- Hệ thống phải cung cấp cho quản trị viên tổ chức khả năng theo dõi mức sử dụng tài nguyên hiện tại so với giới hạn của gói dịch vụ đang đăng ký.
- Hệ thống phải cung cấp tùy chọn đăng ký gói dùng thử miễn phí với thời hạn hoặc giới hạn sử dụng nhất định, đồng thời đảm bảo quy trình nâng cấp lên gói trả phí diễn ra thuận tiện và liền mạch cho người dùng.
- Hệ thống phải cung cấp cho quản trị viên tổ chức quyền truy cập vào lịch sử giao dịch và hóa đơn thanh toán chi tiết, bao gồm ngày tháng, số tiền, gói dịch vụ tương ứng và trạng thái thanh toán.

3.4 Yêu cầu phi chức năng

3.4.1 Hiệu suất (Performance)

- **Thời gian phản hồi Chatbot:** Dưới điều kiện tải thông thường (10-20 người dùng đồng thời), 90% phản hồi của chatbot đối với truy vấn người dùng phải được hoàn thành trong vòng 3 giây.
- **Thời gian xử lý tác vụ nền:** Thời gian trung bình để hệ thống trích xuất và xử lý nội dung từ một URL được cung cấp làm nguồn tri thức không được vượt quá 15 giây cho các trang web có kích thước thông thường (dưới 2MB HTML).
- **Khả năng chịu tải:** Hệ thống phải có khả năng xử lý đồng thời ít nhất 30 người dùng tương tác mà không làm tăng thời gian phản hồi trung bình của chatbot quá 50% so với điều kiện tải thông thường.
- **Khả năng mở rộng:** Kiến trúc hệ thống phải được thiết kế sử dụng các thành phần có khả năng mở rộng theo chiều ngang để cho phép xử lý tải tăng lên trong tương lai bằng cách bổ sung tài nguyên máy chủ, thay vì yêu cầu thiết kế lại toàn bộ hệ thống.

3.4.2 Tính khả dụng (Availability)

- **Độ sẵn sàng:** Hệ thống được thiết kế hướng tới mục tiêu độ sẵn sàng 99.5% trong giờ hoạt động dự kiến, không bao gồm thời gian bảo trì theo lịch trình.
- **Bảo trì theo lịch trình:** Mọi hoạt động bảo trì yêu cầu ngừng hệ thống phải được thông báo trước ít nhất 24 giờ và thực hiện trong khung giờ có lượng truy cập thấp. Tổng thời gian bảo trì theo lịch trình không vượt quá 4 giờ mỗi tháng.

3.4.3 Bảo mật (Security)

- **Chính sách mật khẩu:** Hệ thống phải thực thi chính sách mật khẩu mạnh, đảm bảo mật khẩu có độ dài tối thiểu 8 ký tự, không quá giống với thuộc tính người dùng, không nằm trong danh sách mật khẩu phổ biến, và không chỉ chứa toàn số. Hệ thống phải băm mật khẩu trước khi lưu trữ.
- **Kiểm soát truy cập:** Hệ thống phải triển khai Kiểm soát truy cập dựa trên vai trò với ít nhất 3 vai trò được định nghĩa (ví dụ: Chủ sở hữu, Quản trị viên, Nhân viên), đảm bảo người dùng chỉ có thể truy cập các chức năng và dữ liệu phù hợp với vai trò được gán.
- **Mã hóa dữ liệu truyền:** Toàn bộ giao tiếp giữa trình duyệt người dùng và máy chủ hệ thống phải được mã hóa bằng TLS 1.2 hoặc cao hơn.

- **Mã hóa dữ liệu lưu trữ:** Tất cả thông tin định danh cá nhân của người dùng và thông tin nhạy cảm của tổ chức phải được mã hóa khi lưu trữ trong cơ sở dữ liệu sử dụng thuật toán mã hóa đối xứng mạnh.
- **Ghi log bảo mật:** Hệ thống phải ghi log các sự kiện bảo mật quan trọng, bao gồm: đăng nhập thành công/thất bại, thay đổi mật khẩu, thay đổi quyền, tạo/xóa tài khoản quản trị. Log phải bao gồm dấu thời gian, định danh người dùng, địa chỉ IP nguồn, và mô tả sự kiện. Log phải được lưu trữ an toàn trong tối thiểu 90 ngày.

3.4.4 Khả năng sử dụng (Usability)

- **Hiệu quả tác vụ:** Mục tiêu là ít nhất 80% người dùng quản trị lần đầu, sau khi xem tài liệu hoặc video hướng dẫn dưới 15 phút, có thể hoàn thành thành công các tác vụ cốt lõi sau đây trong vòng 10 phút cho mỗi tác vụ: Tải lên thành công một tệp tài liệu và khởi tạo quá trình xử lý/huấn luyện; thay đổi thành công các thuộc tính giao diện chatbot và lưu lại cấu hình; và hoàn tất việc gửi lời mời tham gia tổ chức đến một địa chỉ email mới.
- **Tính nhất quán giao diện:** Các yếu tố điều hướng chính, thuật ngữ, và giao diện phải nhất quán trên tất cả các màn hình quản trị của hệ thống.
- **Khả năng truy cập:** Giao diện chatbot nhúng và các trang quản trị chính phải tuân thủ Web Content Accessibility Guidelines (WCAG) 2.1 Mức AA cho các tiêu chí sau: độ tương phản màu (4.5:1 cho văn bản thường, 3:1 cho văn bản lớn).

3.4.5 Độ tin cậy (Reliability)

- **Kiểm tra tính hợp lệ đầu vào:** Hệ thống phải thực hiện kiểm tra tính hợp lệ cho tất cả các dữ liệu đầu vào từ người dùng trước khi xử lý. Phải ngăn chặn dữ liệu không hợp lệ hoặc không đúng định dạng gây ra lỗi hoặc làm hỏng dữ liệu. Các lỗi xác thực phải được trả về rõ ràng cho người dùng.
- **Xử lý lỗi và ghi log:** 100% các lỗi không mong muốn ở phía máy chủ phải được ghi log với đầy đủ thông tin stack trace và ngữ cảnh để phục vụ việc sửa chữa. Người dùng cuối không được nhìn thấy thông tin lỗi chi tiết hoặc stack trace, thay vào đó là một thông báo lỗi thân thiện và mã tham chiếu.

3.4.6 Khả năng bảo trì (Maintainability)

- **Chất lượng mã nguồn:** Mã nguồn phía máy chủ phải tuân thủ theo hướng dẫn về phong cách lập trình của ngôn ngữ/framework được sử dụng. Phải sử dụng công cụ phân tích mã tĩnh tích hợp trong quy trình phát triển để phát hiện các vấn đề về phong cách và lỗi tiềm ẩn.
- **Tính mô-đun:** Hệ thống phải được cấu trúc thành các mô-đun logic rõ ràng (ví dụ: quản lý tài khoản, quản lý tri thức, xử lý hội thoại, giao diện nhúng) với các giao diện được định nghĩa tốt giữa chúng để giảm thiểu sự phụ thuộc chéo.
- **Độ bao phủ kiểm thử:** Các unit test tự động phải đạt được độ bao phủ mã nguồn tối thiểu 70% cho các mô-đun logic nghiệp vụ cốt lõi.

3.4.7 Tính di động (Portability)

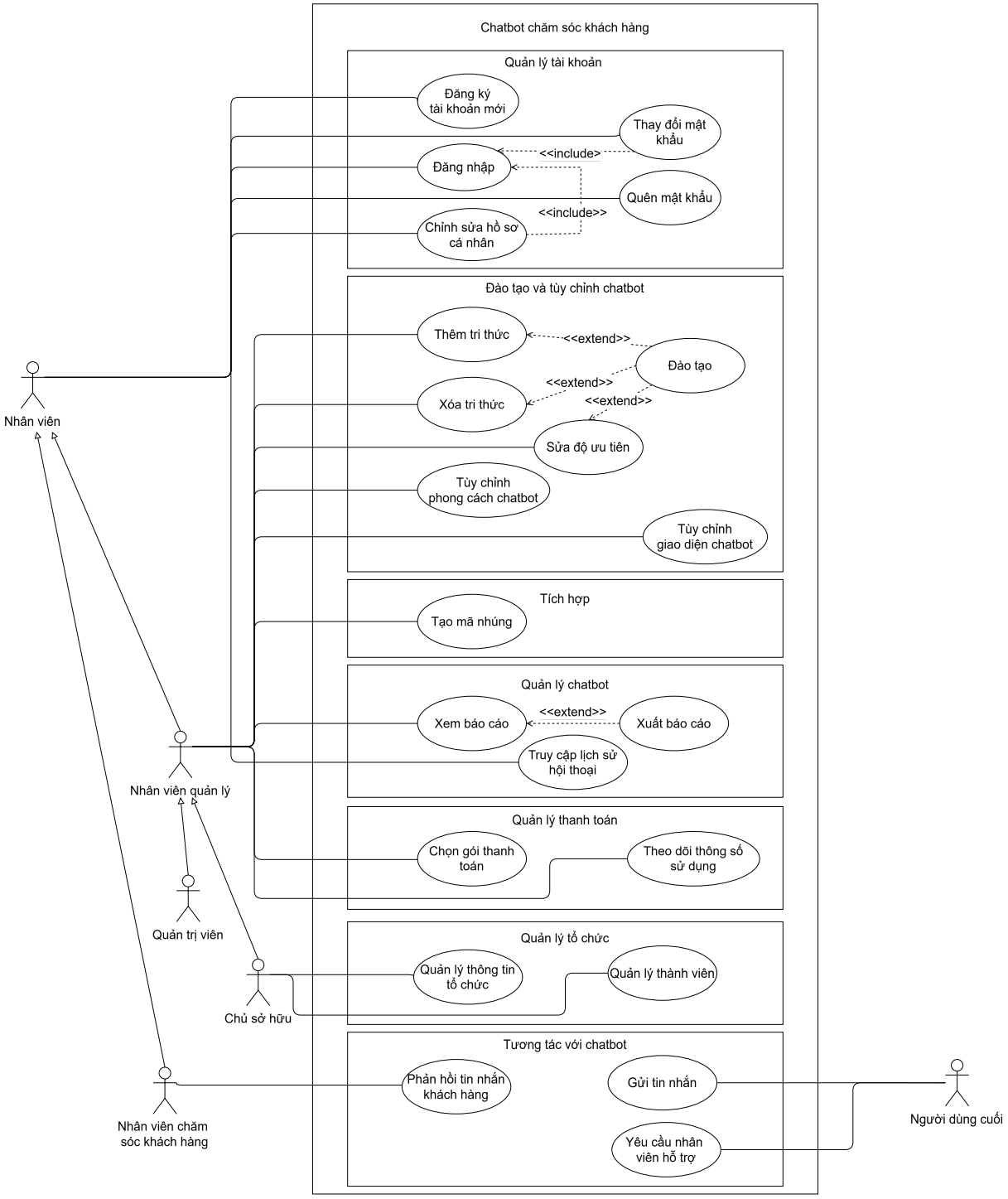
- **Tương thích trình duyệt:** Giao diện quản trị và chatbot nhúng phải hiển thị và hoạt động chính xác trên các phiên bản ổn định mới nhất của Google Chrome, Mozilla Firefox, Apple Safari, và Microsoft Edge tại thời điểm kiểm thử cuối cùng.
- **Thiết kế responsive:** Giao diện quản trị và chatbot nhúng phải tự động điều chỉnh bố cục để có thể sử dụng được trên các kích thước màn hình từ 360px chiều rộng đến 1920px chiều rộng.

3.4.8 Yêu cầu Pháp lý và Đạo đức (Legal & Ethical Requirements)

- **Tuân thủ Quyền riêng tư:** Hệ thống phải tuân thủ các yêu cầu của Nghị định 13/2023/NĐ-CP về bảo vệ dữ liệu cá nhân của Việt Nam. Điều này bao gồm việc cung cấp chính sách bảo mật rõ ràng, cơ chế lấy sự đồng ý của người dùng trước khi thu thập dữ liệu cá nhân, và quy trình xử lý yêu cầu của chủ thể dữ liệu trong vòng 30 ngày.
- **Minh bạch AI:** Giao diện chatbot phải hiển thị rõ ràng cho người dùng rằng họ đang tương tác với một hệ thống trí tuệ nhân tạo chứ không phải con người, ví dụ thông qua một thông báo ban đầu hoặc nhãn hiệu cố định trong giao diện trò chuyện.

3.5 Biểu đồ Use case

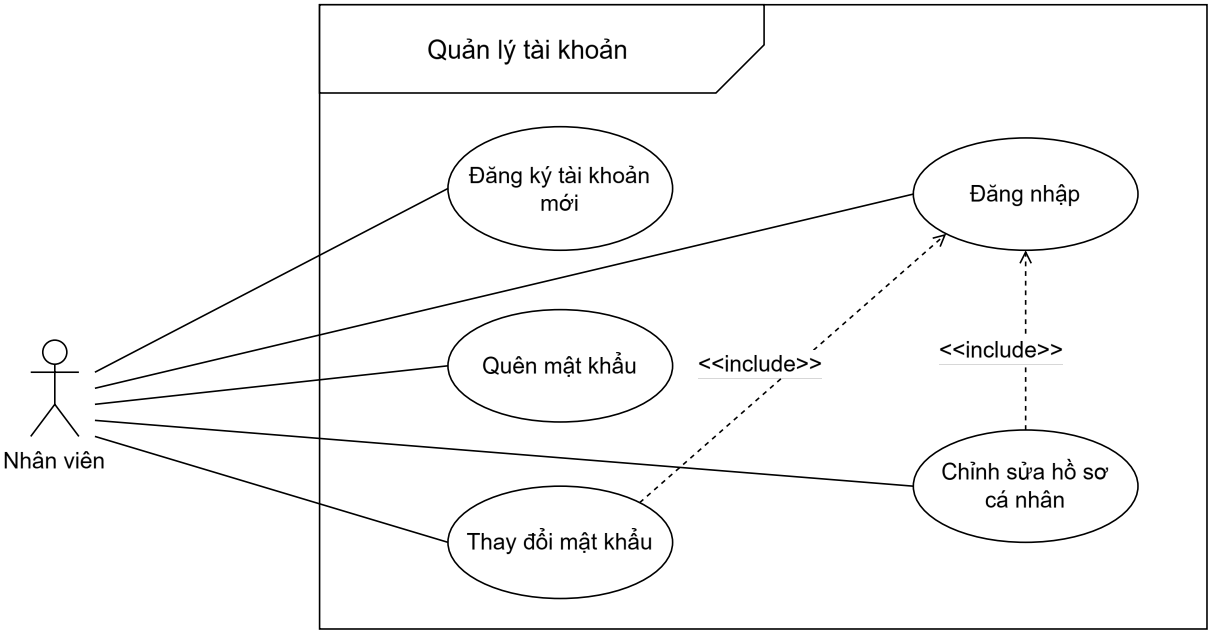
3.5.1 Biểu đồ use case tổng



Hình 18: *Biểu đồ use case tổng*

3.5.2 Quản lý tài khoản

3.5.2.1 Biểu đồ use case



Hình 19: *Biểu đồ use case cho Quản lý tài khoản*

3.5.2.2 Use case scenario

Đăng ký tài khoản mới

Use case name:	Đăng ký tài khoản mới		
Created by:	Nguyễn Đức An	Last updated by:	Nguyễn Đức An
Date created:	19/10/2024	Date last updated:	09/12/2024
Actors:	Người dùng		
Description:	Cho phép người dùng đăng ký tài khoản mới và tham gia vào tổ chức.		
Trigger:	Chọn vào nút "Đăng ký".		
Preconditions:	- Đối với tạo tổ chức mới: Người dùng cần chuẩn bị thông tin tổ chức. - Đối với tham gia tổ chức: Người dùng cần có mã mời.		
Postconditions:	Đăng ký thành công tài khoản và tham gia vào tổ chức.		
Normal Flows:	1. Người dùng truy cập trang đăng ký/đăng nhập. 2. Người dùng chọn một trong hai lựa chọn: A. Tạo tổ chức mới: A1. Điền thông tin và chọn Đăng ký. A2. Hệ thống gửi email xác nhận. A3. Người dùng xác minh email qua link trong thư. A4. Hệ thống xác minh thành công và lưu thông tin vào cơ sở dữ liệu. B. Tham gia tổ chức: B1. Nhập mã mời nhận từ email, thông tin cá nhân và chọn Đăng ký. B2. Hệ thống xác thực mã mời và lưu thông tin. 3. Hệ thống thông báo thành công và chuyển đến trang đăng nhập.		
Alternative Flows:	Không có.		
Exceptions:	**E1. Tại bước A1, thông tin tổ chức đã tồn tại:** A1.1. Hệ thống hiển thị thông báo lỗi. A1.2. Người dùng nhập thông tin khác hoặc chọn tham gia tổ chức. **E2. Tại bước B1, mã mời không hợp lệ hoặc hết hạn:** B1.1. Hệ thống hiển thị thông báo lỗi. B1.2. Người dùng nhập mã mời khác hoặc liên hệ nhân viên quản lý.		
Note and issues:	Không có.		

Bảng 4: *Use case scenario cho Đăng ký tài khoản mới*

Đăng nhập

Use case name:	Đăng nhập		
Created by:	Nguyễn Đức An	Last updated by:	Nguyễn Đức An
Date created:	19/10/2024	Date last updated:	10/11/2024
Actors:	Nhân viên (gồm nhân viên chăm sóc khách hàng, nhân viên quản lý)		
Description:	Cho phép nhân viên đăng nhập với tài khoản đã tạo.		
Trigger:	Chọn vào nút "Đăng nhập".		
Preconditions:	Nhân viên đã đăng ký thành công trước đó.		
Postconditions:	Đăng nhập thành công vào hệ thống.		
Normal Flows:	1. Nhân viên điền đầy đủ email và mật khẩu trong trang nhập. 2. Nhân viên ấn nút "Đăng nhập". 3. Hệ thống thông báo thành công và chuyển đến trang chủ.		
Alternative Flows:	Không có		
Exceptions:	**E1. Tại bước 3, có lỗi email không tồn tại.** 3.1. Hệ thống hiển thị cảnh báo "Email không tồn tại". Kết thúc use case. **E2. Tại bước 3, có lỗi sai mật khẩu.** 2.1. Hệ thống hiển thị cảnh báo "Sai mật khẩu". 2.2. Người dùng nhập lại mật khẩu. 2.3. Use case tiếp tục bước 2. 2.4. Nếu ngoại lệ E2 lại xảy ra, ấn vào nút "Quên mật khẩu". 2.5. Chuyển đến use case "Quên mật khẩu".		
Note and issues:	Không có.		

Bảng 5: *Use case scenario cho Đăng nhập*

Quên mật khẩu

Use case name:	Quên mật khẩu		
Created by:	Nguyễn Đức An	Last updated by:	Nguyễn Đức An
Date created:	19/10/2024	Date last updated:	10/11/2024
Actors:	Nhân viên (gồm nhân viên chăm sóc khách hàng, nhân viên quản lý)		
Description:	Cho phép nhân viên lấy lại mật khẩu khi quên.		
Trigger:	Chọn vào nút "Quên mật khẩu".		
Preconditions:	Nhân viên đã đăng ký thành công trước đó.		
Postconditions:	Đặt lại mật khẩu thành công.		
Normal Flows:	1. Nhân viên nhập email đã đăng ký. 2. Nhân viên ấn nút "Tìm kiếm". 3. Hệ thống gửi mã khôi phục đến email của người dùng. 4. Người dùng mở email và nhấn vào link để xác minh. 5. Nhân viên nhập mật khẩu mới và ấn nhấn "Xác nhận". 6. Hệ thống lưu mật khẩu mới cho tài khoản này. 7. Hệ thống chuyển đến trang đăng nhập, tiếp tục với use case "Đăng nhập".		
Alternative Flows:	Không có.		
Exceptions:	**E1. Tại bước 3, có lỗi email không tồn tại.** 2.1. Hệ thống hiển thị cảnh báo "Email không tồn tại". Kết thúc use case.		
Note and issues:	Không có.		

Bảng 6: *Use case scenario cho Quên mật khẩu*

Chỉnh sửa hồ sơ cá nhân

Use case name:	Chỉnh sửa hồ sơ cá nhân		
Created by:	Nguyễn Đức An	Last updated by:	Nguyễn Đức An
Date created:	19/10/2024	Date last updated:	10/11/2024
Actors:	Nhân viên (gồm nhân viên chăm sóc khách hàng, nhân viên quản lý)		
Description:	Cho phép nhân viên chỉnh sửa các thông tin cá nhân.		
Trigger:	Chọn "Cài đặt" trên thanh điều hướng.		
Preconditions:	Nhân viên đã đăng nhập và đang ở trang "Cài đặt".		
Postconditions:	Chỉnh sửa thông tin cá nhân thành công.		
Normal Flows:	1. Nhân viên chọn chỉnh sửa và tùy chỉnh các thông tin. 2. Nhân viên ấn nút "Lưu". 3. Hệ thống hiển thị thông báo thành công và lưu các thông tin mới.		
Alternative Flows:	Không có		
Exceptions:	**E1. Tại bước 2, nhân viên ấn "Hủy bỏ".** 2.1. Hệ thống khóa chỉnh sửa và use case kết thúc.		
Note and issues:	Không có.		

Bảng 7: *Use case scenario cho Chỉnh sửa hồ sơ cá nhân*

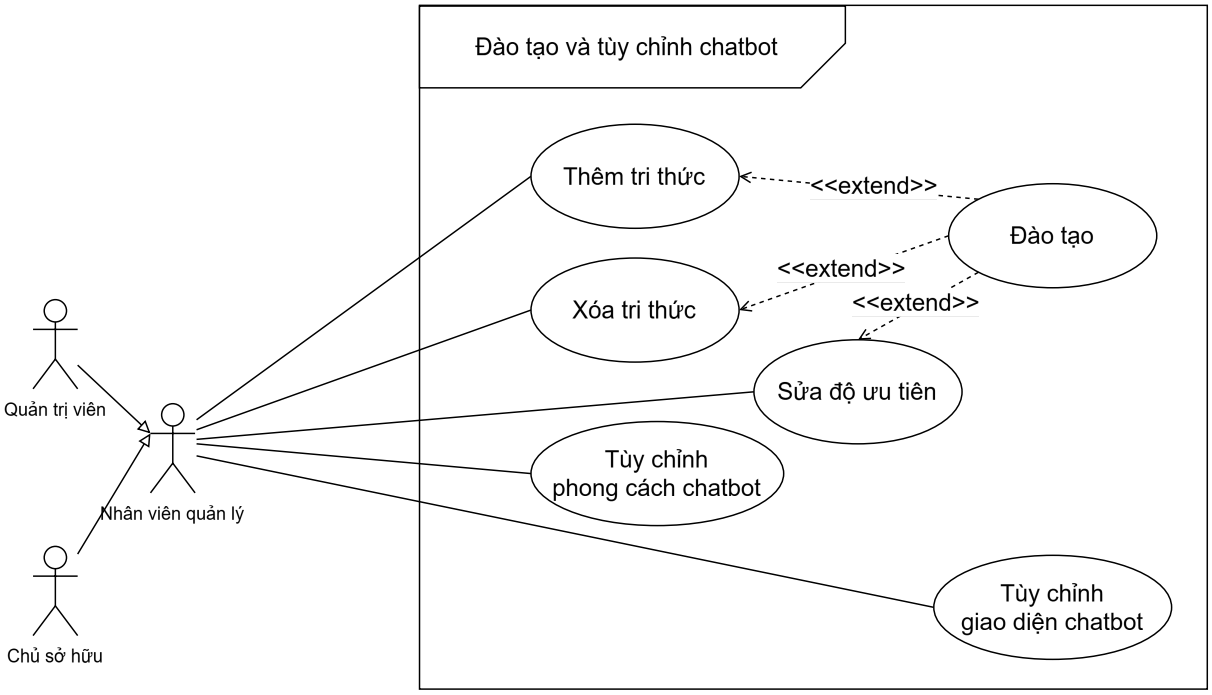
Đổi mật khẩu

Use case name:	Đổi mật khẩu		
Created by:	Nguyễn Đức An	Last updated by:	Nguyễn Đức An
Date created:	19/10/2024	Date last updated:	19/10/2024
Actors:	Nhân viên (gồm nhân viên chăm sóc khách hàng, nhân viên quản lý)		
Description:	Cho phép nhân viên chủ động thay đổi mật khẩu.		
Trigger:	Chọn vào tab "Mật khẩu" của mục "Tài khoản" trong trang "Cài đặt"		
Preconditions:	Nhân viên đã đăng nhập vào hệ thống và đang ở trang "Cài đặt".		
Postconditions:	Đổi mật khẩu thành công.		
Normal Flows:	1. Nhân viên nhập mật khẩu hiện tại và mật khẩu mới. 2. Nhân viên ấn nút "Lưu". 3. Hệ thống lưu mật khẩu mới và đăng xuất tài khoản.		
Alternative Flows:	Không có.		
Exceptions:	**E1. Tại bước 3, lỗi sai mật khẩu hiện tại.** 2.1. Hệ thống báo "Sai mật khẩu hiện tại" Use case kết thúc.		
Note and issues:	Không có.		

Bảng 8: *Use case scenario cho Đổi mật khẩu*

3.5.3 Đào tạo và tùy chỉnh chatbot

3.5.3.1 Biểu đồ use case



Hình 20: *Biểu đồ use case cho Đào tạo và tùy chỉnh chatbot*

3.5.3.2 Use case scenario

Thêm tri thức

Use case name:	Thêm tri thức		
Created by:	Nguyễn Đức An	Last updated by:	Nguyễn Đức An
Date created:	19/10/2024	Date last updated:	19/10/2024
Actors:	Nhân viên quản lý		
Description:	Cho phép công ty thêm tri thức để huấn luyện chatbot.		
Trigger:	Chọn vào mục "Cập nhật tri thức" ở trang "Đào tạo chatbot".		
Preconditions:	Nhân viên quản lý đã đăng nhập thành công. Người dùng đã chuẩn bị các dữ liệu huấn luyện.		
Postconditions:	Thêm tri thức thành công.		
Normal Flows:	1. Người dùng ấn vào biểu tượng tải file để tải tài liệu lên. 2. Hệ thống hiển thị của sổ chọn file chứa các file phù hợp. 3. Người dùng chọn các file muốn tải lên. 4. Hệ thống hiển thị danh sách các file đã nhận lên trang web. 5. Người dùng chọn "Lưu". 6. Hệ thống hiển thị thành công và cập nhật các tri thức mới.		
Alternative Flows:	**A1. Tại bước 1, người dùng chọn tab Nhập tri thức.** 1.1. Người dùng nhập văn bản. Tiếp tục bước 5 trong normal flows. **A2. Tại bước 1, người dùng chọn tab Nhập tri thức web.** 1.1. Người dùng điền các thông tin của website. Tiếp tục bước 5 trong normal flows.		
Exceptions:	**E1. Tại bước 6, có lỗi.** 1.1. Hệ thống báo lỗi tương ứng Kết thúc use case		
Note and issues:	Không có.		

Bảng 9: *Use case scenario cho Thêm tri thức*

Xóa tri thức

Use case name:	Xóa tri thức		
Created by:	Nguyễn Đức An	Last updated by:	Nguyễn Đức An
Date created:	19/10/2024	Date last updated:	19/10/2024
Actors:	Nhân viên quản lý		
Description:	Cho phép công ty xóa tri thức trong kho lưu trữ.		
Trigger:	Chọn vào trang "Đào tạo chatbot" trên thanh điều hướng.		
Preconditions:	Nhân viên quản lý đã đăng nhập thành công. Danh sách tri thức không rỗng		
Postconditions:	Tri thức được xóa vĩnh viễn.		
Normal Flows:	1. Người dùng ấn vào biểu tượng xóa tại một tri thức. 2. Hệ thống hiển thị của sổ xác nhận. 3. Người dùng xác nhận xóa. 4. Hệ thống hiển thị thông báo xóa vĩnh viễn thành công, tri thức không còn hiển thị.		
Alternative Flows:	**A1. Tại bước 4, nếu tri thức có trạng thái chờ hoặc đã đào tạo.** 1.1. Hệ thống báo xóa tạm thời thành công và tri thức bị làm mờ 1.2. Chuyển đến use case Đào tào Tiếp tục bước 4 trong normal flows		
Exceptions:	**E1. Tại bước 3, muốn hủy.** 1.1. Người dùng chọn Hủy bỏ 1.1. Hệ thống đóng cửa sổ xác nhận Kết thúc use case		
Note and issues:	Không có.		

Bảng 10: *Use case scenario cho Xóa tri thức*

Sửa độ ưu tiên

Use case name:	Sửa độ ưu tiên		
Created by:	Nguyễn Đức An	Last updated by:	Nguyễn Đức An
Date created:	19/10/2024	Date last updated:	19/10/2024
Actors:	Nhân viên quản lý		
Description:	Cho phép nhân viên sửa độ ưu tiên cho các tri thức.		
Trigger:	Chọn "Đào tạo chatbot" trên thanh điều hướng.		
Preconditions:	Nhân viên quản lý đăng nhập thành công Tồn tại ít nhất một tri thức		
Postconditions:	Độ ưu tiên của tri thức được cập nhật		
Normal Flows:	1. Nhân viên quản lý thay đổi độ ưu tiên cho các tri thức. 2. Chuyển đến use case Đào tạo 3. Hiển thị thông báo chỉnh sửa thành công, các độ ưu tiên mới được cập nhật		
Alternative Flows:	Không có.		
Exceptions:	Không có.		
Note and issues:	Không có.		

Bảng 11: *Use case scenario cho Sửa độ ưu tiên*

Đào tạo

Use case name:	Đào tạo		
Created by:	Nguyễn Đức An	Last updated by:	Nguyễn Đức An
Date created:	19/10/2024	Date last updated:	19/10/2024
Actors:	Nhân viên quản lý		
Description:	Cho phép nhân viên đào tạo các tri thức.		
Trigger:	Chọn "Đào tạo chatbot" trên thanh điều hướng		
Preconditions:	Nhân viên quản lý đăng nhập thành công Tồn tại sự thay đổi tri thức (thêm, xóa, sửa)		
Postconditions:	Các tri thức được đào tạo thành công		
Normal Flows:	1. Nhân viên chọn Lưu và đào tạo 2. Hệ thống hiển thị Đang chờ đào tạo, các tri thức có trạng thái chờ 3. Hệ thống hiển thị thông báo đào tạo thành công, các tri thức có trạng thái đã đào tạo		
Alternative Flows:	Không có.		
Exceptions:	Không có.		
Note and issues:	Không có.		

Bảng 12: *Use case scenario cho Đào tạo*

Tùy chỉnh giao diện chatbot

Use case name:	Tùy chỉnh giao diện chatbot		
Created by:	Nguyễn Đức An	Last updated by:	Nguyễn Đức An
Date created:	19/10/2024	Date last updated:	19/10/2024
Actors:	Nhân viên quản lý		
Description:	Cho phép nhân viên quản lý thay đổi một số đặc điểm giao diện của chatbot.		
Trigger:	Chọn vào "Tùy chỉnh chatbot" trên thanh điều hướng.		
Preconditions:	Nhân viên quản lý đăng nhập thành công.		
Postconditions:	Cập nhật giao diện chatbot thành công.		
Normal Flows:	1. Nhân viên quản lý chỉnh sửa các thành phần của giao diện chatbot 2. Nhân viên quản lý nhấn "Lưu". 3. Hệ thống lưu lại các thiết lập.		
Alternative Flows:	**A1. Tại bước 2, nếu muốn xem trước.** 1.1. Người dùng chọn Xem trước 1.2. Hệ thống hiển thị giao diện chatbot mới Tiếp tục bước 2 trong normal flows		
Exceptions:	**E1. Tại bước 2, người dùng ấn vào nút "Hủy bỏ". Use-case kết thúc.**		
Note and issues:	Không có.		

Bảng 13: *Use case scenario cho Tùy chỉnh giao diện chatbot.*

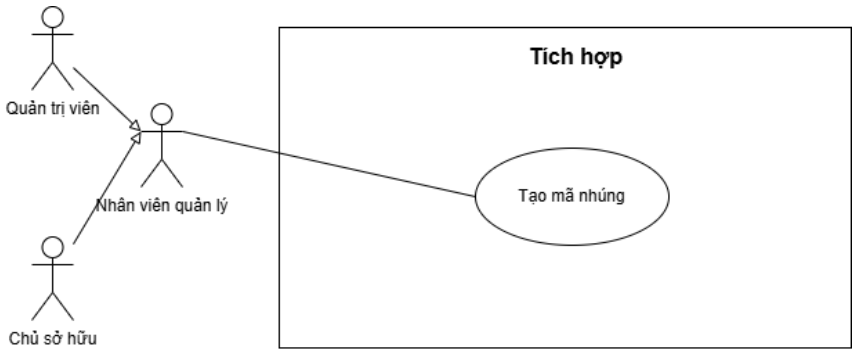
Tùy chỉnh phong cách chatbot

Use case name:	Tùy chỉnh phong cách chatbot		
Created by:	Nguyễn Đức An	Last updated by:	Nguyễn Đức An
Date created:	19/10/2024	Date last updated:	19/10/2024
Actors:	Nhân viên quản lý		
Description:	Cho phép nhân viên quản lý thay đổi một số đặc điểm ứng xử của chatbot.		
Trigger:	Chọn vào "Phong cách" trong mục Cấu hình chatbot.		
Preconditions:	Nhân viên quản lý đăng nhập thành công và đang ở trang Tùy chỉnh chatbot.		
Postconditions:	Phong cách được cập nhật.		
Normal Flows:	1. Người dùng chọn tone giọng khác. 2. Hệ thống hiển thị thông báo thành công và tone giọng mới cập nhật. 3. Người dùng chỉnh sửa tin nhắn mặc định. 4. Người dùng chọn Lưu. 5. Hệ thống hiển thị thông báo thành công và tin nhắn mới cập nhật.		
Alternative Flows:	Không có.		
Exceptions:	**E1. Tại bước 5, người dùng ấn vào nút "Hủy bỏ". Use-case kết thúc.**		
Note and issues:	Không có.		

Bảng 14: *Use case scenario cho Tùy chỉnh phong cách chatbot.*

3.5.4 Tích hợp

3.5.4.1 Biểu đồ usecase



Hình 21: *Biểu đồ use case cho tích hợp*

3.5.4.2 Usecase scenerio

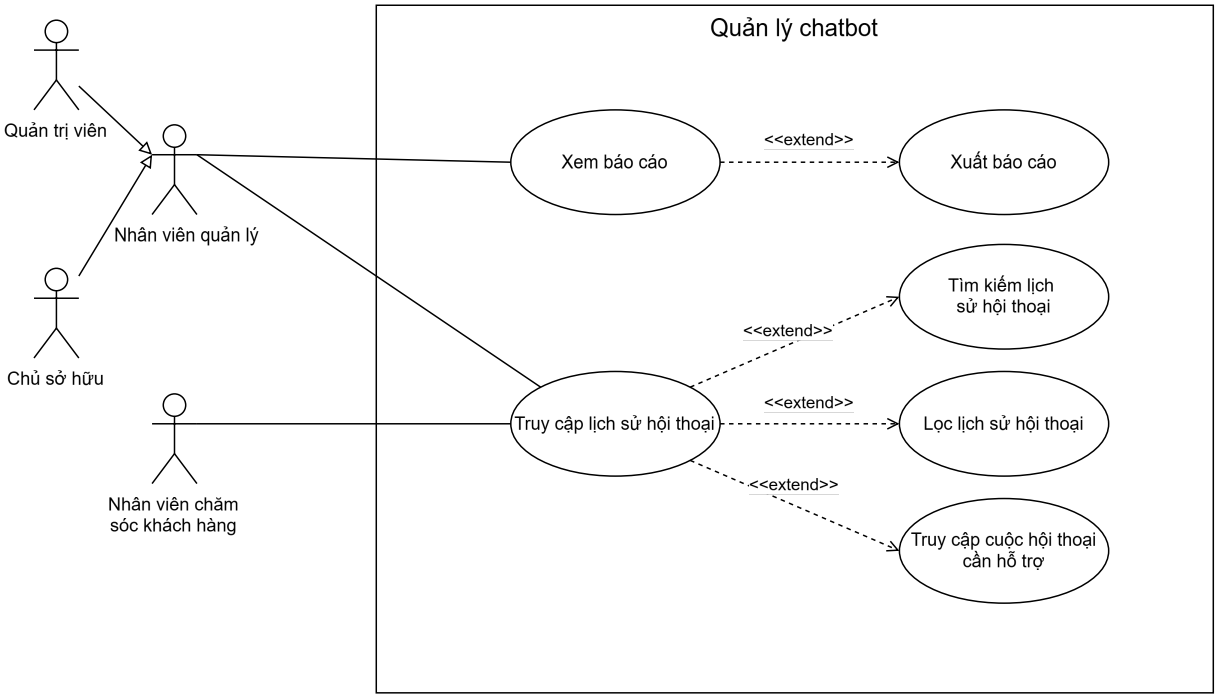
Tạo mã nhúng

Use case name:	Tạo mã nhúng		
Created by:	Phạm Đức Thắng	Last updated by:	Phạm Đức Thắng
Date created:	19/10/2024	Date last updated:	19/10/2024
Actors:	Nhân viên quản lý doanh nghiệp		
Description:	Nhân viên quản lý doanh nghiệp tạo đoạn mã nhúng và gắn vào website để có thể tích hợp chatbot vào website của họ.		
Trigger:	Nhân viên quản lý muốn tạo mã nhúng để tích hợp chatbot vào website.		
Preconditions:	- Nhân viên quản lý đã đăng nhập vào hệ thống. - Doanh nghiệp đã đăng ký dịch vụ chatbot.		
Postconditions:	Đoạn mã nhúng được tạo thành công và sẵn sàng để tích hợp.		
Normal Flows:	1. Nhân viên quản lý đăng nhập vào hệ thống quản lý chatbot. 2. Điều hướng tới trang “Tùy chỉnh chatbot”. 3. Xem trước kích thước, màu sắc, vị trí hiển thị của chatbot. 4. Nhấn nút "Tích hợp". 5. Hệ thống tạo và hiển thị đoạn mã nhúng. 6. Nhân viên quản lý sao chép đoạn mã này để gán vào website.		
Alternative Flows:	Không có		
Exceptions:	**E1: Xảy ra sự cố kỹ thuật trong quá trình tạo đoạn mã nhúng** 5.1. Hệ thống hiển thị thông báo lỗi: "Đã xảy ra lỗi trong quá trình tạo đoạn mã nhúng. Vui lòng thử lại sau."		
Notes and issues:	Không có		

Bảng 15: *Use case scenario cho Tạo mã nhúng*

3.5.5 Quản lý chatbot

3.5.5.1 Biểu đồ use case



Hình 22: *Biểu đồ use case cho Quản lý chatbot*

3.5.5.2 Use case scenario

Xem báo cáo

Use case name:	Xem báo cáo		
Created by:	Lê Đình Huy	Last updated by:	Lê Đình Huy
Date created:	19 / 10 / 2024	Date last updated:	19 / 10 / 2024
Actors:	Nhân viên quản lý		
Description:	Cho phép nhân viên quản lý xem báo cáo thống kê về hệ thống chatbot của mình.		
Trigger:	Chọn "Báo cáo" trên thanh điều hướng.		
Preconditions:	- Nhân viên quản lý có tài khoản trên website. - Nhân viên quản lý đã đăng nhập thành công vào hệ thống. - Thiết bị của nhân viên quản lý có kết nối mạng và kết nối với hệ thống. - Nhân viên quản lý có quyền thực hiện chức năng.		
Postconditions:	Hệ thống hiển thị các thông số và biểu đồ thống kê.		
Normal Flows:	1. Nhân viên quản lý chọn "Báo cáo" trên thanh điều hướng. 2. Hệ thống hiển thị các thông số và báo cáo thống kê.		
Alternative Flows:	**A1: Sau bước 2** 3.1. Nhân viên quản lý chọn "Lọc". 3.2. Hệ thống hiển thị các thông số cho phép của bộ lọc. 3.3. Nhân viên quản lý chọn một thông số. Tiếp tục bước 2 trong Normal flows.		
Exceptions:	Không có		
Note and issues:	Không có		

Bảng 16: *Use case scenario cho Xem báo cáo*

Xuất báo cáo

Use case name:	Xuất báo cáo		
Created by:	Lê Đình Huy	Last updated by:	Lê Đình Huy
Date created:	19 / 10 / 2024	Date last updated:	19 / 10 / 2024
Actors:	Nhân viên quản lý		
Description:	Cho phép nhân viên quản lý xuất báo cáo thống kê về hệ thống chatbot của mình ra file pdf hoặc excel.		
Trigger:	Chọn "Xuất báo cáo" trong giao diện Xem báo cáo.		
Preconditions:	- Nhân viên quản lý có tài khoản trên website. - Nhân viên quản lý đã đăng nhập thành công vào hệ thống. - Thiết bị của nhân viên quản lý có kết nối mạng và kết nối với hệ thống. - Nhân viên quản lý có quyền thực hiện chức năng.		
Postconditions:	File được tạo và tải về thiết bị của nhân viên quản lý.		
Normal Flows:	1. Nhân viên quản lý chọn "Xuất báo cáo" trong giao diện Xem báo cáo. 2. Hệ thống hiển thị các loại tệp cho nhân viên quản lý chọn. 3. Nhân viên quản lý chọn một loại tệp. 4. File được tải về thiết bị của nhân viên quản lý.		
Alternative Flows:	Không có		
Exceptions:	Không có		
Note and issues:	Không có		

Bảng 17: *Use case scenario cho Xem báo cáo*

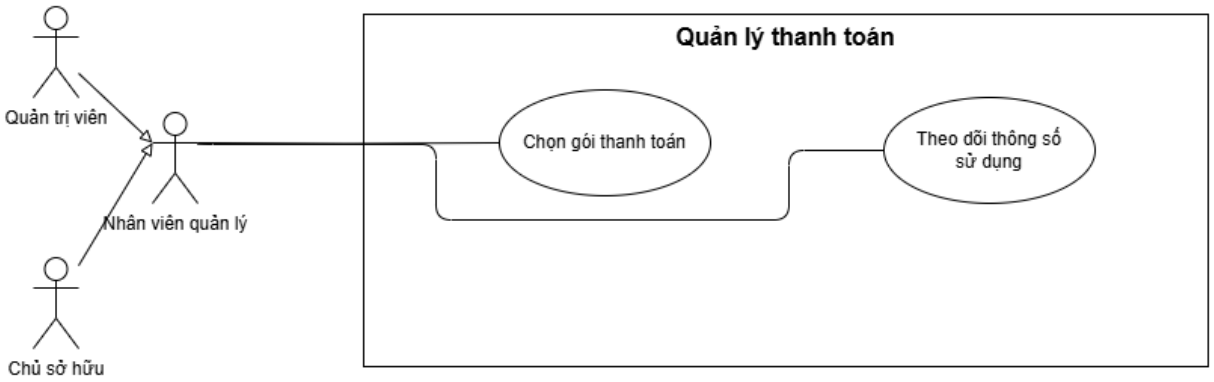
Truy cập lịch sử hội thoại

Use case name:	Truy cập lịch sử hội thoại		
Created by:	Lê Đình Huy	Last updated by:	Lê Đình Huy
Date created:	19 / 10 / 2024	Date last updated:	19 / 10 / 2024
Actors:	Nhân viên		
Description:	Cho phép nhân viên truy cập và xem lịch sử hội thoại.		
Trigger:	Chọn "Chat" trên thanh điều hướng.		
Preconditions:	- Nhân viên có tài khoản trên website. - Nhân viên đã đăng nhập thành công vào hệ thống. - Thiết bị của nhân viên có kết nối mạng và kết nối với hệ thống. - Nhân viên có quyền thực hiện chức năng.		
Postconditions:	Hệ thống hiển thị chi tiết toàn bộ cuộc hội thoại của khách hàng.		
Normal Flows:	1. Nhân viên chọn "Chat" trên thanh điều hướng. 2. Hệ thống hiển thị danh sách các cuộc hội thoại. 3. Nhân viên chọn xem một cuộc hội thoại. 4. Hệ thống hiển thị chi tiết toàn bộ cuộc hội thoại của khách hàng.		
Alternative Flows:	**A1: Tại bước 3** 3.1. Nhân viên chọn vào ô Tìm kiếm và nhập nội dung cần tìm. 3.2. Nhân viên chọn "Tìm". Tiếp tục bước 2 trong Normal flows. **A2: Tại bước 3** 3.1. Nhân viên chọn "Lọc". 3.2. Hệ thống hiển thị các thông số cho phép của bộ lọc. 3.3. Nhân viên chọn một thông số. Tiếp tục bước 2 trong Normal flows. **A3: Tại bước 3** 3.1. Nhân viên chọn "Cần hỗ trợ". Tiếp tục bước 2 trong Normal flows.		
Exceptions:	Không có		
Note and issues:	Không có		

Bảng 18: *Use case scenario cho Xem báo cáo*

3.5.6 Quản lý thanh toán

3.5.6.1 Biểu đồ usecase



Hình 23: *Biểu đồ use case cho Quản lý thanh toán*

3.5.6.2 Usecase scenerio

Chọn gói thanh toán

Use case name:	Chọn gói thanh toán		
Created by:	Phạm Đức Thắng	Last updated by:	Phạm Đức Thắng
Date created:	19/10/2024	Date last updated:	19/10/2024
Actors:	Nhân viên quản lý		
Description:	Người dùng lựa chọn gói đăng ký phù hợp với nhu cầu sử dụng dịch vụ chatbot từ các gói có sẵn trên hệ thống.		
Trigger:	Người dùng muốn chọn hoặc thay đổi gói đăng ký dịch vụ chatbot.		
Preconditions:	- Người dùng đã đăng ký và đăng nhập vào tài khoản của mình trên hệ thống. - Hệ thống đã có các gói đăng ký được cấu hình sẵn.		
Postconditions:	- Người dùng đã chọn và xác nhận gói đăng ký mới. - Hệ thống đã cập nhật gói đăng ký của doanh nghiệp và thực hiện thanh toán nếu cần.		
Normal Flows:	1. Người dùng đăng nhập vào tài khoản của mình trên trang web hoặc ứng dụng. 2. Người dùng điều hướng vào trang "Thanh toán" mục "Gói đăng ký. 3. Hệ thống hiển thị danh sách các gói đăng ký hiện có với các tính năng và giá cả tương ứng. 4. Người dùng chọn gói đăng ký mong muốn bằng cách nhấp vào nút "Mua". 5. Hệ thống chuyển đến trang thanh toán chứa thông tin và mã QR. 6. Người dùng tiến hành thanh toán. 7. Hệ thống thanh toán xác nhận thanh toán thành công và quay về hệ thống. Các thông tin mới được cập nhật.		
Alternative Flows:	Không có		
Exceptions:	**E1: Tại bước 6, người dùng hủy thanh toán.** 6.1. Người dùng chọn Hủy và xác nhận. 6.2. Hệ thống thanh toán xác nhận hủy và quay về hệ thống. Kết thúc use case.		
Notes and issues:	Không có		

Bảng 19: *Use case scenario cho Chọn gói thanh toán*

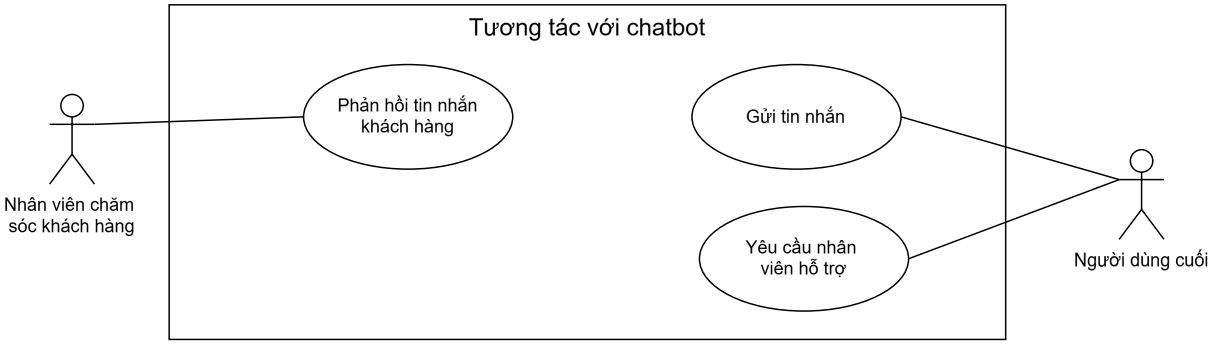
Theo dõi thông số sử dụng

Use case name:	Theo dõi thông số sử dụng		
Created by:	Phạm Đức Thắng	Last updated by:	Phạm Đức Thắng
Date created:	19/10/2024	Date last updated:	19/10/2024
Actors:	Nhân viên quản lý		
Description:	Người dùng có thể theo dõi các thông số giới hạn của gói đã đăng ký, bao gồm thời hạn gói đăng ký, số lượng yêu cầu, bộ nhớ và tài khoản nhân viên CSKH đã sử dụng.		
Trigger:	Người dùng muốn xem thông tin về mức độ sử dụng dịch vụ chatbot.		
Preconditions:	- Người dùng đã đăng ký và sử dụng dịch vụ chatbot. - Hệ thống đã ghi nhận dữ liệu sử dụng dịch vụ của doanh nghiệp.		
Postconditions:	Người dùng đã xem được các thông số sử dụng dịch vụ chi tiết.		
Normal Flows:	1. Người dùng đăng nhập vào tài khoản của mình. 2. Người dùng truy cập vào trang "Thanh toán". 3. Hệ thống hiển thị các thông số sử dụng hiện tại như số lượng yêu cầu đã xử lý, thời gian gói còn lại và bộ nhớ đã sử dụng.		
Alternative Flows:	Không có		
Exceptions:	**E1: Hệ thống chưa có dữ liệu sử dụng do mới bắt đầu dịch vụ hoặc vấn đề kỹ thuật** 5.1. Hệ thống kiểm tra và phát hiện thiếu dữ liệu. 5.2. Hiển thị thông báo: "Chưa có dữ liệu sử dụng để hiển thị. Vui lòng quay lại sau."		
Notes and issues:	Không có		

Bảng 20: *Use case scenario cho Theo dõi sử dụng*

3.5.7 Tương tác với chatbot

3.5.7.1 Biểu đồ use case



Hình 24: *Biểu đồ use case cho Tương tác với chatbot*

3.5.7.2 Use case scenario

Phản hồi tin nhắn khách hàng

Use case name:	Phản hồi tin nhắn khách hàng		
Created by:	Lê Đình Huy	Last updated by:	Lê Đình Huy
Date created:	19 / 10 / 2024	Date last updated:	19 / 10 / 2024
Actors:	Nhân viên chăm sóc khách hàng		
Description:	Cho phép nhân viên chăm sóc khách hàng phản hồi trực tiếp tin nhắn của khách hàng.		
Trigger:	Chọn vào xem đoạn hội thoại với một khách hàng.		
Preconditions:	- Nhân viên có tài khoản trên website. - Nhân viên đã đăng nhập thành công vào hệ thống. - Thiết bị của nhân viên có kết nối mạng và kết nối với hệ thống. - Nhân viên có quyền thực hiện chức năng.		
Postconditions:	Tin nhắn được gửi thành công và hiển thị ở hai phía.		
Normal Flows:	1. Nhân viên CSKH chọn vào xem đoạn hội thoại với một khách hàng. 2. Hệ thống hiển thị màn hình chat với khách hàng đó. 3. Nhân viên CSKH chọn vào khung điền tin nhắn và nhập nội dung tin nhắn. 4. Nhân viên CSKH chọn "Gửi". 5. Hệ thống lưu dữ liệu vào database và hiển thị tin nhắn vừa gửi lên màn hình.		
Alternative Flows:	Không có		
Exceptions:	Không có		
Note and issues:	Không có		

Bảng 21: *Use case scenario cho Phản hồi tin nhắn khách hàng*

Gửi tin nhắn

Use case name:	Gửi tin nhắn		
Created by:	Lê Đình Huy	Last updated by:	Lê Đình Huy
Date created:	19 / 10 / 2024	Date last updated:	19 / 10 / 2024
Actors:	Người dùng cuối (Khách hàng)		
Description:	Cho phép người dùng cuối gửi tin nhắn trong chatbot.		
Trigger:	Chọn vào nút chatbot trên website của doanh nghiệp.		
Preconditions:	- Thiết bị của người dùng có kết nối mạng. - Cuộc hội thoại ở chế độ tự động.		
Postconditions:	Tin nhắn được gửi thành công và hiển thị trên màn hình. Đồng thời, tin nhắn từ chatbot sẽ phản hồi lại trong một khoảng thời gian nhất định.		
Normal Flows:	1. Người dùng chọn vào nút chatbot trên website của doanh nghiệp. 2. Hệ thống hiển thị màn hình chatbot. 3. Người dùng chọn vào khung điền tin nhắn và nhập nội dung tin nhắn. 4. Người dùng chọn "Gửi". 5. Hệ thống lưu dữ liệu vào database và hiển thị tin nhắn vừa gửi lên màn hình. 6. Hệ thống chatbot tạo và phản hồi người dùng trong một khoảng thời gian nhất định.		
Alternative Flows:	Không có		
Exceptions:	Không có		
Note and issues:	Không có		

Bảng 22: *Use case scenario cho Gửi tin nhắn*

Yêu cầu nhân viên hỗ trợ

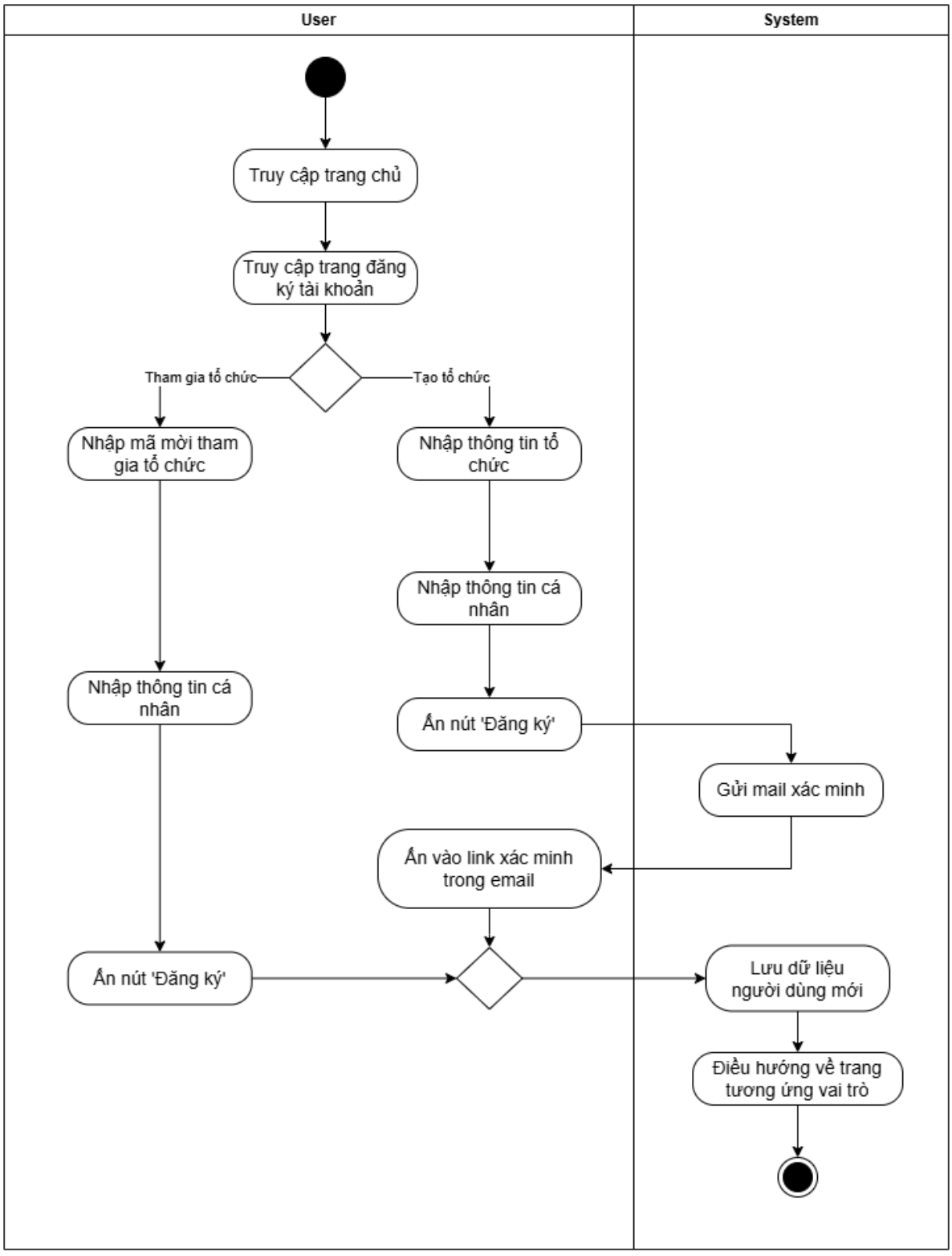
Use case name:	Yêu cầu nhân viên hỗ trợ		
Created by:	Lê Đình Huy	Last updated by:	Lê Đình Huy
Date created:	19 / 10 / 2024	Date last updated:	19 / 10 / 2024
Actors:	Người dùng cuối (Khách hàng)		
Description:	Cho phép người dùng cuối có thể yêu cầu nhân viên hỗ trợ.		
Trigger:	Chọn vào nút yêu cầu hỗ trợ trong khung chatbot.		
Preconditions:	- Thiết bị của người dùng có kết nối mạng.		
Postconditions:	Yêu cầu được gửi lên hệ thống và màn hình hiển thị thông báo thành công. Đồng thời, tính năng chat tự động sẽ được tạm dừng.		
Normal Flows:	1. Người dùng chọn vào nút yêu cầu hỗ trợ trong khung chatbot. 2. Hệ thống hiển thị pop up Hỗ trợ khách hàng. 3. Người dùng điền thông tin và chọn "Gửi yêu cầu". 4. Yêu cầu được gửi lên hệ thống và màn hình hiển thị thông báo thành công. Đồng thời, tính năng chat tự động sẽ được tạm dừng.		
Alternative Flows:	Không có		
Exceptions:	**E1: Tại bước 3** 3.1. Người dùng chọn "Đóng". 3.2. Pop up Hỗ trợ khách hàng được đóng. Use case dừng lại.		
Note and issues:	Không có		

Bảng 23: *Use case scenario cho Yêu cầu nhân viên hỗ trợ*

3.6 Biểu đồ Activity

3.6.1 Quản lý tài khoản

3.6.1.1 Đăng ký



Hình 25: *Biểu đồ activity cho Đăng ký*

Mô tả: Biểu đồ hoạt động Đăng ký gồm 2 swimlane: Người dùng và Hệ thống.

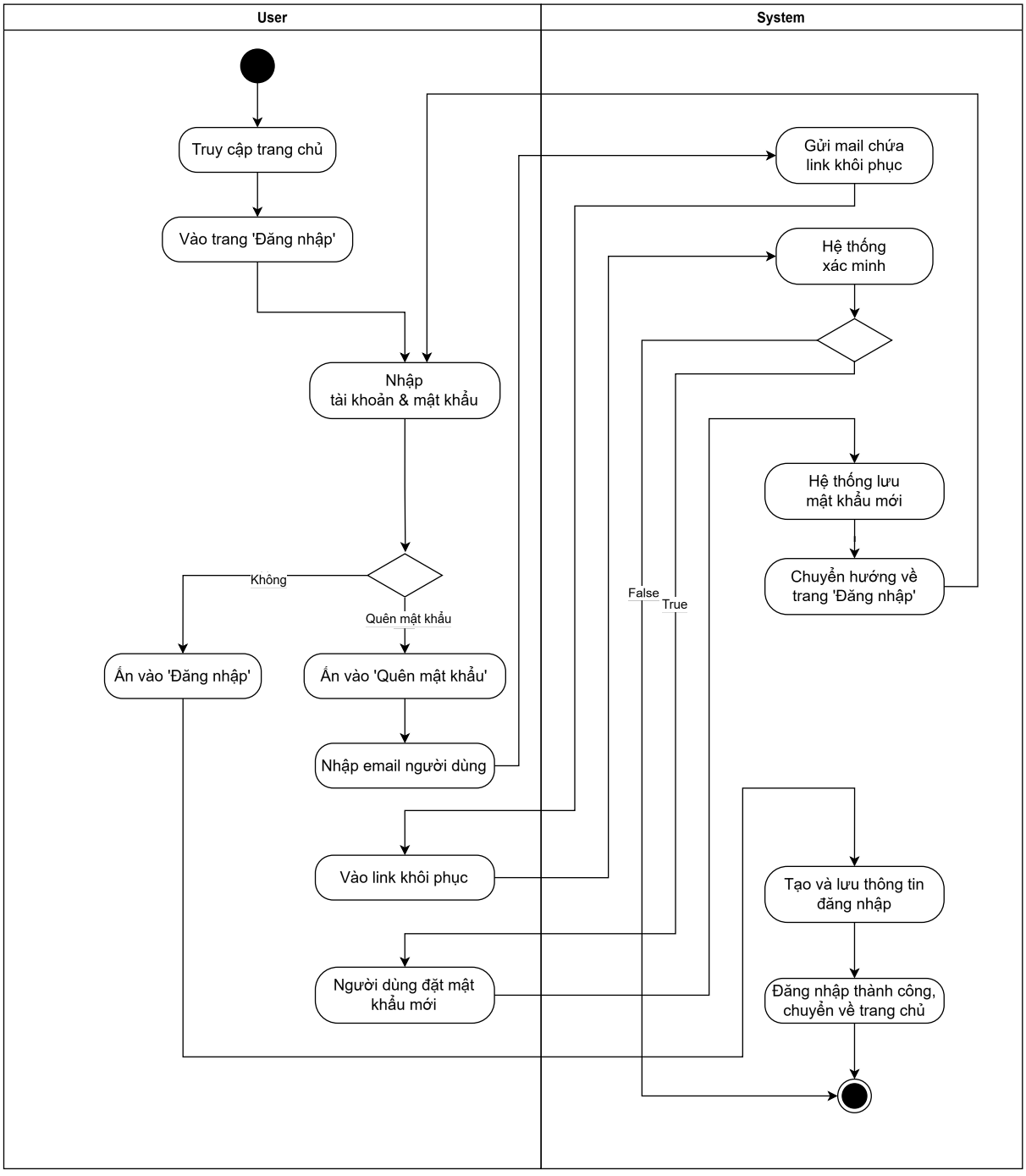
Đầu tiên, người dùng truy cập trang đăng ký/đăng nhập và được hiển thị hai lựa chọn: "Tạo tổ chức mới" hoặc "Tham gia tổ chức".

Luồng tạo tổ chức mới: Khi chọn "Tạo tổ chức mới", người dùng cần điền các thông tin tổ chức như tên tổ chức, email domain và thông tin cá nhân bao gồm họ tên, email, mật khẩu. Sau đó, hệ thống sẽ gửi email xác minh đến địa chỉ email đã đăng ký. Người dùng cần xác minh email bằng cách nhấp vào link trong thư. Sau khi xác minh thành công, hệ thống sẽ tạo tổ chức mới và tự động gán người dùng làm quản trị viên của tổ chức.

Luồng tham gia tổ chức: Khi chọn "Tham gia tổ chức", người dùng cần nhập mã mời và điền thông tin cá nhân bao gồm họ tên, email, mật khẩu. Hệ thống sẽ kiểm tra tính hợp lệ của mã mời, lưu trữ thông tin người dùng vào cơ sở dữ liệu và gán vai trò theo cấu hình trong lời mời.

Khi hoàn tất quá trình đăng ký theo một trong hai luồng trên, người dùng sẽ được điều hướng đến trang chính với các chức năng tương ứng với vai trò được phân trong tổ chức.

3.6.1.2 Đăng nhập và quên mật khẩu



Hình 26: *Biểu đồ activity cho Đăng nhập và quên mật khẩu*

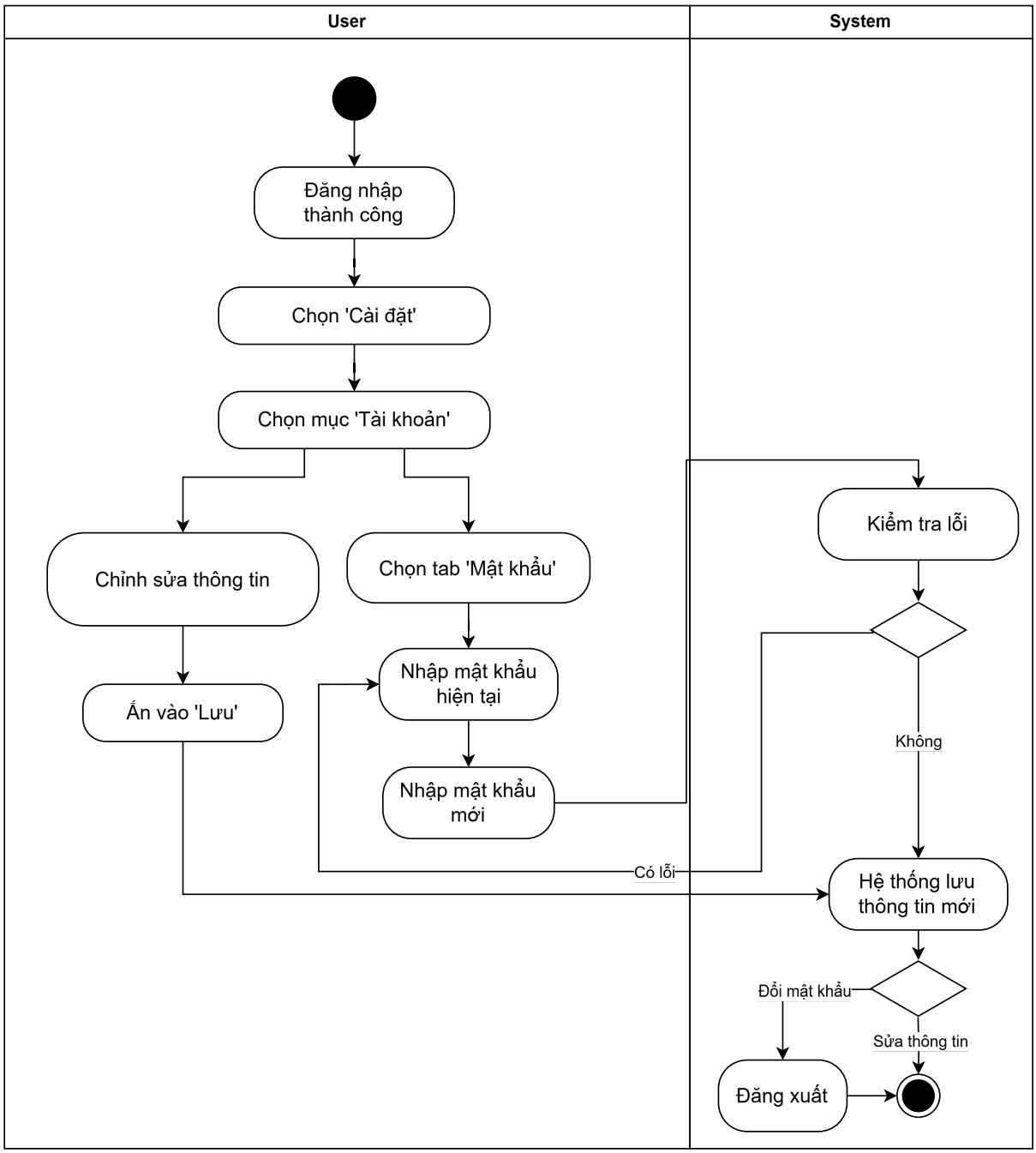
Mô tả: Biểu đồ hoạt động gồm 2 swimlane: Người dùng và hệ thống.

Đầu tiên, nhân viên chọn vào nút "Đăng nhập" ở trang chủ. Nhân viên nhập email và mật khẩu để đăng nhập (nếu đã đăng ký trước đó). Nếu người dùng quên mật khẩu:

1. Nhập email đã đăng ký.
2. Hệ thống gửi email chứa link khôi phục đến người dùng.
3. Nhân viên vào link khôi phục.
4. Nếu hệ thống xác minh là đúng, nhân viên sẽ được cho phép đổi mật khẩu.
5. Nhân viên đổi xong sẽ được chuyển về trang đăng nhập để tiếp tục.

Sau khi đăng nhập thành công, hệ thống sẽ lưu dữ liệu đăng nhập và chuyển nhân viên về trang chủ.

3.6.1.3 Chỉnh sửa hồ sơ cá nhân



Hình 27: *Biểu đồ activity cho Chỉnh sửa hồ sơ cá nhân*

Mô tả: Biểu đồ hoạt động gồm 2 swimlane: Người dùng và hệ thống.

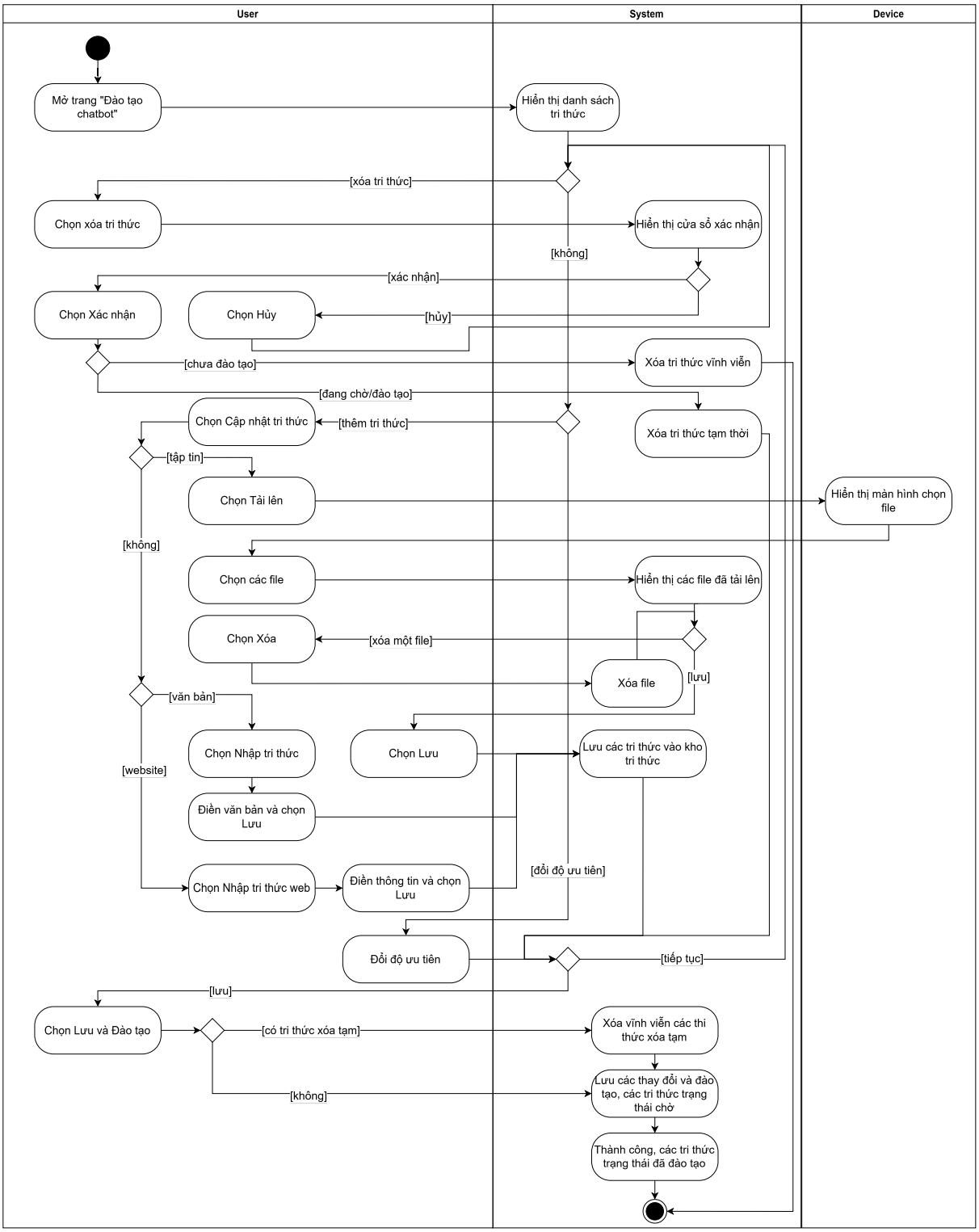
Đầu tiên, nhân viên chọn vào nút "Cài đặt" ở thanh bên. Chọn mục "Tài khoản" để thực hiện chức năng này. Nhân viên có thể:

- Chỉnh sửa thông tin cá nhân: Nhân viên thay đổi được ảnh đại diện, họ tên và số điện thoại. Tuy nhiên email và vai trò của tài khoản sẽ không thể thay đổi.
- Thay đổi mật khẩu chủ động bằng cách nhập mật khẩu hiện tại và mật khẩu mới. Nếu mật khẩu hiện tại không đúng, người dùng không thể tiếp tục tiến trình.

Dù chỉnh sửa thông tin nào, sau khi thay đổi xong và nhấn lưu, hệ thống sẽ lưu lại các thông tin mới này. Khi đổi mật khẩu thành công, hệ thống sẽ đăng xuất tài khoản và cho đăng nhập lại.

3.6.2 Đào tạo và tùy chỉnh chatbot

3.6.2.1 Đào tạo chatbot



Hình 28: *Biểu đồ activity cho Đào tạo chatbot*

Mô tả: Biểu đồ hoạt động gồm 3 swimlane: Người dùng, hệ thống và thiết bị.

Đầu tiên, người dùng vào trang "Đào tạo chatbot", hệ thống hiển thị danh sách tri thức. Tại trang này, nhân viên có thể có các lựa chọn:

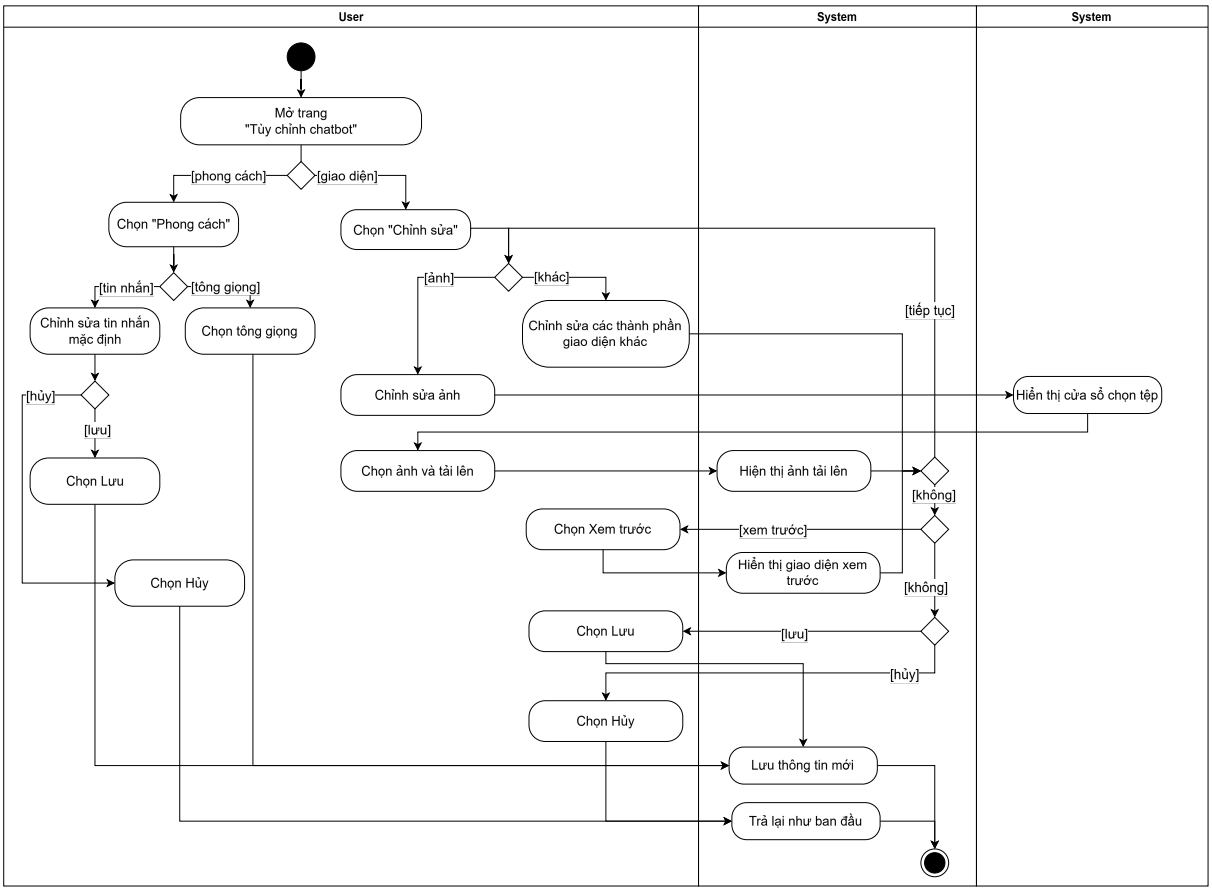
- Xóa tri thức, khi bấm xóa, màn hình hiển thị cửa sổ xác nhận, người dùng có 2 lựa chọn là xác nhận và hủy. Khi xác nhận, đối với tri thức chưa đào tạo sẽ được xóa vĩnh viễn, còn lại sẽ chỉ xóa tạm thời.

- Thêm tri thức: người dùng chọn Cập nhật tri thức, sau đó có thể chọn một trong ba loại tri thức:
  - Dạng tập tin: người dùng tải lên các tập tin và có thể thêm hoặc xóa các tập tin đã tải lên.
  - Dạng văn bản: người dùng vào tab nhập tri thức và nhập văn bản.
  - Dạng website: người dùng vào tab nhập tri thức web và điền thông tin.

  Khi chọn Lưu thì các tri thức mới sẽ được thêm.
- Đổi độ ưu tiên: người dùng đổi độ ưu tiên cho tri thức

Sau đó người dùng có thể lặp lại các bước trên hoặc chọn Lưu và đào tạo. Khi đó, nếu tồn tại tri thức xóa tạm, hệ thống sẽ xóa vĩnh viễn tri thức đó rồi mới lưu các thay đổi và đào tạo. Khi đào tạo xong, các tri thức sẽ có trạng thái đã đào tạo.

3.6.2.2 Tùy chỉnh chatbot



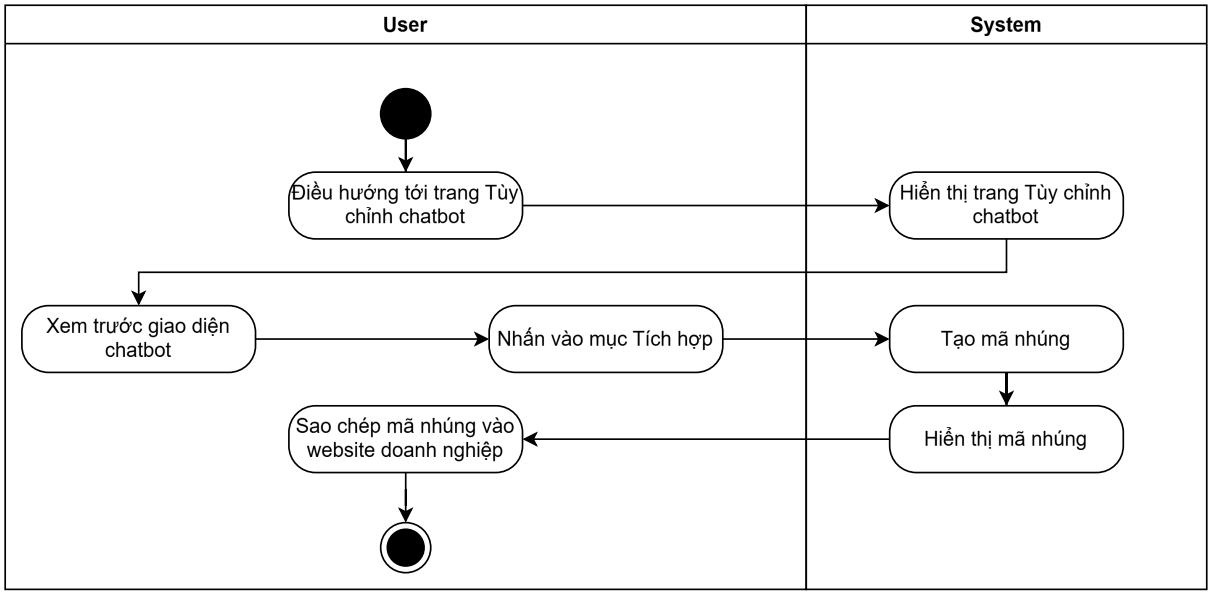
Hình 29: *Biểu đồ activity cho Tùy chỉnh chatbot*

Mô tả: Biểu đồ hoạt động gồm 3 swimlane: Người dùng, hệ thống và thiết bị.
Đầu tiên, nhân viên vào trang Tùy chỉnh chatbot. Tại trang này, nhân viên có 2 lựa chọn:

- Tùy chỉnh phong cách: tin nhắn mặc định (lời chào, lời chuyển giao, lời tạm biệt) cùng với tông giọng (lịch sự, vui vẻ, thân thiện, trực tiếp).
- Tùy chỉnh giao diện: ảnh nền chat, ảnh đại diện chatbot và một số đặc tính CSS: màu nền chat, phông chữ, màu chữ, độ bo góc tin nhắn,...

Nhân viên nhấn Lưu để hoàn tất chỉnh sửa hoặc Hủy để hủy bỏ thao tác. Ở tùy chỉnh giao diện, nhân viên có thể xem trước giao diện đã chỉnh sửa trước khi Lưu.

3.6.3 Tích hợp



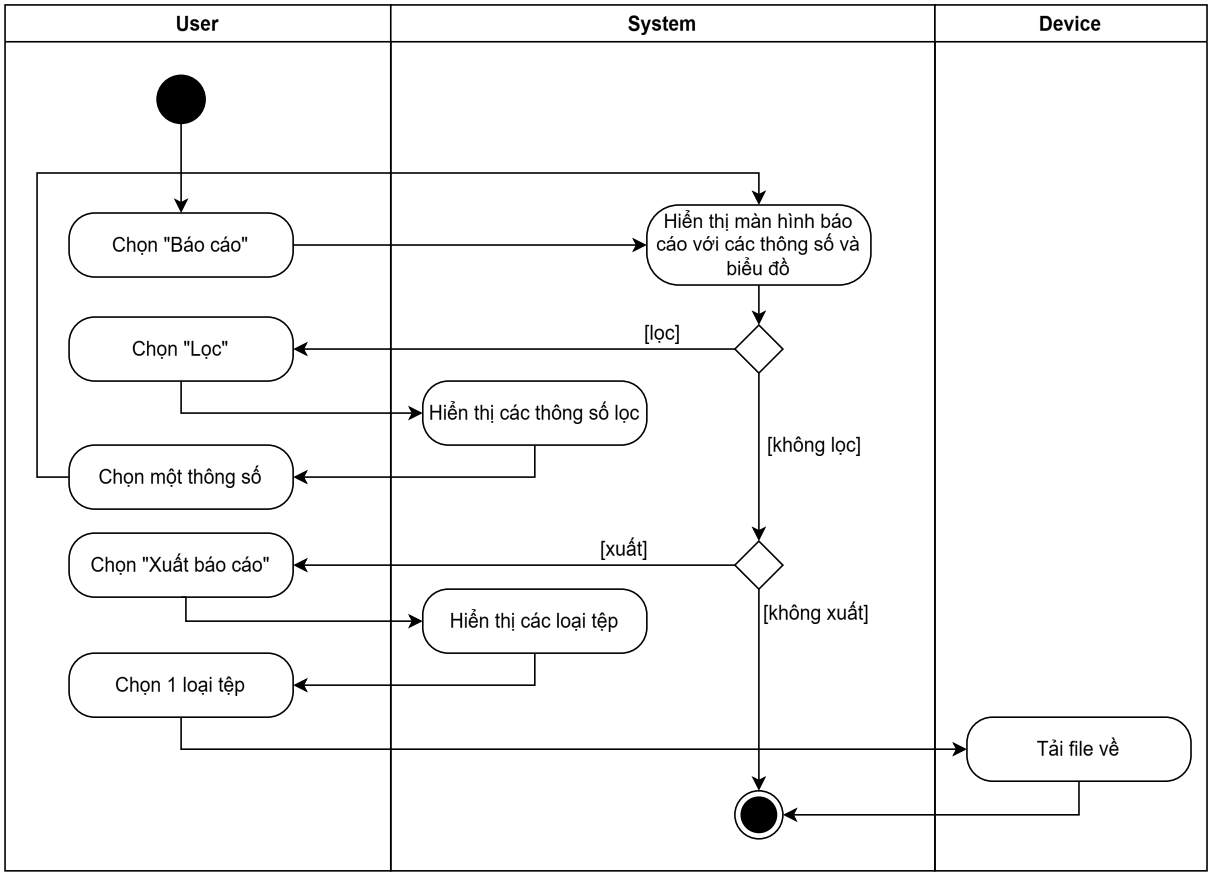
Hình 30: *Biểu đồ activity cho Tích hợp*

Mô tả: Biểu đồ hoạt động gồm 2 swimlane: Người dùng và hệ thống.

Quá trình tạo mã nhúng cho chatbot bắt đầu bằng việc người dùng điều hướng tới trang Tùy chỉnh chatbot. Sau khi truy cập thành công, hệ thống sẽ hiển thị giao diện Tùy chỉnh chatbot và cho phép xem trước giao diện chatbot để đảm bảo rằng thiết kế và các chức năng hoạt động như mong muốn trước khi triển khai. Khi đã hài lòng với bản xem trước, người dùng sẽ nhấn mở mục Tích hợp để hệ thống bắt đầu quá trình tạo mã nhúng. Sau khi hoàn thành, mã nhúng này sau đó sẽ được hiển thị trên giao diện người dùng, cho phép người dùng dễ dàng thao tác. Cuối cùng, người dùng chỉ cần sao chép mã nhúng và dán vào website doanh nghiệp của mình để hoàn tất quá trình tích hợp chatbot.

3.6.4 Quản lý chatbot

3.6.4.1 Báo cáo



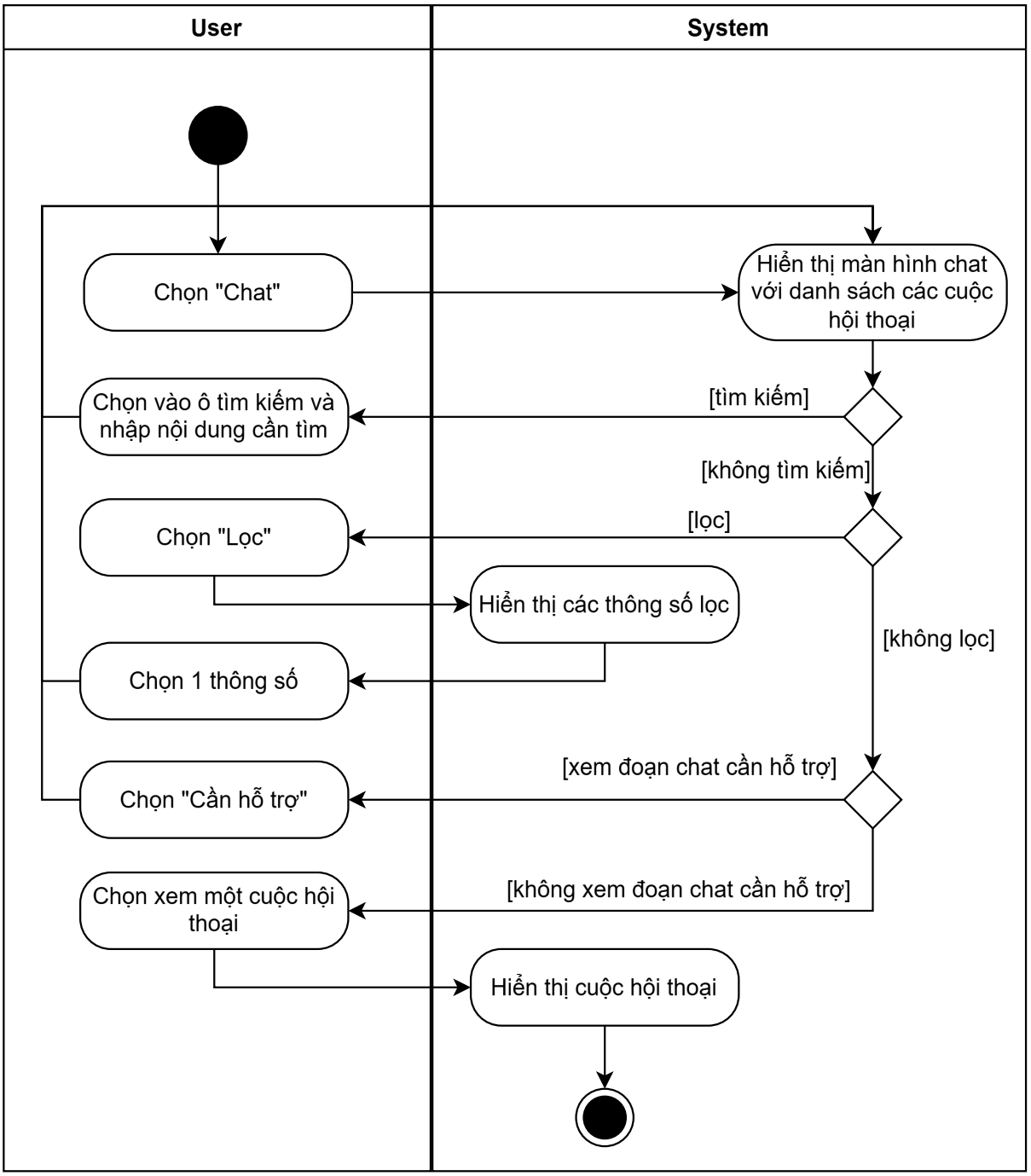
Hình 31: *Biểu đồ activity cho Báo cáo*

Mô tả: Biểu đồ hoạt động gồm 3 swimlane: Người dùng, hệ thống và thiết bị.

Ban đầu quản trị viên doanh nghiệp chọn vào nút "Báo cáo" trên thanh điều hướng. Khi đó màn hình sẽ chuyển qua trang Báo cáo và hiển thị các thông số và biểu đồ.

- Nếu quản trị viên chọn "Lọc", hệ thống sẽ hiển thị các thông số lọc và quản trị viên chọn một thông số. Khi đó, hệ thống sẽ hiển thị lại các thông số và biểu đồ tương ứng với thông số lọc đó. Rồi sau đó quản trị viên có thể chọn "Lọc" tiếp hoặc chọn "Xuất" hoặc dừng lại.
- Nếu quản trị viên chọn "Xuất", hệ thống sẽ hiển thị các loại tệp. Sau đó người dùng sẽ phải chọn một. Khi đó, file sẽ được tải về thiết bị.
- Nếu quản trị viên không chọn gì thì sẽ kết thúc.

3.6.4.2 Lịch sử hội thoại



Hình 32: *Biểu đồ activity cho Lịch sử hội thoại*

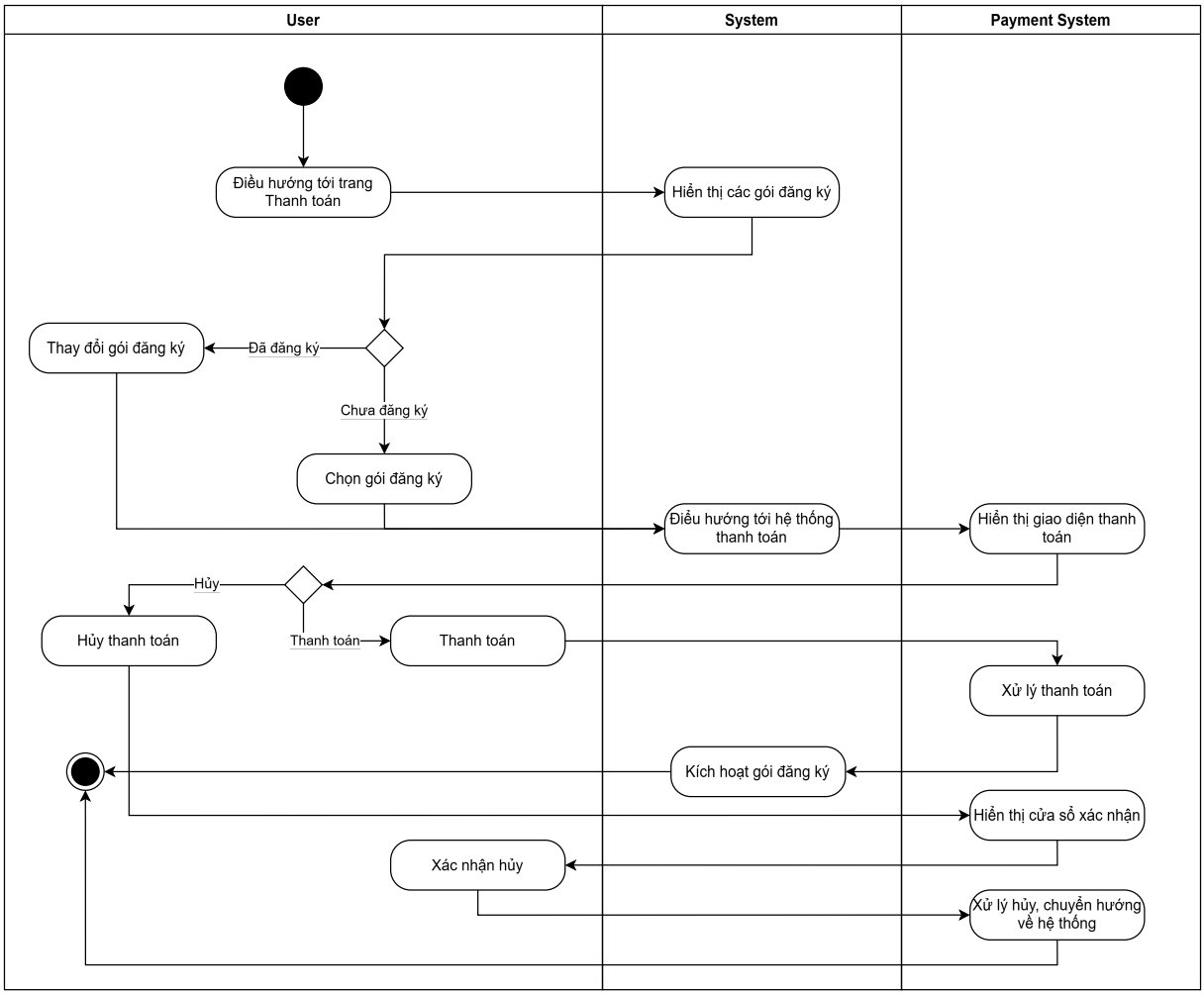
Mô tả: Biểu đồ hoạt động gồm 2 swimlane: Người dùng và hệ thống.

Ban đầu người dùng chọn vào nút "Chat" trên thanh điều hướng. Khi đó màn hình sẽ chuyển qua trang Chat và hiển thị danh sách các cuộc hội thoại của khách hàng.

- Nếu người dùng chọn vào ô tìm kiếm, nhập nội dung tìm kiếm thì khi đó hệ thống sẽ hiển thị các cuộc hội thoại có tên tương ứng với nội dung tìm kiếm.
- Nếu người dùng chọn "Lọc", hệ thống sẽ hiển thị các thông số lọc và người dùng chọn một thông số. Khi đó, hệ thống sẽ hiển thị lại các cuộc hội thoại tương ứng với thông số lọc đó.
- Nếu người dùng chọn "Cần hỗ trợ" thì hệ thống sẽ chỉ hiển thị các cuộc hội thoại mà khách hàng đã gửi yêu cầu hỗ trợ.

Sau đó người dùng có thể lặp lại các bước trên hoặc chọn xem một cuộc hội thoại. Khi quản trị viên chọn xem một cuộc hội thoại thì hệ thống sẽ hiển thị chi tiết cuộc hội thoại đó.

3.6.5 Quản lý thanh toán



Hình 33: *Biểu đồ activity cho Quản lý thanh toán*

Mô tả: Biểu đồ hoạt động gồm 3 swimlane: Người dùng, hệ thống và hệ thống thanh toán PayOS.

Quá trình đăng ký gói dịch vụ bắt đầu bằng việc người dùng điều hướng tới trang thanh toán. Khi truy cập thành công, hệ thống sẽ hiển thị các gói đăng ký hiện có, cho phép người dùng lựa chọn phù hợp với nhu cầu của mình.

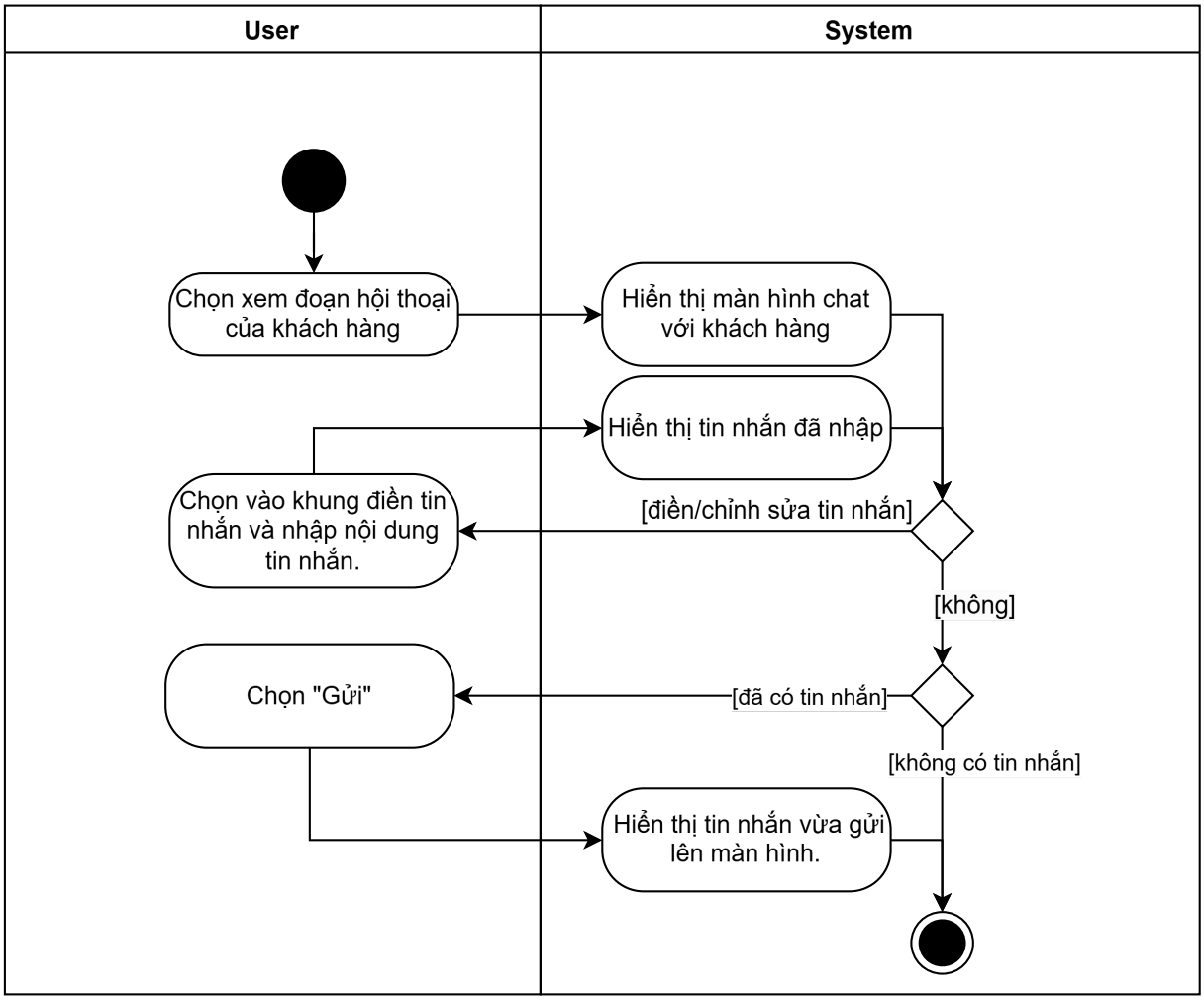
Tiếp theo, người dùng có thể thay đổi hoặc chọn gói đăng ký mới nếu muốn nâng cấp hoặc điều chỉnh dịch vụ. Sau khi lựa chọn gói phù hợp và chọn mua, hệ thống sẽ điều hướng tới hệ thống thanh toán để thực hiện giao dịch. Giao diện thanh toán sẽ được hiển thị, nơi người dùng có thể quét mã QR để thanh toán.

Khi người dùng thực hiện thanh toán xong, hệ thống sẽ tiến hành xử lý thanh toán. Nếu thanh toán thành công, hệ thống sẽ kích hoạt gói đăng ký mà người dùng đã chọn, đảm bảo rằng dịch vụ được cung cấp đầy đủ theo gói đã mua.

Trong trường hợp người dùng không thanh toán mà muốn hủy, người dùng chọn Hủy bỏ và xác nhận. Hệ thống thanh toán sẽ xử lý yêu cầu hủy, khi thành công sẽ chuyển về hệ thống.

3.6.6 Tương tác với chatbot

3.6.6.1 Phản hồi tin nhắn khách hàng



Hình 34: *Biểu đồ activity cho Phản hồi tin nhắn khách hàng*

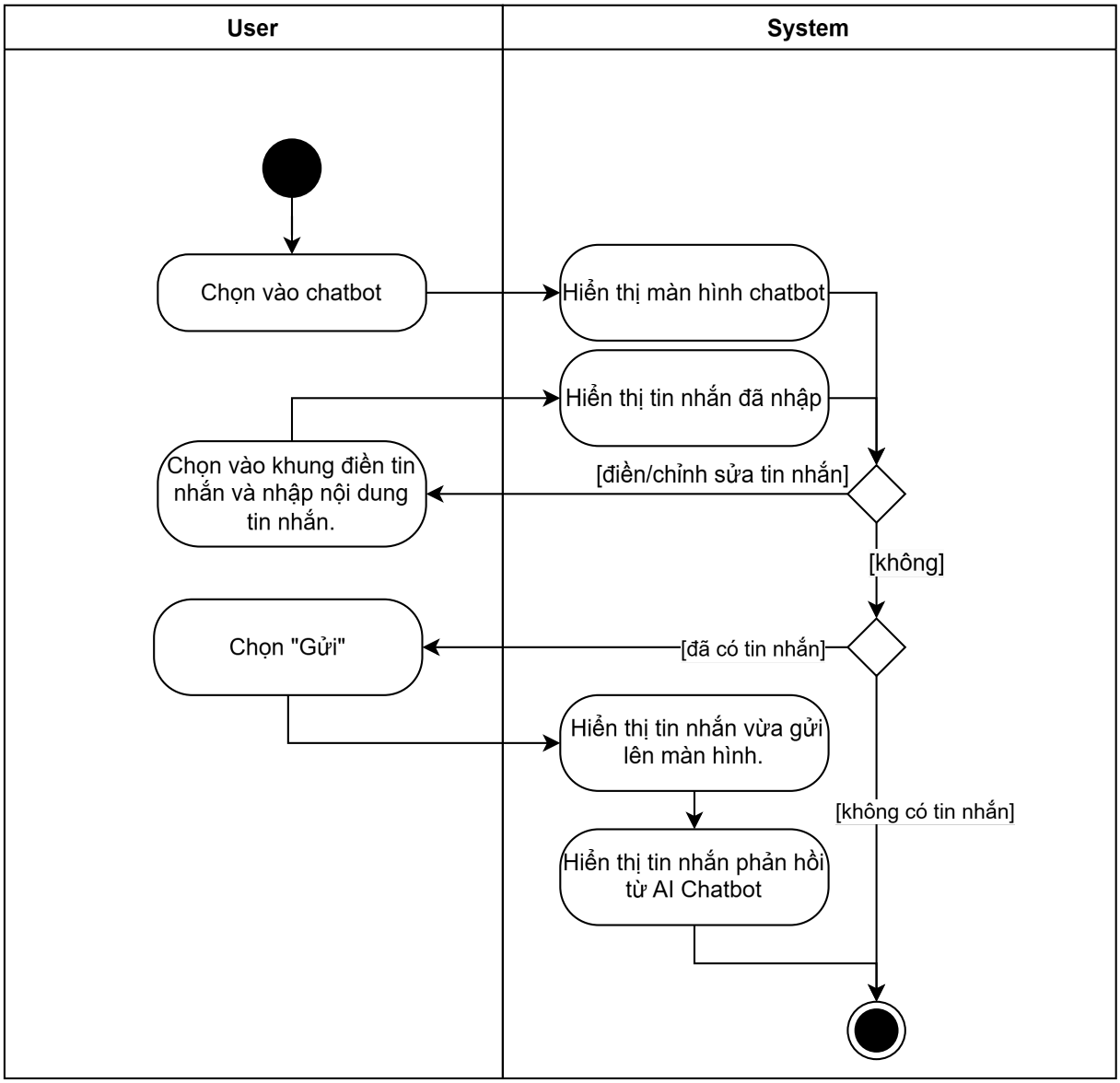
Mô tả: Biểu đồ hoạt động gồm 2 swimlane: Người dùng và hệ thống.

Ban đầu nhân viên chăm sóc khách hàng chọn xem đoạn hội thoại của khách hàng tại trang Chat. Hệ thống sẽ hiển thị màn hình chat với khách hàng đó.

Nếu muốn điền hoặc chỉnh sửa tin nhắn, nhân viên chăm sóc khách hàng chọn vào khung điền tin nhắn và nhập nội dung tin nhắn. Khi đó hệ thống sẽ hiển thị đoạn tin nhắn mà nhân viên chăm sóc khách hàng đã nhập.

Sau đó nhân viên chăm sóc khách hàng có thể lặp lại các bước trên. Hoặc nếu đã có tin nhắn và nhân viên chăm sóc khách hàng chọn gửi thì hệ thống sẽ hiển thị tin nhắn vừa gửi lên màn hình. Hoặc nếu chưa có tin nhắn thì sẽ dừng lại.

3.6.6.2 Gửi tin nhắn



Hình 35: *Biểu đồ activity cho Gửi tin nhắn*

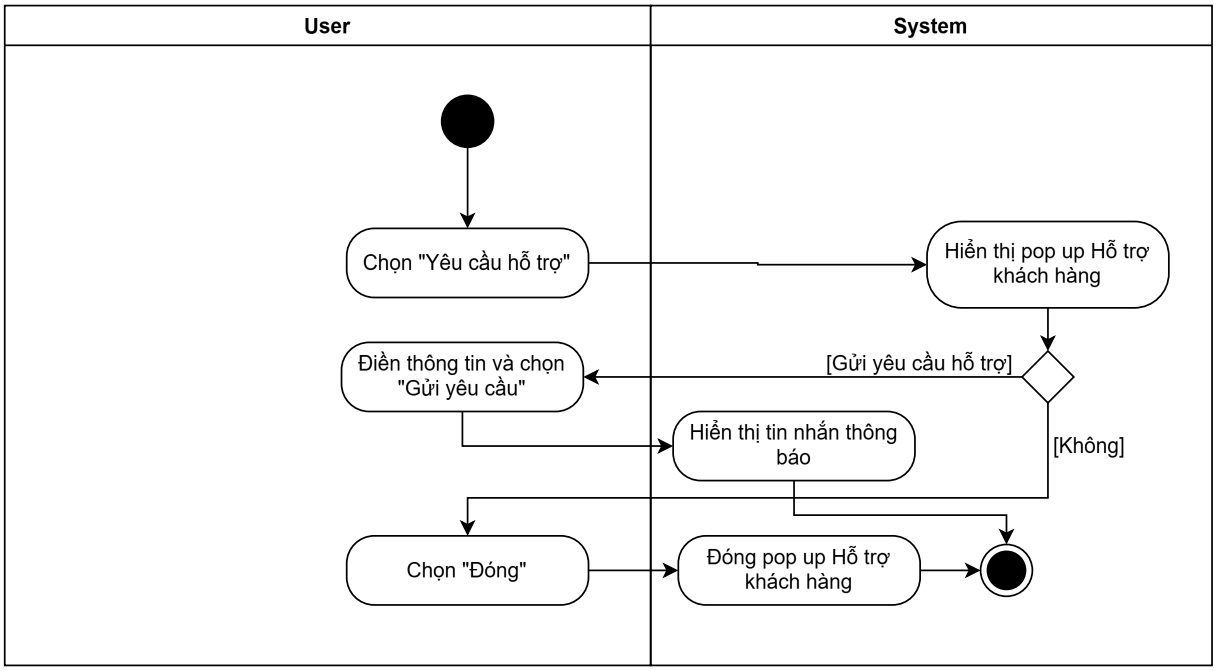
Mô tả: Biểu đồ hoạt động gồm 2 swimlane: Người dùng và hệ thống.

Ban đầu khách hàng chọn vào chatbot tại màn hình website của doanh nghiệp. Hệ thống sẽ hiển thị màn hình chatbot.

Nếu muốn điền hoặc chỉnh sửa tin nhắn, khách hàng chọn vào khung điền tin nhắn và nhập nội dung tin nhắn. Khi đó hệ thống sẽ hiển thị đoạn tin nhắn mà khách hàng đã nhập.

Sau đó khách hàng có thể lặp lại các bước trên. Hoặc nếu đã có tin nhắn và khách hàng chọn gửi thì hệ thống sẽ hiển thị tin nhắn vừa gửi lên màn hình. Đồng thời, hệ thống sẽ tạo và gửi lại tin nhắn phản hồi từ AI Chatbot. Hoặc nếu chưa có tin nhắn thì sẽ dừng lại.

3.6.6.3 Yêu cầu hỗ trợ



Hình 36: *Biểu đồ activity cho Yêu cầu hỗ trợ*

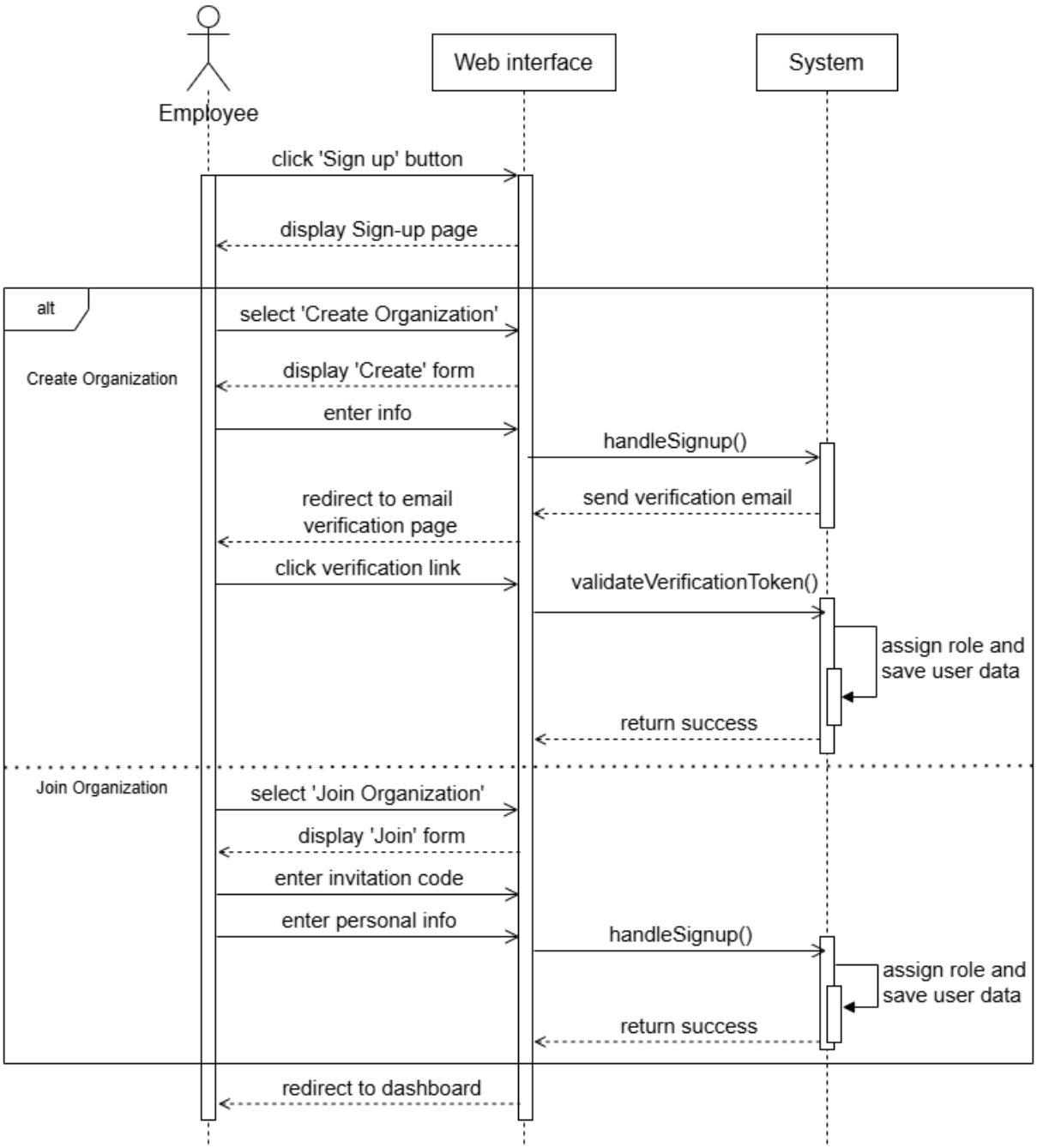
Mô tả: Biểu đồ hoạt động gồm 2 swimlane: Người dùng và hệ thống.
Tại màn hình chatbot, khách hàng chọn "Yêu cầu hỗ trợ". Hệ thống sẽ hiển thị popup hỗ trợ khách hàng.

- Nếu muốn gửi yêu cầu hỗ trợ, khách hàng điền thông tin và chọn "Gửi yêu cầu". Hệ thống sẽ hiển thị các tin nhắn thông báo, cuộc hội thoại chuyển sang chế độ hỗ trợ.
- Nếu khách hàng chọn "Đóng", hệ thống sẽ đóng popup Hỗ trợ khách hàng.

3.7 Biểu đồ Sequence

3.7.1 Quản lý tài khoản

3.7.1.1 Đăng ký



Hình 37: *Biểu đồ sequence cho Đăng ký*

Mô tả: Biểu đồ tuần tự Đăng ký mô tả sự tương tác giữa các thực thể sau:

- Actor: Người dùng
- Giao diện trang web (Web interface)
- Hệ thống

Các bước trong quá trình tương tác như sau:

1. Người dùng truy cập trang đăng ký.

2. Giao diện hiển thị hai lựa chọn: "Tạo tổ chức mới" hoặc "Tham gia tổ chức".
3. **Luồng tạo tổ chức mới:**

3a. Người dùng chọn "Tạo tổ chức mới".

3b. Người dùng điền thông tin tổ chức và thông tin cá nhân.

3c. Gửi yêu cầu tạo tổ chức.

3d. Hệ thống gửi email xác minh cho người dùng.

3e. Người dùng xác minh email.

3f. Hệ thống xác thực và lưu thông tin người dùng và tổ chức.

4. **Luồng tham gia tổ chức:**

4a. Người dùng chọn "Tham gia tổ chức".

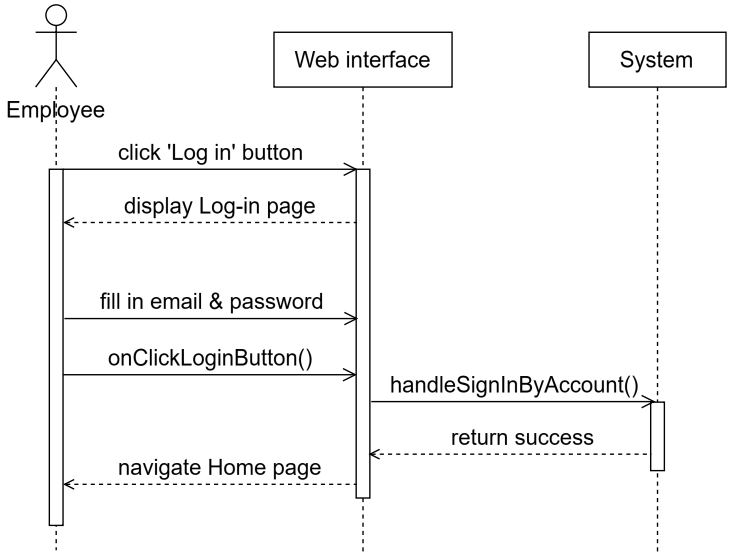
4b. Người dùng nhập mã mời và thông tin cá nhân.

4c. Gửi yêu cầu tham gia tổ chức.

4d. Hệ thống xác thực mã mời và lưu thông tin người dùng.

5. Hệ thống gán vai trò tương ứng:
   - Người tạo tổ chức được gán vai trò quản trị viên
   - Người tham gia được gán vai trò theo mã mời
6. Giao diện nhận thông báo thành công và chuyển hướng người dùng đến trang chính theo vai trò.

3.7.1.2 Đăng nhập



Hình 38: *Biểu đồ sequence cho Đăng nhập*

Mô tả: Biểu đồ tuần tự Đăng nhập mô tả sự tương tác giữa các thực thể sau:

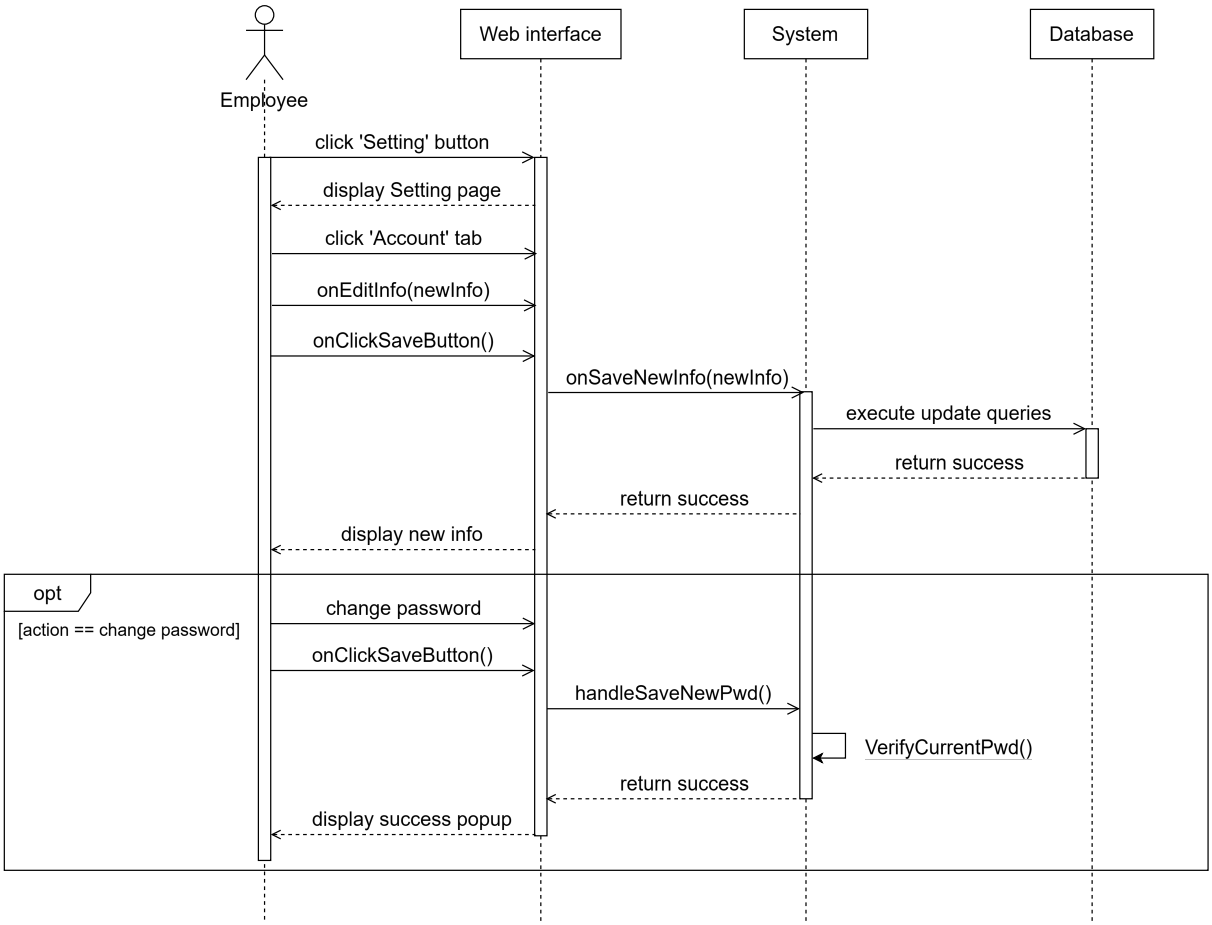
- Actor: Nhân viên (gồm cả quản trị viên và nhân viên chăm sóc khách hàng).
- Giao diện trang web (Web interface).
- Hệ thống.

Các bước trong quá trình tương tác như sau:

1. Nhân viên ấn nút Đăng nhập.
2. Giao diện hiển thị trang Đăng nhập.

3. Nhân viên nhập hai thông tin trên. Sau đó ấn nút "Đăng nhập" để gọi phương thức onClickLoginButton() trên giao diện.
4. Phương thức handleSignInByAccount() của hệ thống được gọi.
5. Hệ thống trả về mã thông báo thành công cho giao diện web.
6. Nhân viên được điều hướng về trang chủ.

3.7.1.3 Chỉnh sửa hồ sơ cá nhân



Hình 39: *Biểu đồ sequence cho Chỉnh sửa hồ sơ cá nhân*

Mô tả: Biểu đồ tuần tự Chỉnh sửa hồ sơ cá nhân mô tả sự tương tác giữa các thực thể sau:

- Actor: Nhân viên (gồm cả quản trị viên và nhân viên chăm sóc khách hàng).
- Giao diện trang web (Web interface).
- Hệ thống.
- Database.

Các bước trong quá trình tương tác như sau:

1. Nhân viên ấn nút "Cài đặt".
2. Giao diện hiển thị trang Cài đặt. Nhân viên ấn vào mục Tài khoản.

   Trường hợp chỉnh sửa thông tin cá nhân:

   (a) Nhân viên chỉnh sửa các thông tin: ảnh đại diện, họ tên, số điện thoại và ấn nút "Đăng nhập" để gọi phương thức onClickSaveButton() trên giao diện.

   (b) Phương thức onSaveNewInfo(newInfo) của hệ thống được gọi.

(c) Các lệnh update được thực hiện trong cơ sở dữ liệu.

(d) Mã thông báo thành công được trả về, các thông tin mới được hiển thị lên giao diện web.

Trường hợp sử dụng thay đổi mật khẩu:

(a) Nhân viên nhập mật khẩu hiện tại, mật khẩu mới và ấn "Lưu" để gọi phương thức onClickSaveButton() trên giao diện.

(b) Phương thức handleSaveNewPwd() của hệ thống được gọi.

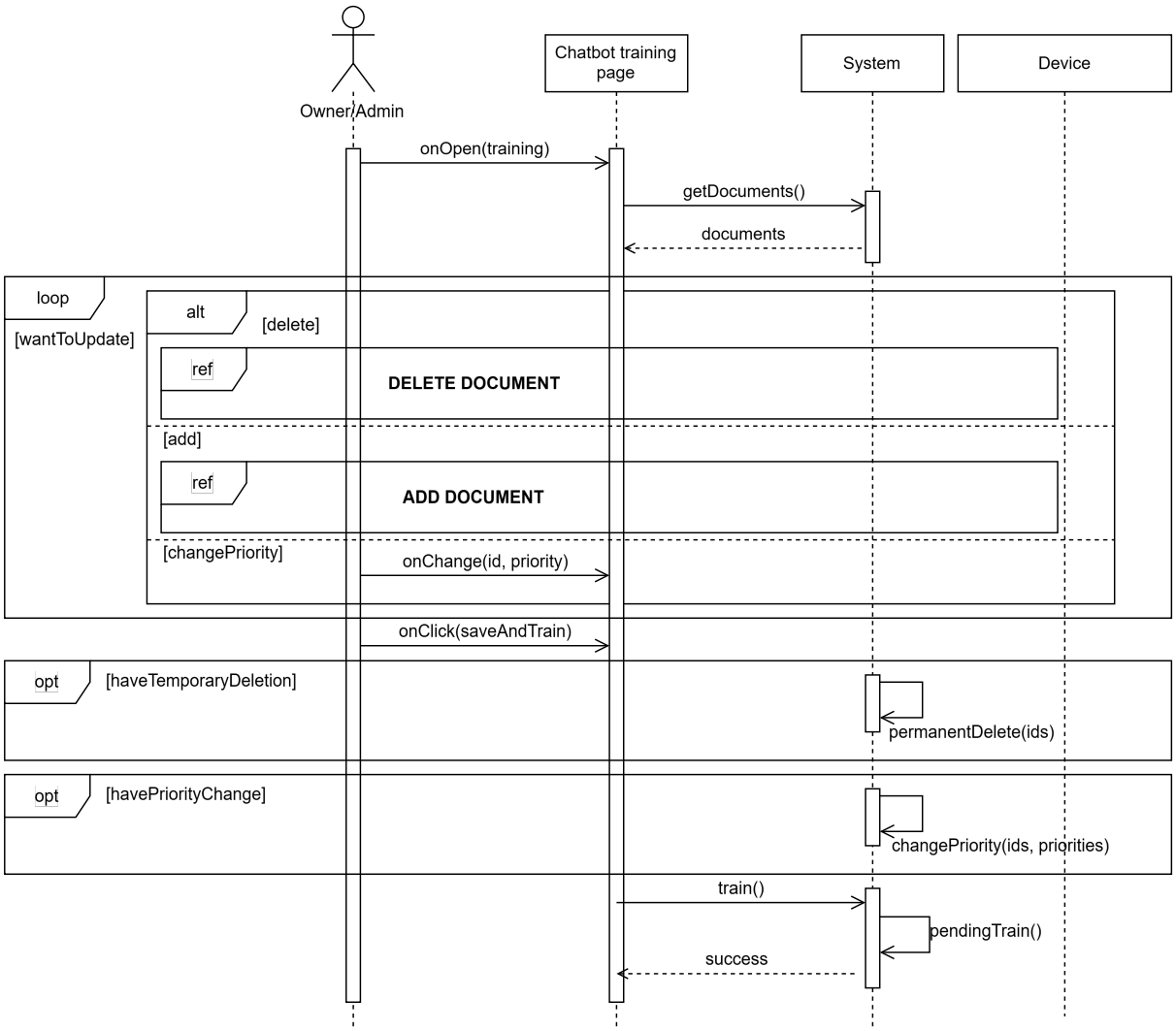
(c) Hệ thống kiểm tra mật khẩu hiện tại đã đúng chưa (VerifyCurrentPwd()).

(d) Mã thông báo thành công được trả về.

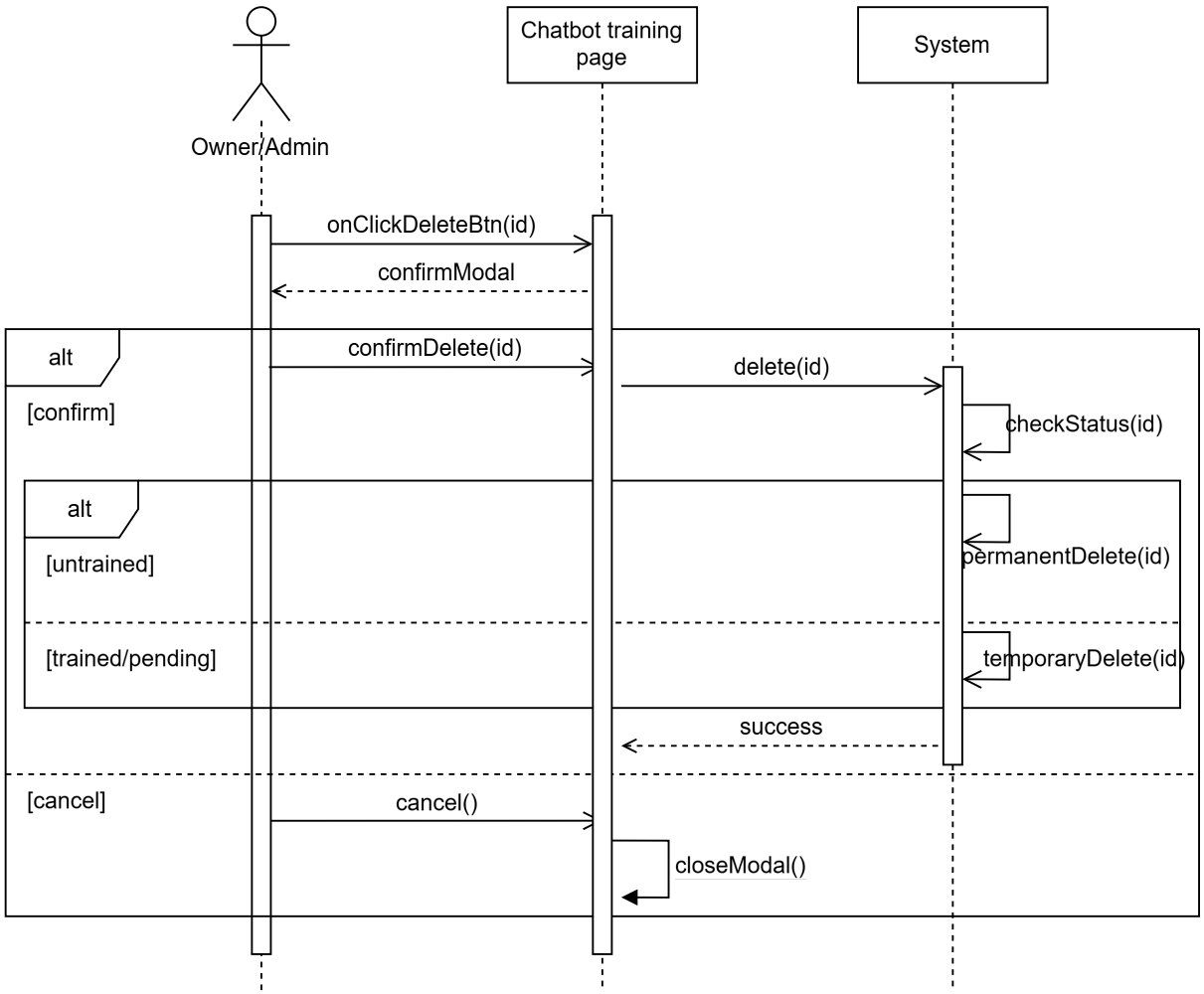
(e) Giao diện web hiển thị thông báo thành công.

3.7.2 Đào tạo và tùy chỉnh chatbot

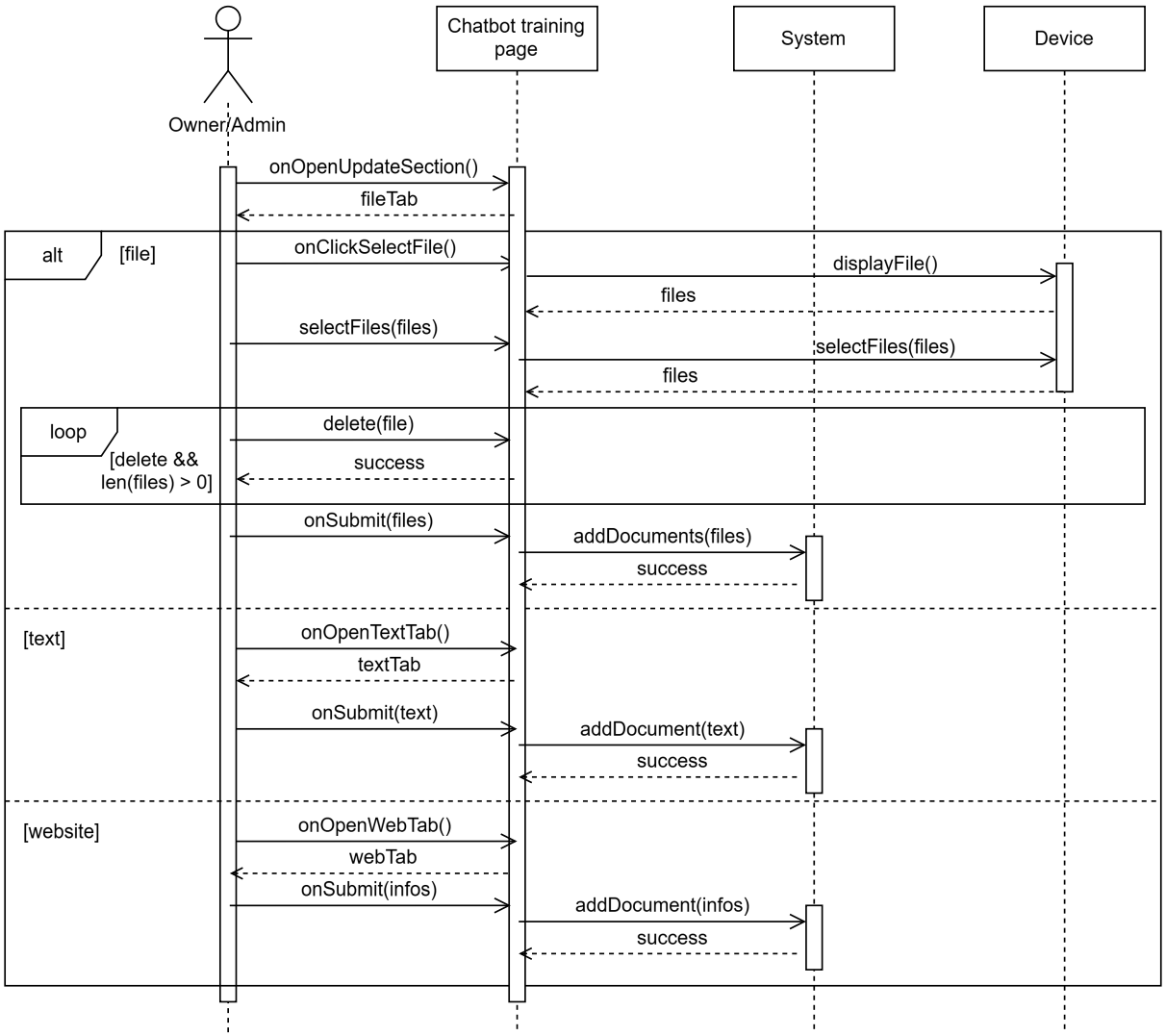
3.7.2.1 Đào tạo chatbot



Hình 40: *Biểu đồ sequence cho Đào tạo chatbot*



Hình 41: *Biểu đồ sequence cho Xóa tri thức*



Hình 42: *Biểu đồ sequence cho Thêm tri thức*

Mô tả: Biểu đồ tuần tự Đào tạo chatbot mô tả sự tương tác giữa các thực thể sau:

- Actor: Nhân viên quản lý (chủ sở hữu/quản trị viên).
- Giao diện trang web (trang chatbot training).
- Hệ thống.
- Thiết bị.

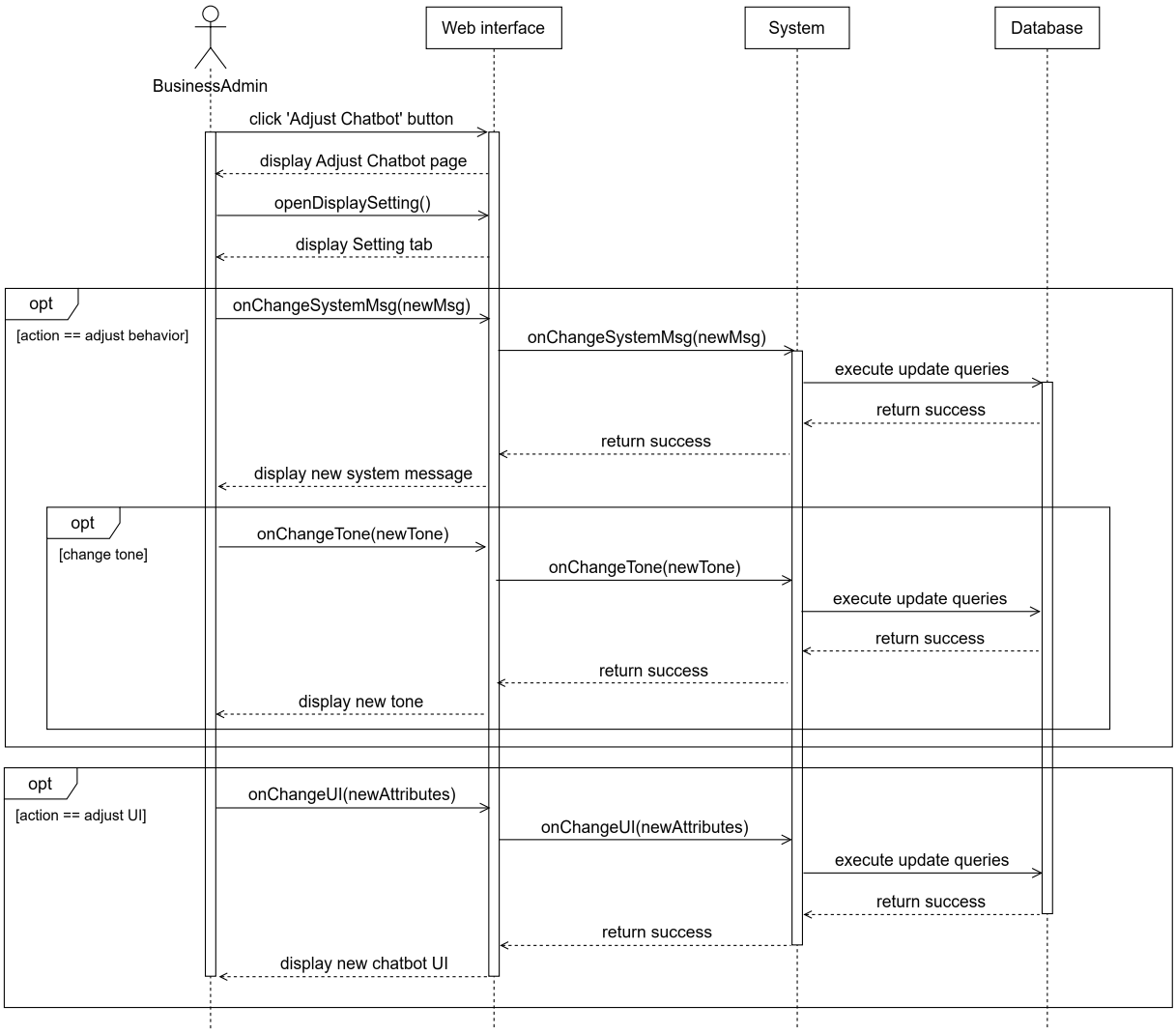
Các bước trong quá trình tương tác như sau:

1. Người dùng mở trang Đào tạo chatbot, khi đó Hệ thống sẽ gọi getDocuments() và trả về giao diện danh sách các tri thức.
2. Tiếp theo người dùng có thể thực hiện nhiều lần các lựa chọn sau:
   - Xóa tri thức: người dùng chọn delete cho tri thức có id trên màn hình. Giao diện hiển thị confirm-Modal. Nếu người dùng chọn xác nhận, hệ thống gọi delete(id) để xóa tri thức, khi đó hệ thống sẽ gọi checkStatus(id) để kiểm tra trạng thái, nếu untrained thì gọi permanentDelete(id), còn không gọi temporaryDelete(id) sau đó sẽ trả về thông báo thành công. Nếu người dùng chọn Hủy, giao diện đóng modal và kết thúc.
   - Thêm tri thức: Người dùng mở Cập nhật tri thức. Khi đó có 3 lựa chọn để thêm tri thức:
     - Tập tin: Người dùng nhấn tải file, thiết bị người dùng gọi displayFile() và trả files về giao diện. Người dùng chọn các file cần tải lên, thiết bị gọi hàm selectFiles(files) để chọn file và trả files là các file đã chọn. Người dùng có thể xóa các file đã tải và cuối cùng lưu, giao diện gọi onSubmit(files), hệ thống gọi addDocument(files) và trả về thông báo thành công.

  - Văn bản: Người dùng mở tab thêm tri thức, điền và lưu, giao diện gọi onSubmit(text), hệ thống gọi addDocument(text) và trả về thông báo thành công.
  - Website: Người dùng mở tab thêm tri thức web, điền thông tin và lưu, giao diện gọi onSubmit(infos), hệ thống gọi addDocument(infos) và trả về thông báo thành công.
- Sửa độ ưu tiên: người dùng sửa độ ưu tiên cho tri thức, giao diện gọi onChange(id, priority) với id là id của tri thức, priority là độ ưu tiên mới.

3. Sau khi hoàn thành quá trình chỉnh sửa, người dùng chọn Lưu và đào tạo. Trước hết, nếu tồn tại tri thức bị xóa tạm thời, hệ thống gọi permanentDelete(ids) để xóa các tri thức. Nếu tồn tại sự thay đổi độ ưu tiên, hệ thống gọi changePriority(ids, priorities) để đổi độ ưu tiên mới, sau đó hệ thống sẽ gọi hàm train() để bắt đầu đào tạo và pendingTrain() để đặt trạng thái chờ. Khi xong, hệ thống trả về thông báo thành công.

3.7.2.2 Tùy chỉnh chatbot



Hình 43: *Biểu đồ sequence cho Tùy chỉnh chatbot*

Mô tả: Biểu đồ tuần tự Tùy chỉnh chatbot mô tả sự tương tác giữa các thực thể sau:

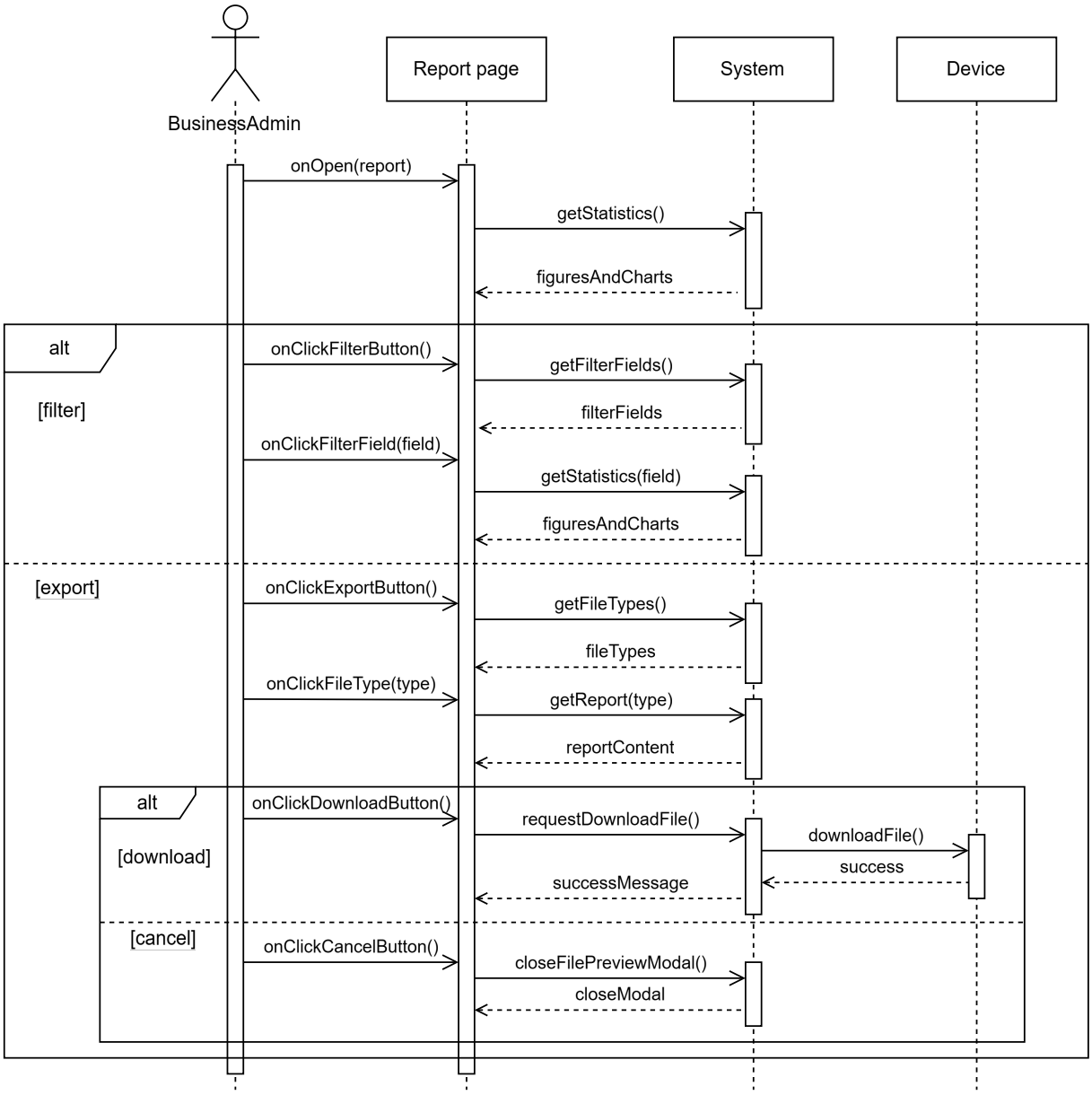
- Actor: Quản trị viên.
- Giao diện trang web (Web interface).
- Hệ thống.
- Cơ sở dữ liệu.

Các bước trong quá trình tương tác như sau: Đầu tiên quản trị viên vào mục Tùy chỉnh chatbot và mở tab cài đặt (openDisplaySetting() trên giao diện)

1. **Trường hợp cần chỉnh sửa phong cách ứng xử:**
   (a) Người dùng thay đổi tin nhắn mặc định (gọi onChangeSystemMsg(address) của giao diện web) hoặc onChangeTone(newTone) trong trường hợp thay đổi tông giọng.
   (b) Lời gọi onChangeSystemMsg(address) hoặc onChangeTone(newTone) của hệ thống để thay đổi địa chỉ, tông giọng.
   (c) Các câu truy vấn cập nhật được thực hiện trong cơ sở dữ liệu và trả về thông tin kết quả.
   (d) Mã thông báo thành công được trả về cho giao diện web.
   (e) Tin nhắn mặc định / tông giọng mới được hiển thị.
2. **Trường hợp cần sửa địa chỉ giao diện chatbot:**
   (a) Người dùng thay đổi các thuộc tính giao diện. Sau đó nhấn lưu để gọi onChangeUI(newAttributes) của giao diện.
   (b) Lời gọi onChangeUI(newAttributes) của hệ thống để chỉnh sửa các thuộc tính mục tiêu.
   (c) Các câu truy vấn cập nhật được thực hiện trong cơ sở dữ liệu và trả về thông tin kết quả.
   (d) Mã thông báo thành công được trả về cho giao diện web.
   (e) Giao diện hiển thị chatbot với UI mới.

3.7.3 Quản lý chatbot

3.7.3.1 Báo cáo



Hình 44: *Biểu đồ sequence cho Báo cáo*

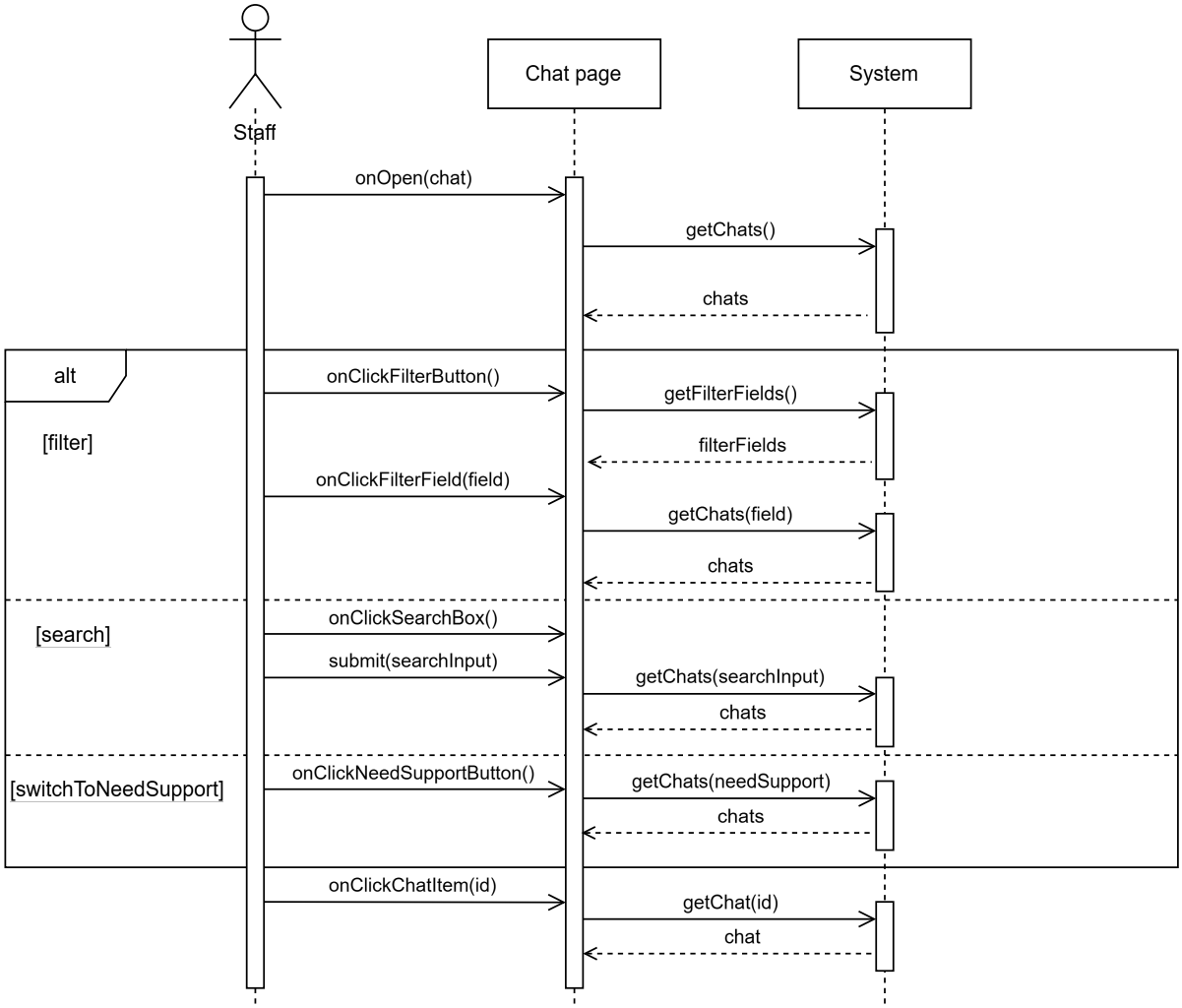
Mô tả:

- Quản trị viên doanh nghiệp mở trang Báo cáo qua hàm onOpen(report).
- Giao diện trang Báo cáo sẽ gọi hàm getStatistics() đến Hệ thống để lấy các dữ liệu thống kê.
- Hệ thống phản hồi và trả lại các dữ liệu thống kê cho trang Báo cáo.
- Có 2 tùy chọn khả dụng là quản trị viên có thể chọn lọc hoặc xuất:
  - Lọc: quản trị viên click nút Lọc với onClickFilterButton(), trang Báo cáo gọi hàm getFilterFields() đến hệ thống, hệ thống trả lại dữ liệu các filterFields, quản trị viên tiếp tục chọn trường cần lọc với onClickFilterField(field) có biến field truyền vào, trang Báo cáo gọi hàm getStatistics(field) đến hệ thống để lấy dữ liệu thống kê theo trường lọc đó, hệ thống trả lại các figuresAndCharts tương ứng.
  - Xuất: quản trị viên click nút Xuất với onClickExportButton(), trang Báo cáo gọi hàm getFileTypes() đến hệ thống, hệ thống trả lại dữ liệu các loại file, quản trị viên tiếp tục chọn loại file với OnClickFileType(type) có biến type được truyền vào, trang Báo cáo gọi hàm getReport(type) đến hệ thống để lấy

dữ liệu file báo cáo, hệ thống trả lại file báo cáo phù hợp. Tiếp tục có 2 tùy chọn là quản trị viên có thể chọn Tải xuống hoặc Hủy bỏ:

* Tải xuống: Quản trị viên click nút Tải xuống với onClickDownloadButton(), trang báo cáo gọi hàm requestDownloadFile() lên hệ thống, hệ thống gọi hàm downloadFile() lên thiết bị để file được tải xuống thiết bị, khi thành công thiết bị trả tin nhắn thành công lên hệ thống và hệ thống trả lại trang báo cáo.
* Hủy bỏ: Quản trị viên click nút Hủy bỏ với onClickCancelButton(), trang báo cáo gọi hàm closeFilePreviewModal() lên hệ thống để đóng modal xem trước, hệ thống trả lại tin nhắn closeModal.

3.7.3.2 Lịch sử hội thoại



Hình 45: *Biểu đồ sequence cho Lịch sử hội thoại*

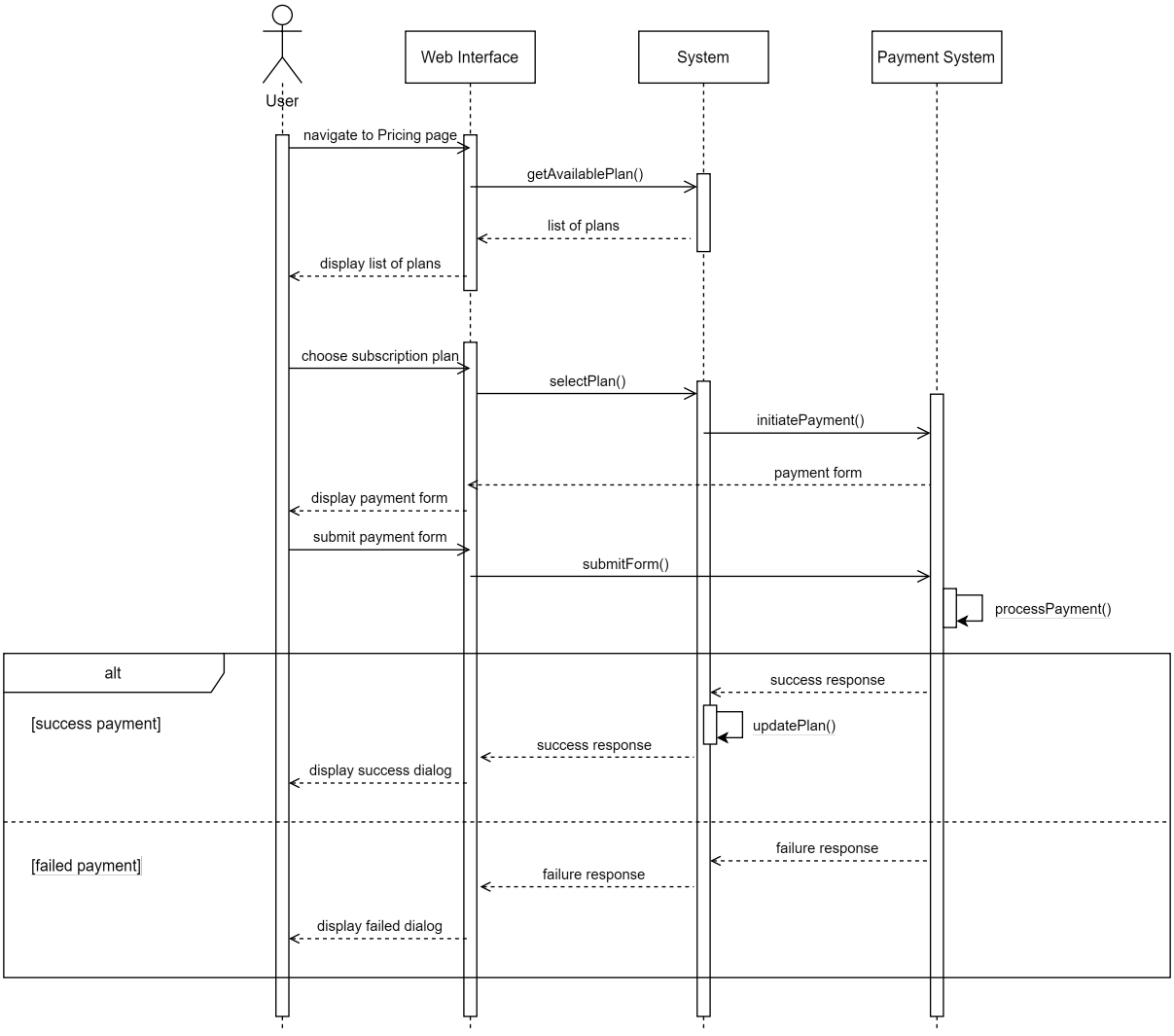
Mô tả:

- Nhân viên doanh nghiệp mở trang Hội thoại qua hàm onOpen(chat).
- Giao diện trang Hội thoại sẽ gọi hàm getChats() đến Hệ thống để lấy danh sách các cuộc hội thoại.
- Hệ thống phản hồi và trả lại danh sách các cuộc hội thoại cho trang Hội thoại.
- Có 3 tùy chọn khả dụng là nhân viên có thể chọn lọc, tìm kiếm hoặc chọn xem các cuộc hội thoại cần hỗ trợ:
  - Lọc: nhân viên click nút Lọc với onClickFilterButton(), trang Hội thoại gọi hàm getFilterFields() đến hệ thống, hệ thống trả lại dữ liệu các filterFields, nhân viên tiếp tục chọn trường cần lọc với onClickFilterField(field) có biến field truyền vào, trang Hội thoại gọi hàm getChats(field) đến hệ thống để lấy dữ liệu hội thoại theo trường lọc đó, hệ thống trả lại các chats tương ứng.

- Tìm kiếm: nhân viên click khung Search với onClickSearchBox(), tiếp tục nhập và hoàn thành nội dung tìm kiếm với submit(searchInput), trang Hội thoại gọi hàm getChats(searchInput) đế hệ thống với biến searchInput từ nhân viên truyền tới để lấy dữ liệu hội thoại theo nội dung tìm kiếm đó, hệ thống trả lại các chats tương ứng.
- Xem các cuộc hội thoại cần hỗ trợ: nhân viên click nút Cần hỗ trợ với onClickNeedSupportBtn(), trang Hội thoại gọi hàm getChats(needSupport) đến hệ thống để lấy danh sách các cuộc hội thoại cần hỗ trợ, hệ thống trả lại các chats tương ứng.

- Nhân viên tiếp tục chọn xem 1 cuộc hội thoại với onClickChatItem(id).
- Trang Hội thoại gọi hàm getChat(id) đến hệ thống với biến id truyền vào.
- Hệ thống trả lại chat tương ứng.

3.7.4 Quản lý thanh toán



Hình 46: *Biểu đồ sequence cho Quản lý thanh toán*

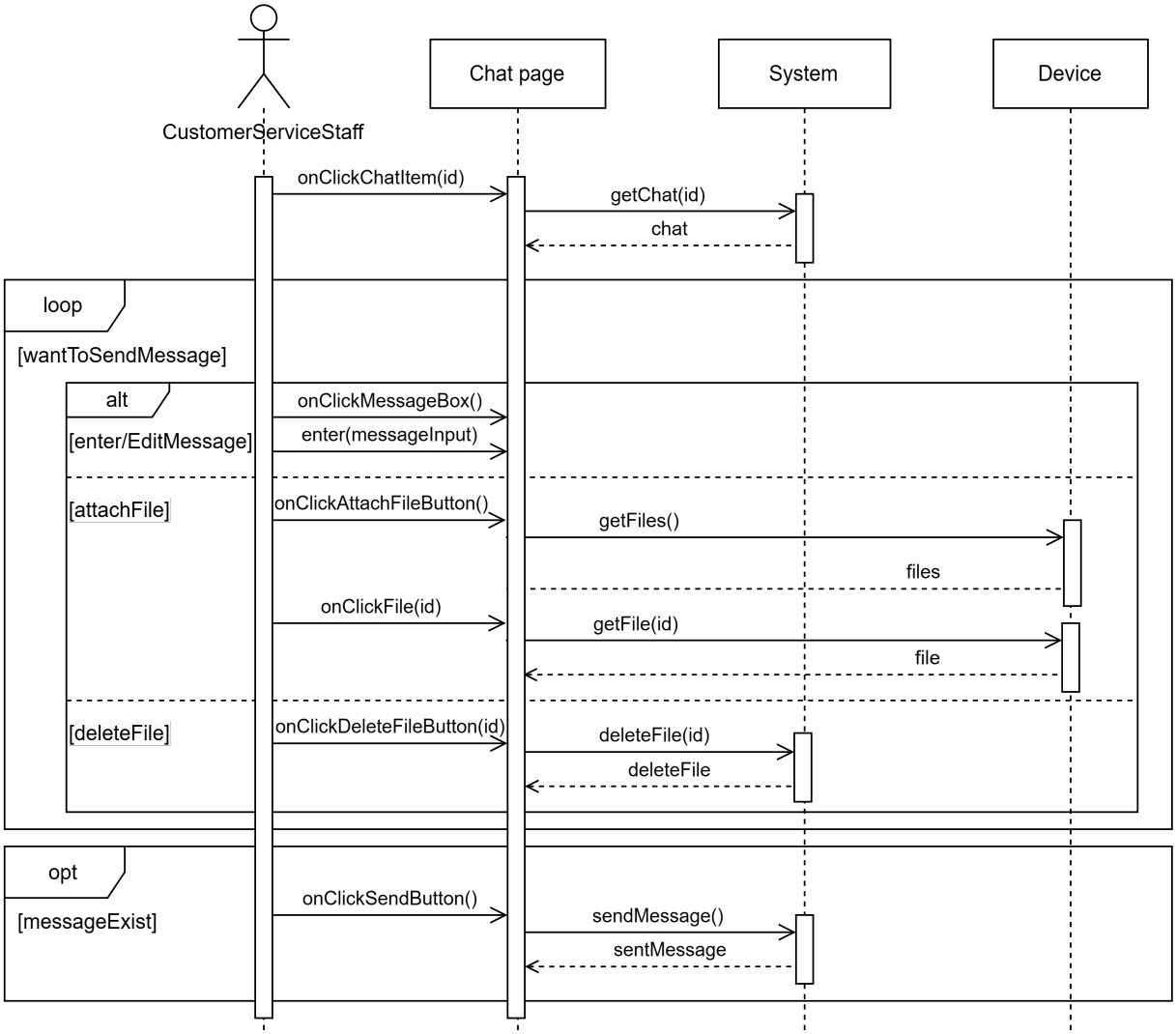
Mô tả:

- Người dùng điều hướng đến trang thanh toán.
- Trang thanh toán sẽ gọi hàm getAvailablePlan() đến hệ thống để lấy các gói dịch vụ.
- Hệ thống trả về danh sách các gói dịch vụ và hiển thị với người dùng.
- Người dùng chọn một gói dịch vụ.

- Trang thanh toán gọi hàm selectPlan() đến hệ thống.
- Hệ thống gọi hàm initiatePayment() đến hệ thống thanh toán.
- Hệ thống thanh toán trả về giao diện và giao diện trả về người dùng form thanh thanh toán.
- Người dùng thực hiện nhập các thông tin và submit.
- Trang thanh toán gọi hàm submitForm() đến hệ thống thanh toán.
- Hệ thống thanh toán tự gọi hàm processPayment() để tiến hành quá trình thanh toán.
- Tiếp theo ngừi dùng có thể gặp 2 tình huống
  - Nếu thanh toán thành công: hệ thống thanh toán trả về hệ thống phản hồi thành công. Hệ thống gọi hàm updatePlan() để cập nhật và trả về phản hồi thành công cho giao diện và hiển thị với người dùng.
  - Nếu thnah toán thất bại: phản hồi thất bại sẽ được truyền từ hệ thống thanh toán đến hệ thống rồi đến giao diện rồi sẽ hiển thị đến người dùng.

3.7.5 Tương tác với chatbot

3.7.5.1 Phản hồi tin nhắn khách hàng



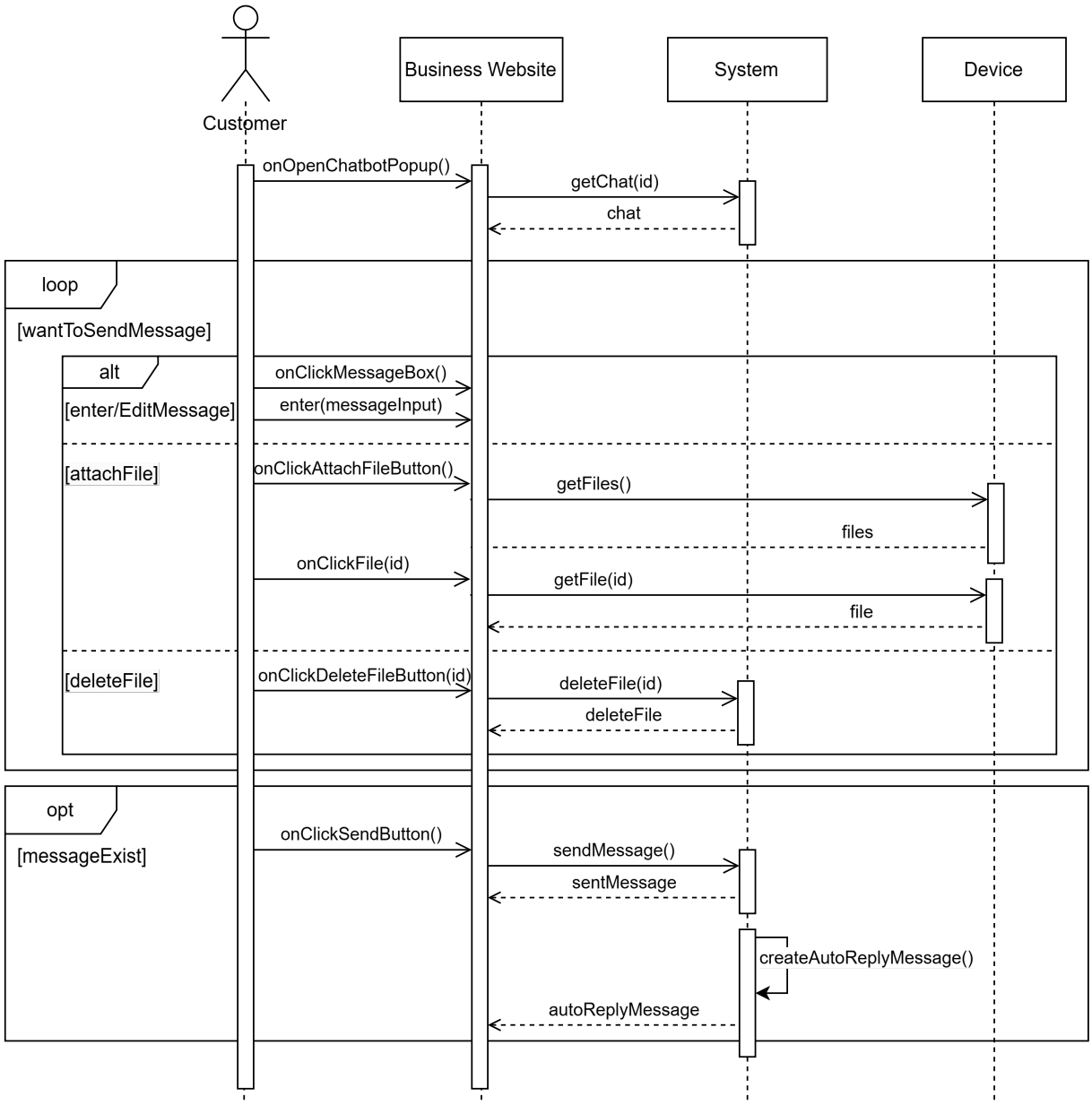
Hình 47: *Biểu đồ sequence cho Phản hồi tin nhắn khách hàng*

Mô tả:

- Nhân viên CSKH chọn xem 1 cuộc hội thoại với onClickChatItem(id).

- Trang Hội thoại gọi hàm getChat(id) đến hệ thống với biến id truyền vào.
- Hệ thống trả lại chat tương ứng.
- Có một vòng lặp với điều kiện nhân viên CSKH tiếp tục quá trình nhập tin nhắn, trong vòng lặp có 3 lựa chọn là nhập/sửa tin nhắn, đính kèm tệp hoặc xóa tệp:
  - Nhập/sửa tin nhắn: nhân viên CSKH click vào khung tin nhắn với onClickMessageBox(), tiếp tục nhập nội dung tin nhắn với enter(messageInput) có biến messageInput truyền vào.
  - Đính kèm tệp: nhân viên CSKH click vào nút Đính kèm tệp với onClickAttachFileButton(), trang Hội thoại gọi hàm getFiles(). đến thiết bị để lấy danh sách các file có thể tải lên, thiết bị tả lại các files cho trang Hội thoại, nhân viên CSKH chọn một file onClickFile(id), trang Hội thoại gọi hàm getFile(id) đến thiết bị với biến id truyền vào, thiết bị trả lại file tương ứng.
  - Xóa tệp: nhân viên CSKH click nút xóa cho tệp cần xóa với onClickDeleteFileButton(id), trang Hội thoại gọi hàm deleteFile(id) đến hệ thống với biến id truyền vào, hệ thống xóa tệp và trả lại tin nhắn deleteFile tương ứng.
- Có một tùy chọn với điều kiện tin nhắn tồn tại: nhân viên CSKH click Gửi với onClickSendButton(), trang Hội thoại gọi hàm sendMessage() đến hệ thống, hệ thống trả lại tin nhắn sendMessage tương ứng.

3.7.5.2 Gửi tin nhắn



Hình 48: *Biểu đồ sequence cho Gửi tin nhắn*

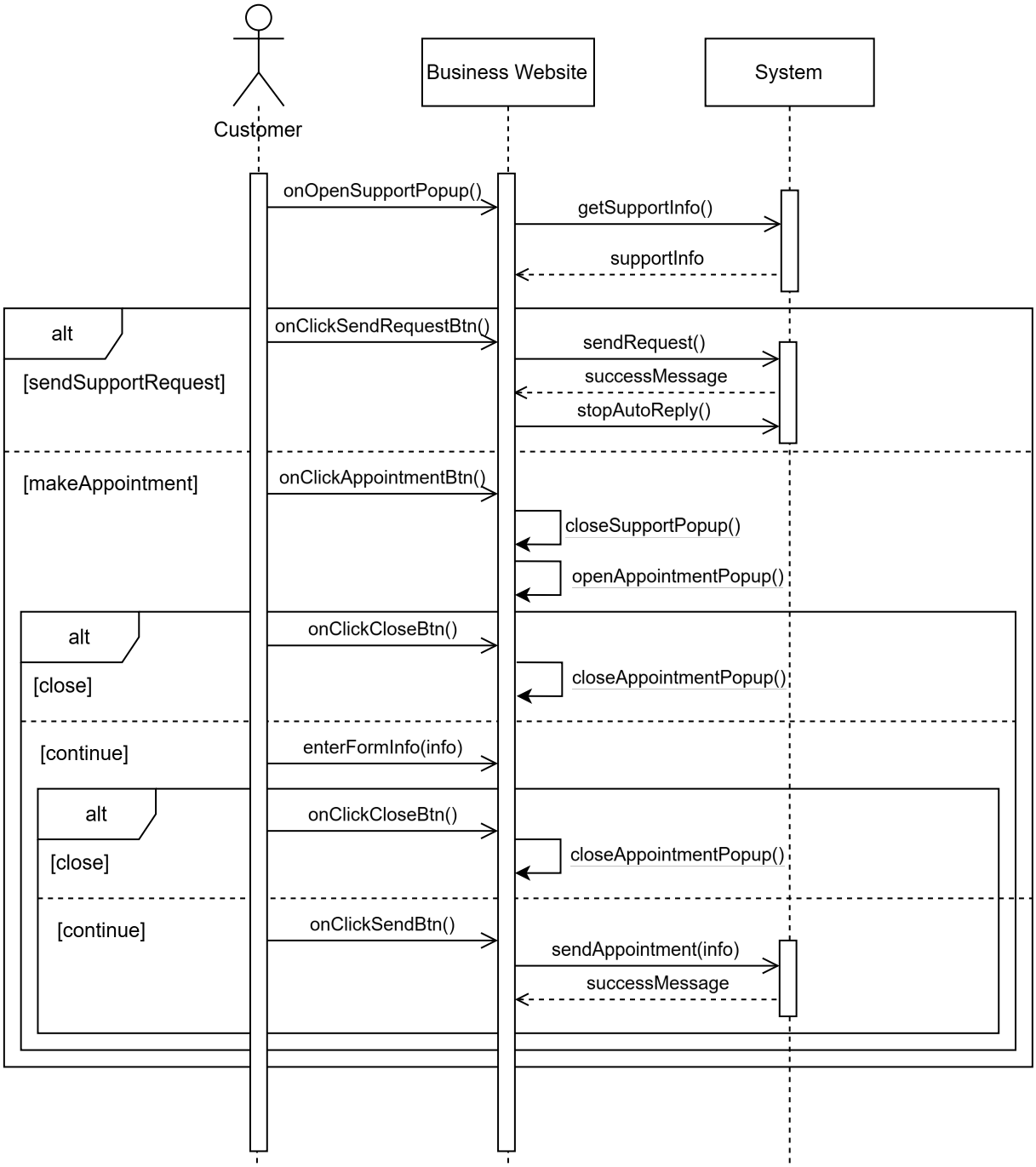
Mô tả:

- Khách hàng click vào nút Chatbot với onOpenChatbotPopup().
- Giao diện gọi hàm getChat(id) đến hệ thống với biến id truyền vào.
- Hệ thống trả lại chat tương ứng.
- Có một vòng lặp với điều kiện khách hàng tiếp tục quá trình nhập tin nhắn, trong vòng lặp có 3 lựa chọn là nhập/sửa tin nhắn, đính kèm tệp hoặc xóa tệp:
  - Nhập/sửa tin nhắn: khách hàng click vào khung tin nhắn với onClickMessageBox(), tiếp tục nhập nội dung tin nhắn với enter(messageInput) có biến messageInput truyền vào.
  - Đính kèm tệp: khách hàng click vào nút Đính kèm tệp với onClickAttachFileButton(), giao diện gọi hàm getFiles(). đến thiết bị để lấy danh sách các file có thể tải lên, thiết bị tả lại các files cho giao diện, khách hàng chọn một file onClickFile(id), giao diện gọi hàm getFile(id) đến thiết bị với biến id truyền vào, thiết bị trả lại file tương ứng.

– Xóa tệp: khách hàng click nút xóa cho tệp cần xóa với onClickDeleteFileButton(id), giao diện gọi hàm deleteFile(id) đến hệ thống với biến id truyền vào, hệ thống xóa tệp và trả lại tin nhắn deleteFile tương ứng.

- Có một tùy chọn với điều kiện tin nhắn tồn tại: khách hàng click Gửi với onClickSendButton(), giao diện gọi hàm sendMessage() đến hệ thống, hệ thống trả lại tin nhắn sendMessage tương ứng, hệ thống tự gọi hàm createAutoReplyMessage() để tạo tin nhắn tự động từ hệ thống chatbot, hệ thống trả lại giao diện autoReplyMessage.

3.7.5.3 Yêu cầu hỗ trợ



Hình 49: *Biểu đồ sequence cho Yêu cầu hỗ trợ*

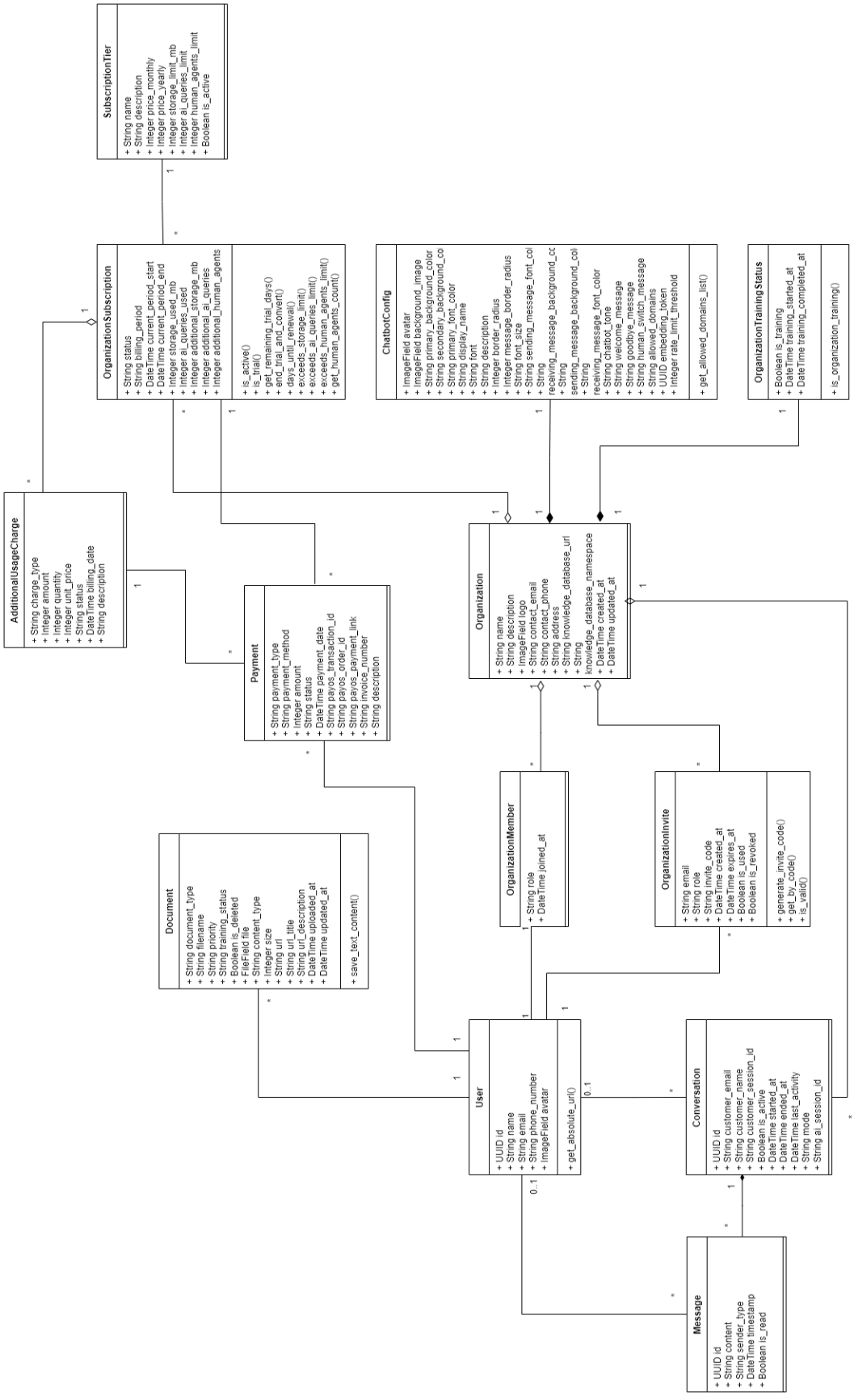
Mô tả:

- Khách hàng click nút Hỗ trợ với hàm onOpenSupportPoppup().
- Giao diện gọi hàm getSupportInro() đế hệ thống để lấy thông tin hỗ trợ.

- Hệ thống trả về giao diện supportInfo.
- Có 2 tùy chọn khả dụng là gửi yêu cầu hỗ trợ hoặc đặt lịch hẹn:
  - Gửi yêu cầu hỗ trợ: khách hàng click Gửi yêu cầu với onClickSendRequestBtn(), giao diện gọi hàm sendRequest() đến hệ thống để gửi yêu cầu lên hệ thống, hệ thống trả lại về tin nhắn thành công successMessage, giao diện gọi hàm stopAutoReply() để tắt tính năng trả lời tự động.
  - Đặt lịch hẹn: khách hàng click Đặt lịch hẹn với onClickAppointmentBtn(), giao diện tự gọi hàm closeSupportPoppup() để đóng popup Hỗ trợ, giao diện tự gọi hàm openAppointmentPopup() để mở popup Đặt lịch hẹn, tiếp tục có 2 tùy chọn khả dụng là đóng popup hoặc tiếp tục:
    * Đóng: khách hàng click Đóng với onClickCloseBtn(), giao diện tự gọi closeAppointmentPopup() để đóng popup Đặt lịch hẹn.
    * Tiếp tục: khách hàng nhập các thông tin cho cuộc hẹn với enterFornInfo(info), tiếp tục có 2 tùy chọn là đóng popup hoặc tiếp tục:
      · Đóng: khách hàng click Đóng với onClickCloseBtn(), giao diện tự gọi closeAppointmentPopup() để đóng popup Đặt lịch hẹn.
      · Tiếp tục: khách hàng click Gửi với onClickSendBtn(), giao diện gọi hàm sendAppointment(info) đến hệ thống với biến info, hệ thống nhận và trả lại tin nhắn thành công successMessage.

3.8 Biểu đồ Class

- **Quản lý Người dùng (User Management)**
  - **User**
    * *Mục đích*: Quản lý các thông tin định danh và cơ bản của người dùng hệ thống.
    * *Thuộc tính chính*: `id`, `name`, `email`, `phone_number`, `avatar`.
    * *Quan hệ (Relationships)*:
      · Liên kết đơn nhất tới một thực thể `OrganizationMember`.
      · Có thể đóng vai trò nhân viên hỗ trợ tham gia vào nhiều thực thể `Conversation`.
      · Là người khởi tạo của nhiều thực thể `Message`.
      · Là người khởi tạo nhiều thực thể `OrganizationInvite`.
      · Là người tải lên của nhiều thực thể `Document`.
      · Là người khởi tạo nhiều giao dịch `Payment`.
    * *Chức năng chính*: Lưu trữ và quản lý thông tin định danh của người dùng.
- **Quản lý Tổ chức (Organization Management)**
  - **Organization**
    * *Mục đích*: Quản lý thông tin, cấu hình và các tài nguyên liên quan đến một tổ chức sử dụng hệ thống.
    * *Thuộc tính chính*: `name`, `description`, `logo`, `contact_email`, `contact_phone`, `address`, `knowledge_database_url`, `knowledge_database_namespace`, `created_at`, `updated_at`.
    * *Quan hệ (Relationships)*:
      · Bao gồm nhiều thành viên thông qua thực thể `OrganizationMember`.
      · Quản lý nhiều lời mời tham gia thông qua thực thể `OrganizationInvite`.
      · Liên kết đơn nhất tới một cấu hình chatbot (`ChatbotConfig`).
      · Bao gồm nhiều tài liệu huấn luyện (`Document`).
      · Liên kết đơn nhất tới một trạng thái huấn luyện (`OrganizationTrainingStatus`).
      · Quản lý nhiều gói đăng ký dịch vụ thông qua thực thể `OrganizationSubscription`.
      · Bao gồm nhiều cuộc hội thoại (`Conversation`).
      · Quản lý nhiều giao dịch thanh toán (`Payment`).
    * *Chức năng chính*: Đóng vai trò trung tâm quản lý thông tin, cấu hình và tài nguyên liên quan đến tổ chức.
  - **OrganizationMember**
    * *Mục đích*: Quản lý vai trò và thông tin tham gia của một người dùng cụ thể trong tổ chức.
    * *Thuộc tính chính*: `role` [OWNER, ADMIN, AGENT], `joined_at`.



Hình 50: *Biểu đồ class các thực thể cho tầng dữ liệu*

    - *Quan hệ (Relationships)*:
        - Liên kết đơn nhất tới một thực thể `User`.
        - Thuộc về một thực thể `Organization` duy nhất.
    - *Chức năng chính*: Xác định vai trò của thành viên và ghi nhận thời điểm bắt đầu tham gia tổ chức.
- **OrganizationInvite**
    - *Mục đích*: Quản lý các lời mời dành cho người dùng mới tham gia vào tổ chức.
    - *Thuộc tính chính*: `email`, `role`, `invite_code`, `created_at`, `expires_at`, `is_used`, `is_revoked`.
    - *Quan hệ (Relationships)*:
        - Thuộc về một thực thể `Organization` duy nhất.
        - Được tạo bởi một `User` (người mời).
    - *Chức năng chính*: Cung cấp các chức năng tạo mã mời, truy xuất lời mời theo mã, và kiểm tra tính hợp lệ của lời mời.

- **Quản lý Gói cước & Thanh toán (Subscription & Billing)**
    - **SubscriptionTier**
        - *Mục đích*: Định nghĩa các cấp độ gói dịch vụ, bao gồm các giới hạn tài nguyên và chi phí tương ứng.
        - *Thuộc tính chính*: `name` [FREE, STARTER, PROFESSIONAL], `description`, `price_monthly`, `price_yearly`, `storage_limit_mb`, `ai_queries_limit`, `human_agents_limit`, `is_active`.
        - *Quan hệ (Relationships)*:
            - Là cơ sở định nghĩa cho nhiều thực thể `OrganizationSubscription`.
        - *Chức năng chính*: Cung cấp thông tin chi tiết về các gói dịch vụ khả dụng.
    - **OrganizationSubscription**
        - *Mục đích*: Quản lý gói dịch vụ hiện tại của một tổ chức, bao gồm trạng thái, chu kỳ thanh toán và mức độ sử dụng tài nguyên.
        - *Thuộc tính chính*: `status`, `billing_period`, `current_period_start`, `current_period_end`, `storage_used_mb`, `ai_queries_used`, `additional_storage_mb`, `additional_ai_queries`, `additional_human_agents`.
        - *Quan hệ (Relationships)*:
            - Thuộc về một thực thể `Organization` duy nhất.
            - Liên kết đơn nhất tới một thực thể `SubscriptionTier`.
            - Liên quan đến nhiều thực thể phí phát sinh (`AdditionalUsageCharge`).
            - Liên quan đến nhiều giao dịch thanh toán (`Payment`).
        - *Chức năng chính*: Quản lý trạng thái đăng ký, xử lý gia hạn, kiểm tra giới hạn sử dụng và theo dõi tài nguyên đã tiêu thụ.
    - **AdditionalUsageCharge**
        - *Mục đích*: Ghi nhận các khoản phí phát sinh từ việc sử dụng tài nguyên vượt mức hoặc mua thêm tài nguyên bổ sung.
        - *Thuộc tính chính*: `charge_type`, `amount`, `quantity`, `unit_price`, `status`, `billing_date`, `description`.
        - *Quan hệ (Relationships)*:
            - Liên kết tới một thực thể `OrganizationSubscription`.
            - Liên quan đến một hoặc nhiều giao dịch thanh toán (`Payment`).
            - Thuộc về một thực thể `Organization` (thông qua `OrganizationSubscription`).
        - *Chức năng chính*: Ghi nhận chi tiết các khoản phí phát sinh, phục vụ cho việc tạo hóa đơn.
    - **Payment**
        - *Mục đích*: Ghi nhận thông tin chi tiết về các giao dịch thanh toán được thực hiện.
        - *Thuộc tính chính*: `payment_type`, `payment_method`, `amount`, `status`, `payment_date`, `payos_transaction_id`, `payos_order_id`, `payos_payment_link`, `invoice_number`, `description`.
        - *Quan hệ (Relationships)*:
            - Thuộc về một thực thể `Organization` duy nhất.
            - Có thể được khởi tạo bởi một `User`.
            - Liên quan đến một thực thể `OrganizationSubscription`.

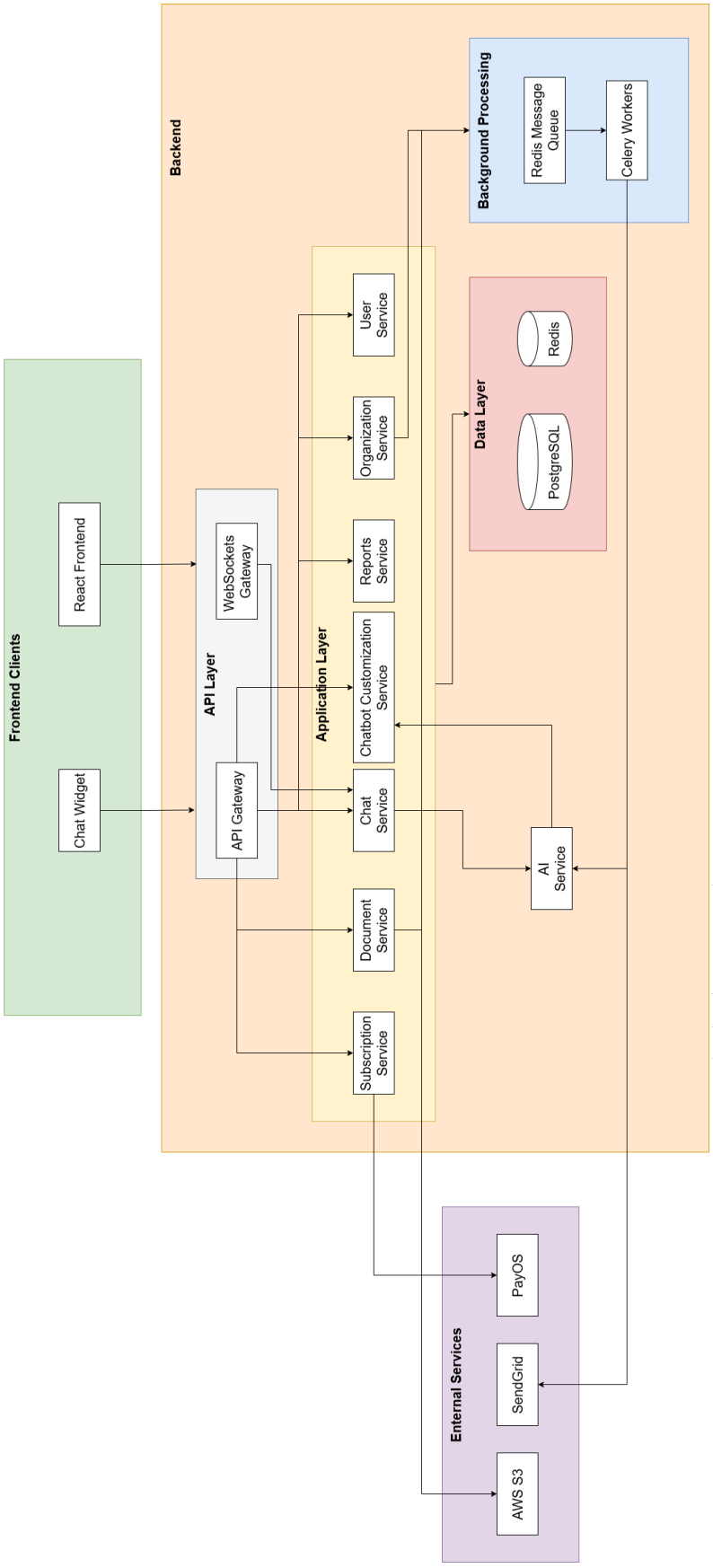
        - Liên quan đến một hoặc nhiều bản ghi `AdditionalUsageCharge`.
    - *Chức năng chính*: Lưu trữ lịch sử giao dịch và theo dõi trạng thái hoàn thành của các khoản thanh toán.

- **Cấu hình Chatbot & AI (Chatbot & AI Configuration)**
  - **ChatbotConfig**
    - *Mục đích*: Lưu trữ các thiết lập liên quan đến giao diện hiển thị và hành vi hoạt động của chatbot.
    - *Thuộc tính chính*: `avatar`, `background_image`, `colors`, `fonts`, `display_name`, `messages`, `allowed_domains`, `embedding_token`, `rate_limit_threshold`.
    - *Quan hệ (Relationships)*:
      - Thuộc về một thực thể `Organization` duy nhất.
    - *Chức năng chính*: Cho phép tùy chỉnh giao diện và hành vi chatbot, quản lý danh sách tên miền được phép nhúng, và hỗ trợ kiểm soát truy cập.
  - **Document**
    - *Mục đích*: Quản lý các tài liệu được sử dụng làm cơ sở tri thức để huấn luyện mô hình AI của chatbot.
    - *Thuộc tính chính*: `document_type` [FILE, TEXT, URL], `filename`, `priority`, `training_status`, `is_deleted`, `file`, `content_type`, `size`, `url`, `url_title`, `url_description`, `uploaded_at`, `updated_at`.
    - *Quan hệ (Relationships)*:
      - Thuộc về một thực thể `Organization` duy nhất.
      - Được tải lên bởi một `User` (người tải lên).
    - *Chức năng chính*: Lưu trữ tài liệu huấn luyện, quản lý trạng thái xử lý, và thực hiện các tác vụ liên quan đến nội dung.
  - **OrganizationTrainingStatus**
    - *Mục đích*: Theo dõi trạng thái quá trình huấn luyện mô hình AI dựa trên cơ sở tri thức của toàn bộ tổ chức.
    - *Thuộc tính chính*: `is_training`, `training_started_at`, `training_completed_at`.
    - *Quan hệ (Relationships)*:
      - Thuộc về một thực thể `Organization` duy nhất.
    - *Chức năng chính*: Cung cấp thông tin về trạng thái huấn luyện hiện tại và ghi nhận lịch sử các lần huấn luyện đã hoàn thành.

- **Quản lý Hội thoại (Conversation Management)**
  - **Conversation**
    - *Mục đích*: Đại diện và quản lý một phiên hội thoại hoàn chỉnh giữa khách hàng cuối và hệ thống chatbot hoặc nhân viên hỗ trợ.
    - *Thuộc tính chính*: `id`, `customer_email`, `customer_name`, `customer_session_id`, `is_active`, `started_at`, `ended_at`, `last_activity`, `mode`, `ai_session_id`.
    - *Quan hệ (Relationships)*:
      - Bao gồm một chuỗi các thực thể `Message` thể hiện nội dung trao đổi.
      - Thuộc về một thực thể `Organization` duy nhất.
      - Có thể được tiếp nhận và xử lý bởi một `User` đóng vai trò nhân viên hỗ trợ (agent).
    - *Chức năng chính*: Lưu trữ thông tin định danh khách hàng, toàn bộ lịch sử tin nhắn, và quản lý trạng thái của cuộc hội thoại.
  - **Message**
    - *Mục đích*: Đại diện cho một tin nhắn trao đổi trong một cuộc hội thoại.
    - *Thuộc tính chính*: `id`, `content`, `sender_type` [CUSTOMER, AGENT, AI, SYSTEM], `timestamp`, `is_read`.
    - *Quan hệ (Relationships)*:
      - Thuộc về một thực thể `Conversation` duy nhất.
      - Có thể được gửi bởi một `User` (khách hàng hoặc nhân viên hỗ trợ), hoặc được tạo bởi AI hoặc hệ thống.
    - *Chức năng chính*: Lưu trữ nội dung tin nhắn, thông tin người gửi (khách hàng, nhân viên hỗ trợ, AI, hệ thống) và thời gian.

4 Thiết kế hệ thống

4.1 Kiến trúc hệ thống



Hình 51: *Biểu đồ kiến trúc hệ thống*

4.1.1 Tổng quan kiến trúc

Hệ thống được thiết kế dựa trên kiến trúc nguyên khối có cấu trúc lớp (Layered Monolithic Architecture) chủ đạo, kết hợp với các thành phần xử lý bất đồng bộ và tương tác với dịch vụ AI nội bộ chuyên biệt được triển khai độc lập. Trung tâm của hệ thống là một ứng dụng Django tập trung, đảm nhiệm vai trò xử lý các yêu cầu HTTP, triển khai logic nghiệp vụ cốt lõi và quản lý dữ liệu. Các thành phần chức năng chính được tổ chức dưới dạng các module bên trong ứng dụng này.

Lựa chọn kiến trúc này mang lại một số lợi ích chính. Thứ nhất, việc áp dụng kiến trúc nguyên khối cùng với hệ sinh thái toàn diện của Django cho phép tối ưu hóa hiệu quả phát triển và đẩy nhanh tốc độ triển khai tính năng. Thứ hai, hệ thống tích hợp Django Channels và công nghệ WebSocket để đáp ứng yêu cầu giao tiếp thời gian thực, đặc biệt là cho chức năng trò chuyện, mà không đòi hỏi sự phức tạp của một kiến trúc phân tán hoàn toàn. Cuối cùng, mặc dù duy trì một lõi nguyên khối, kiến trúc vẫn đảm bảo khả năng mở rộng thông qua việc sử dụng Celery và Redis để xử lý các tác vụ nền và tác vụ định kỳ một cách bất đồng bộ, cho phép mở rộng quy mô của các tiến trình xử lý một cách độc lập khi tải tăng cao.

Giao tiếp với các ứng dụng phía người dùng được thực hiện chủ yếu thông qua API Gateway dựa trên Django REST Framework cho các yêu cầu REST API và WebSocket Gateway dựa trên Django Channels cho giao tiếp thời gian thực. Hệ thống được triển khai trên nền tảng Heroku, sử dụng Gunicorn/Uvicorn làm máy chủ WSGI/ASGI và Whitenoise để phục vụ các tệp tĩnh.

4.1.2 Chi tiết các thành phần chính

4.1.2.1 Lớp ứng dụng (Application Layer)

Trung tâm của hệ thống là ứng dụng Django đóng vai trò điều phối chính. Bên trong ứng dụng này, các chức năng nghiệp vụ được phân tách thành các module dịch vụ chuyên biệt, bao gồm:

- **Module quản lý người dùng (User Service):** Chịu trách nhiệm xác thực, đăng ký, quản lý thông tin hồ sơ người dùng (bao gồm cả quản trị viên và nhân viên hỗ trợ), và thực thi cơ chế phân quyền truy cập trong toàn hệ thống.
- **Module quản lý tổ chức (Organization Service):** Quản lý cấu trúc nhóm/tổ chức, liên kết người dùng với tổ chức, và các quyền hạn, cài đặt riêng biệt áp dụng cho từng tổ chức sử dụng dịch vụ.
- **Module hội thoại (Chat Service):** Quản lý toàn bộ luồng giao tiếp thời gian thực trong hệ thống, bao gồm các cuộc hội thoại giữa người dùng cuối với **chatbot AI**, cũng như giữa người dùng cuối và **nhân viên hỗ trợ (human agents)**. Module này điều phối việc gửi/nhận tin nhắn qua WebSocket, quản lý trạng thái phiên chat, và có thể bao gồm logic phân phối chat cho nhân viên hoặc chuyển giao hội thoại giữa AI và người.
- **Module tùy chỉnh chatbot (Chatbot Service):** Chịu trách nhiệm cấu hình và vận hành các chatbot. Module này cho phép quản trị viên **tùy chỉnh** các khía cạnh của chatbot như giao diện hiển thị, hành vi, tính cách, và các kịch bản hội thoại cơ bản. Đồng thời, nó điều phối logic tương tác của chatbot trong cuộc hội thoại, phối hợp chặt chẽ với **dịch vụ AI** để thực hiện các tác vụ xử lý ngôn ngữ tự nhiên và sinh phản hồi thông minh.
- **Module xử lý tài liệu (Document Service):** Quản lý vòng đời của tài liệu tri thức được sử dụng bởi hệ thống và chatbot. Chức năng bao gồm tải lên, lưu trữ, phân loại, lập chỉ mục và truy xuất tài liệu. Quan trọng hơn, module này cung cấp logic để **kích hoạt quá trình huấn luyện** các mô hình AI dựa trên nội dung tài liệu được chọn lọc, đồng thời **theo dõi trạng thái** của các tiến trình huấn luyện này.
- **Module quản lý gói (Subscription Service):** Xử lý các nghiệp vụ liên quan đến thanh toán (tích hợp với PayOS) và quản lý các gói dịch vụ, bao gồm đăng ký, gia hạn, nâng cấp/hạ cấp gói, và theo dõi giới hạn sử dụng tài nguyên theo gói.
- **Module báo cáo (Report Service):** Thu thập dữ liệu từ các module khác để tổng hợp, phân tích và **tạo ra các báo cáo** chi tiết về hiệu suất hoạt động, cung cấp thông tin giá trị cho việc đánh giá và cải tiến hệ thống.

Cách tổ chức theo module này, tận dụng cấu trúc ứng dụng của Django, giúp duy trì sự tách biệt rõ ràng về chức năng trong một mã nguồn thống nhất, thuận lợi cho việc phát triển, kiểm thử và bảo trì.

4.1.2.2 Lớp giao tiếp (Interface Layer)

- **API Gateway:** Được xây dựng trên nền tảng Django REST Framework, cung cấp giao diện REST API chuẩn hóa cho các frontend tương tác với backend.
- **WebSocket Gateway:** Sử dụng Django Channels, thiết lập kênh giao tiếp hai chiều, thời gian thực, chủ yếu phục vụ cho tính năng trò chuyện.

4.1.2.3 Xử lý tác vụ nền (Background Processing)

- **Celery Workers:** Các tiến trình độc lập thực thi các tác vụ tốn thời gian hoặc tài nguyên một cách bất đồng bộ, giúp giải phóng luồng xử lý chính của ứng dụng web và cải thiện độ phản hồi.
- **Redis Message Queue:** Đóng vai trò là hàng đợi tin nhắn, trung gian giao tiếp giữa ứng dụng Django và các Celery workers.

4.1.2.4 Lớp dữ liệu (Data Layer)

- **PostgreSQL:** Hệ quản trị cơ sở dữ liệu quan hệ chính, lưu trữ hầu hết dữ liệu có cấu trúc của hệ thống như thông tin người dùng, tổ chức, lịch sử trò chuyện, cấu hình chatbot, thông tin thuê bao, và dữ liệu tài liệu. item **Redis:** Đóng vai trò đa năng quan trọng trong hệ thống, phục vụ ba mục đích chính:
  - Làm **bộ nhớ đệm** để tối ưu hóa tốc độ truy xuất các dữ liệu thường dùng.
  - Làm **backend cho lớp channels** của Django Channels, hỗ trợ việc quản lý và truyền tải tin nhắn hiệu quả cho giao tiếp thời gian thực qua WebSocket.
  - Làm **hàng đợi tin nhắn** cho hệ thống xử lý tác vụ nền Celery, điều phối việc thực thi các tác vụ bất đồng bộ.

4.1.2.5 Dịch vụ AI

Hệ thống bao gồm một dịch vụ AI chuyên biệt, triển khai độc lập và tách biệt với backend. Dịch vụ này chuyên trách việc cung cấp các khả năng xử lý ngôn ngữ tự nhiên và trí tuệ đối thoại tiên tiến, tận dụng các mô hình ngôn ngữ lớn. Module chat bên trong ứng dụng lõi sẽ giao tiếp với Dịch vụ AI này thông qua giao diện REST API.

Việc lựa chọn triển khai Dịch vụ AI một cách độc lập mang lại một số lợi ích về mặt kiến trúc và vận hành:

- **Tối ưu tài nguyên:** Cho phép dịch vụ AI chạy trên hạ tầng phần cứng chuyên biệt và được cấp phát tài nguyên tối ưu cho các tác vụ tính toán nặng của AI.
- **Khả năng mở rộng độc lập:** Quy mô của Dịch vụ AI có thể được điều chỉnh một cách linh hoạt dựa trên tải xử lý AI, độc lập với việc mở rộng quy mô của ứng dụng web lõi.
- **Linh hoạt về công nghệ và cập nhật:** Cho phép sử dụng các thư viện, framework, hoặc thậm chí ngôn ngữ lập trình khác (phù hợp hơn cho AI/ML) và có thể cập nhật các mô hình AI thường xuyên mà không cần phải triển khai lại toàn bộ ứng dụng lõi.
- **Tách biệt lỗi:** Sự cố tiềm ẩn trong dịch vụ AI ít có khả năng ảnh hưởng trực tiếp đến sự ổn định của ứng dụng web chính và ngược lại.

4.1.2.6 Tích hợp dịch vụ bên ngoài

Ngoài các thành phần nội bộ, hệ thống còn tích hợp với các dịch vụ của bên thứ ba chuyên biệt để mở rộng chức năng và tận dụng các giải pháp đã được tối ưu hóa trên thị trường:

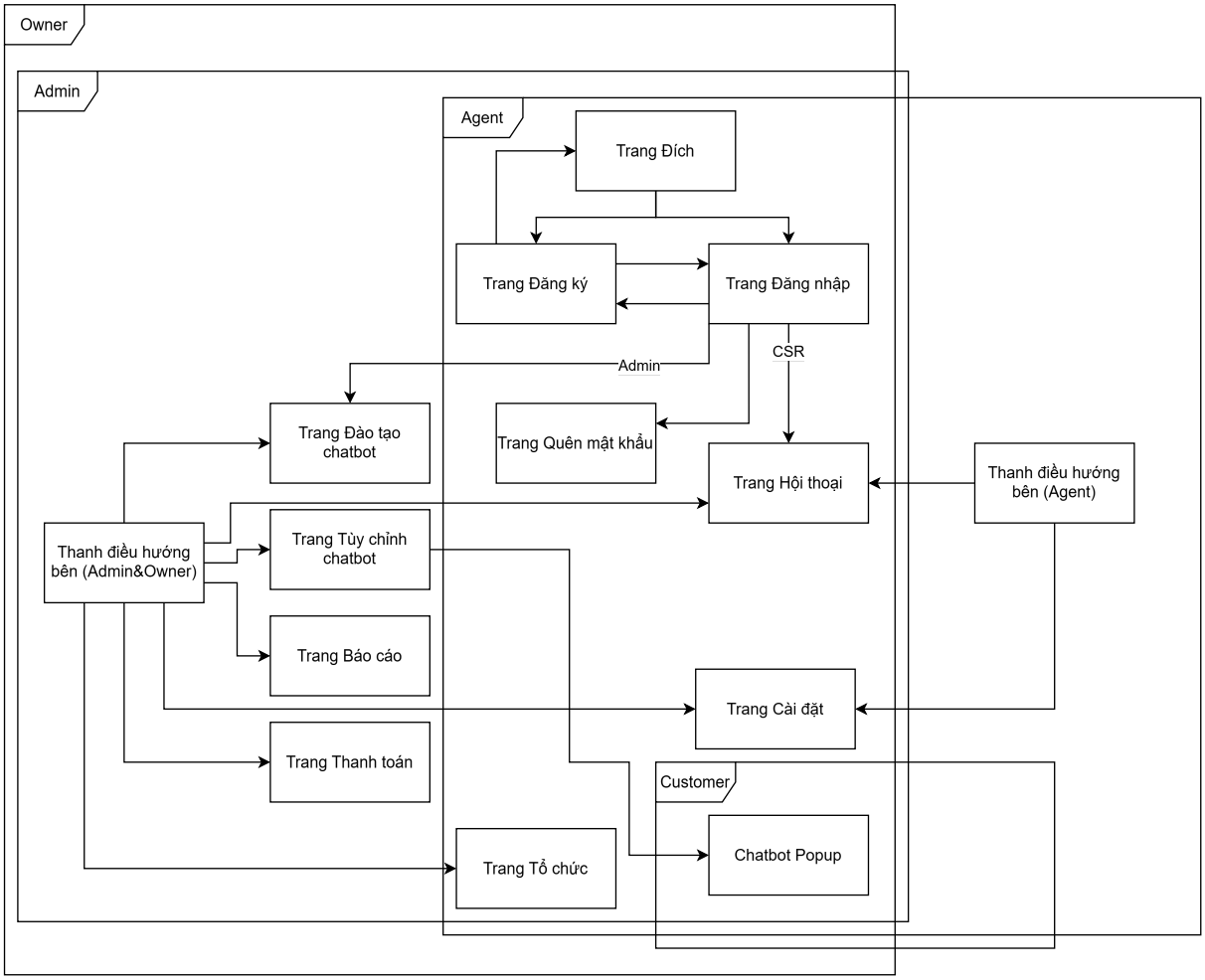
- **AWS S3 (Simple Storage Service):** Dịch vụ lưu trữ đối tượng trên nền tảng đám mây của Amazon Web Services, được sử dụng để lưu trữ bền vững và có khả năng mở rộng cao cho các tệp tài liệu, hình ảnh, video và các dữ liệu media khác do người dùng tải lên hoặc hệ thống tạo ra.
- **PayOS:** Một cổng thanh toán của bên thứ ba, cung cấp giao diện và xử lý các giao dịch thanh toán trực tuyến một cách an toàn, hỗ trợ nhiều phương thức thanh toán khác nhau, phục vụ cho việc quản lý đăng ký và thuê bao dịch vụ.
- **SendGrid:** Dịch vụ chuyên dụng cho việc gửi email, đảm bảo khả năng gửi email giao dịch (ví dụ: xác nhận đăng ký, đặt lại mật khẩu) và email thông báo đến người dùng với tỉ lệ gửi thành công cao và tuân thủ các tiêu chuẩn về email.

Tóm lại, kiến trúc nguyên khối có cấu trúc lớp này cân bằng giữa hiệu quả phát triển, khả năng bảo trì và các yêu cầu về hiệu năng, thời gian thực. Việc tích hợp các thành phần xử lý bất đồng bộ và các dịch vụ bên ngoài chuyên biệt giúp hệ thống giải quyết các bài toán phức tạp về xử lý nền, lưu trữ và trí tuệ nhân tạo mà vẫn duy trì được sự đơn giản tương đối trong cấu trúc lõi.

4.2 Giao diện

4.2.1 Lược đồ Screen-flow

Lược đồ này cung cấp cái nhìn tổng quan về trải nghiệm người dùng: cách các trang giao diện kết nối với nhau, thứ tự hiển thị, và ai là người kiểm soát trang giao diện đó (phân chia vai trò). Cụ thể, quản trị viên và các nhân viên chăm sóc khách hàng quản lý hầu hết các trang, và người dùng khi tiếp cận với trang web doanh nghiệp, chỉ cần tương tác với chatbot popup là đủ.



Hình 52: *Lược đồ Screen-flow*

4.2.2 Logo ứng dụng



Hình 53: *Logo của ứng dụng CoolChat*

a) **Tên ứng dụng**

- Ứng dụng được triển khai có tên là CoolChat.

- CoolChat hướng đến việc trở thành thương hiệu số 1 Việt Nam trong công cuộc đồng hành cùng các doanh nghiệp trên hành trình tư vấn và chăm sóc khách hàng của họ.

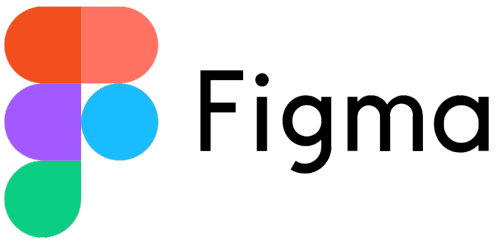
b) **Biểu tượng**

Đây là hình ảnh logo đầy đủ của hệ thống cung cấp chatbot CoolChat.

- Logo là hình ảnh một biểu tượng chatbot được cách điệu một cách hiện đại và trẻ trung. Màu xanh dương được sử dụng làm màu chủ đạo, mang đến cảm giác tươi mát, tin cậy và chuyên nghiệp.
- Slogan "Make it cool": CoolChat cam kết mang đến những trải nghiệm giao tiếp thú vị và mới mẻ cho người dùng trong lĩnh vực giao tiếp và tư vấn trực tuyến.

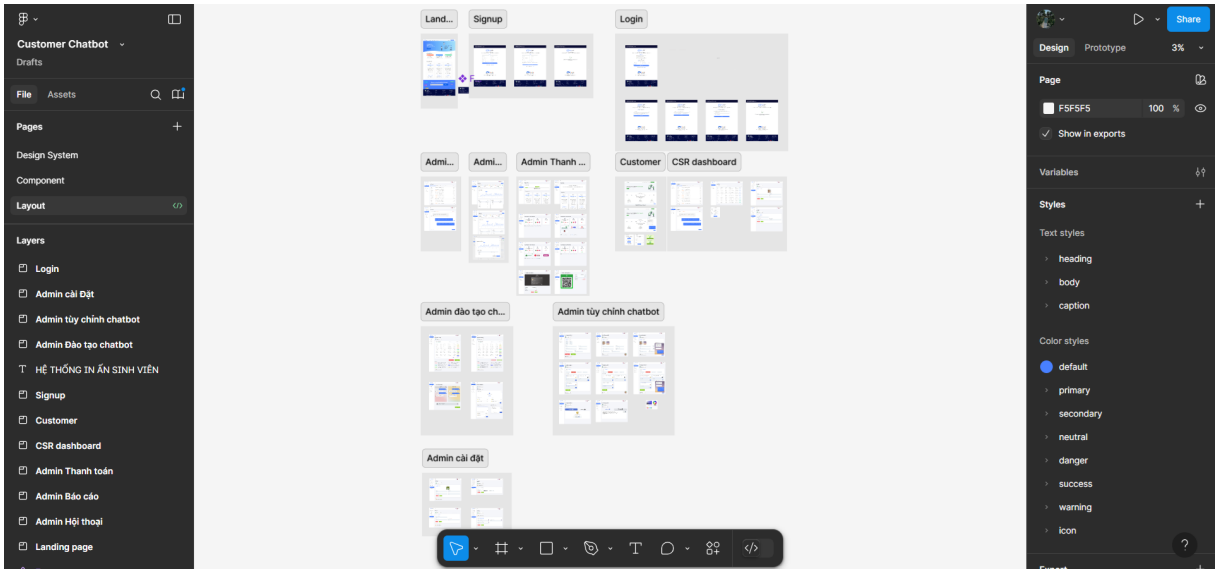
4.2.3 Thiết kế giao diện với Figma

Figma là một công cụ UI/UX trực tuyến được phát triển bởi Figma Inc. Công cụ này hỗ trợ các nhóm làm việc cộng tác trong việc tạo ra các sản phẩm thiết kế kỹ thuật số, và được sử dụng rất phổ biến trong thiết kế giao diện cho ứng dụng web hay mobile. Công cụ này nổi bật với khả năng làm việc trực tuyến, cho phép nhiều người dùng cùng lúc chỉnh sửa và tương tác với các bản thiết kế trong thời gian thực.



Hình 54: *Công cụ Figma*

Nhóm đã hoàn thiện bản thiết kế cho tất cả màn hình trên Figma để có thể thuận tiện cho quá trình hiện thực hệ thống.



Hình 55: *Các màn hình được thiết kế trên Figma*

4.3 Cơ sở dữ liệu

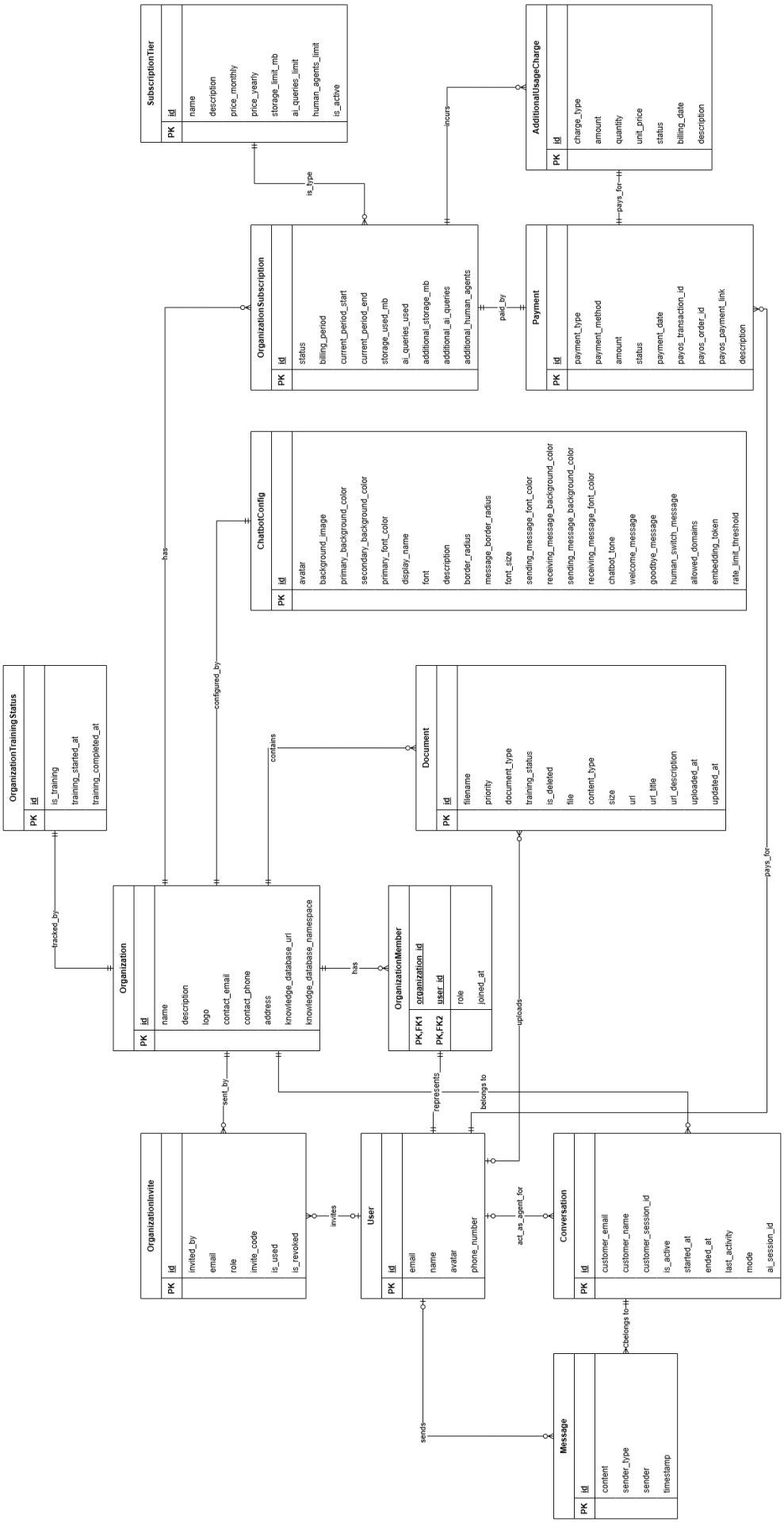
4.3.1 Quản lý Người dùng

4.3.1.1 Bảng `users_user`:

Bảng cơ sở cho tất cả các người dùng trong hệ thống, kế thừa từ AbstractUser của Django.

Thuộc tính:

- **id** (PK): Định danh duy nhất của người dùng (mặc định là INTEGER auto-increment).



Hình 56: *Biểu đồ ERD hệ thống*

- **password** (VARCHAR): Mật khẩu đã được mã hóa (kế thừa).
- **last_login** (TIMESTAMP WITH TIME ZONE): Thời điểm đăng nhập gần nhất (kế thừa).
- **name** (VARCHAR(255)): Tên đầy đủ của người dùng.
- **email** (VARCHAR): Địa chỉ email đăng nhập, không được trùng lặp.
- **phone_number** (VARCHAR(17)): Số điện thoại người dùng.
- **avatar** (VARCHAR): Đường dẫn đến tệp ảnh đại diện người dùng (có thể null).
- **is_staff** (BOOLEAN): Trạng thái có quyền truy cập trang admin (kế thừa).
- **is_active** (BOOLEAN): Trạng thái tài khoản có đang hoạt động (kế thừa).
- **date_joined** (TIMESTAMP WITH TIME ZONE): Thời điểm người dùng tham gia hệ thống (kế thừa).

Quan hệ:

- Kế thừa từ `django.contrib.auth.models.AbstractUser`.
- Quan hệ một-một với bảng `organization_organizationmember`.
- Quan hệ một-nhiều với bảng `chat_conversation` (vai trò agent).
- Quan hệ một-nhiều với bảng `chat_message` (vai trò sender là AGENT).
- Quan hệ một-nhiều với bảng `document_document` (vai trò uploader).
- Quan hệ một-nhiều với bảng `organization_organizationinvite` (vai trò invited_by).
- Quan hệ một-nhiều với bảng `subscription_payment` (vai trò initiating_user).

4.3.2 Quản lý Tổ chức

4.3.2.1 Bảng `organization_organization`:

Lưu trữ thông tin cơ bản về các tổ chức sử dụng hệ thống.

Thuộc tính:

- **id** (PK): Định danh duy nhất của tổ chức (mặc định là INTEGER auto-increment).
- **name** (VARCHAR(255)): Tên tổ chức.
- **description** (TEXT): Mô tả về tổ chức (có thể trống).
- **logo** (VARCHAR): Đường dẫn đến tệp logo của tổ chức (có thể null).
- **contact_email** (VARCHAR): Email liên hệ chính.
- **contact_phone** (VARCHAR(20)): Số điện thoại liên hệ (có thể trống).
- **address** (TEXT): Địa chỉ tổ chức (có thể trống).
- **knowledge_database_url** (VARCHAR(2000)): URL đến cơ sở tri thức (có thể null).
- **knowledge_database_namespace** (VARCHAR(255)): Namespace trong cơ sở tri thức (có thể null).
- **created_at** (TIMESTAMP WITH TIME ZONE): Thời điểm tạo tổ chức.
- **updated_at** (TIMESTAMP WITH TIME ZONE): Thời điểm cập nhật tổ chức lần cuối.

Quan hệ:

- Quan hệ một-nhiều với bảng `organization_organizationmember`.
- Quan hệ một-nhiều với bảng `organization_organizationinvite`.
- Quan hệ một-nhiều với bảng `chat_conversation`.
- Quan hệ một-nhiều với bảng `document_document`.

- Quan hệ một-một với bảng `chatbot_chatbotconfig`.
- Quan hệ một-một với bảng `document_organizationtrainingstatus`.
- Quan hệ một-nhiều với bảng `subscription_organizationsubscription`.
- Quan hệ một-nhiều với bảng `subscription_payment`.

4.3.2.2 Bảng `organization_organizationmember`:

Quản lý thành viên và vai trò của họ trong tổ chức.

Thuộc tính:

- **id** (PK): Định danh duy nhất của bản ghi thành viên.
- **organization_id** (FK - `organization.Organization`): Tham chiếu đến tổ chức.
- **user_id** (OneToOne - `users.User`): Tham chiếu duy nhất đến người dùng là thành viên.
- **role** (VARCHAR(10)): Vai trò của thành viên (OWNER, ADMIN, AGENT).
- **joined_at** (TIMESTAMP WITH TIME ZONE): Thời điểm tham gia tổ chức.

Quan hệ:

- Quan hệ nhiều-một với bảng `organization_organization`.
- Quan hệ một-một với bảng `users_user`.
- Ràng buộc duy nhất kết hợp (`organization_id`, `user_id`).

4.3.2.3 Bảng `organization_organizationinvite`:

Quản lý các lời mời tham gia tổ chức.

Thuộc tính:

- **id** (PK): Định danh duy nhất của lời mời.
- **organization_id** (FK - `organization.Organization`): Tham chiếu đến tổ chức gửi lời mời.
- **email** (VARCHAR): Email của người được mời.
- **role** (VARCHAR(10)): Vai trò được mời (ADMIN, AGENT).
- **invited_by_id** (FK - `users.User`): Tham chiếu đến người gửi lời mời (có thể null).
- **invite_code** (VARCHAR(12)): Mã mời duy nhất.
- **created_at** (TIMESTAMP WITH TIME ZONE): Thời điểm tạo lời mời.
- **expires_at** (TIMESTAMP WITH TIME ZONE): Thời điểm hết hạn lời mời (mặc định: 7 ngày sau khi tạo).
- **is_used** (BOOLEAN): Trạng thái lời mời đã được sử dụng hay chưa (mặc định: False).
- **is_revoked** (BOOLEAN): Trạng thái lời mời đã bị thu hồi hay chưa (mặc định: False).

Quan hệ:

- Quan hệ nhiều-một với bảng `organization_organization`.
- Quan hệ nhiều-một với bảng `users_user` (cho invited_by).

4.3.3 Quản lý Trò chuyện

4.3.3.1 Bảng `chat_conversation`:

Quản lý thông tin về các cuộc trò chuyện giữa khách hàng và hệ thống (AI hoặc nhân viên hỗ trợ).

Thuộc tính:

- **id** (UUID): Định danh duy nhất của cuộc trò chuyện.
- **organization_id** (FK - `organization.Organization`): Tham chiếu đến tổ chức sở hữu cuộc trò chuyện.
- **customer_email** (VARCHAR): Địa chỉ email của khách hàng (có thể trống).
- **customer_name** (VARCHAR(255)): Tên khách hàng (có thể trống).
- **customer_session_id** (VARCHAR(255)): Định danh phiên làm việc của khách hàng.
- **agent_id** (FK - `users.User`): Tham chiếu đến nhân viên hỗ trợ đang xử lý (nếu có).
- **is_active** (BOOLEAN): Trạng thái cuộc trò chuyện còn hoạt động hay không (mặc định: True).
- **started_at** (TIMESTAMP WITH TIME ZONE): Thời điểm bắt đầu cuộc trò chuyện.
- **ended_at** (TIMESTAMP WITH TIME ZONE): Thời điểm kết thúc cuộc trò chuyện (có thể null).
- **last_activity** (TIMESTAMP WITH TIME ZONE): Thời điểm hoạt động cuối cùng trong cuộc trò chuyện.
- **mode** (VARCHAR(10)): Chế độ xử lý cuộc trò chuyện (AI, HUMAN, AI_ONLY, mặc định: AI).
- **ai_session_id** (VARCHAR(255)): Định danh phiên làm việc với AI (có thể trống).

Quan hệ:

- Quan hệ nhiều-một với bảng `organization_organization`.
- Quan hệ nhiều-một với bảng `users_user` (cho agent).
- Quan hệ một-nhiều với bảng `chat_message`.

4.3.3.2 Bảng `chat_message`:

Lưu trữ nội dung các tin nhắn trong mỗi cuộc trò chuyện.

Thuộc tính:

- **id** (UUID): Định danh duy nhất của tin nhắn.
- **conversation_id** (FK - `chat.Conversation`): Tham chiếu đến cuộc trò chuyện chứa tin nhắn.
- **content** (TEXT): Nội dung tin nhắn.
- **sender_type** (VARCHAR(10)): Loại người gửi (CUSTOMER, AGENT, AI, SYSTEM).
- **sender_id** (FK - `users.User`): Tham chiếu đến người gửi nếu là AGENT (có thể null).
- **timestamp** (TIMESTAMP WITH TIME ZONE): Thời điểm tin nhắn được gửi.

Quan hệ:

- Quan hệ nhiều-một với bảng `chat_conversation`.
- Quan hệ nhiều-một với bảng `users_user` (cho sender nếu là AGENT).

4.3.4 Quản lý Chatbot

4.3.4.1 Bảng `chatbot_chatbotconfig`:

Lưu trữ cấu hình chi tiết cho chatbot của mỗi tổ chức.

Thuộc tính:

- **organization_id** (OneToOne - `organization.Organization`): Tham chiếu duy nhất đến tổ chức sở hữu cấu hình.
- **avatar** (VARCHAR): Đường dẫn đến tệp ảnh đại diện chatbot (có thể null, max 2MB, jpg/jpeg/png).
- **background_image** (VARCHAR): Đường dẫn đến tệp ảnh nền chatbot (có thể null, max 2MB, jpg/jpeg/png).
- **primary_background_color** (VARCHAR(7)): Mã màu hex cho nền chính (mặc định: #FFFFFF).
- **secondary_background_color** (VARCHAR(7)): Mã màu hex cho nền phụ (mặc định: #F0F0F0).
- **primary_font_color** (VARCHAR(7)): Mã màu hex cho màu chữ chính (mặc định: #000000).
- **display_name** (VARCHAR(50)): Tên hiển thị của chatbot (tối thiểu 2 ký tự).
- **font** (VARCHAR(20)): Font chữ sử dụng (mặc định: ARIAL, lựa chọn từ danh sách định sẵn).
- **description** (VARCHAR(200)): Mô tả ngắn về chatbot (có thể trống).
- **border_radius** (INTEGER): Bán kính bo góc của khung chat (mặc định: 8).
- **message_border_radius** (INTEGER): Bán kính bo góc của bong bóng tin nhắn (mặc định: 12).
- **font_size** (VARCHAR(2)): Kích thước font chữ (mặc định: 16, lựa chọn từ danh sách định sẵn).
- **sending_message_font_color** (VARCHAR(7)): Màu chữ tin nhắn gửi đi (mặc định: #000000).
- **receiving_message_background_color** (VARCHAR(7)): Màu nền tin nhắn nhận được (mặc định: #FFFFFF).
- **sending_message_background_color** (VARCHAR(7)): Màu nền tin nhắn gửi đi (mặc định: #E3E3E3).
- **receiving_message_font_color** (VARCHAR(7)): Màu chữ tin nhắn nhận được (mặc định: #000000).
- **chatbot_tone** (VARCHAR(20)): Giọng điệu của chatbot (FRIENDLY, PROFESSIONAL, HUMOROUS, CONCISE, mặc định: PROFESSIONAL).
- **welcome_message** (VARCHAR(500)): Tin nhắn chào mừng (mặc định: "Xin chào! Tôi có thể giúp gì cho bạn?", 10-500 ký tự).
- **goodbye_message** (VARCHAR(500)): Tin nhắn tạm biệt (mặc định: "Cảm ơn bạn đã trò chuyện...", 10-500 ký tự).
- **human_switch_message** (VARCHAR(500)): Tin nhắn khi chuyển sang nhân viên hỗ trợ (mặc định: "Tôi sẽ kết nối bạn...", 10-500 ký tự).
- **allowed_domains** (TEXT): Danh sách các tên miền được phép nhúng widget (phân tách bằng dấu phẩy, trống nghĩa là cho phép tất cả).
- **embedding_token** (UUID): Token duy nhất để nhúng (không thể chỉnh sửa).
- **rate_limit_threshold** (INTEGER): Số lượng yêu cầu AI tối đa mỗi giờ (mặc định: 20).
- **created_at** (TIMESTAMP WITH TIME ZONE): Thời điểm tạo cấu hình.
- **updated_at** (TIMESTAMP WITH TIME ZONE): Thời điểm cập nhật cấu hình lần cuối.

Quan hệ:

- Quan hệ một-một với bảng `organization_organization`.

4.3.5 Quản lý Tài liệu Huấn luyện

4.3.5.1 Bảng `document_document`:

Quản lý các tài liệu dùng để huấn luyện AI cho tổ chức.

Thuộc tính:

- **id** (PK): Định danh duy nhất của tài liệu (mặc định là INTEGER auto-increment).
- **document_type** (VARCHAR(4)): Loại tài liệu (FILE, TEXT, URL, mặc định: FILE).
- **filename** (VARCHAR(255)): Tên tệp hoặc định danh tài liệu.
- **priority** (VARCHAR(6)): Mức độ ưu tiên huấn luyện (HIGH, MEDIUM, LOW, NONE, mặc định: NONE).
- **training_status** (VARCHAR(10)): Trạng thái huấn luyện (UNTRAINED, PENDING, TRAINED, mặc định: UNTRAINED).
- **is_deleted** (BOOLEAN): Trạng thái tài liệu đã bị xóa mềm hay chưa (mặc định: False).
- **file** (VARCHAR): Đường dẫn đến tệp tài liệu (nếu document_type=FILE hoặc TEXT, max 20MB, pdf/doc/-docx/txt).
- **content_type** (VARCHAR(100)): Kiểu nội dung của tệp (có thể null).
- **size** (INTEGER): Kích thước tệp tính bằng byte (có thể null).
- **url** (VARCHAR(2000)): Địa chỉ URL của tài liệu (nếu document_type=URL, có thể null).
- **url_title** (VARCHAR(255)): Tiêu đề trang web từ URL (có thể null).
- **url_description** (TEXT): Mô tả trang web từ URL (có thể null).
- **organization_id** (FK - `organization.Organization`): Tham chiếu đến tổ chức sở hữu tài liệu.
- **uploader_id** (FK - `users.User`): Tham chiếu đến người tải lên tài liệu (có thể null nếu người dùng bị xóa).
- **uploaded_at** (TIMESTAMP WITH TIME ZONE): Thời điểm tải lên tài liệu.
- **updated_at** (TIMESTAMP WITH TIME ZONE): Thời điểm cập nhật tài liệu lần cuối.

Quan hệ:

- Quan hệ nhiều-một với bảng `organization_organization`.
- Quan hệ nhiều-một với bảng `users_user`.

4.3.5.2 Bảng `document_organizationtrainingstatus`:

Theo dõi trạng thái huấn luyện chung của tổ chức.

Thuộc tính:

- **organization_id** (OneToOne - `organization.Organization`): Tham chiếu duy nhất đến tổ chức.
- **is_training** (BOOLEAN): Trạng thái tổ chức có đang trong quá trình huấn luyện hay không (mặc định: False).
- **training_started_at** (TIMESTAMP WITH TIME ZONE): Thời điểm bắt đầu quá trình huấn luyện gần nhất (có thể null).
- **training_completed_at** (TIMESTAMP WITH TIME ZONE): Thời điểm hoàn thành quá trình huấn luyện gần nhất (có thể null).

Quan hệ:

- Quan hệ một-một với bảng `organization_organization`.

4.3.6 Quản lý Gói đăng ký và Thanh toán

4.3.6.1 Bảng `subscription_subscriptiontier`:

Định nghĩa các gói đăng ký (tiers) và giới hạn của chúng.

Thuộc tính:

- **id** (PK): Định danh duy nhất của gói.
- **name** (VARCHAR(20)): Tên gói (FREE, STARTER, PROFESSIONAL), duy nhất.
- **description** (TEXT): Mô tả gói (có thể trống).
- **price_monthly** (INTEGER): Giá hàng tháng (VND, mặc định: 0).
- **price_yearly** (INTEGER): Giá hàng năm (VND, mặc định: 0).
- **storage_limit_mb** (INTEGER): Giới hạn dung lượng lưu trữ (MB, mặc định: 100).
- **ai_queries_limit** (INTEGER): Giới hạn truy vấn AI hàng tháng (mặc định: 500).
- **human_agents_limit** (INTEGER): Giới hạn số lượng nhân viên hỗ trợ (mặc định: 2).
- **is_active** (BOOLEAN): Trạng thái gói có đang được cung cấp hay không (mặc định: True).

Quan hệ:

- Quan hệ một-nhiều với bảng `subscription_organizationsubscription`.

4.3.6.2 Bảng `subscription_organizationsubscription`:

Quản lý gói đăng ký hiện tại của mỗi tổ chức và việc sử dụng tài nguyên.

Thuộc tính:

- **id** (PK): Định danh duy nhất của bản ghi đăng ký.
- **organization_id** (FK - `organization.Organization`): Tham chiếu đến tổ chức.
- **tier_id** (FK - `subscription.SubscriptionTier`): Tham chiếu đến gói đăng ký.
- **status** (VARCHAR(20)): Trạng thái thanh toán/đăng ký (ACTIVE, PENDING, FAILED, CANCELED, TRIAL, mặc định: ACTIVE).
- **billing_period** (VARCHAR(10)): Chu kỳ thanh toán (MONTHLY, YEARLY, mặc định: MONTHLY).
- **current_period_start** (TIMESTAMP WITH TIME ZONE): Thời điểm bắt đầu chu kỳ hiện tại.
- **current_period_end** (TIMESTAMP WITH TIME ZONE): Thời điểm kết thúc chu kỳ hiện tại.
- **storage_used_mb** (INTEGER): Dung lượng đã sử dụng (MB, mặc định: 0).
- **ai_queries_used** (INTEGER): Số lượng truy vấn AI đã sử dụng trong chu kỳ (mặc định: 0).
- **additional_storage_mb** (INTEGER): Dung lượng lưu trữ bổ sung đã mua (MB, mặc định: 0).
- **additional_ai_queries** (INTEGER): Số lượng truy vấn AI bổ sung đã mua (mặc định: 0).
- **additional_human_agents** (INTEGER): Số lượng nhân viên hỗ trợ bổ sung đã mua (mặc định: 0).

Quan hệ:

- Quan hệ nhiều-một với bảng `organization_organization`.
- Quan hệ nhiều-một với bảng `subscription_subscriptiontier`.
- Quan hệ một-nhiều với bảng `subscription_additionalusagecharge`.
- Quan hệ một-nhiều với bảng `subscription_payment`.

4.3.6.3 Bảng `subscription_additionalusagecharge`:

Ghi nhận các khoản phí phát sinh do sử dụng vượt mức gói đăng ký.

Thuộc tính:

- **id** (PK): Định danh duy nhất của khoản phí.
- **organization_subscription_id** (FK - `subscription.OrganizationSubscription`): Tham chiếu đến bản ghi đăng ký liên quan.
- **charge_type** (VARCHAR(20)): Loại phí (AI_QUERIES, STORAGE, HUMAN_AGENTS).
- **amount** (INTEGER): Số tiền phí (VND).
- **quantity** (INTEGER): Số lượng tài nguyên vượt mức.
- **unit_price** (INTEGER): Đơn giá mỗi đơn vị tài nguyên (VND).
- **status** (VARCHAR(20)): Trạng thái thanh toán phí (PENDING, PAID, FAILED, mặc định: PENDING).
- **billing_date** (TIMESTAMP WITH TIME ZONE): Ngày xuất hóa đơn/phí (mặc định: now).
- **description** (TEXT): Mô tả chi tiết (có thể trống).

Quan hệ:

- Quan hệ nhiều-một với bảng `subscription_organizationsubscription`.
- Quan hệ một-nhiều với bảng `subscription_payment`.

4.3.6.4 Bảng `subscription_payment`:

Lưu trữ lịch sử các giao dịch thanh toán.

Thuộc tính:

- **id** (PK): Định danh duy nhất của giao dịch.
- **organization_id** (FK - `organization.Organization`): Tham chiếu đến tổ chức thực hiện thanh toán.
- **initiating_user_id** (FK - `users.User`): Tham chiếu đến người dùng khởi tạo thanh toán (có thể null).
- **payment_type** (VARCHAR(20)): Loại thanh toán (SUBSCRIPTION, ADDITIONAL_CHARGE).
- **payment_method** (VARCHAR(20)): Phương thức thanh toán (PAYOS, BANK_TRANSFER, CREDIT_CARD).
- **amount** (INTEGER): Số tiền thanh toán (VND).
- **subscription_id** (FK - `subscription.OrganizationSubscription`): Tham chiếu đến đăng ký liên quan (có thể null).
- **additional_charge_id** (FK - `subscription.AdditionalUsageCharge`): Tham chiếu đến phí bổ sung liên quan (có thể null).
- **status** (VARCHAR(20)): Trạng thái giao dịch (PENDING, COMPLETED, FAILED, REFUNDED, mặc định: PENDING).
- **payment_date** (TIMESTAMP WITH TIME ZONE): Thời điểm thanh toán thành công (có thể null).
- **payos_transaction_id** (VARCHAR(255)): ID giao dịch từ PayOS (có thể null).
- **payos_order_id** (VARCHAR(255)): ID đơn hàng từ PayOS (có thể null).
- **payos_payment_link** (VARCHAR(512)): Link thanh toán PayOS (có thể null).
- **invoice_number** (VARCHAR(50)): Số hóa đơn (có thể null).
- **description** (TEXT): Mô tả giao dịch (có thể trống).
- **created_at** (TIMESTAMP WITH TIME ZONE): Thời điểm tạo bản ghi giao dịch.

- **updated_at** (TIMESTAMP WITH TIME ZONE): Thời điểm cập nhật bản ghi giao dịch lần cuối.

Quan hệ:

- Quan hệ nhiều-một với bảng `organization_organization`.
- Quan hệ nhiều-một với bảng `users_user` (cho initiating_user).
- Quan hệ nhiều-một với bảng `subscription_organizationsubscription`.
- Quan hệ nhiều-một với bảng `subscription_additionalusagecharge`.

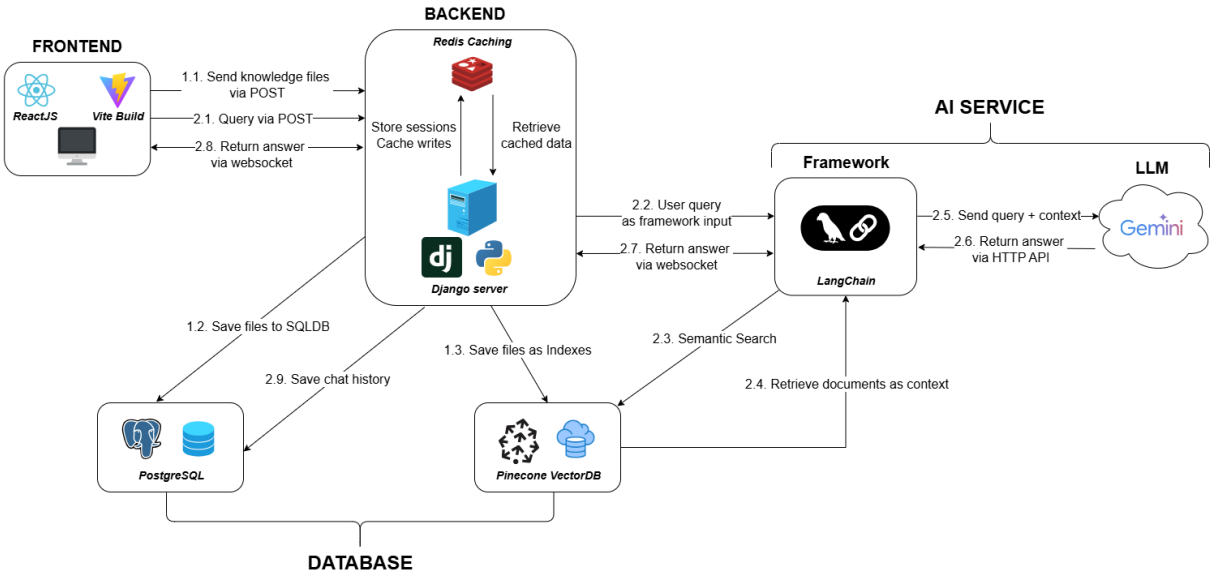
5 Hiện thực

5.1 Công nghệ

5.1.1 Lược đồ tổng quan

Lược đồ này cung cấp góc nhìn tổng quát về cách các công nghệ được ứng dụng trong hệ thống dịch vụ chatbot, bao gồm Frontend (nhân viên và người dùng), Backend, AI, và hệ cơ sở dữ liệu (SQL và Vector). Có hai luồng đi chính được mô tả trên hình:

1. Luồng thứ nhất (các mũi tên có dòng miêu tả bắt đầu bằng chữ số 1): quy trình thực hiện lưu và đánh chỉ mục dữ liệu tri thức.
2. Luồng thứ hai (các mũi tên có dòng miêu tả bắt đầu bằng chữ số 2): quy trình giao tiếp giữa người dùng thực tế và chatbot của doanh nghiệp.



Hình 57: *Lược đồ tổng quan về mối quan hệ giữa các công nghệ của hệ thống*

Các bộ phận của hệ thống trao đổi dữ liệu thông qua các giao thức REST API (Post, HTTP API) và Websocket. Các hệ cơ sở dữ liệu sẽ đảm nhận các nhiệm vụ đọc, ghi những loại dữ liệu khác nhau. Ngoài ra, Django Server sử dụng Redis Caching Server để tăng tốc việc truy xuất dữ liệu sẵn có.

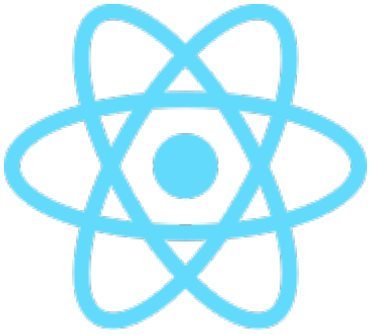
5.1.2 Frontend

5.1.2.1 ReactJS

a **Giới thiệu**

React (hay còn được gọi là React.js hoặc ReactJS) là một thư viện JavaScript front-end mã nguồn mở và miễn phí để xây dựng giao diện người dùng dựa trên các thành phần UI riêng lẻ. Nó được phát triển và duy trì bởi Meta (trước đây là Facebook) và cộng đồng các nhà phát triển và công ty cá nhân. React có thể được sử dụng làm cơ sở để phát triển các ứng dụng SPA (Single page application), thiết bị di động hoặc ứng dụng

được kết xuất bằng máy chủ với các thư viện khác như Next.js. Tuy nhiên, React chỉ hướng tới việc quản lý trạng thái và hiển thị trạng thái đó cho DOM, vì vậy việc tạo ứng dụng bằng React thường yêu cầu sử dụng thêm các thư viện bổ sung để thực hiện định tuyến trang, cũng như thêm một số chức năng ở phía máy khách.



Hình 58: *Logo React - Nguồn: Internet*

b **Đặc điểm công nghệ**

Sau đây là một số đặc điểm nổi bật của React:

- Component (Thành phần): Thành phần trong React là các đơn vị cấu trúc và logic độc lập, tạo nên các phần tử UI. Mỗi thành phần có thể kết hợp cấu trúc HTML, styling CSS và hành vi JavaScript. Thành phần nhận các thuộc tính (props) để cấu hình vẻ ngoài và hành vi, giúp tăng khả năng tái sử dụng. Thành phần có thể đơn giản như một nút hoặc phức tạp như một tiện ích chọn ngày hoặc biểu đồ tương tác.
- JSX: JSX là một cú pháp giống HTML được sử dụng trong mã JavaScript của React. JSX giúp việc hình dung cấu trúc UI trở nên trực quan hơn, tạo cảm giác gần gũi với HTML hơn là gọi hàm JavaScript phức tạp. JSX cho phép nhúng biểu thức JavaScript trực tiếp trong cấu trúc giống HTML, tạo ra nội dung tùy chỉnh dựa trên các biến và logic. JSX sau cùng sẽ được biên dịch thành mã JavaScript thông thường bởi các công cụ như Babel.
- Thuộc tính (Props): Thuộc tính là cơ chế chính để truyền dữ liệu từ thành phần cha sang thành phần con trong React. Chúng tạo ra mối quan hệ rõ ràng giữa thành phần cha và con, nơi thành phần cha quyết định những gì một thành phần con nên hiển thị hoặc cách nó nên hoạt động. Props chỉ nên được đọc trong một thành phần, không nên sửa đổi chúng trực tiếp để đảm bảo dòng chảy dữ liệu (data flow) dễ đoán. Công cụ như PropTypes hoặc TypeScript có thể được sử dụng để kiểm tra kiểu của props, giúp phát hiện sớm các lỗi tiềm ẩn.
- State (Trạng thái): Trạng thái là dữ liệu được quản lý trong một thành phần, có thể kích hoạt việc tái xây dựng thành phần khi được cập nhật. Trạng thái cho phép một thành phần thay đổi vẻ ngoài hoặc hành vi theo thời gian phản ứng với tương tác của người dùng hoặc các sự kiện khác. Trạng thái hoạt động như bộ nhớ ngắn hạn của thành phần cho các giá trị mà nó cần theo dõi nội bộ. Khi trạng thái của một thành phần được cập nhật, React sẽ biết rằng thành phần và UI của nó có thể cần phải thay đổi, kích hoạt một lần tái hiển thị.

c **So sánh các công nghệ tương tự trên thị trường**

Bên dưới sẽ là bảng so sánh sách công nghệ tương tự React theo các tiêu chí cụ thể do nhóm tự đánh giá.

Tiêu chí	React	Angular	Vue	Svelte
Learning curve	Trung bình	Khó	Dễ	Trung bình
Tốc độ	Nhanh	Nhanh	Nhanh	Nhanh nhất
Kích thước	Lớn	Lớn	Nhỏ	Nhỏ
Cộng đồng	Lớn	Lớn	Phát triển	Nhỏ
Tài liệu	Xuất sắc	Tốt	Tốt	Tốt
Khả năng mở rộng	Tốt	Tốt	Tốt	Tốt

d **Tại sao chọn công nghệ này cho dự án?**

Nhóm đã lựa chọn công nghệ React vì những lý do sau:

- Hiệu suất cao nhờ Virtual DOM.
- Component-based Architecture: React chia UI thành các thành phần (components) nhỏ, tái sử dụng được.

- Thư viện lớn và cộng đồng mạnh.
- JSX (JavaScript XML): JSX giúp lập trình viên viết mã dễ dàng và trực quan hơn bằng cách kết hợp JavaScript với cú pháp HTML.
- Tích hợp dễ dàng với công nghệ khác: React có thể tích hợp với bất kỳ backend nào (Node.js, Django, Spring Boot, v.v.) thông qua API, hỗ trợ cả RESTful API và GraphQL.
- Dễ học: Với kiến thức cơ bản về JavaScript, lập trình viên có thể nhanh chóng làm quen với React.

5.1.2.2 Vite

a) **Giới thiệu**

Vite là một công cụ xây dựng (build tool) hiện đại, được phát triển bởi Evan You, người sáng lập Vue.js. Vite được thiết kế để cải thiện trải nghiệm phát triển frontend bằng cách cung cấp tốc độ cao và hiệu quả vượt trội so với các công cụ truyền thống như Webpack.



Hình 59: *Logo Vite - Nguồn Internet*

b) **Đặc điểm công nghệ**

Vite gồm có 2 thành phần quan trọng:

- Dev server (Máy chủ phát triển): Cung cấp các tính năng nâng cao phong phú cho các module ES gốc, chẳng hạn như tính năng Hot Module Replacement (HMR) cực kỳ nhanh.
- Lệnh build: Đóng gói mã bằng Rollup, được cấu hình sẵn để xuất các tài nguyên tĩnh được tối ưu hóa cao cho môi trường production.

c) **So sánh các công nghệ tương tự trên thị trường**

Bên dưới sẽ là bảng so sánh sách công nghệ tương tự Vite theo các tiêu chí cụ thể do nhóm tự đánh giá.

Tiêu chí	Webpack	Parcel	Vite
Learning curve	Khó	Dễ	Dễ nhất
Tốc độ	Chậm	Nhanh	Nhanh nhất
Kích thước	Lớn nhất	Lớn	Nhỏ nhất
Cộng đồng	Lớn nhất	Phát triển	Nhỏ
Tài liệu	Tốt	Tốt	Tốt
Khả năng mở rộng	Tốt	Tốt	Tốt

d) **Tại sao chọn công nghệ này cho dự án?**

Nhóm chọn Vite vì những lý do sau:

- Tốc độ khởi chạy cực nhanh bằng cách sử dụng ES Modules (ESM) và native browser support.
- HMR (Hot Module Replacement) hiệu suất cao: Vite có cơ chế HMR được tối ưu hóa, chỉ tải lại các module thực sự thay đổi.
- Build nhanh chóng với Rollup.
- Tương thích đa framework: ở đây nhóm kết hợp với thư viện React qua plugin @vitejs/plugin-react.
- Cấu hình đơn giản.
- Hỗ trợ plugin phong phú.

5.1.2.3 NextUI

NextUI là một thư viện giao diện người dùng (UI library) hiện đại được thiết kế để xây dựng các ứng dụng web với phong cách đẹp mắt và hiệu năng cao. NextUI sử dụng React làm nền tảng, giúp lập trình viên dễ dàng tạo ra các thành phần giao diện (components) tái sử dụng và nhanh chóng.



Hình 60: *Logo NextUI - Nguồn: Internet*

5.1.2.4 TailwindCSS

Tailwind CSS là một utility-first CSS framework, cho phép lập trình viên xây dựng giao diện trực tiếp bằng cách sử dụng các lớp tiện ích (utility classes) thay vì viết mã CSS tùy chỉnh. Framework này không cung cấp các thành phần giao diện dựng sẵn mà tập trung vào việc tối ưu hóa quá trình tạo và tùy chỉnh giao diện.



Hình 61: *Logo TailwindCSS - Nguồn: Internet*

5.1.3 Backend

5.1.3.1 Django và Django REST Framework (DRF)

1. Giới thiệu

   Django là một framework phát triển web bậc cao, mã nguồn mở, được xây dựng trên ngôn ngữ lập trình Python, tuân thủ theo biến thể của kiến trúc MVC (Model-View-Controller) là MVT (Model-View-Template). Kiến trúc này phân tách rõ ràng giữa logic nghiệp vụ, logic xử lý yêu cầu và phản hồi, và logic trình bày, thúc đẩy việc phát triển module hóa và tái sử dụng mã nguồn. Django REST Framework là một bộ công cụ mạnh mẽ và linh hoạt được xây dựng dựa trên Django, chuyên biệt hóa cho việc xây dựng các giao diện lập trình ứng dụng Web. Sự kết hợp giữa Django và DRF không chỉ được cộng đồng phát triển phần mềm quốc tế công nhận về hiệu quả và tính bảo mật, mà còn đặc biệt phù hợp cho việc xây dựng backend của các ứng dụng phức tạp như hệ thống chatbot, nơi đòi hỏi khả năng mở rộng, quản lý dữ liệu linh hoạt và tích hợp dễ dàng với các dịch vụ khác.



Hình 62: *Logo chính thức của Django Framework*

2. Đặc điểm Công nghệ Nổi bật

   Django và DRF cung cấp một hệ sinh thái tính năng toàn diện, đáp ứng các yêu cầu khắt khe của phát triển web hiện đại:

- **Hệ thống Object-Relational Mapping:** Cung cấp một lớp trừu tượng hóa mạnh mẽ, cho phép tương tác với cơ sở dữ liệu (như PostgreSQL, MySQL, SQLite) thông qua các đối tượng Python thay vì viết truy vấn SQL thủ công. Điều này giúp đơn giản hóa đáng kể việc thao tác dữ liệu, tăng cường tính độc lập với hệ quản trị cơ sở dữ liệu cụ thể và giảm thiểu lỗi liên quan đến SQL injection.
- **Hệ thống xác thực và phân quyền:** Tích hợp sẵn cơ chế xác thực người dùng mạnh mẽ (bao gồm quản lý phiên, mật khẩu băm) và hệ thống phân quyền linh hoạt dựa trên quyền hoặc nhóm. DRF mở rộng khả năng này với hỗ trợ các phương thức xác thực API phổ biến như Token Authentication và JWT, cùng với các lớp phân quyền tùy chỉnh cao.
- **Cơ chế tuần tự hóa:** DRF cung cấp cơ chế `serializers` hiệu quả để chuyển đổi các kiểu dữ liệu phức tạp của Python (ví dụ: queryset, model instance) thành các định dạng dữ liệu chuẩn như JSON hoặc XML để truyền qua API, và ngược lại để phân tích cú pháp và xác thực dữ liệu đầu vào.
- **ViewSets và Routers:** Tự động hóa việc ánh xạ các URL tới logic xử lý cho các thao tác CRUD (Create, Read, Update, Delete) thông qua ViewSets và Routers, giúp giảm thiểu mã lặp lại và đảm bảo tính nhất quán cấu trúc API theo chuẩn RESTful.
- **Giao diện quản trị tự động:** Một trong những tính năng đặc trưng của Django là khả năng tự động tạo ra một giao diện quản trị web (admin site) đầy đủ chức năng dựa trên các model dữ liệu đã định nghĩa, cho phép quản lý dữ liệu ứng dụng một cách trực quan mà không cần phát triển thêm.
- **Hệ thống Middleware:** Cung cấp một kiến trúc hook linh hoạt trong chu trình xử lý request/response, cho phép can thiệp để thực hiện các tác vụ xuyên suốt như bảo mật (ví dụ: CSRF protection), quản lý phiên, ghi log, nén dữ liệu một cách module hóa.

3. So sánh với các Framework Backend Tương tự

Bảng 26: *So sánh Django/DRF với các framework backend phổ biến*

Tiêu chí	**Django + DRF**	**Express.js (Node.js)**	**Spring Boot (Java)**	**FastAPI (Python)**
Ngôn ngữ nền tảng	Python	JavaScript	Java	Python
Kiến trúc	MVT (Batteries-included)	Tối giản (Minimalist)	MVC (Comprehensive)	Tối giản, Hiện đại
Tốc độ phát triển ban đầu	Rất cao	Cao	Trung bình	Rất cao
Hiệu năng (Raw Performance)	Tốt	Rất tốt	Xuất sắc	Xuất sắc (Async)
Tính năng tích hợp sẵn	Rất nhiều (ORM, Admin, Auth)	Rất ít (Middleware-based)	Nhiều (DI, Security, Data)	Trung bình (Validation, Async)
Hệ sinh thái	Lớn, Trưởng thành	Rất lớn	Rất lớn, Doanh nghiệp	Đang phát triển nhanh
Độ phức tạp học	Trung bình-Cao	Thấp-Trung bình	Cao-Rất cao	Thấp-Trung bình
Khả năng mở rộng	Tốt	Rất tốt	Xuất sắc	Rất tốt (Async)
Tài liệu & Cộng đồng	Rất tốt	Xuất sắc	Rất tốt	Tốt, Tích cực

Django/DRF nổi bật với triết lý "batteries-included", cung cấp nhiều thành phần thiết yếu, giúp đẩy nhanh tốc độ phát triển ban đầu, đặc biệt với các ứng dụng yêu cầu tính năng quản trị và ORM phức tạp. Mặc dù FastAPI có hiệu năng thô cao hơn nhờ kiến trúc bất đồng bộ, Django vẫn đáp ứng tốt yêu cầu hiệu năng cho đa số ứng dụng web, bao gồm cả hệ thống chatbot, và có hệ sinh thái trưởng thành hơn.

4. Cơ sở Lựa chọn Django và DRF cho Dự án

Việc lựa chọn Django và DRF làm nền tảng backend cho dự án chatbot tư vấn khách hàng dựa trên các luận điểm sau:

- **Tính toàn diện và phát triển nhanh:** Hệ sinh thái "batteries-included" với ORM, Admin, Authentication tích hợp sẵn giúp giảm thiểu đáng kể thời gian và công sức phát triển các thành phần cơ bản, cho phép tập trung vào logic nghiệp vụ cốt lõi của chatbot.

- **Bảo mật mặc định mạnh mẽ:** Cung cấp các cơ chế phòng chống hiệu quả các lỗ hổng phổ biến như XSS (Cross-Site Scripting), CSRF (Cross-Site Request Forgery), SQL injection, là yếu tố then chốt cho một ứng dụng xử lý dữ liệu người dùng.
- **ORM linh hoạt và mạnh mẽ:** Django ORM tích hợp mượt mà với PostgreSQL, hỗ trợ các kiểu dữ liệu nâng cao (như JSONField) và các truy vấn phức tạp, rất phù hợp để quản lý dữ liệu hội thoại và hồ sơ người dùng đa dạng của chatbot.
- **Xây dựng RESTful API chuyên nghiệp:** DRF cung cấp một khung làm việc chuẩn hóa, dựa trên class-based views, serializers, và permissions, giúp xây dựng các API nhất quán, dễ bảo trì và dễ dàng tích hợp với frontend hoặc các dịch vụ bên ngoài.
- **Khả năng mở rộng và cộng đồng lớn:** Django có kiến trúc hỗ trợ tốt cho việc mở rộng quy mô (ví dụ: caching, tối ưu hóa cơ sở dữ liệu) và được hỗ trợ bởi một cộng đồng lớn mạnh, cung cấp nhiều thư viện bên thứ ba chất lượng, tài liệu chi tiết và diễn đàn hỗ trợ tích cực.

5.1.3.2 Hệ Quản trị Cơ sở dữ liệu PostgreSQL

1. Giới thiệu

   PostgreSQL, thường được gọi là "Postgres", là một hệ quản trị cơ sở dữ liệu quan hệ đối tượng (Object-Relational Database Management System - ORDBMS) mã nguồn mở tiên tiến. Được phát triển và bảo trì bởi một cộng đồng toàn cầu đa dạng trong hơn 30 năm, PostgreSQL nổi bật với kiến trúc vững chắc, độ tin cậy đã được kiểm chứng, tính toàn vẹn dữ liệu cao và sự tuân thủ chặt chẽ các chuẩn SQL. Không chỉ hỗ trợ đầy đủ các tính năng của cơ sở dữ liệu quan hệ truyền thống và đảm bảo các thuộc tính ACID (Atomicity, Consistency, Isolation, Durability), PostgreSQL còn cung cấp các khả năng xử lý dữ liệu phức tạp và phi cấu trúc mạnh mẽ, bao gồm hỗ trợ kiểu dữ liệu JSON/JSONB, dữ liệu không gian với PostGIS extension, và tìm kiếm toàn văn (full-text search). Các tính năng nâng cao như materialized views, table inheritance, và cơ chế kiểm soát tương tranh đa phiên bản (Multi-Version Concurrency Control - MVCC) làm cho PostgreSQL trở thành lựa chọn hàng đầu cho nhiều ứng dụng đòi hỏi cao.



Hình 63: *Logo chính thức của PostgreSQL*

2. Đặc điểm Công nghệ Nổi bật

   PostgreSQL cung cấp một loạt các tính năng mạnh mẽ, đáp ứng nhu cầu đa dạng của các ứng dụng hiện đại:

   - **Tuân thủ ACID và độ tin cậy cao:** Đảm bảo tính toàn vẹn dữ liệu ở mức độ cao nhất thông qua việc thực thi nghiêm ngặt các thuộc tính ACID, cực kỳ quan trọng cho các giao dịch trong hệ thống chatbot (ví dụ: cập nhật trạng thái hội thoại, lưu trữ thông tin khách hàng).
   - **Xử lý Dữ liệu JSON/JSONB nâng cao:** Cung cấp hỗ trợ gốc cho kiểu dữ liệu JSON và định dạng nhị phân tối ưu hóa JSONB. JSONB cho phép lập chỉ mục (indexing) hiệu quả trên dữ liệu JSON, giúp truy vấn nhanh chóng các cấu trúc dữ liệu phi cấu trúc hoặc bán cấu trúc, rất phù hợp để lưu trữ lịch sử hội thoại, metadata hoặc các thuộc tính linh hoạt của người dùng.
   - **Tìm kiếm Toàn văn (Full-Text Search):** Tích hợp sẵn cơ chế tìm kiếm văn bản mạnh mẽ, hỗ trợ nhiều ngôn ngữ, stemming (chuẩn hóa từ), quản lý từ điển đồng nghĩa (thesaurus), xếp hạng kết quả tìm kiếm (ranking), và các kiểu chỉ mục chuyên biệt (GIN, GiST) để tăng tốc độ tìm kiếm trong các tập dữ liệu văn bản lớn (ví dụ: tìm kiếm trong kho tri thức của chatbot).
   - **Kiểm soát Tương tranh Đa phiên bản (MVCC):** Sử dụng cơ chế MVCC, cho phép các giao dịch đọc không bị chặn bởi các giao dịch ghi và ngược lại. Điều này giúp tăng đáng kể khả năng xử lý đồng thời (concurrency), cho phép hệ thống chatbot phục vụ nhiều người dùng cùng lúc mà không làm giảm hiệu năng.
   - **Khả năng Mở rộng (Extensibility):** PostgreSQL có tính mở rộng cao thông qua các extensions (ví dụ: PostGIS cho dữ liệu địa lý), kiểu dữ liệu tùy chỉnh, hàm tùy chỉnh, và hỗ trợ các kỹ thuật scaling như phân vùng bảng (table partitioning), truy vấn song song (parallel query), và sao chép dữ liệu (replication).

- **Bảo mật Đa lớp:** Cung cấp các tính năng bảo mật mạnh mẽ như kiểm soát truy cập dựa trên vai trò (role-based access control), mã hóa dữ liệu ở tầng cột (column-level encryption), mã hóa kết nối SSL/TLS, và các phương thức xác thực đa dạng.

3. So sánh với các Hệ Quản trị Cơ sở dữ liệu Tương tự

Bảng 27: *So sánh PostgreSQL với các hệ quản trị CSDL khác*

Tiêu chí	PostgreSQL	MySQL	MongoDB (NoSQL)	SQLite
Loại CSDL	ORDBMS (Quan hệ đối tượng)	RDBMS (Quan hệ)	Document (NoSQL)	RDBMS (File-based)
Tuân thủ ACID	Đầy đủ	Đầy đủ (với InnoDB)	Từng phần (cải thiện gần đây)	Đầy đủ
Hỗ trợ JSON	Rất mạnh (JSONB, indexing)	Tốt (JSON type)	Native (BSON)	Hỗ trợ cơ bản (JSON1 ext)
Tính năng nâng cao	Rất nhiều (MVCC, FTS, GIS)	Nhiều (Replication, Clustering)	Schema linh hoạt, Sharding	Đơn giản, Nhúng
Hiệu năng (chung)	Rất tốt (đặc biệt với tải phức tạp)	Rất tốt (đặc biệt với tải đọc nhiều)	Xuất sắc (với workload phù hợp)	Tốt (cho single-user/low-concurrency)
Khả năng mở rộng (Scale)	Tốt (Vertical, Replication, Partitioning)	Rất tốt (Replication, Clustering)	Xuất sắc (Horizontal Sharding)	Hạn chế (chủ yếu vertical)
Độ phức tạp quản trị	Trung bình-Cao	Trung bình	Trung bình	Rất thấp
Trường hợp sử dụng phù hợp	Ứng dụng phức tạp, cần tính toàn vẹn cao, dữ liệu đa dạng	Web apps phổ biến, CMS, e-commerce	Dữ liệu lớn, phi cấu trúc, cần scale ngang nhanh	Ứng dụng nhúng, mobile, testing, CSDL nhỏ

PostgreSQL nổi trội trong việc xử lý các truy vấn phức tạp và hỗ trợ các kiểu dữ liệu đa dạng, đặc biệt là JSONB, làm cho nó phù hợp hơn MySQL cho các ứng dụng cần lưu trữ và truy vấn dữ liệu bán cấu trúc như hội thoại chatbot. So với MongoDB, PostgreSQL cung cấp tính toàn vẹn dữ liệu mạnh mẽ hơn theo chuẩn ACID, mặc dù MongoDB có lợi thế về khả năng mở rộng ngang và schema linh hoạt.

4. Cơ sở Lựa chọn PostgreSQL cho Dự án

PostgreSQL được chọn làm hệ quản trị cơ sở dữ liệu chính cho dự án chatbot này vì những lý do sau:

- **Tích hợp Tối ưu với Django:** Django ORM có sự hỗ trợ hạng nhất (first-class support) cho PostgreSQL, bao gồm các kiểu dữ liệu đặc thù như `JSONField`, `ArrayField`, giúp tận dụng tối đa sức mạnh của cả hai công nghệ.
- **Khả năng Xử lý Dữ liệu Hội thoại Linh hoạt:** Hỗ trợ JSONB mạnh mẽ với khả năng lập chỉ mục và truy vấn hiệu quả là yếu tố then chốt để lưu trữ, phân tích và truy xuất lịch sử hội thoại, thông tin ngữ cảnh và các thuộc tính người dùng thay đổi linh hoạt của chatbot.
- **Tính năng Tìm kiếm Toàn văn Tích hợp:** Cho phép xây dựng chức năng tìm kiếm trong cơ sở tri thức hoặc lịch sử hội thoại của chatbot một cách hiệu quả mà không cần tích hợp thêm giải pháp tìm kiếm bên ngoài phức tạp cho các nhu cầu cơ bản.
- **Độ tin cậy và Tính Toàn vẹn Dữ liệu:** Đảm bảo ACID là yếu tố quan trọng để duy trì sự nhất quán của trạng thái hội thoại và thông tin người dùng, đặc biệt khi có nhiều tương tác đồng thời.
- **Khả năng Mở rộng và Tính Bền vững:** PostgreSQL cung cấp các tùy chọn mở rộng quy mô khi cần thiết và được hỗ trợ bởi một cộng đồng mã nguồn mở lớn mạnh, đảm bảo sự phát triển liên tục, tài liệu phong phú và không bị phụ thuộc vào nhà cung cấp (vendor lock-in).
- **Sẵn có trên Nền tảng Triển khai:** Dễ dàng tích hợp và quản lý trên các nền tảng PaaS như Heroku (thông qua Heroku Postgres) hoặc các nhà cung cấp đám mây lớn khác.

5.1.3.3 Redis (Remote Dictionary Server)

1. Giới thiệu

   Redis là một kho lưu trữ cấu trúc dữ liệu trong bộ nhớ (in-memory data structure store) mã nguồn mở, hiệu năng cao, thường được sử dụng làm cơ sở dữ liệu key-value, bộ đệm (cache), và trình môi giới thông điệp (message broker). Redis nổi bật với tốc độ truy cập dữ liệu cực nhanh do hoạt động chủ yếu trên RAM, đồng thời hỗ trợ nhiều loại cấu trúc dữ liệu phức tạp (như strings, hashes, lists, sets, sorted sets) và cung cấp các tính năng như persistence (lưu trữ bền vững tùy chọn), replication (sao chép), và clustering (phân cụm).

2. Đặc điểm Công nghệ Nổi bật

   - **Hiệu năng Cực cao:** Do lưu trữ dữ liệu trong bộ nhớ chính (RAM), Redis cho phép thực hiện hàng triệu thao tác đọc/ghi mỗi giây với độ trễ rất thấp (sub-millisecond latency), lý tưởng cho các tác vụ yêu cầu phản hồi nhanh.
   - **Hỗ trợ Đa dạng Cấu trúc Dữ liệu:** Ngoài cặp key-value đơn giản, Redis hỗ trợ các cấu trúc dữ liệu phong phú như lists (hàng đợi), sets (tập hợp duy nhất), sorted sets (tập hợp có thứ tự), hashes (đối tượng), bitmaps, hyperloglogs, và geospatial indexes.
   - **Bộ đệm (Caching):** Là trường hợp sử dụng phổ biến nhất, Redis giúp giảm tải đáng kể cho cơ sở dữ liệu chính bằng cách lưu trữ các dữ liệu thường xuyên truy cập (ví dụ: kết quả truy vấn CSDL, kết quả render trang, session người dùng) trong bộ nhớ.
   - **Quản lý Phiên (Session Management):** Cung cấp giải pháp lưu trữ session người dùng nhanh chóng và có khả năng mở rộng, thay thế cho việc lưu session trong CSDL quan hệ hoặc file hệ thống.
   - **Hàng đợi Tác vụ và Message Broker:** Có thể sử dụng Redis (đặc biệt là cấu trúc List hoặc Pub/Sub) để triển khai các hàng đợi tác vụ đơn giản hoặc hệ thống publish/subscribe cho giao tiếp giữa các thành phần của ứng dụng.
   - **Persistence Tùy chọn:** Hỗ trợ hai cơ chế lưu trữ dữ liệu xuống đĩa là RDB (snapshots định kỳ) và AOF (ghi lại mọi thao tác ghi), giúp đảm bảo dữ liệu không bị mất hoàn toàn khi khởi động lại server.

3. So sánh với các Công nghệ Tương tự

   Bảng 28: *So sánh Redis với Memcached*

Tiêu chí	Redis	Memcached
Loại hình	In-memory data structure store	In-memory key-value cache
Cấu trúc dữ liệu	Đa dạng (strings, lists, sets, hashes, etc.)	Chỉ key-value (strings)
Persistence	Có (RDB, AOF)	Không (chỉ cache)
Replication/Clustering	Có (Built-in)	Hạn chế (thường qua client-side)
Hiệu năng (đơn giản Get/Set)	Rất cao	Rất cao (có thể nhỉnh hơn một chút)
Sử dụng bộ nhớ	Cao hơn (do tính năng)	Tối ưu hơn (cho cache đơn giản)
Tính năng bổ sung	Pub/Sub, Lua scripting, Transactions	Ít tính năng hơn
Trường hợp sử dụng	Caching, Session store, Message queue, Leaderboards, Real-time analytics	Caching đơn giản

   Redis cung cấp bộ tính năng phong phú hơn Memcached, đặc biệt là khả năng persistence và hỗ trợ nhiều cấu trúc dữ liệu, làm cho nó linh hoạt hơn cho nhiều tác vụ ngoài caching đơn thuần.

4. Cơ sở Lựa chọn Redis cho Dự án Redis được tích hợp vào kiến trúc backend của chatbot vì các lý do chính sau:

   - **Tăng tốc Độ phản hồi:** Sử dụng Redis làm bộ đệm (cache) cho các truy vấn CSDL thường xuyên (ví dụ: thông tin cấu hình chatbot, dữ liệu người dùng hay truy cập), các phản hồi API phổ biến, giúp giảm độ trễ và cải thiện trải nghiệm người dùng cuối.

- **Giảm tải Cơ sở dữ liệu:** Việc phục vụ các yêu cầu đọc từ cache Redis giúp giảm đáng kể số lượng truy vấn đến PostgreSQL, bảo vệ CSDL khỏi tình trạng quá tải, đặc biệt khi lượng người dùng tăng cao.
- **Quản lý Trạng thái Hội thoại Tạm thời:** Có thể sử dụng Redis để lưu trữ các thông tin trạng thái tạm thời của phiên hội thoại (ví dụ: ngữ cảnh hiện tại, dữ liệu đang chờ xử lý), giúp duy trì luồng tương tác mượt mà mà không cần ghi liên tục vào CSDL chính.
- **Hỗ trợ Hệ thống Hàng đợi (Optional):** Nếu cần xử lý các tác vụ bất đồng bộ (ví dụ: gửi thông báo, xử lý dữ liệu nền), Redis có thể đóng vai trò là một message broker đơn giản và hiệu quả khi tích hợp với các thư viện như Celery.
- **Tích hợp Dễ dàng:** Có nhiều thư viện client chất lượng cao cho Python (ví dụ: `redis-py`) và dễ dàng tích hợp với Django (qua `django-redis`). Các nền tảng PaaS như Heroku cũng cung cấp Redis add-on tiện lợi.

5.1.3.4 Dịch vụ Lưu trữ Amazon S3 (Simple Storage Service)

1. Giới thiệu

   Amazon S3 (Simple Storage Service) là một dịch vụ lưu trữ đối tượng trên nền tảng đám mây của Amazon Web Services (AWS). S3 cung cấp khả năng lưu trữ và truy xuất dữ liệu với độ bền, tính sẵn sàng, hiệu năng và khả năng mở rộng hàng đầu trong ngành. Dịch vụ này được thiết kế để lưu trữ gần như không giới hạn số lượng đối tượng với kích thước từ vài byte đến hàng terabyte, phù hợp cho nhiều trường hợp sử dụng như lưu trữ website tĩnh, sao lưu và phục hồi dữ liệu, lưu trữ dữ liệu lớn, và đặc biệt là lưu trữ các tệp đa phương tiện cho các ứng dụng web và di động.

2. Đặc điểm Công nghệ Nổi bật

   - **Độ bền và tính sẵn sàng cao:** Amazon S3 được thiết kế để đạt độ bền 99.999999999% (11 số 9) cho các đối tượng, lưu trữ dữ liệu trên nhiều thiết bị và nhiều Availability Zones (AZs) khác nhau, đảm bảo an toàn và khả năng truy cập dữ liệu liên tục.
   - **Khả năng mở rộng gần như vô hạn:** Người dùng có thể lưu trữ lượng dữ liệu không giới hạn mà không cần lo lắng về việc quản lý hạ tầng lưu trữ vật lý. S3 tự động điều chỉnh quy mô để đáp ứng nhu cầu.
   - **Lớp lưu trữ đa dạng (Storage Classes):** Cung cấp nhiều lớp lưu trữ khác nhau (S3 Standard, S3 Intelligent-Tiering, S3 Standard-IA, S3 One Zone-IA, S3 Glacier Instant Retrieval, S3 Glacier Flexible Retrieval, S3 Glacier Deep Archive) với các mức giá và hiệu năng truy cập khác nhau, cho phép tối ưu chi phí dựa trên tần suất truy cập và yêu cầu về độ trễ.
   - **Bảo mật và quản lý truy cập mạnh mẽ:** Hỗ trợ mã hóa dữ liệu cả khi lưu trữ (at rest) và khi truyền (in transit), cùng với các cơ chế kiểm soát truy cập chi tiết thông qua IAM policies, bucket policies, Access Control Lists (ACLs), và điểm truy cập S3 (S3 Access Points).
   - **Quản lý vòng đời dữ liệu:** Cho phép tự động chuyển đổi đối tượng giữa các lớp lưu trữ khác nhau hoặc xóa đối tượng sau một khoảng thời gian nhất định để tối ưu chi phí và tuân thủ quy định.
   - **Tích hợp hệ sinh thái AWS:** Tích hợp chặt chẽ với hàng loạt dịch vụ AWS khác như EC2, CloudFront (CDN), Lambda, Athena, Redshift, phục vụ cho nhiều kiến trúc ứng dụng phức tạp.
   - **Hỗ trợ website tĩnh và CDN:** Có thể cấu hình S3 bucket để host một website tĩnh và dễ dàng tích hợp với Amazon CloudFront để phân phối nội dung trên toàn cầu với độ trễ thấp.

3. So sánh với các Giải pháp Lưu trữ Tương tự

Bảng 29: *So sánh Amazon S3 với các dịch vụ lưu trữ đối tượng khác*

Tiêu chí	Amazon S3	Google Cloud Storage (GCS)	Azure Blob Storage	Self-hosting (e.g., MinIO)
Nhà cung cấp	AWS	Google Cloud	Microsoft Azure	Tự quản lý
Độ bền	Rất cao (11 số 9)	Rất cao (11 số 9)	Rất cao (tùy chọn cấu hình)	Tùy thuộc hạ tầng
Tính sẵn sàng	Cao (tùy storage class)	Cao (tùy storage class)	Cao (tùy storage class)	Tùy thuộc hạ tầng
Chi phí	Cạnh tranh, trả theo sử dụng	Cạnh tranh, trả theo sử dụng	Cạnh tranh, trả theo sử dụng	Chi phí hạ tầng và vận hành

Tiếp tục ở trang sau

Bảng 29 – Tiếp tục

Tiêu chí	Amazon S3	Google Cloud Storage (GCS)	Azure Blob Storage	Self-hosting (e.g., MinIO)
Hệ sinh thái	Rất lớn (AWS)	Lớn (GCP)	Lớn (Azure)	Hạn chế (tùy giải pháp)
Tính năng	Rất phong phú	Phong phú	Phong phú	Cơ bản đến nâng cao (tùy giải pháp)
Độ phức tạp quản trị	Thấp (dịch vụ quản lý)	Thấp (dịch vụ quản lý)	Thấp (dịch vụ quản lý)	Cao (tự quản lý hạ tầng)

Các nhà cung cấp đám mây lớn (AWS, GCP, Azure) đều cung cấp dịch vụ lưu trữ đối tượng mạnh mẽ và tương đồng về các tính năng cốt lõi. Lựa chọn thường phụ thuộc vào hệ sinh thái đám mây đang sử dụng, yêu cầu cụ thể về tính năng hoặc chi phí. Giải pháp tự host như MinIO mang lại sự kiểm soát cao hơn nhưng đòi hỏi chi phí đầu tư và vận hành hạ tầng đáng kể.

4. Cơ sở Lựa chọn Amazon S3 cho Dự án Việc sử dụng Amazon S3 để lưu trữ các tệp tĩnh và dữ liệu người dùng trong dự án chatbot được dựa trên các lý do sau:

   - **Khả năng lưu trữ mở rộng và tin cậy:** Đảm bảo khả năng lưu trữ các tệp đa phương tiện (ví dụ: hình ảnh sản phẩm, tài liệu hướng dẫn mà chatbot cung cấp, ảnh đại diện người dùng) hoặc các bản ghi log lớn một cách tin cậy và có thể mở rộng dễ dàng khi ứng dụng phát triển.
   - **Giảm tải cho server ứng dụng:** Việc phục vụ các tệp tĩnh (CSS, JavaScript, hình ảnh) trực tiếp từ S3 (thường kết hợp với CDN như CloudFront) giúp giảm tải đáng kể cho server ứng dụng Django, cải thiện hiệu năng và khả năng đáp ứng của backend.
   - **Chi phí hiệu quả:** Mô hình định giá trả theo dung lượng lưu trữ và lượng dữ liệu truyền đi giúp tối ưu chi phí, đặc biệt với các lớp lưu trữ chi phí thấp cho dữ liệu ít truy cập.
   - **Độ bền dữ liệu cao:** Đảm bảo an toàn cho các dữ liệu quan trọng như tệp người dùng tải lên hoặc các bản sao lưu dữ liệu.

5.1.3.5 PayOS

1. Giới thiệu

   PayOS là một nhà cung cấp dịch vụ cổng thanh toán tại Việt Nam, cung cấp giải pháp tích hợp đa dạng các phương thức thanh toán trực tuyến phổ biến trong nước vào các ứng dụng web và di động. Dịch vụ này tập trung vào việc đơn giản hóa quy trình tích hợp thanh toán cho các doanh nghiệp và nhà phát triển, hỗ trợ các phương thức như thẻ ATM nội địa, thẻ tín dụng quốc tế, và các ví điện tử thông dụng tại Việt Nam. Trong bối cảnh dự án chatbot này, PayOS được lựa chọn để xử lý các giao dịch tài chính liên quan, ví dụ như quản lý gói đăng ký dịch vụ hoặc các thanh toán phát sinh khác.

2. Đặc điểm Công nghệ và Dịch vụ Nổi bật

   - **API tích hợp linh hoạt:** Cung cấp bộ giao diện lập trình ứng dụng (API) và tài liệu kỹ thuật (documentation) cho phép lập trình viên tích hợp cổng thanh toán vào backend (ví dụ: Django) để khởi tạo giao dịch, xử lý kết quả thanh toán, và truy vấn lịch sử giao dịch.
   - **Quy trình thanh toán an toàn:** Tuân thủ các tiêu chuẩn bảo mật ngành thanh toán (ví dụ: PCI DSS), áp dụng các cơ chế mã hóa và xác thực để bảo vệ thông tin giao dịch nhạy cảm.
   - **Giao diện thanh toán tùy chỉnh:** Cung cấp tùy chọn hiển thị cổng thanh toán dưới dạng redirect sang trang của PayOS hoặc tích hợp trực tiếp trên website/ứng dụng của người bán.
   - **Dashboard quản lý giao dịch:** Cung cấp giao diện quản trị cho phép theo dõi, quản lý các giao dịch, đối soát và rút tiền.

3. Cơ sở Lựa chọn (hoặc xem xét) PayOS cho Dự án Việc tích hợp PayOS vào hệ thống chatbot được cân nhắc dựa trên các yếu tố sau:

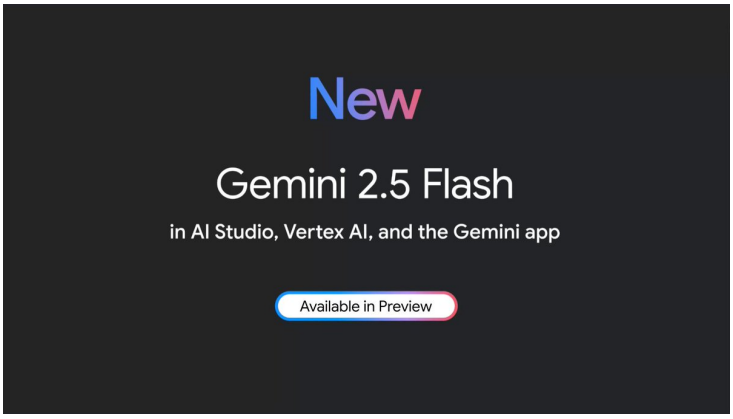
   - **Phù hợp thị trường Việt Nam:** Cung cấp các phương thức thanh toán quen thuộc và được ưa chuộng bởi người dùng trong nước, tăng khả năng tiếp cận và tỷ lệ thành công của giao dịch.
   - **Đáp ứng nhu cầu chức năng:** Cung cấp các tính năng cần thiết cho việc xử lý thanh toán của dự án.
   - **Đơn giản hóa tích hợp:** Mục tiêu của các cổng thanh toán như PayOS là cung cấp API và quy trình tích hợp tương đối đơn giản, giúp giảm thời gian và độ phức tạp khi triển khai chức năng thanh toán.

5.1.4 AI

5.1.4.1 Mô hình Gemini-2.5-Flash

a) **Giới thiệu**

Gemini 2.5 Flash là phiên bản mô hình ngôn ngữ (tầm trung) mới nhất của Google, kế thừa nền tảng Gemini 2.0 Flash với khả năng reasoning cải tiến, đồng thời ưu tiên tốc độ và tiết kiệm chi phí. Mô hình suy nghĩ lai này cho phép nhà phát triển bật hoặc tắt quá trình "thinking" để điều chỉnh độ phức tạp của nhiệm vụ. Thông qua tham số thinking_budget, người dùng có thể kiểm soát ngân sách token dành cho reasoning, cân bằng chất lượng và độ trễ theo nhu cầu . Với tỷ lệ hiệu năng/chi phí vượt trội, Gemini 2.5 Flash mang đến giải pháp AI linh hoạt, giúp tối ưu chi phí và hiệu quả triển khai trong nhiều ứng dụng.



Hình 64: *Mô hình ngôn ngữ lớn Gemini-2.5-flash preview. Nguồn: Internet*

b) **Đặc điểm công nghệ**

Gemini 2.5 Flash cung cấp các tính năng nổi bật như:

- Hybrid reasoning với khả năng bật/tắt quá trình "thinking" tùy theo độ phức tạp nhiệm vụ.
- Tham số 'thinking_budget' cho phép kiểm soát số token tối đa dành cho reasoning, cân bằng chất lượng và độ trễ.
- Tốc độ xử lý cao với độ trễ thấp, tối ưu chi phí cho các tác vụ khối lượng lớn.
- Hỗ trợ đa modal (văn bản, hình ảnh, âm thanh, video) cho các ứng dụng đa dạng.
- Tích hợp dễ dàng qua Gemini API, Google AI Studio và Vertex AI, giúp triển khai nhanh chóng.

Ưu điểm:

- Cân bằng chất lượng và chi phí tốt, dẫn đầu về hiệu năng/giá trong phân khúc Flash.
- Khả năng reasoning nâng cao giúp giải quyết các tác vụ phức tạp như toán học, phân tích và lập trình chính xác hơn.
- Mô hình lai linh hoạt cho phép tối ưu tham số và độ phức tạp cho từng use case.
- Độ trễ thấp, phù hợp cho kịch bản cần phản hồi nhanh như dịch vụ khách hàng hay xử lý thông tin theo thời gian thực.

Nhược điểm:

- Chi phí triển khai ban đầu có thể cao, đặc biệt khi sử dụng 'thinking_budget' lớn.
- Phụ thuộc vào kết nối đám mây ổn định, có thể gặp gián đoạn khi mạng yếu.
- Cấu hình reasoning nhiều tham số có thể phức tạp, đòi hỏi nhà phát triển làm quen.
- Trong những tác vụ đơn giản, nên tắt reasoning để tiết kiệm chi phí và thời gian.

c) **So sánh các công nghệ tương tự trên thị trường**

Các thông số trong bảng sau được trích từ bài báo của White và cộng sự (2024) [5].

Model	Global Average (point)	Reasoning Average (point)	Language Average (point)	Instruction Following (point)
Grok 3 Mini Beta	68.33	87.61	59.09	78.70
Meta-Llama-3.1-8B-Instruct-Turbo	24.95	14.78	21.10	54.90
Claude 3.5 Haiku	38.49	26.19	39.71	61.88
Gemini 2.5 Flash Preview	71.21	73.47	59.43	79.02
GPT 4o mini	37.63	25.64	29.88	56.80

trong đó:

- Global Average (Hiệu suất tổng thể): Phản ánh hiệu suất tổng thể của model trong mọi khía cạnh. Điều này quan trọng vì chatbot chăm sóc khách hàng cần xử lý đa dạng yêu cầu, từ ngôn ngữ đến dữ liệu phân tích, dù không nhất thiết phải xuất sắc ở tất cả.
- Reasoning Average (Khả năng suy luận): Khả năng suy luận rất quan trọng vì chatbot cần hiểu ngữ cảnh, giải quyết vấn đề của khách hàng và đưa ra câu trả lời hợp lý. Ví dụ, khi khách hàng trình bày một vấn đề, chatbot phải phân tích đúng và đề xuất giải pháp.
- Language Average (Khả năng xử lý ngôn ngữ): Đây là yếu tố cốt lõi cho chatbot chăm sóc khách hàng, vì chatbot cần giao tiếp tự nhiên, hiểu câu hỏi của khách hàng, và trả lời một cách rõ ràng và chính xác.
- IF Average (Instruction Following - Khả năng làm theo hướng dẫn): Đảm bảo rằng chatbot có thể hiểu và tuân theo các yêu cầu cụ thể từ khách hàng, chẳng hạn như "Liệt kê danh sách điểm đến" hoặc "Cập nhật thông tin liên lạc".

Với Instruction Following cao nhất (79.02), Gemini 2.5 Flash Preview đảm bảo chatbot có thể hiểu và đáp ứng chính xác yêu cầu của khách hàng, yếu tố quan trọng nhất cho chăm sóc khách hàng. Reasoning: Đạt mức cao (73.47), đủ để xử lý các tình huống cần suy luận. Language: Đứng đầu, giúp chatbot giao tiếp tự nhiên, hiểu ý người dùng và bám sát yêu cầu.

d) **Tại sao chọn công nghệ này cho dự án?**

Gemini 2.5 Flash là mô hình phù hợp nhất cho dự án xây dựng dịch vụ cung cấp chatbot đám mây nhờ:

- Hiệu năng vượt trội: Khả năng phản hồi nhanh và mượt mà giúp cải thiện trải nghiệm người dùng, khả năng tư duy tốt hơn người tiền nhiệm 2.0 Flash.
- Tối ưu chi phí vận hành: Với khả năng sử dụng tài nguyên hiệu quả, Gemini 2.5 Flash giúp giảm chi phí vận hành trên nền tảng đám mây (chi phí thấp nhất trong các model đối thủ có dùng api), đồng thời rẻ hơn 15 lần so với phiên bản cao cấp Gemini 2.5 Pro.
- Khả năng mở rộng: Hỗ trợ xử lý đồng thời nhiều yêu cầu, phù hợp với các dịch vụ có quy mô lớn.
- Đa dạng ứng dụng: Thích hợp cho nhiều ngành công nghiệp, từ thương mại, giáo dục, y tế đến tài chính, giúp tăng tính cạnh tranh của dịch vụ.
- Dễ dàng triển khai: Triển khai dễ dàng trên Google AI Studio, Vertex AI, và Firebase mà không cần cơ sở hạ tầng on-premise như chuỗi mô hình Llama.

5.1.4.2 LangChain

a) **Giới thiệu**

LangChain là một framework mã nguồn mở, được phát triển nhằm tối ưu hóa việc xây dựng ứng dụng sử dụng Large Language Models (LLMs) như GPT-4 hoặc các mô hình tương tự. Ra đời vào năm 2022 bởi Harrison Chase, LangChain tập trung vào việc đơn giản hóa các quy trình tích hợp LLMs vào ứng dụng bằng cách cung cấp các công cụ hỗ trợ lập trình viên xử lý luồng hội thoại, quản lý bộ nhớ, và truy cập dữ liệu ngoài. LangChain đã nhanh chóng trở thành một công cụ phổ biến trong lĩnh vực AI nhờ khả năng trừu tượng hóa nhiều thành phần phức tạp khi làm việc với LLMs.



Hình 65: *Framework LangChain - Nguồn: Internet*

b) **Đặc điểm công nghệ**

LangChain cung cấp một bộ công cụ mạnh mẽ để xây dựng ứng dụng sử dụng LLMs, bao gồm:

- Kết nối với LLMs: Dễ dàng tích hợp với GPT (OpenAI), Hugging Face, hay các mô hình khác.
- Bộ nhớ (Memory): Quản lý ngữ cảnh trong các cuộc hội thoại dài, giúp ứng dụng "ghi nhớ" thông tin người dùng cung cấp.
- Kết nối dữ liệu bên ngoài: Tích hợp với cơ sở dữ liệu, API, hoặc tìm kiếm qua tài liệu để tăng tính chính xác và cá nhân hóa phản hồi.
- Chain: Tạo các chuỗi thao tác (pipeline) để xử lý dữ liệu đầu vào và kết hợp nhiều mô hình AI khác nhau.
- Agent: Xây dựng các tác nhân (agents) tự động thực hiện nhiệm vụ như tìm kiếm thông tin hoặc phân tích dữ liệu.
- Tool: Hỗ trợ phát triển các công cụ tự động xử lý nhiều loại dữ liệu, từ văn bản đến bảng tính.

Ưu điểm:

- Tính linh hoạt cao: LangChain có thể tích hợp với nhiều loại LLMs và công cụ khác nhau, giúp tùy chỉnh dễ dàng cho từng dự án.
- Quản lý ngữ cảnh tốt: Bộ nhớ giúp duy trì ngữ cảnh trong các ứng dụng hội thoại, nâng cao trải nghiệm người dùng.
- Tăng cường khả năng truy xuất thông tin: LangChain cho phép truy cập dữ liệu bên ngoài, làm cho các chatbot thông minh và thực tế hơn.
- Hỗ trợ phát triển nhanh: Cung cấp thư viện API và công cụ phát triển toàn diện, giúp rút ngắn thời gian xây dựng ứng dụng.
- Mã nguồn mở: Cộng đồng phát triển mạnh mẽ, tài liệu đầy đủ và được cập nhật liên tục.

Nhược điểm:

- Độ phức tạp: Đối với người mới tiếp cận, việc làm quen với các khái niệm như Chain, Memory, hay Agent có thể mất thời gian.
- Phụ thuộc vào LLMs: Hiệu suất của LangChain phụ thuộc vào các mô hình AI mà nó tích hợp, có thể bị giới hạn bởi chi phí hoặc năng lực của LLM.
- Khả năng mở rộng: Khi ứng dụng lớn, việc quản lý bộ nhớ và xử lý luồng dữ liệu có thể trở nên phức tạp, đòi hỏi tối ưu hóa kỹ thuật.

c) **So sánh các công nghệ tương tự trên thị trường**

Tiêu chí	LangChain	LlamaIndex
Ngôn ngữ hỗ trợ	Python và TypeScript/JavaScript với mức độ phát triển mạnh mẽ; cập nhật thường xuyên trên cả hai nền tảng, vì thế cộng đồng support mạnh	Python là trọng tâm phát triển; TypeScript ít được cập nhật hơn và chậm so với phiên bản Python, cộng đồng support ít đa dạng hơn
Chức năng chính	Hỗ trợ xây dựng workflow phức tạp với các LLM, cung cấp công cụ quản lý chain, memory, agent và tích hợp các công nghệ khác như Pinecone, Weaviate	Tập trung vào quản lý và truy xuất dữ liệu phi cấu trúc thông qua index thông minh, hỗ trợ nhiều loại index như tree, list, và vector
Khả năng mở rộng	Tốt, hỗ trợ tích hợp đa dạng các công nghệ backend (ví dụ: vector DB, API, công cụ tìm kiếm)	Mở rộng tốt, nhưng thiên về xử lý và truy xuất dữ liệu với cấu trúc index hiện có
Tính dễ sử dụng	Giao diện API thân thiện, tài liệu chi tiết, cộng đồng lớn hỗ trợ tích cực	Đơn giản khi làm việc với dữ liệu phi cấu trúc, nhưng yêu cầu kiến thức nền tảng về cấu trúc index
Hiệu suất	Cao khi làm việc với workflow phức tạp, nhờ khả năng quản lý memory và agents hiệu quả	Cao khi làm việc với dữ liệu phi cấu trúc và truy vấn qua index; tuy nhiên, hiệu suất có thể giảm nếu dữ liệu quá lớn

d) **Tại sao chọn công nghệ này cho dự án?**

LangChain là lựa chọn lý tưởng cho dự án xây dựng dịch vụ cung cấp chatbot đám mây nhờ:

- Khả năng quản lý ngữ cảnh: Với tính năng Memory, LangChain giúp chatbot ghi nhớ và xử lý các cuộc hội thoại dài mà không mất ngữ cảnh, mang lại trải nghiệm tự nhiên cho người dùng.
- Tích hợp dữ liệu khách hàng: LangChain cho phép truy cập tài liệu và cơ sở dữ liệu của từng khách hàng (RAG - Retrieval-Augmented Generation), giúp cá nhân hóa phản hồi và tăng tính chính xác.
- Xây dựng nhanh và hiệu quả: Với các tính năng như Chain và Tool, LangChain giảm đáng kể thời gian phát triển, giúp triển khai dịch vụ nhanh hơn.
- Khả năng mở rộng đa dạng: LangChain hỗ trợ tích hợp với các mô hình LLMs khác nhau, giúp dễ dàng tùy chỉnh dịch vụ theo yêu cầu của từng khách hàng, đồng thời, framework này cung cấp cập nhật liên tục trên cả 2 ngôn ngữ Python và Typescript, giúp các lập trình viên có đa dạng lựa chọn và không lo ngại vấn đề cập nhật phiên bản trên hai ngôn ngữ lập trình.
- Phù hợp với mô hình SaaS (Software-as-a-Service): Dễ dàng triển khai trên đám mây và tối ưu chi phí, phù hợp cho hệ thống chatbot đa khách hàng.

5.1.4.3 Pinecone Vector Database

a) **Giới thiệu**

Pinecone là một nền tảng cơ sở dữ liệu vector được phát triển bởi công ty Pinecone Systems, Inc., ra mắt vào năm 2021. Công nghệ này được thiết kế để giải quyết nhu cầu lưu trữ và tìm kiếm hiệu quả trong các ứng dụng sử dụng biểu diễn dữ liệu dưới dạng vector (vector embeddings). Pinecone tập trung vào việc đơn giản hóa quy trình triển khai hệ thống tìm kiếm vector, hỗ trợ các ứng dụng AI như tìm kiếm ngữ nghĩa, gợi ý nội dung, và chatbot.



Hình 66: *Hệ cơ sở dữ liệu vector Pinecone - Nguồn: Internet*

b) **Đặc điểm công nghệ**

Pinecone cung cấp các tính năng nổi bật giúp nó trở thành lựa chọn hàng đầu cho hệ thống cơ sở dữ liệu vector trên thị trường hiện tại:

- Tìm kiếm gần đúng (Approximate Nearest Neighbor - ANN): Cung cấp khả năng tìm kiếm nhanh chóng và chính xác các vector tương tự trong tập dữ liệu lớn.
- Quản lý dữ liệu vector: Tự động hóa quy trình lưu trữ, cập nhật, và xóa vector.
- Khả năng mở rộng (Scalability): Hỗ trợ xử lý các tập dữ liệu lớn với độ trễ thấp.
- Tích hợp dễ dàng: Pinecone hỗ trợ tích hợp với các framework phổ biến như LangChain, Hugging Face, và OpenAI, giúp triển khai nhanh chóng trong các dự án AI.
- Index tối ưu hóa: Hỗ trợ các cấu trúc dữ liệu index (như HNSW) để tăng tốc độ tìm kiếm vector.
- Quản lý metadata: Lưu trữ và truy xuất thông tin bổ sung liên quan đến từng vector, giúp tăng tính linh hoạt trong ứng dụng.

Ưu điểm:

- Hiệu suất cao: Tìm kiếm gần đúng với độ trễ thấp ngay cả khi làm việc với dữ liệu lớn.
- Dịch vụ được quản lý hoàn toàn (Fully Managed): Không yêu cầu cấu hình và bảo trì phức tạp, tiết kiệm thời gian cho đội ngũ phát triển.
- Khả năng tích hợp mạnh mẽ: Hỗ trợ các công cụ và framework AI phổ biến, giúp dễ dàng triển khai trong các ứng dụng học sâu và xử lý ngôn ngữ tự nhiên.
- Tính sẵn sàng cao: Hoạt động ổn định trên môi trường đám mây với khả năng mở rộng dễ dàng.

Nhược điểm:

- Chi phí: Pinecone có thể tốn kém hơn so với các giải pháp vector database tự quản lý hoặc mã nguồn mở.
- Khả năng tùy chỉnh hạn chế: Là dịch vụ đám mây, người dùng ít quyền kiểm soát đối với cấu trúc nội bộ so với các giải pháp tự quản lý.
- Phụ thuộc vào internet: Hiệu năng giảm khi tốc độ mạng không ổn định.

c) **So sánh các công nghệ tương tự trên thị trường**

Tiêu chí	Pinecone	Weaviate	Milvus	ChromaDB
Loại hình triển khai	Dịch vụ đám mây được quản lý hoàn toàn	Open-source, hỗ trợ đám mây và on-prem	Open-source, hỗ trợ Kubernetes	Open-source, cài đặt nhẹ, dành cho ứng dụng nhỏ
Khả năng mở rộng	Cao, tự động mở rộng	Cao, nhưng cần cấu hình thủ công	Cao, hỗ trợ Kubernetes	Hạn chế, phù hợp cho các ứng dụng nhỏ gọn
Tính dễ sử dụng	Thân thiện, triển khai nhanh	Dễ dàng nhưng cần kiến thức kỹ thuật	Yêu cầu kỹ năng cấu hình cao	Dễ sử dụng, cài đặt nhanh, tích hợp tốt với Python
Quản lý metadata	Có hỗ trợ đầy đủ	Hỗ trợ mạnh mẽ	Hỗ trợ cơ bản	Có hỗ trợ nhưng không mạnh mẽ như Pinecone
Hiệu suất tìm kiếm	Tốt với độ trễ thấp	Tốt, có thể tùy chỉnh	Tốt, hỗ trợ nhiều kiểu index	Tốt cho dữ liệu nhỏ, không tối ưu với dữ liệu lớn
Chi phí	Cao, do là dịch vụ SaaS	Miễn phí, nhưng cần chi phí triển khai	Miễn phí, cần chi phí hạ tầng	Miễn phí, yêu cầu ít tài nguyên

d) **Tại sao chọn công nghệ này cho dự án?**

Pinecone là lựa chọn hợp lý để lưu trữ dạng vector cho dự án xây dựng dịch vụ cung cấp chatbot đám mây nhờ:

- Tìm kiếm ngữ nghĩa nhanh chóng: Pinecone cho phép chatbot tìm kiếm thông tin liên quan dựa trên vector embeddings, tăng khả năng hiểu ngữ nghĩa và phản hồi chính xác.
- Đơn giản hóa triển khai: Là một dịch vụ được quản lý hoàn toàn, Pinecone giảm đáng kể khối lượng công việc bảo trì và quản trị, cho phép tập trung phát triển ứng dụng.
- Tích hợp dễ dàng với RAG (Retrieval-Augmented Generation): Tối ưu cho việc kết hợp dữ liệu bên ngoài, hỗ trợ khách hàng tải lên tài liệu và tích hợp vào quy trình hội thoại.
- Mở rộng quy mô dịch vụ: Với khả năng xử lý dữ liệu lớn và tìm kiếm nhanh, Pinecone hỗ trợ mở rộng ứng dụng chatbot để phục vụ hàng ngàn người dùng đồng thời.
- Quản lý metadata linh hoạt: Pinecone cho phép lưu trữ thông tin phụ trợ, giúp chatbot cung cấp phản hồi được cá nhân hóa dựa trên từng khách hàng.

5.2 Mã nguồn

5.2.1 Quản lý mã nguồn

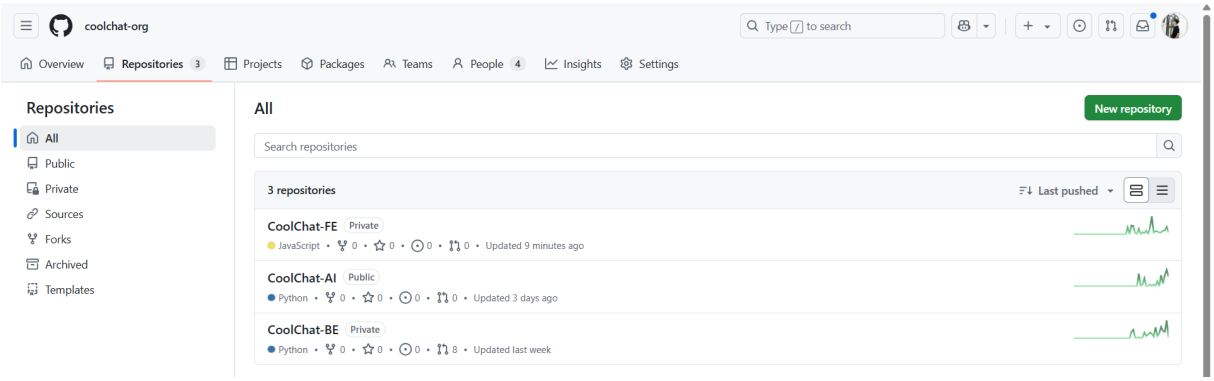
Trong quá trình phát triển hệ thống tạo chatbot chăm sóc khách hàng, nhóm sử dụng Git - một hệ thống quản lý phiên bản phân tán phổ biến, nhằm đảm bảo việc kiểm soát và theo dõi quá trình thay đổi mã nguồn hiệu quả. Git cho phép các thành viên trong nhóm làm việc song song trên nhiều nhánh (branch), dễ dàng tích hợp mã (merge), theo dõi lịch sử thay đổi (commit history) và xử lý xung đột khi làm việc nhóm.

Để lưu trữ và quản lý mã nguồn từ xa, nhóm lựa chọn GitHub - một nền tảng lưu trữ mã nguồn trực tuyến dựa trên Git. GitHub hỗ trợ nhiều tính năng hữu ích cho phát triển cộng tác như pull request, issue tracking, CI/CD tích hợp, và khả năng quản lý phân quyền truy cập theo từng repository.

Để tổ chức cấu trúc mã nguồn một cách rõ ràng và dễ quản lý, nhóm đã tạo một GitHub Organization với tên là CoolChat. Trong đó, mã nguồn của hệ thống được chia thành ba repository riêng biệt tương ứng với từng thành phần chức năng:

- CoolChat-FE: chứa mã nguồn frontend, được phát triển bằng React và Vite.
- CoolChat-BE: chứa mã nguồn backend, được xây dựng bằng Django REST Framework.

- CoolChat-AI: chứa mã nguồn liên quan đến xử lý AI và tích hợp chatbot, bao gồm mô hình xử lý ngôn ngữ tự nhiên.



Hình 67: *CoolChat Organization*

Cấu trúc tổ chức này giúp nhóm phân tách rõ ràng trách nhiệm phát triển, triển khai và bảo trì các thành phần hệ thống, đồng thời thuận tiện cho việc phân quyền truy cập, kiểm soát phiên bản và triển khai CI/CD độc lập cho từng phần.

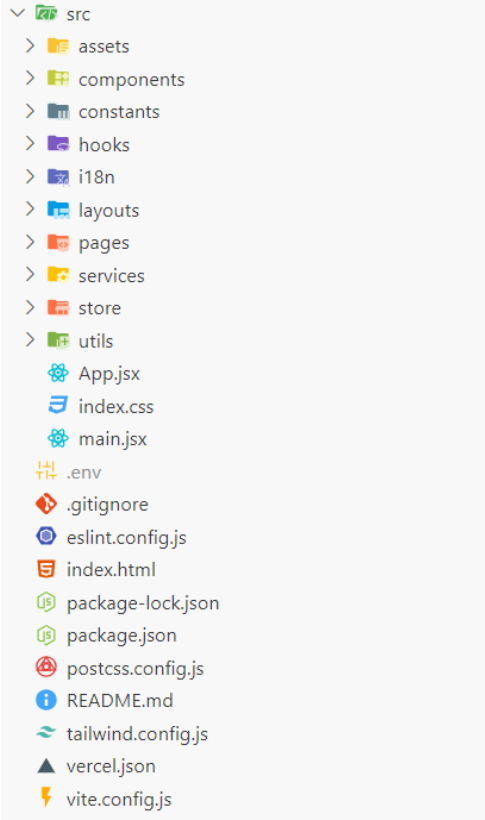
5.2.2 Cấu trúc mã nguồn

5.2.2.1 Frontend

Cấu trúc mã nguồn Frontend như sau:

- Các thư mục
  - assets: Lưu trữ tài nguyên tĩnh như hình ảnh, icon, font,...
  - components: Các thành phần UI tái sử dụng trong toàn bộ ứng dụng
  - constants: Chứa các hằng số được dùng chung trong dự án, ví dụ: các giá trị cố định, config,...
  - hooks: Các custom React hooks để tái sử dụng logic giữa các component
  - i18n: Cấu hình và dữ liệu liên quan đến đa ngôn ngữ
  - layouts: Các layout chung như layout cho trang chính, layout dashboard, layout xác thực,...
  - pages: Các trang chính tương ứng với route trong ứng dụng
  - services: Chứa các dịch vụ để giao tiếp với API với axios
  - store: Quản lý trạng thái toàn cục với Redux
  - utils: Các hàm tiện ích hỗ trợ cho nhiều phần trong dự án
- Các file quan trọng
  - App.jsx: File chính chứa cấu trúc cơ bản của ứng dụng (bao gồm router, provider,...)
  - main.jsx: Điểm khởi chạy của ứng dụng React (ReactDOM.render) - nơi mount App vào DOM
  - index.css: File CSS tổng cho toàn bộ app (chứa các style chung hoặc import từ Tailwind, ...)
  - .env: File lưu trữ các biến môi trường
  - .gitignore: Định nghĩa các file hoặc thư mục sẽ bị Git bỏ qua (không commit lên repository)
  - eslint.config.js: Cấu hình ESLint để kiểm tra chuẩn code và format tự động
  - index.html: File HTML gốc để React mount vào
  - package.json: Khai báo các dependency, script, metadata của dự án
  - package-lock.json: Ghi lại chính xác phiên bản các gói (dependencies) đã cài đặt để đảm bảo build đồng nhất
  - postcss.config.js: Cấu hình PostCSS để xử lý CSS (kết hợp với TailwindCSS)
  - README.md: Tài liệu hướng dẫn sử dụng, mô tả dự án
  - tailwind.config.js: Cấu hình riêng cho TailwindCSS như theme màu sắc, breakpoint, plugin,...

- vercel.json: Cấu hình deploy trên nền tảng Vercel (hosting tự động)
- vite.config.js: Cấu hình Vite - quản lý alias, server, optimize,...



Hình 68: *Cấu trúc mã nguồn Frontend*

5.2.2.2 Backend

Phần mã nguồn backend của hệ thống, được lưu trữ trong repository `CoolChat-BE`, được phát triển dựa trên framework Django, một framework web Python mạnh mẽ và phổ biến, cùng với Django REST Framework để xây dựng các API hiệu quả. Cấu trúc dự án được tổ chức theo mô hình "app-per-feature", giúp tăng tính module hoá, dễ quản lý và bảo trì.

Cấu trúc thư mục chính của dự án backend (`coolchat_be/`) bao gồm các ứng dụng (apps) Django riêng biệt, mỗi ứng dụng đảm nhiệm một nhóm chức năng cụ thể:

- `authentication/`: Chịu trách nhiệm xử lý các quy trình liên quan đến xác thực và phân quyền người dùng (ví dụ: đăng nhập, đăng ký, quản lý token).
- `users/`: Quản lý thông tin chi tiết của người dùng trong hệ thống.
- `chat/`: Xử lý logic cốt lõi liên quan đến các phiên trò chuyện, bao gồm lưu trữ tin nhắn, trạng thái cuộc hội thoại.
- `chatbot/`: Tích hợp và quản lý các chức năng liên quan đến chatbot, giao tiếp với dịch vụ AI và xử lý logic đặc thù của bot.
- `document/`: Quản lý các tài liệu, văn bản hoặc cơ sở tri thức được sử dụng bởi chatbot để trả lời câu hỏi.
- `organization/`: Quản lý thông tin về các tổ chức hoặc doanh nghiệp sử dụng hệ thống chatbot.
- `subscription/`: Xử lý các vấn đề liên quan đến gói dịch vụ, đăng ký và thanh toán.
- `reports/`: Chịu trách nhiệm tạo và quản lý các báo cáo, thống kê về hoạt động của chatbot và tương tác người dùng.

Bên cạnh các ứng dụng chức năng, cấu trúc dự án còn bao gồm các thư mục hỗ trợ quan trọng khác:

- `core/`: Tập hợp các mã nguồn nền tảng và tiện ích dùng chung, bao gồm các lớp trừu tượng cơ sở, mixins, các hàm tiện ích, và các hằng số hoặc cấu hình cốt lõi. Mục đích của thư mục này là thúc đẩy tái sử dụng mã nguồn và đảm bảo tính nhất quán giữa các ứng dụng thành phần trong dự án.

- `templates/`: Lưu trữ các tệp mẫu định dạng HTML, được Django sử dụng để tạo các phản hồi động phía máy chủ. Các mẫu này chủ yếu phục vụ cho việc kết xuất giao diện quản trị (Django Admin) và định dạng nội dung cho các email được hệ thống gửi đi.
- `static/`: Tập hợp các tệp tĩnh như tệp CSS, mã JavaScript, và các tệp hình ảnh. Các tài sản này được sử dụng để định dạng và bổ trợ chức năng cho giao diện quản trị Django và các tài nguyên đã được xây dựng cần thiết để hiển thị thành phần chat widget.
- `docs/`: Chứa tài liệu đặc tả API của hệ thống, tuân theo chuẩn OpenAPI (được trực quan hóa thông qua giao diện Swagger UI). Tài liệu này đóng vai trò then chốt, cung cấp hướng dẫn chi tiết cho đội ngũ phát triển frontend và các bên thứ ba trong việc tương tác và tích hợp với các API endpoints do backend cung cấp.

Ở cấp độ root repository `CoolChat-BE`, các tệp và thư mục cấu hình, quản lý môi trường và phụ thuộc cũng được tổ chức một cách khoa học:

- `config/`: Thư mục chứa các tệp cấu hình chính của ứng dụng Django (ví dụ: `settings.py`, `urls.py` cấp dự án, `wsgi.py`, `asgi.py`).
- `requirements/`: Thư mục chứa các tệp định nghĩa danh sách các gói thư viện Python phụ thuộc. Việc chia nhỏ tệp requirements theo môi trường (ví dụ: `base.txt`, `development.txt`, `production.txt`) giúp quản lý dependencies hiệu quả và đảm bảo môi trường triển khai nhất quán.
- `manage.py`: Tệp tiện ích dòng lệnh chuẩn của Django, dùng để thực thi các tác vụ quản trị dự án (ví dụ: chạy server phát triển, tạo migrations, thực thi câu lệnh tùy chỉnh).
- `.env`, `.env.example`: Các tệp dùng để quản lý biến môi trường, chứa các thông tin nhạy cảm như khóa API, thông tin kết nối cơ sở dữ liệu.

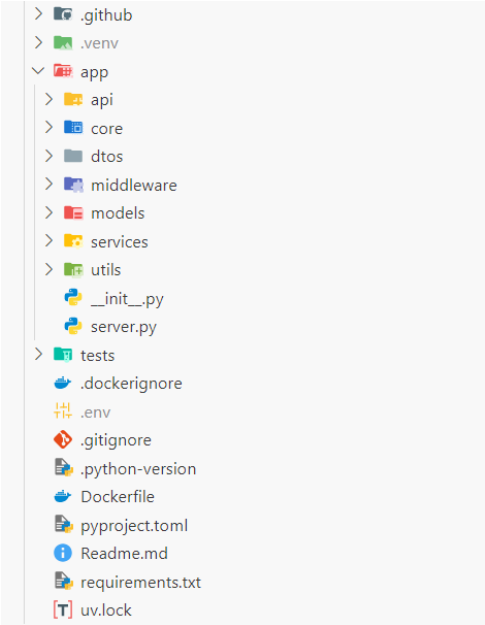
Ngoài ra, dự án còn có thư mục `widget/`, chứa mã nguồn cho thành phần widget trò chuyện có thể nhúng trực tiếp vào trang web của khách hàng. Mặc dù được đặt trong repository backend, cấu trúc bên trong của nó (`src/`, `components/`, `styles/`, `utils/`) gợi ý đây có thể là một ứng dụng frontend độc lập (ví dụ: xây dựng bằng JavaScript thuần hoặc một thư viện/framework nhẹ) được phát triển và đóng gói riêng biệt để dễ dàng tích hợp.

5.2.2.3 AI

Cấu trúc mã nguồn của dịch vụ AI như sau:

- Các thư mục
  - .github: Chứa file .yml, đảm nhận việc CI/CD cho service trên Github Action.
  - .venv: Chứa các gói thư viện cài đặt trong service.
  - app: Là thư mục chính của service, chứa toàn bộ hiện thực mã nguồn.
  - test: Chứa các bài kiểm thử về độ tin cậy của phản hồi từ chatbot.
- Các module bên trong app:
  - api: Là nơi định nghĩa các route và controller cho service. Các route xác định cách service phản hồi với các yêu cầu HTTP, trong khi controller xử lý logic để xử lý các yêu cầu đó.
  - core: Là nơi khởi tạo các thiết lập, sự kết nối database. Đây là phần nền tảng để thiết lập môi trường cho service hoạt động.
  - dtos: Là nơi tập trung của các data transfer object (DTO), dùng để giao tiếp giữa backend với service này. DTOs giúp chuẩn hóa định dạng dữ liệu trao đổi giữa các phần của hệ thống.
  - middleware: Chứa các hàm middleware, có thể can thiệp vào yêu cầu và phản hồi để thực hiện xác thực trước khi yêu cầu đến controller.
  - models: Chứa các model, đại diện cho cấu trúc dữ liệu và logic nghiệp vụ của ứng dụng. Các model thường tương ứng với các bảng trong cơ sở dữ liệu và bao gồm các phương thức để thao tác và xác thực dữ liệu.
  - services: Chứa logic nghiệp vụ của service. Các service xử lý các hoạt động hoặc luồng công việc cụ thể, thường tương tác với các model để truy xuất hoặc cập nhật dữ liệu và thực hiện các tính toán hoặc xử lý cần thiết.
  - utils: Chứa các hàm và lớp tiện ích. Đây là các đoạn mã có thể tái sử dụng cung cấp các chức năng chung, chẳng hạn như hàm trợ giúp cho việc thao tác chuỗi, định dạng ngày tháng, hoặc xử lý lỗi, có thể được sử dụng trong toàn bộ ứng dụng.

- Các file quan trọng
  - Dockerfile: Đây là file chỉ định quy cách đóng gói ứng dụng. Nó chứa các lệnh cần thiết để xây dựng một hình ảnh Docker cho ứng dụng, giúp đơn giản hóa việc triển khai và chạy ứng dụng trong một container.
  - server.py: File khởi động server, đóng vai trò là điểm bắt đầu của ứng dụng.
  - uv.lock: File sinh bởi trình quản lý gói 'uv', đảm bảo rằng các phiên bản của phụ thuộc được sử dụng là nhất quán giữa các môi trường khác nhau.
  - pyproject.toml: File cấu hình cho dự án Python, thường được sử dụng bởi các công cụ như Poetry. File này chứa thông tin về phụ thuộc, cấu hình dự án, và metadata, giúp quản lý dự án khi không thể cài đặt uv.
  - __init__.py: File này được sử dụng để biến một thư mục thành một gói Python, cho phép import các module từ thư mục đó. Nó có thể để trống hoặc chứa mã khởi tạo cho gói.

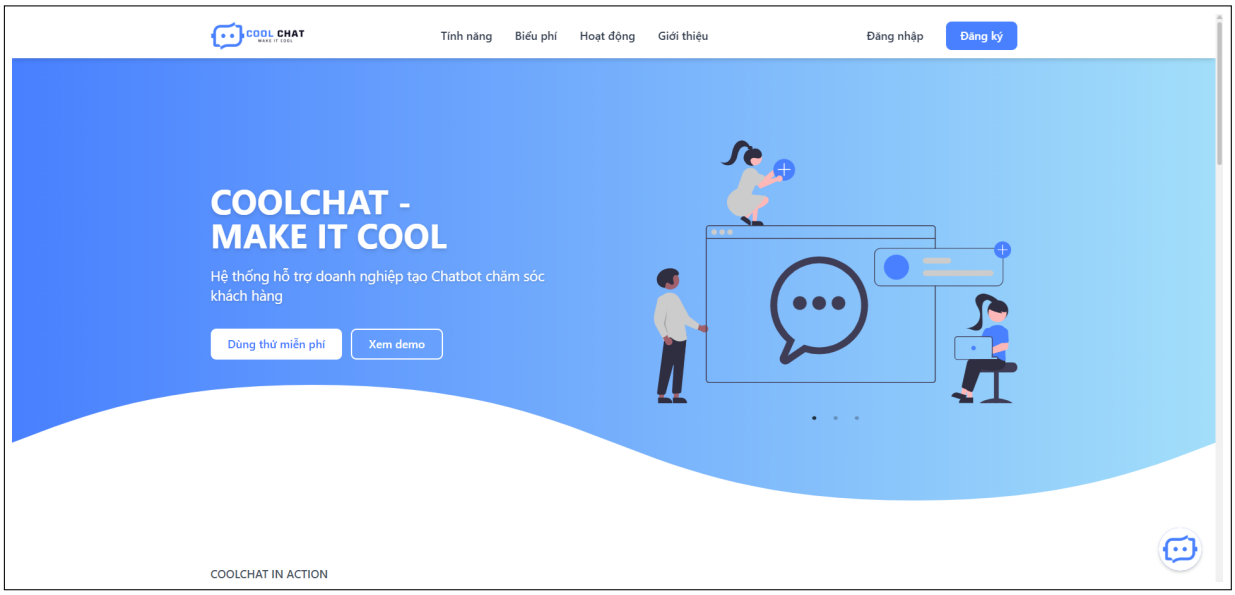


Hình 69: *Cấu trúc mã nguồn AI Service*

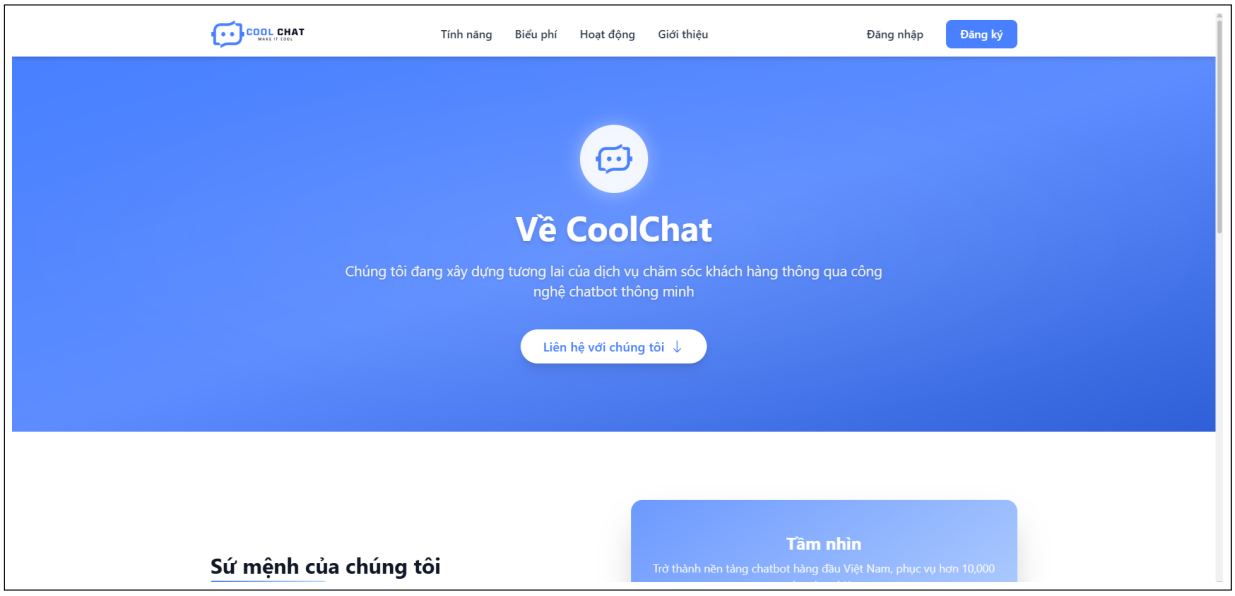
5.3 Giao diện

5.3.1 Trang Đích

Giao diện trang đích sẽ được hiển thị với mọi người dùng. Nội dung sẽ hiển thị các thông tin quan trọng và cần thiết nhất của của hệ thống, giúp người dùng hình dung được giá trị mà hệ thống mang lại. Trang đích gồm có trang chính và trang giới thiệu. Trang chính chứa màn hình chào, các tính năng nổi bật, các gói dịch vụ, cách hoạt động và đăng ký ngay. Trang giới thiệu chứa thông tin công ty, sứ mệnh và đội ngũ. Ngoài ra còn có thanh header chứa logo, các nút điều hướng và thanh footer chứa các thông tin liên hệ.



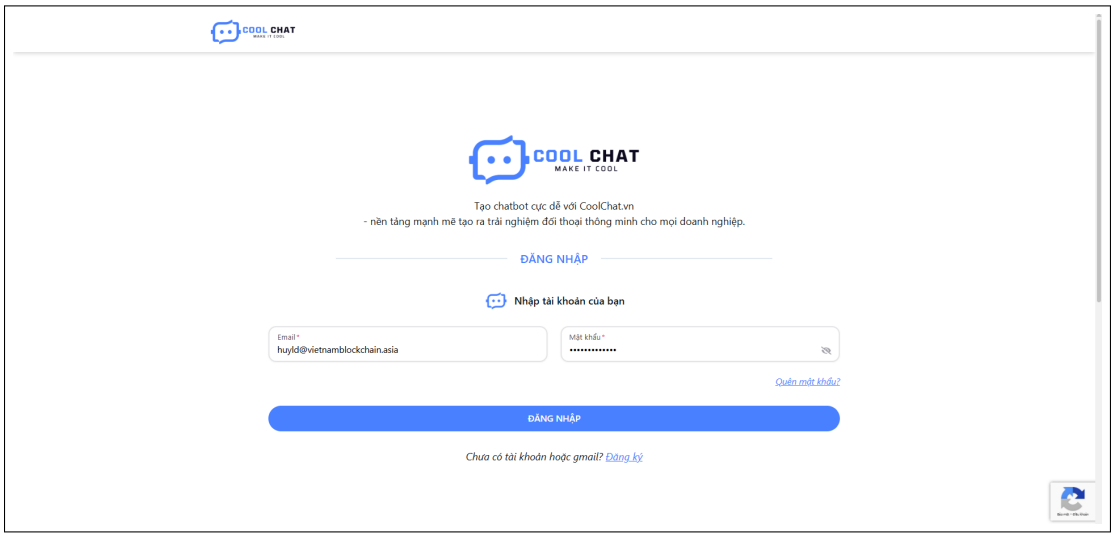
Hình 70: *Giao diện trang đích chính*



Hình 71: *Giao diện trang giới thiệu*

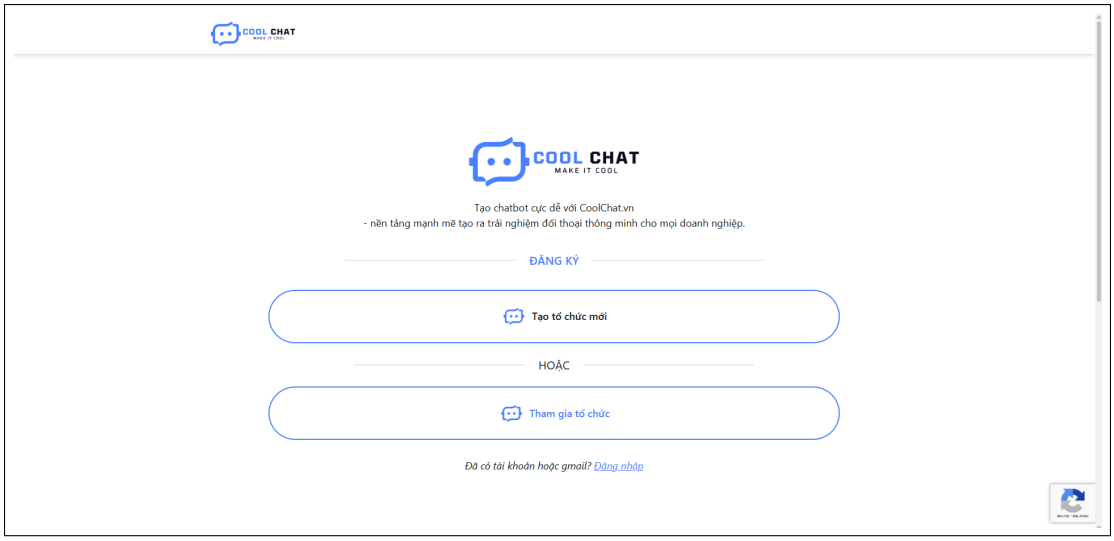
5.3.2 Trang Xác thực

Hệ thống sẽ cung cấp các chức năng xác thực như đăng nhập, tạo tổ chức, tham gia tổ chức, quên mật khẩu cho ba loại người dùng là chủ sở hữu tổ chức, quản trị viên tổ chức và nhân viên chăm sóc khách hàng.



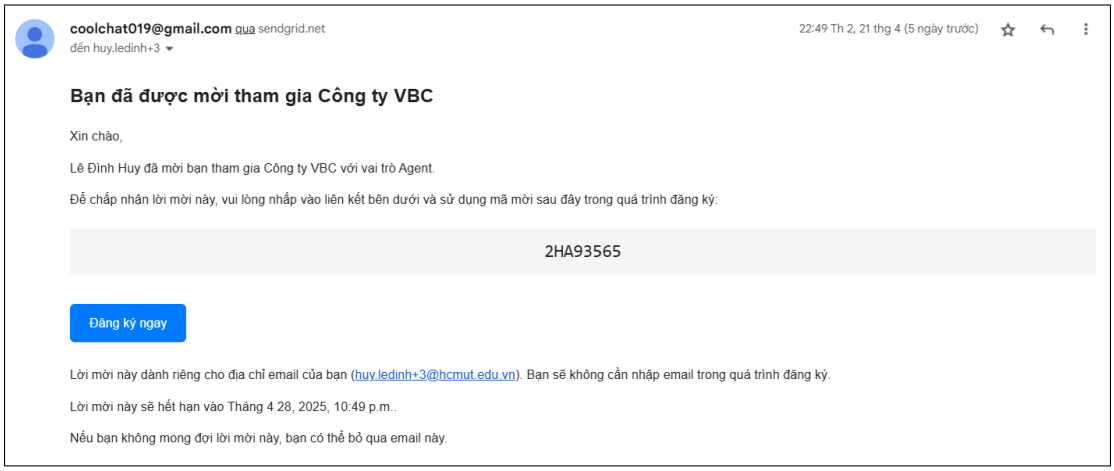
Hình 72: *Giao diện đăng nhập*

Đối với giao diện đăng ký, người dùng có thể chọn tạo một tổ chức mới hoặc tham gia một tổ chức đã tồn tại sử dụng mã giới thiệu được gửi qua địa chỉ email.



Hình 73: *Giao diện đăng ký*

Khi người dùng tạo tổ chức, được mời tham gia tổ chức, quên mật khẩu, người dùng sẽ nhận được một địa chỉ email để xác minh.



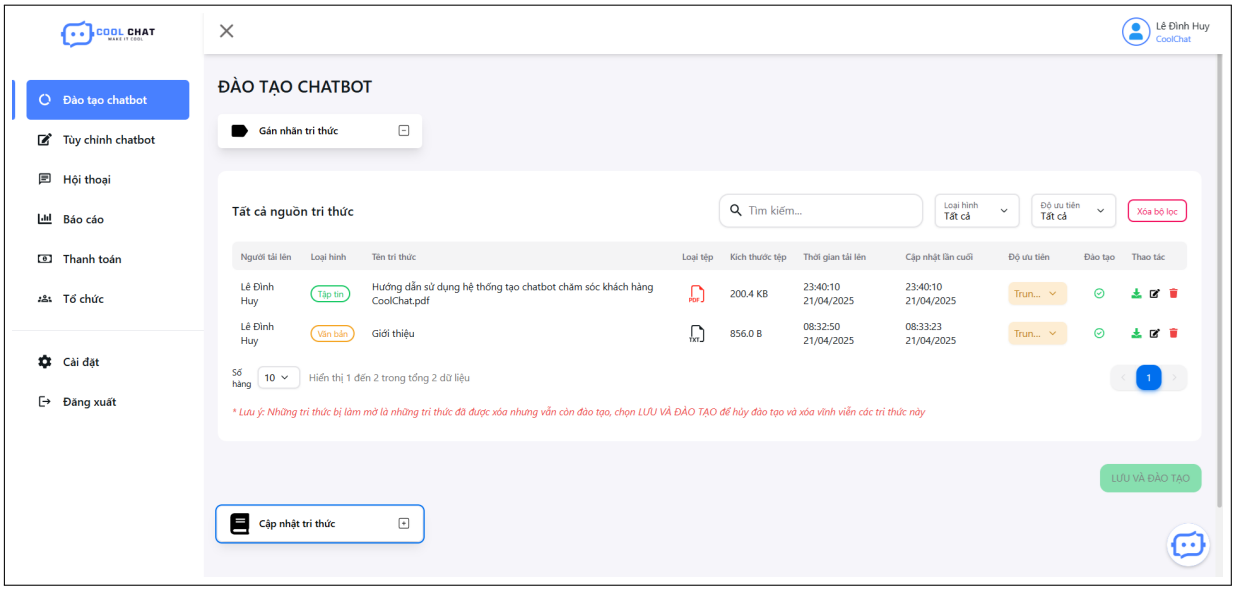
Hình 74: *Email tham gia tổ chức được gửi bởi hệ thống*

Mỗi trang đều chứa đầy đủ các thành phần cần thiết, bố cục hài hòa, màu sắc bắt mắt.

5.3.3 Trang Đào tạo chatbot

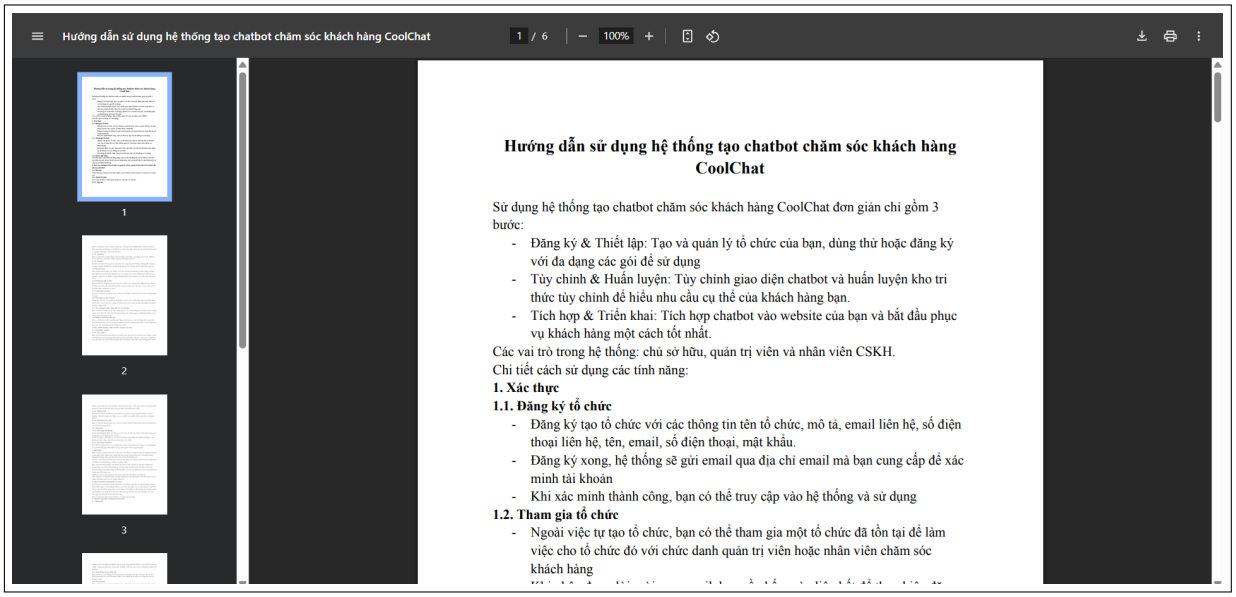
Sau khi đăng nhập thành công với vai trò Chủ sở hữu/Quản trị viên, người dùng sẽ vào trang Đào tạo chatbot. Trang gồm có mục gán nhãn tri thức và mục cập nhật tri thức.

Đối với gán nhãn tri thức, người dùng quan sát thấy một danh sách các tri thức hiện có với các thông tin của từng tri thức. Người dùng có thể tìm kiếm, lọc, xem trang, chỉnh sửa, xem chi tiết, tải xuống, xóa và nhấn nút lưu và đào tạo tri thức.



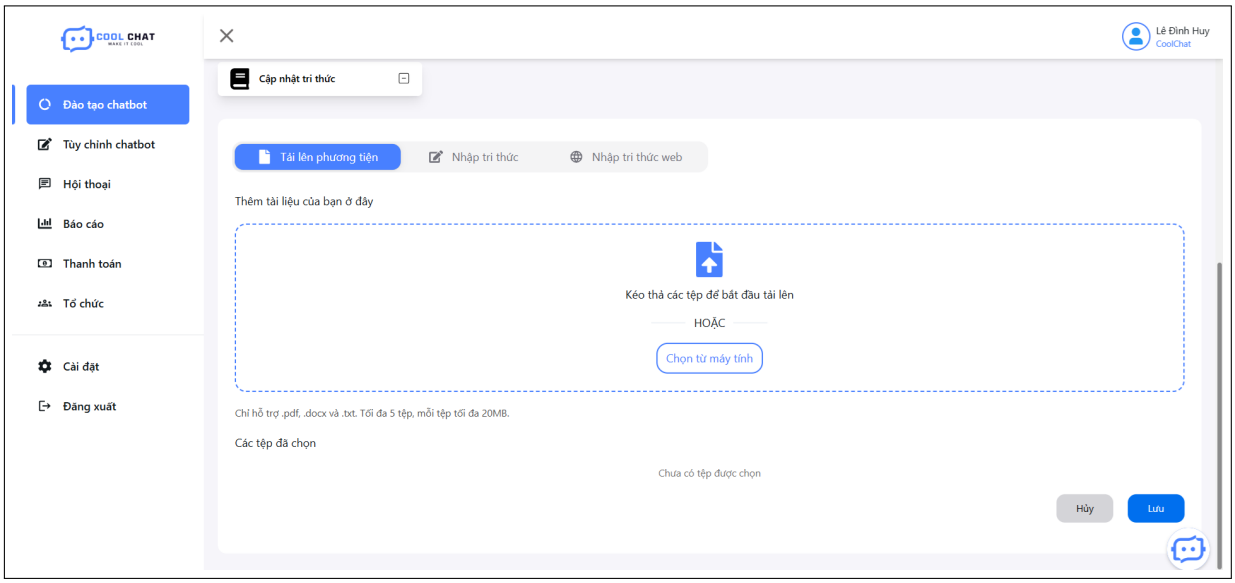
Hình 75: *Giao diện mục Gán nhãn tri thức*

Khi bấm tải xuống một tri thức có dạng tập tin hoặc văn bản, hệ thống sẽ hiện thị bản xem trước cho tri thức có loại tệp .txt và .pdf, loại tệp .docx sẽ được tải ngay xuống thiết bị người dùng.

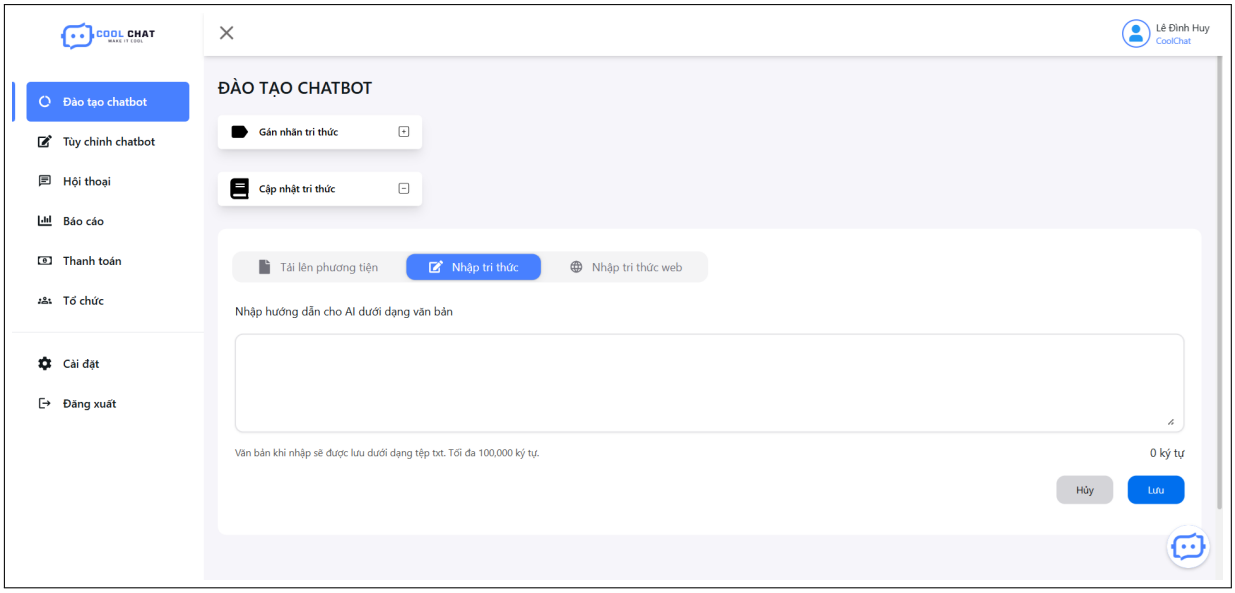


Hình 76: *Bản xem trước khi tải xuống tri thức*

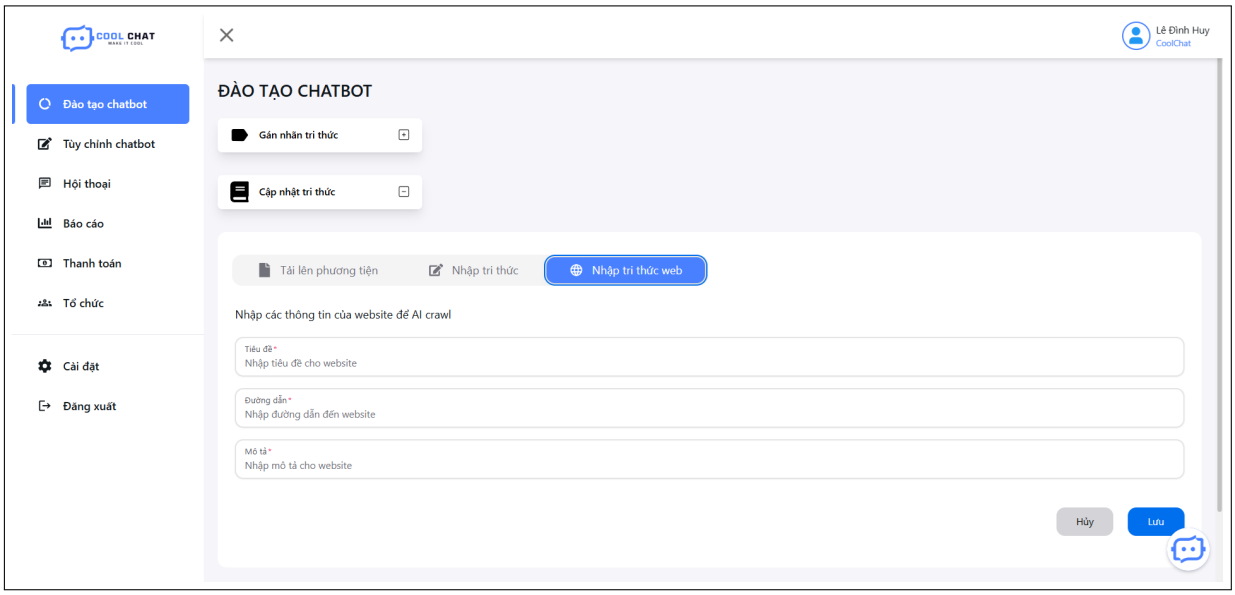
Đối với cập nhật tri thức, người dùng có thể chọn một trong ba loại tập tin, văn bản hoặc website để thêm tri thức mới.



Hình 77: *Giao diện thêm tri thức dạng tập tin*



Hình 78: *Giao diện thêm tri thức dạng văn bản*



Hình 79: *Giao diện thêm tri thức dạng website*

Ngoài ra, người dùng có thể kiểm tra hoạt động của chatbot của mình trước khi tích hợp bằng hộp chat nằm bên góc phải dưới cùng.



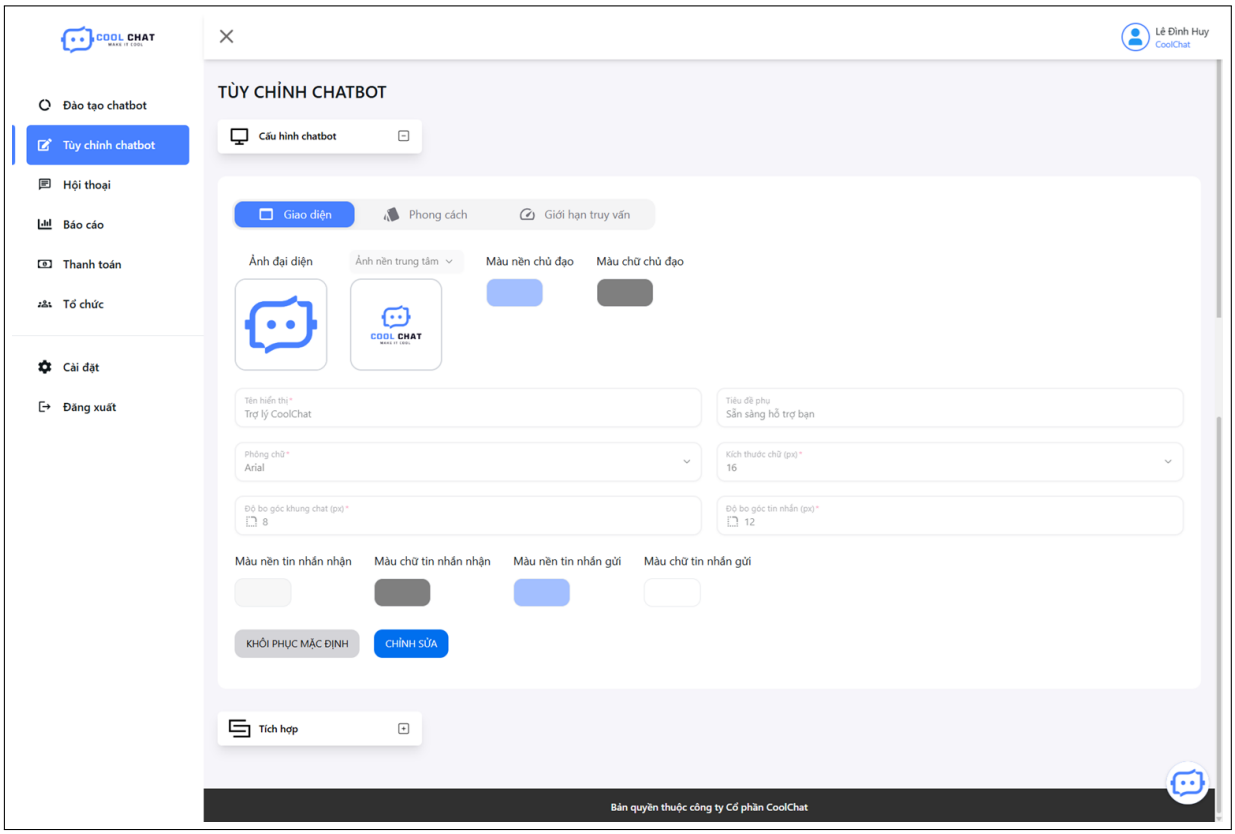
Hình 80: *Giao diện hộp chat kiểm thử*

Giao diện sẽ chứa đầy đủ các nội dung cần thiết, kể cả các lưu ý cho người dùng.

5.3.4 Trang Tùy chỉnh chatbot

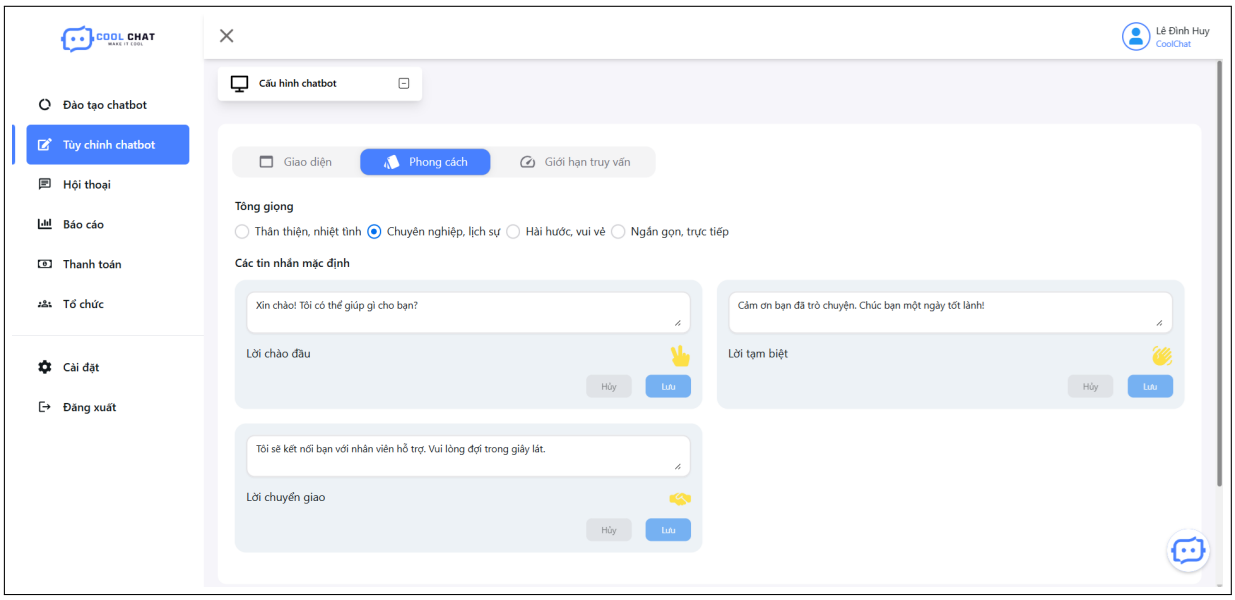
Trang Tùy chỉnh chatbot dành cho vai trò Chủ sở hữu hoặc Quản trị viên.

Người dùng có thể cấu hình giao diện chatbot của tổ chức bằng cách chỉnh sửa đa dạng các thông tin, màu sắc, kích thước, ảnh,... Việc người dùng muốn xem giao diện trước khi lưu hoặc khôi phục giao diện mặc định cũng được hệ thống hỗ trợ đầy đủ.

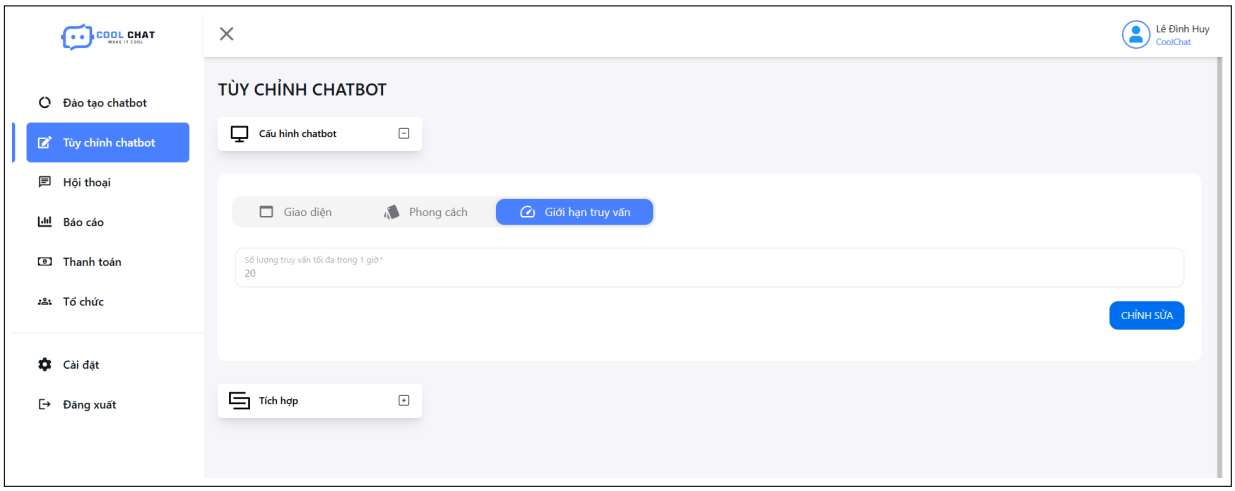


Hình 81: *Giao diện tùy chỉnh giao diện chatbot*

Thêm vào đó, người dùng còn có thể tùy chỉnh phong cách chatbot với tone giọng, các tin nhắn mặc định và cài đặt giới hạn truy vấn, giúp cho chatbot được hoạt động một cách hiệu quả, tránh các trường hợp phá hoại.

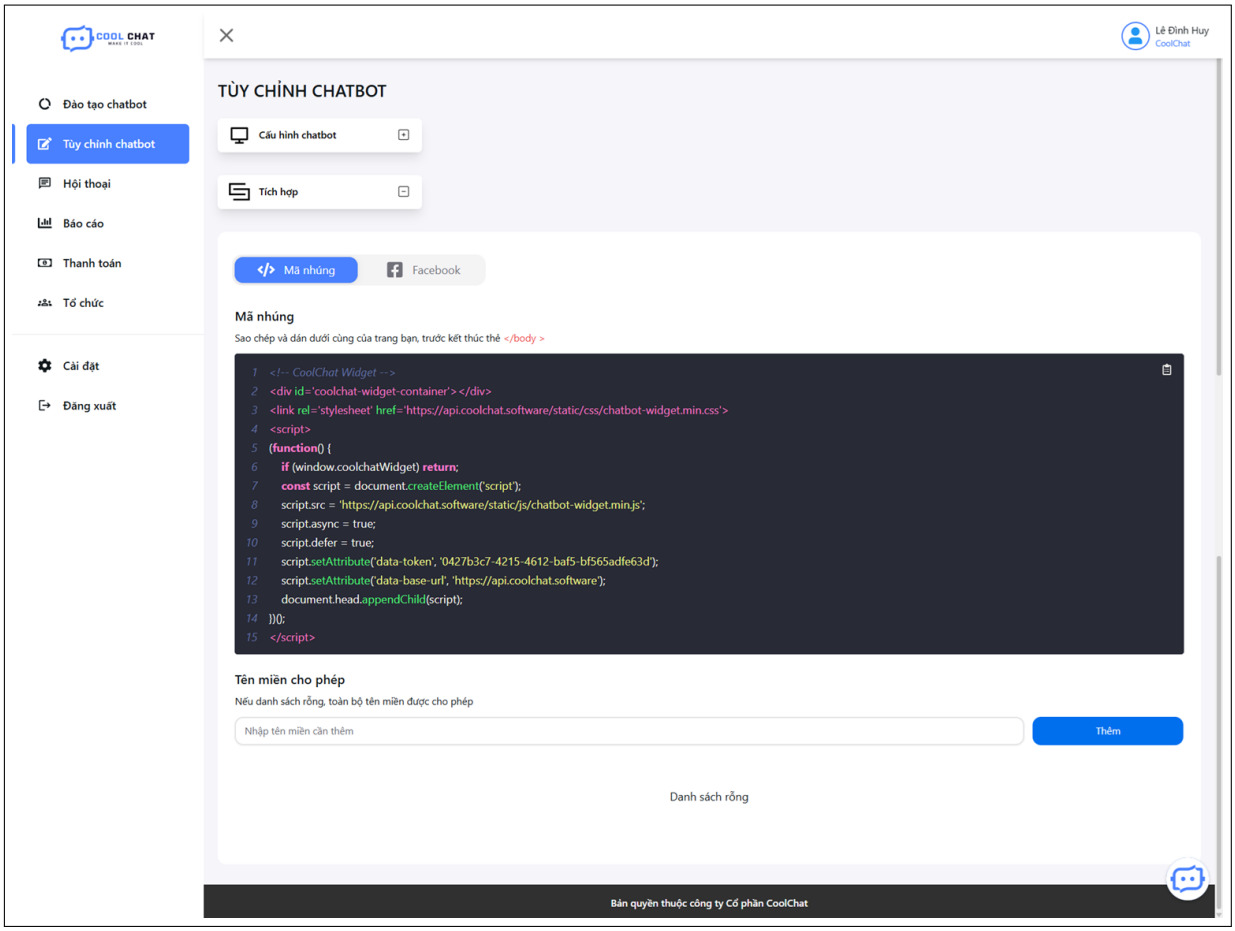


Hình 82: *Giao diện tùy chỉnh phong cách chatbot*



Hình 83: *Giao diện cài đặt giới hạn truy vấn*

Mục Tích hợp sẽ nằm trong trang Tùy chỉnh chatbot. Màn hình sẽ hiển thị đoạn mã nhúng và hướng dẫn. Để giới hạn tích hợp, người dùng có thể quản lý các tên miền cho phép hiển thị.

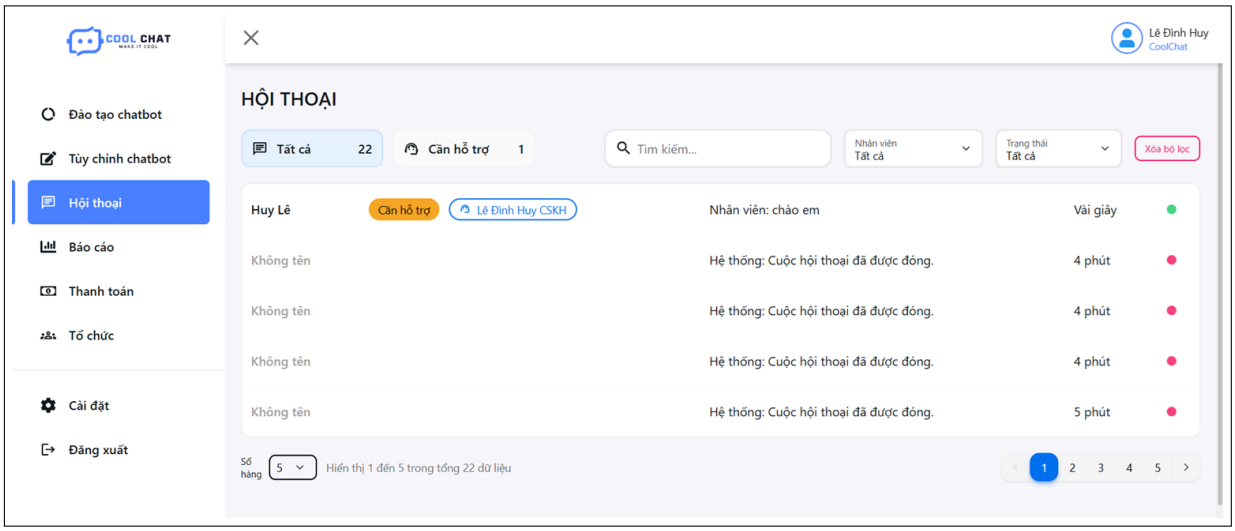


Hình 84: *Giao diện tích hợp*

5.3.5 Trang Hội thoại

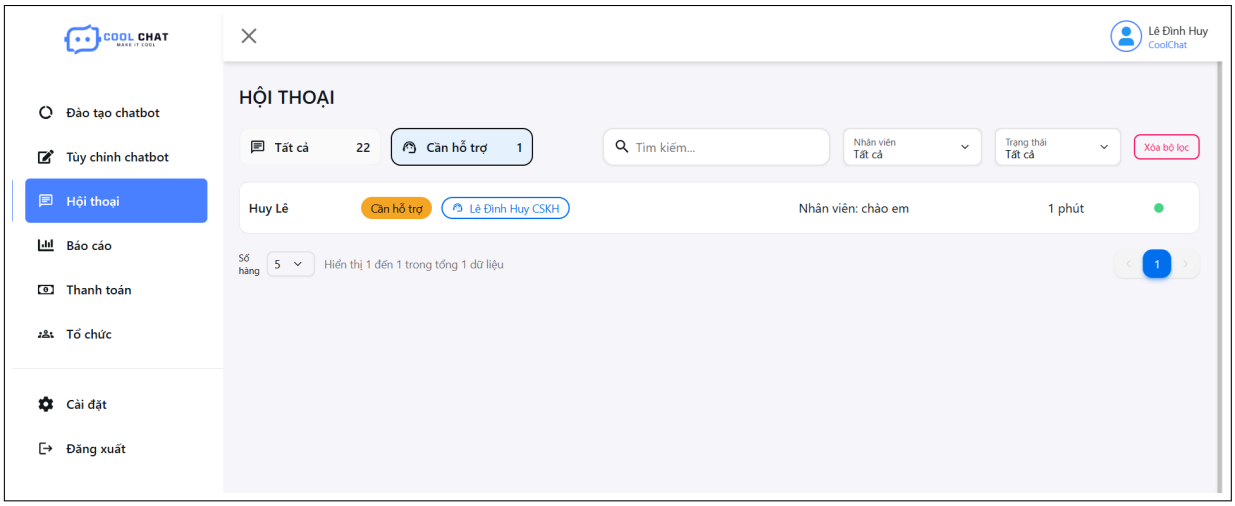
Chức năng hội thoại sẽ dành cho cả 3 role, nhưng sẽ khác nhau giữa Chủ sở hữu và Quản trị viên với Nhân viên CSKH.

Màn hình hội thoại sẽ chứa tất cả cuộc hội thoại của tổ chức dành cho Chủ sở hữu và Quản trị viên, còn Nhân viên chỉ được xem những đoạn hội thoại được giao cho mình. Người dùng có thể tìm kiếm và lọc để tìm kiếm hội thoại một cách nhanh nhất.



Hình 85: *Giao diện hội thoại*

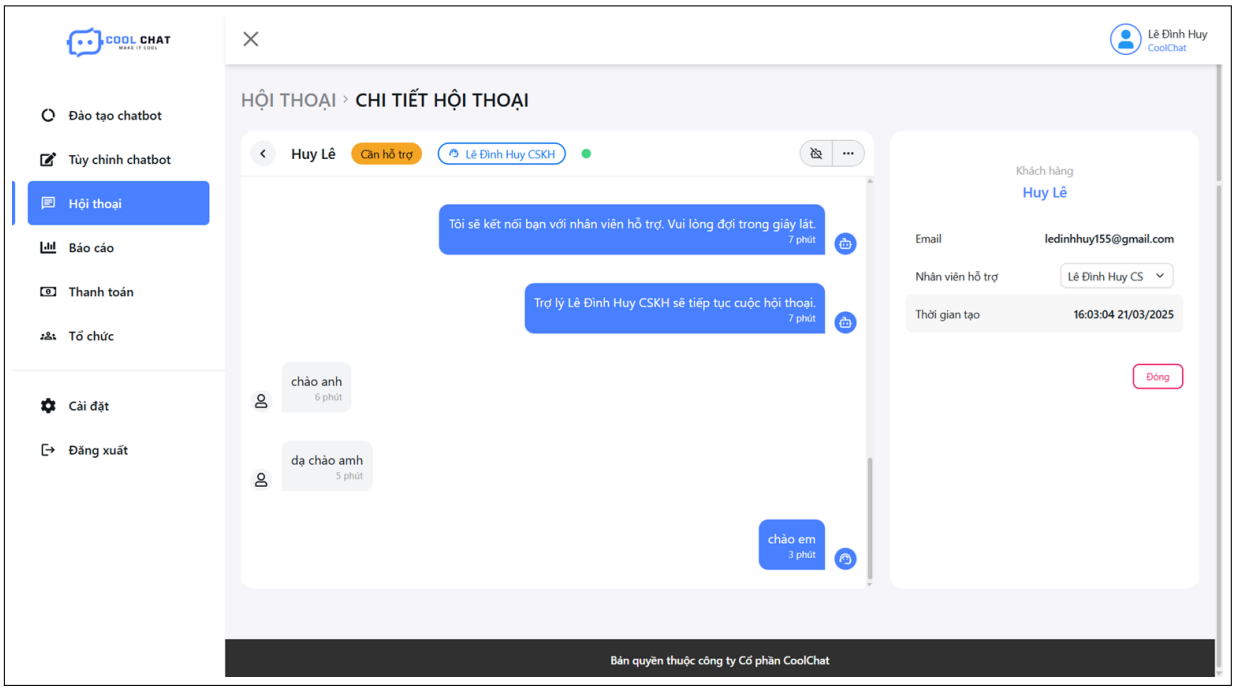
Khi chuyển sang tab Cần hộ trợ, màn hình sẽ lọc ra những hội thoại cần hỗ trợ để hỗ trợ khách hàng một cách nhanh nhất.



Hình 86: *Giao diện hội thoại cần hỗ trợ*

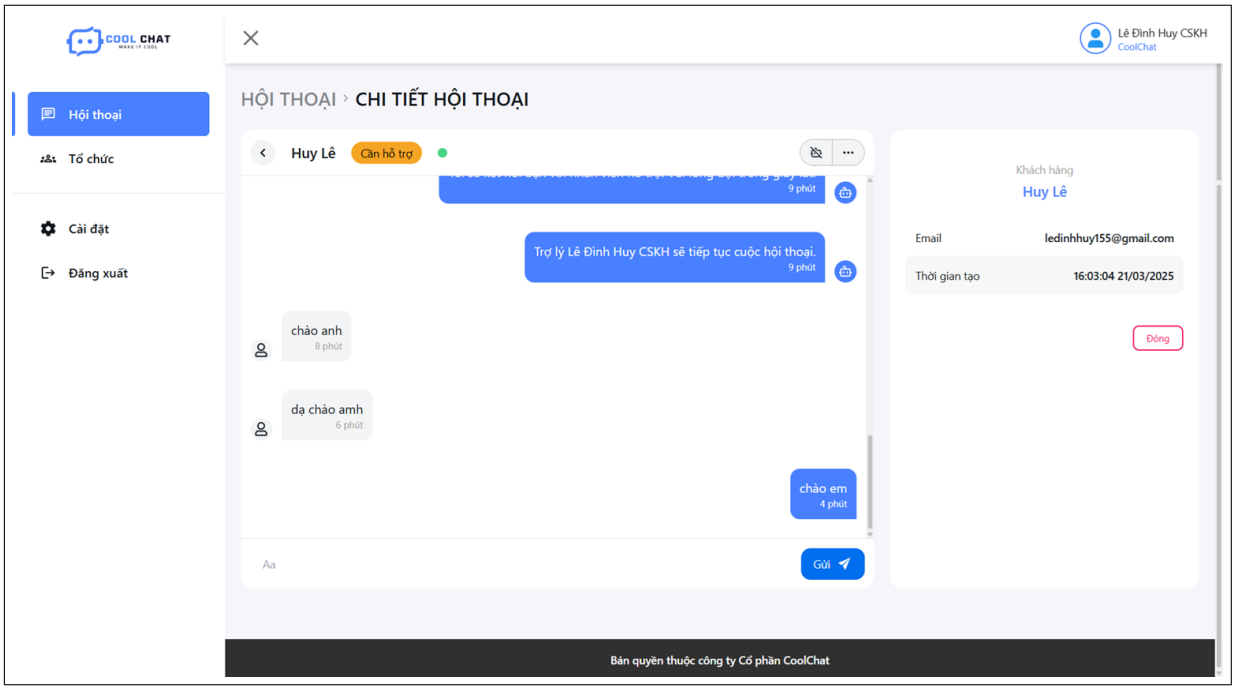
Màn hình xem chi tiết hội thoại cũng tương ứng với từng vai trò với các chức năng cho phép. Màn hình sẽ hiển thị tên khách hàng, trạng thái, tên nhân viên (nếu có), tất cả lịch sử tin nhắn của hệ thống, chatbot và nhân viên. Tin nhắn của bản thân sẽ màu xanh, còn không phải sẽ màu xám. Bên cạnh đó sẽ là thanh thông tin bên phải có thể đóng mở chứa các thông tin chi tiết.

Chủ sở hữu và quản trị viên có thể chuyển hội thoại thành tự động, gán nhân viên và đóng cuộc hội thoại.



Hình 87: *Giao diện chi tiết hội thoại cho chủ sở hữu và quản trị viên*

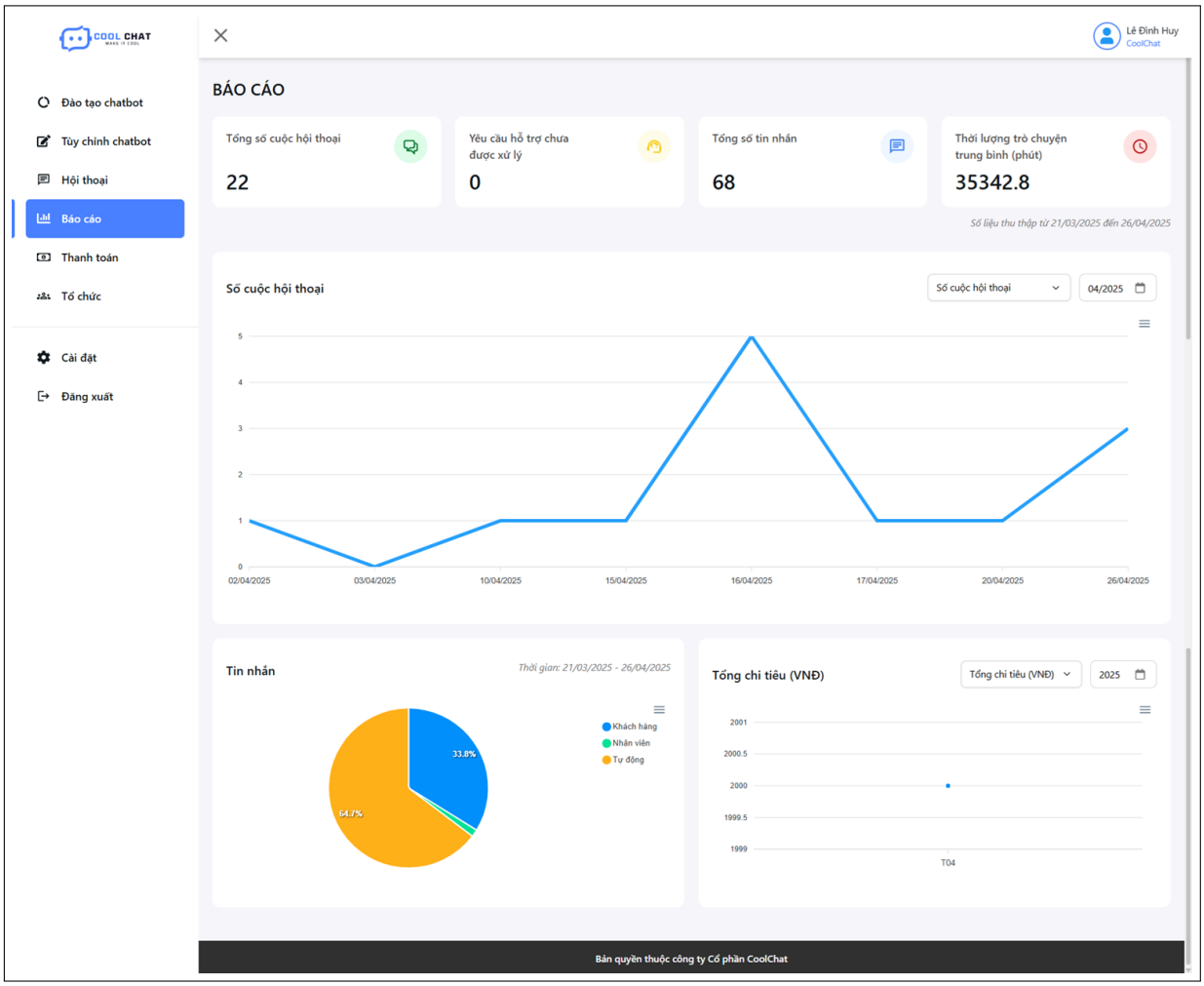
Nhân viên có thể nhắn tin với khách hàng cần hỗ trợ, chuyển hội thoại thành tự động và đóng cuộc hội thoại.



Hình 88: *Giao diện chi tiết hội thoại cho chủ sở hữu và quản trị viên*

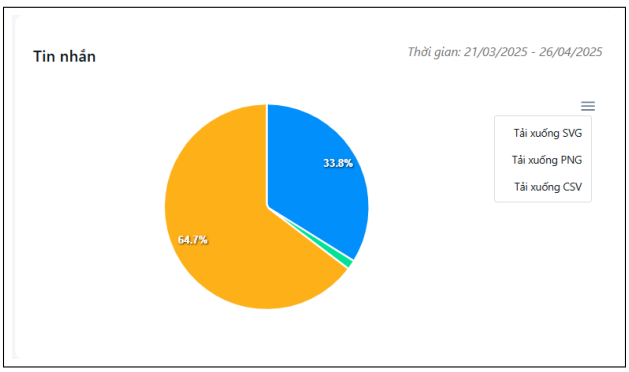
5.3.6 Trang Báo cáo

Giao diện trang Báo cáo dành cho Chủ sở hữu và Quản trị viên chỉ đơn giản là hiện thị các số liệu hoạt động của chatbot như số cuộc hội thoại, số yêu cầu hỗ trợ, số tin nhắn và thời lượng các cuộc hội thoại trung bình. Bên dưới sẽ là các biểu đồ nhằm cung cấp cho người dùng cái nhìn khách quan về quá trình sử dụng hệ thống và hoạt động của chatbot để có thể đánh giá và đưa ra các quyết định phù hợp.



Hình 89: *Giao diện báo cáo*

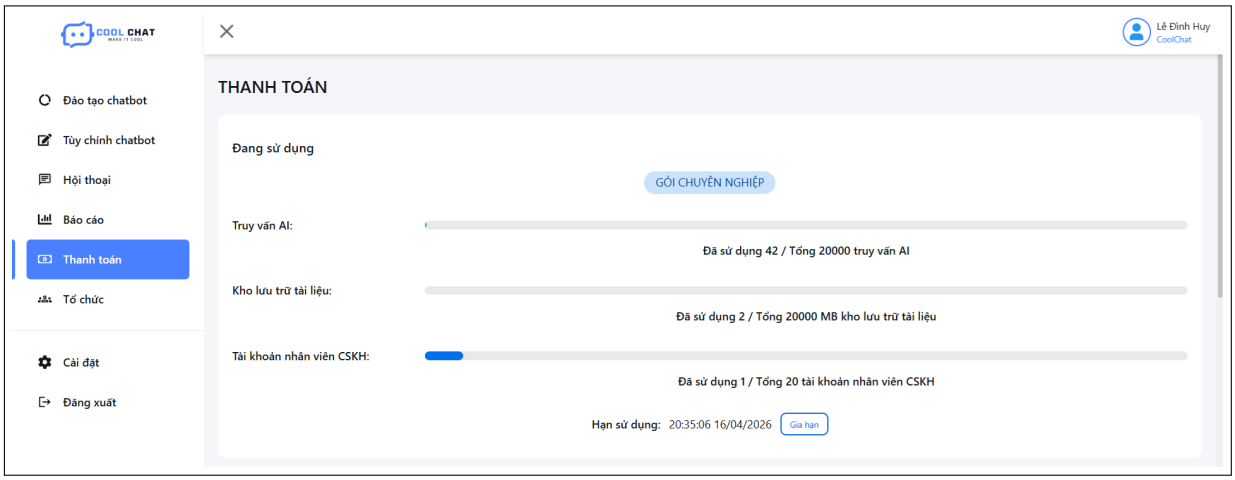
Ngoài ra, người dùng có thể lọc theo thời gian hoặc đổi thông tin khác cần hiển thị với các biểu đồ về hội thoại và thanh toán. Việc xuất biểu đồ là cho phép với cả 3 loại biểu đồ, người dùng có thể tùy chọn xuất svg, png hoặc csv.



Hình 90: *Giao diện xuất báo cáo*

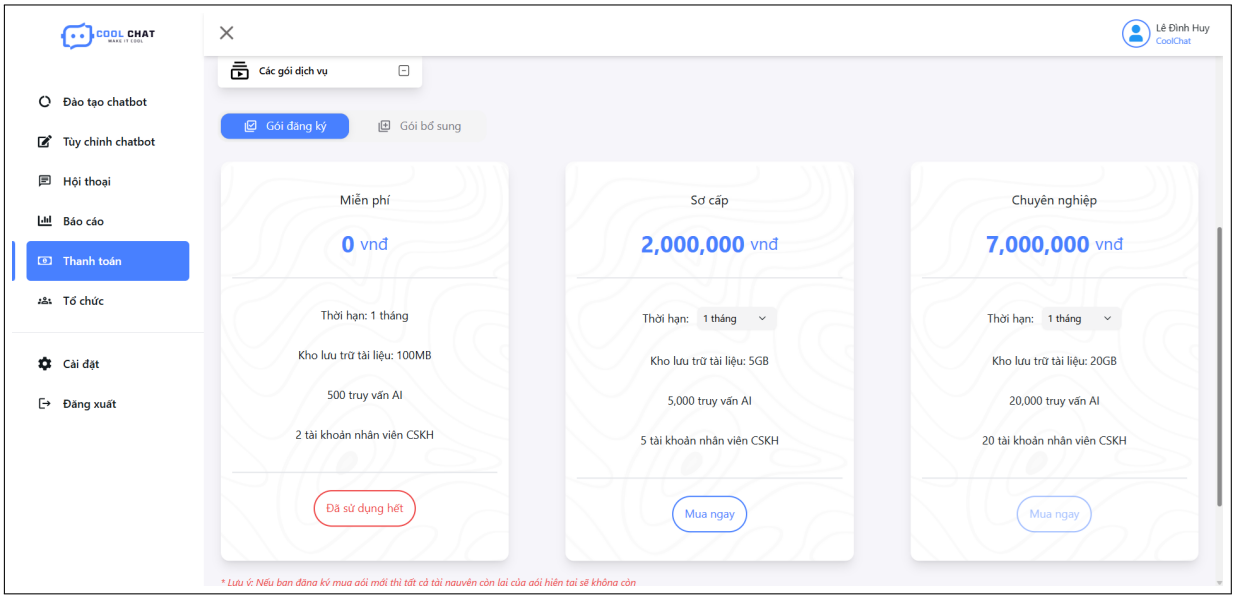
5.3.7 Trang Thanh toán

Trang Thanh toán dành cho Chủ sở hữu và Quản trị viên sẽ hiển thị thông tin của gói đăng ký hiện tại bao gồm tên gói, các mức độ sử dụng của truy vấn AI, kho lưu trữ tài liệu, tài khoản nhân viên và trạng thái gói (nếu còn hạn thì để hạn sử dụng, nếu hết hạn thì để đã hết hạn). Ngoài ra còn có nút gia hạn để người dùng có thể gia hạn thêm thời gian cho gói hiện tại.

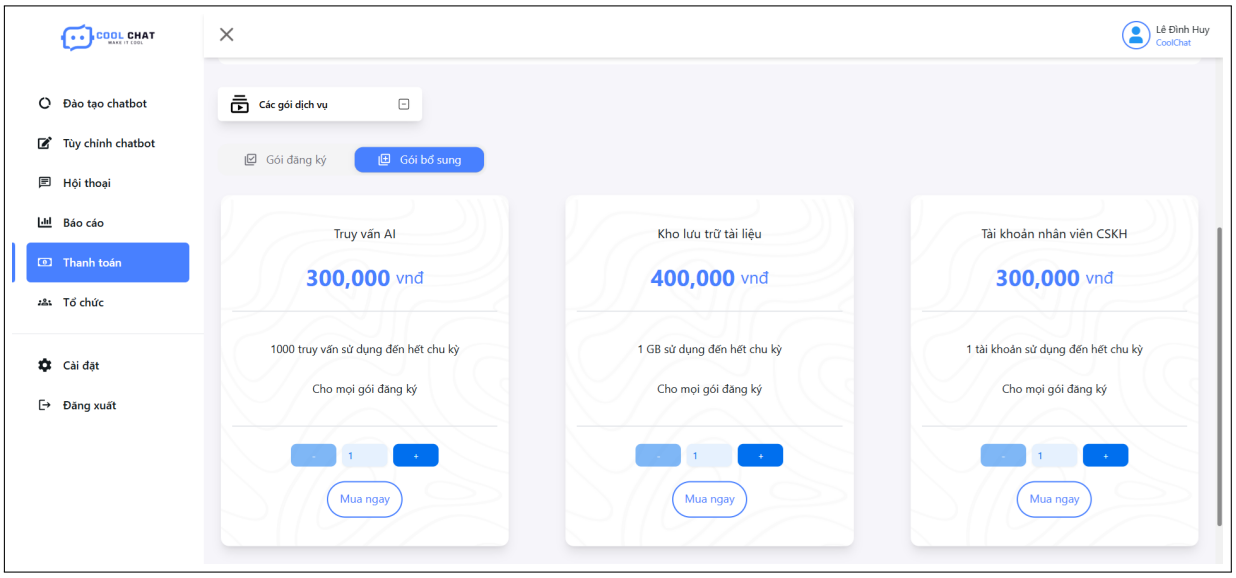


Hình 91: *Giao diện thông tin gói đăng ký hiện tại*

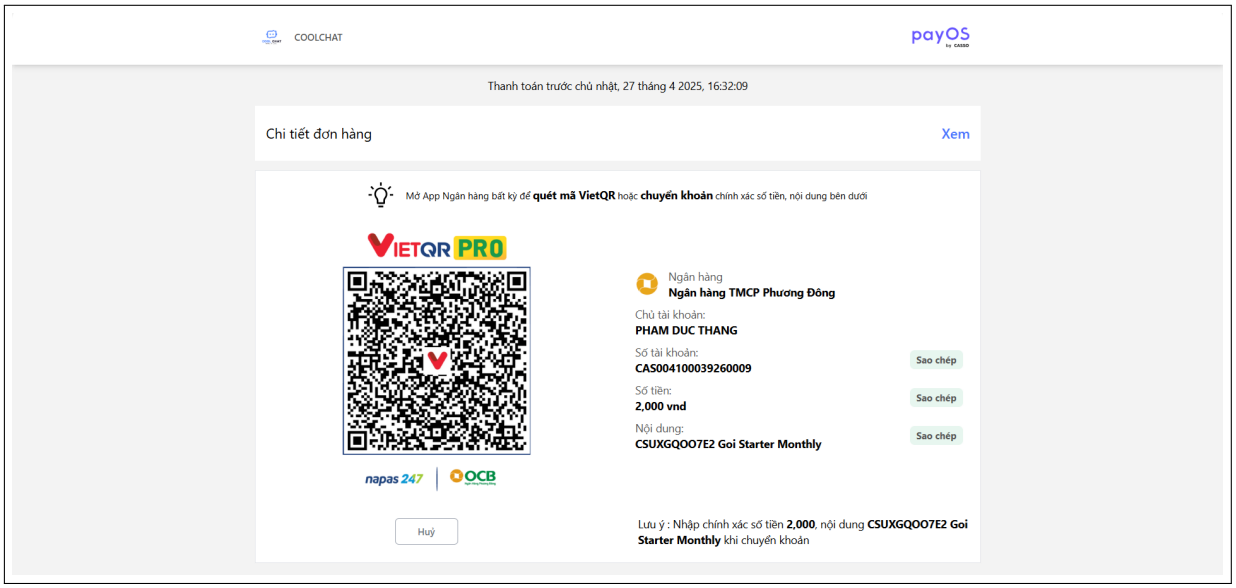
Bên dưới sẽ là thông tin của các gói đăng ký và bổ sung. Người dùng có thể chọn mua và thanh toán nếu được phép.



Hình 92: *Các gói đăng ký*

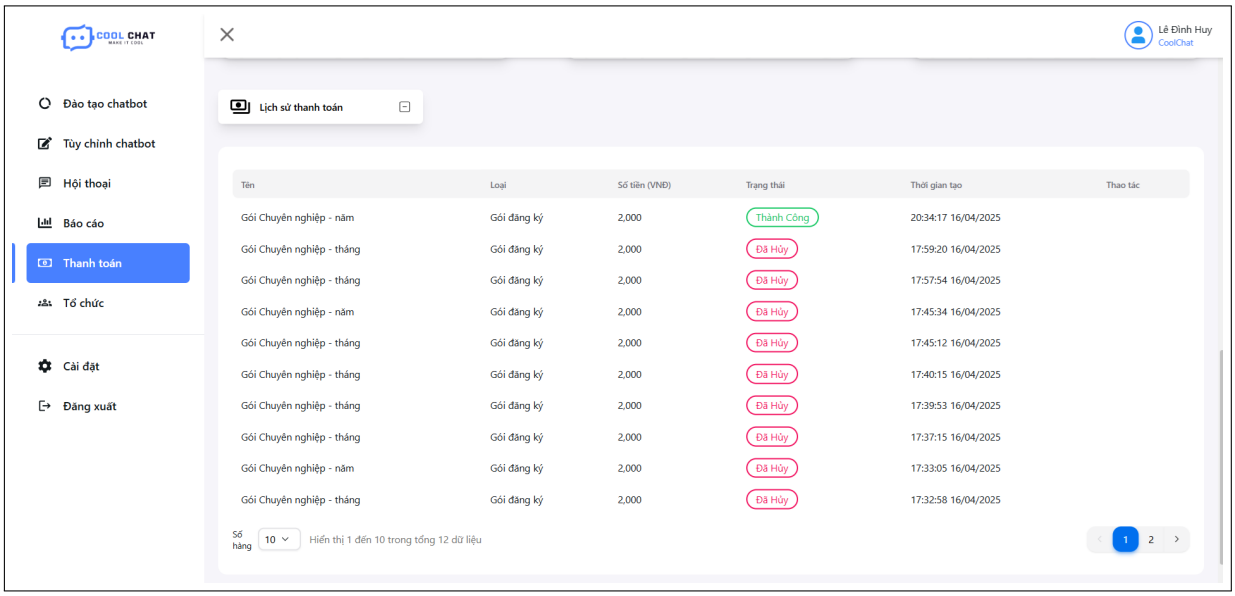


Hình 93: *Các gói bổ sung*



Hình 94: *Giao diện thanh toán với PayOS*

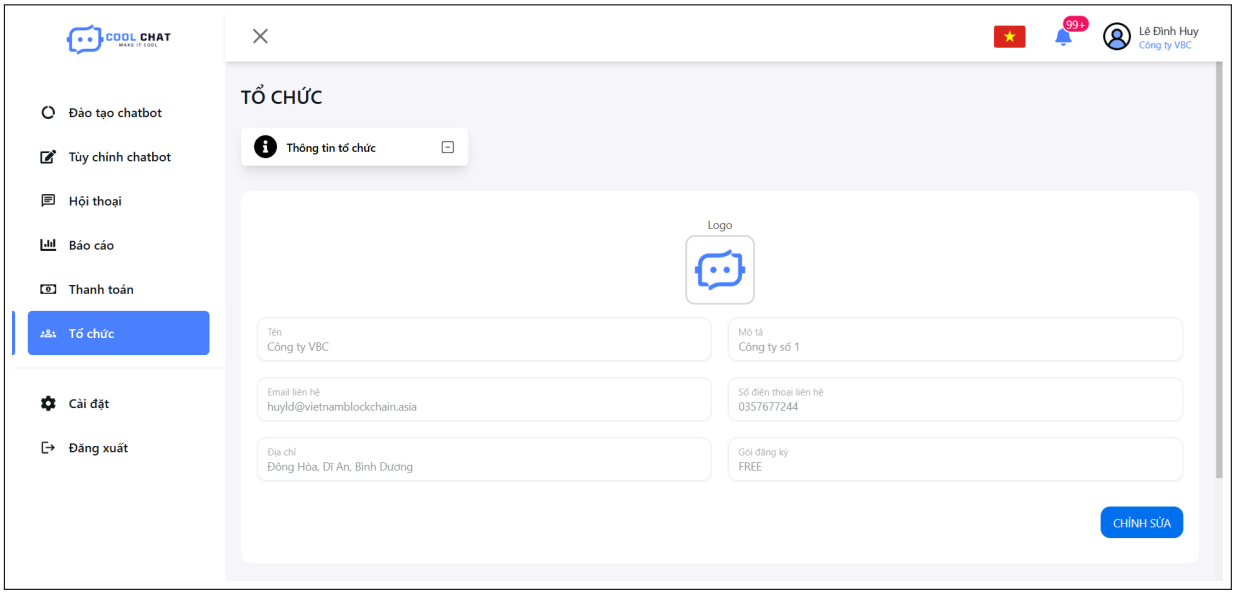
Hệ thống còn hiển thị danh sách các lịch sử thanh toán với các thông tin tương ứng, đối với hóa đơn ở trạng thái chờ, người dùng có thể tiếp tục thanh toán hoặc hủy gói.



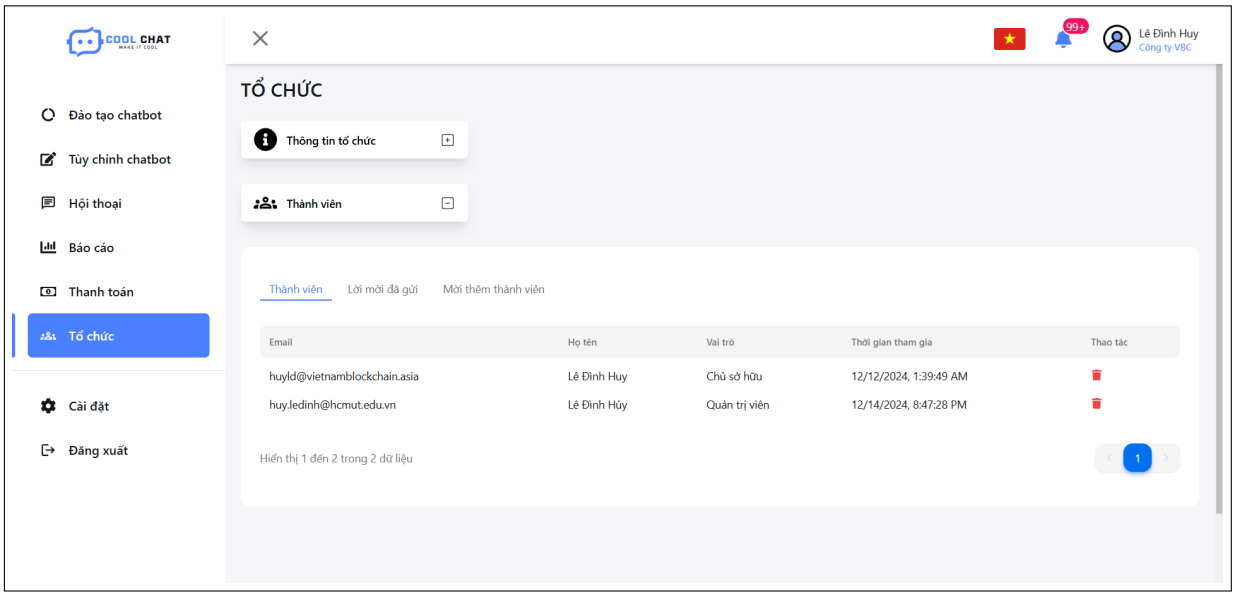
Hình 95: *Giao diện thanh toán được hiện thực*

5.3.8 Trang Tổ chức

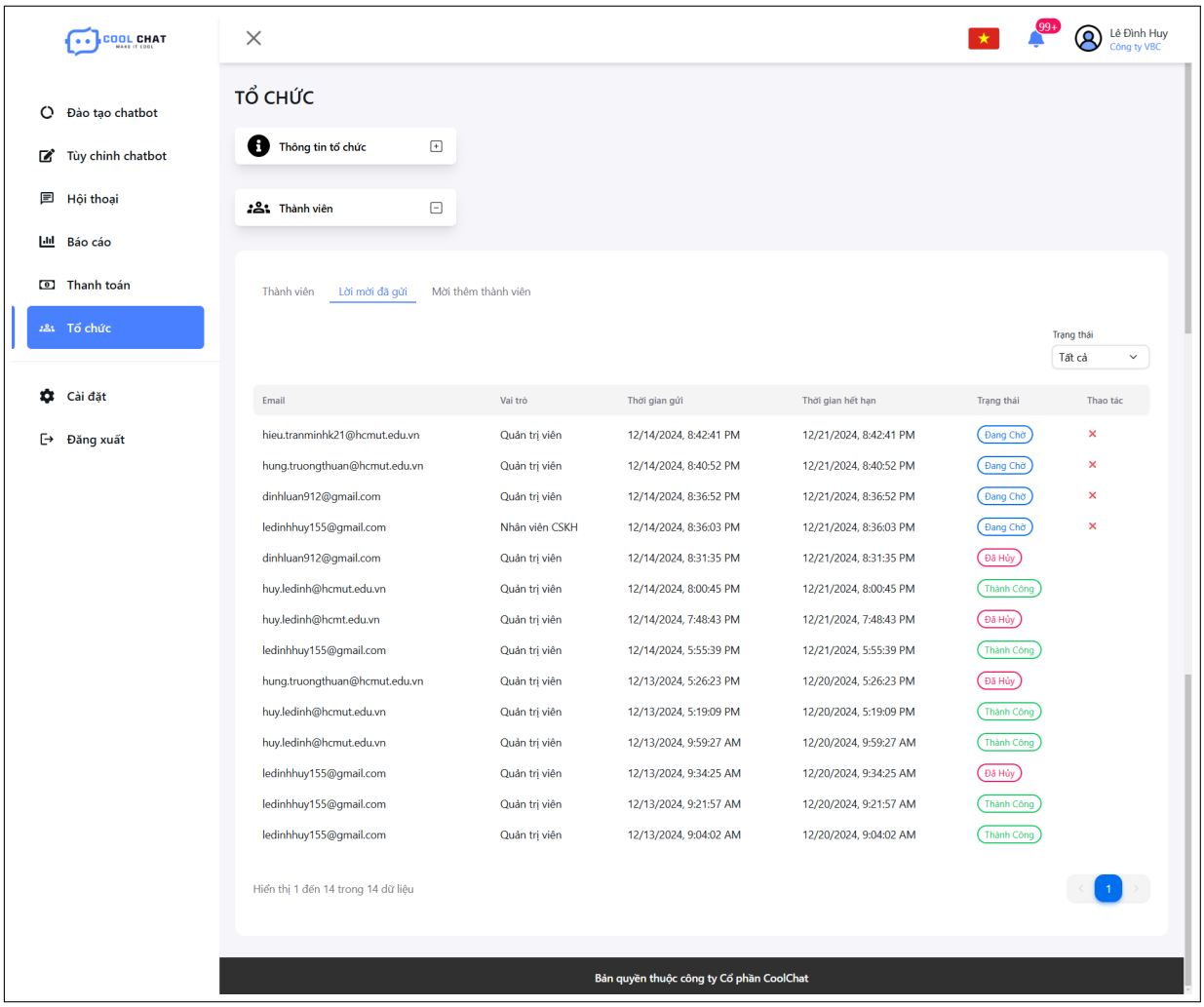
Trang tổ chức là trang cho phép quản lý tổ chức đối với chủ sở hữu tổ chức và chỉ hiển thị thông tin tổ chức đối với quản trị viên và nhân viên. Chủ sở hữu có thể chỉnh sửa thông tin tổ chức, xem danh sách thành viên, danh sách lời mời và mời thêm thành viên vào tổ chức.



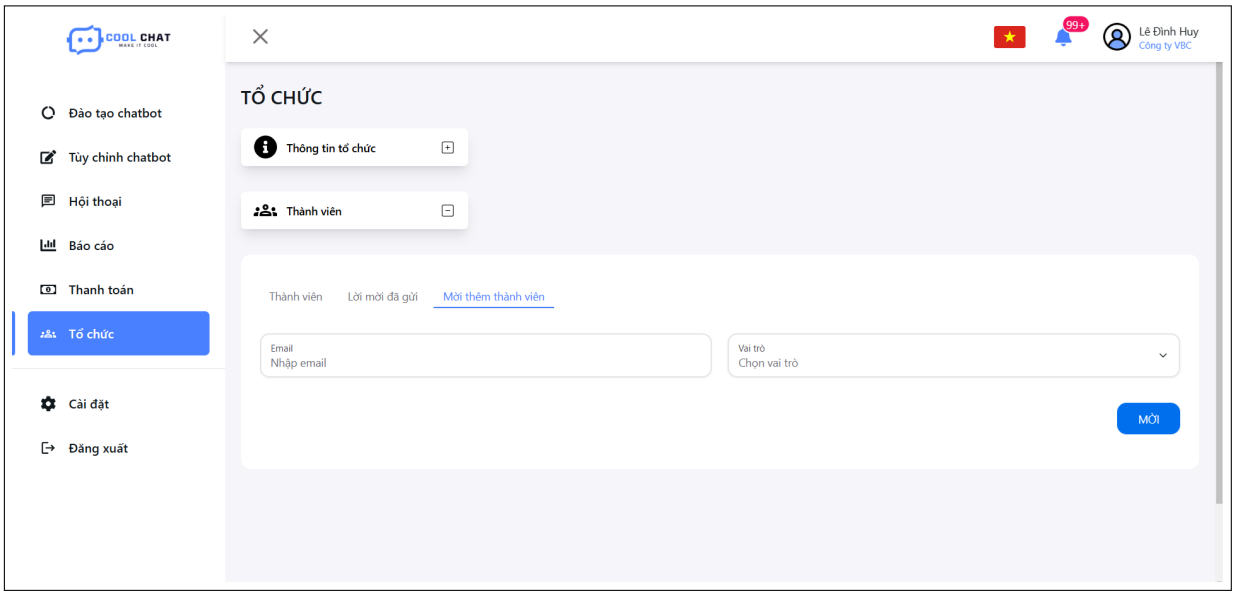
Hình 96: *Giao diện thông tin tổ chức*



Hình 97: *Giao diện danh sách thành viên tổ chức*



Hình 98: *Giao diện danh sách lời mời tham gia tổ chức*

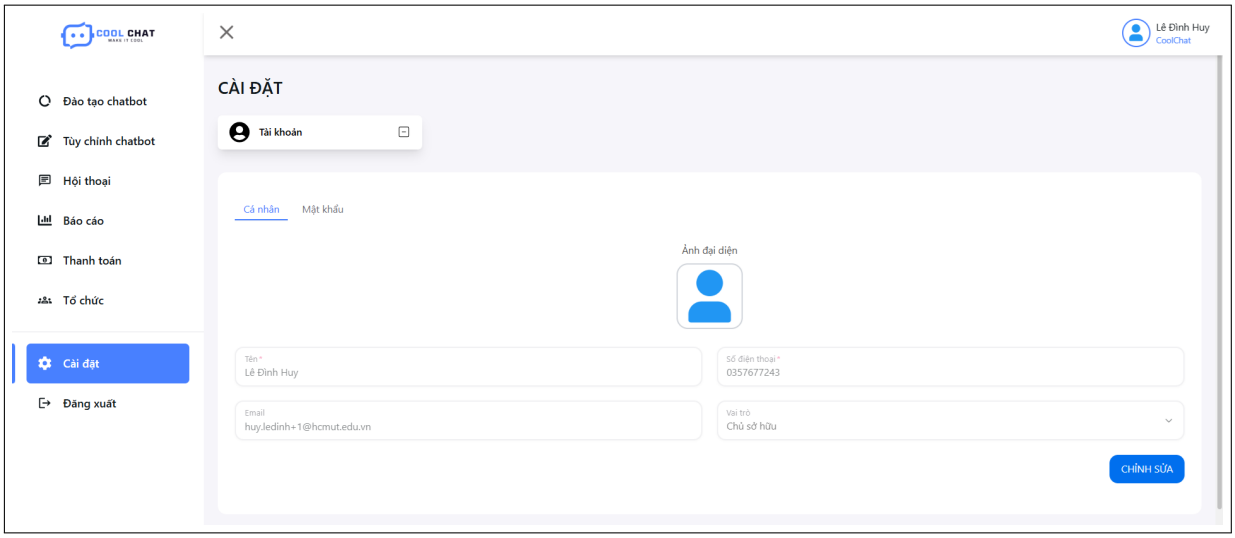


Hình 99: *Giao diện mời thêm thành viên vào tổ chức*

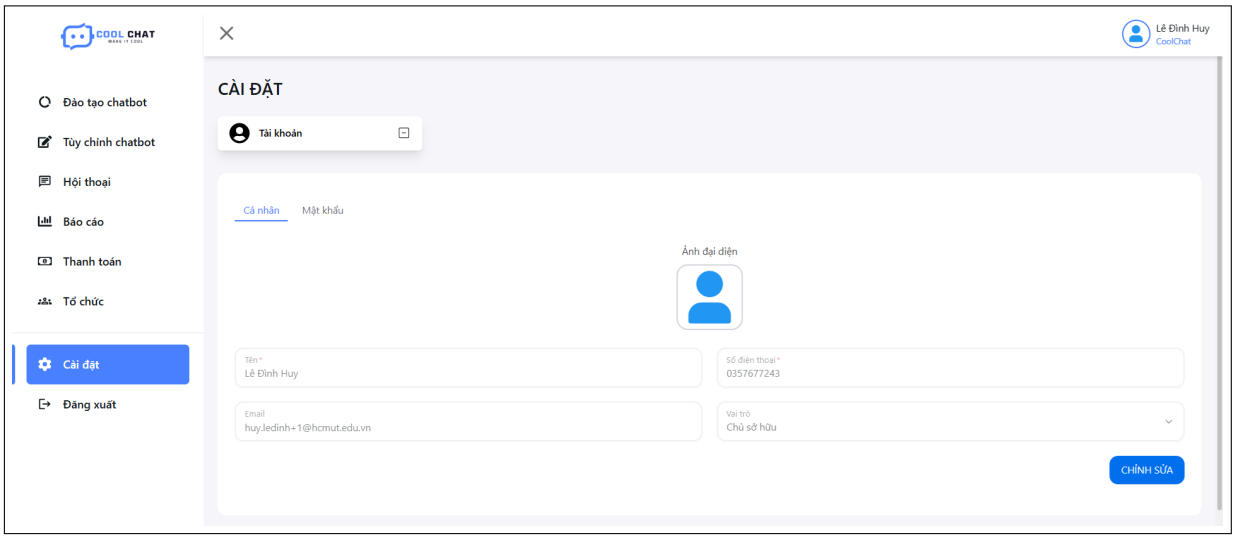
Khi chủ sở hữu điền thông tin và chọn gửi lời mời đến thành viên mới, thành viên mới sẽ nhận được một email tham gia tổ chức bao gồm các thông tin liên quan, mã giới thiệu và nút đăng ký dẫn đến trang đăng ký của hệ thống.

5.3.9 Trang Cài đặt

Mọi người dùng đều có thể chỉnh sửa thông tin cá nhân hoặc đổi mật khẩu ở trang Cài đặt.



Hình 100: *Giao diện chỉnh sửa thông tin cá nhân*



Hình 101: *Giao diện đổi mật khẩu*

5.3.10 Giao diện hộp chat dành cho khách hàng

Mỗi hộp chat của mỗi tổ chức sẽ được tùy chỉnh khác nhau, tuy nhiên đều có những chức năng chung và bố cục giống nhau. Bên dưới là giao diện bình thường được công ty CoolChat cấu hình và giao diện gửi yêu cầu hỗ trợ.



Hình 102: *Giao diện chatbot phía khách hàng*

6 Kiểm thử và đánh giá

6.1 Kiểm thử hệ thống

6.1.1 Kiểm thử API

6.1.2 Tổng quan về công cụ và cấu trúc kiểm thử

Công cụ kiểm thử Quá trình kiểm thử cho backend của hệ thống được xây dựng dựa trên nền tảng khung kiểm thử tích hợp sẵn có của Django, cụ thể là các lớp cơ sở như `django.test.TestCase` và `django.test.APITestCase`. Tuy nhiên, để tăng cường hiệu quả và tính linh hoạt, nhóm đã lựa chọn tích hợp và thực thi các bộ kiểm thử thông qua thư viện `pytest`, kết hợp với plugin `pytest-django`.

`Pytest-django` đóng vai trò cầu nối quan trọng, cho phép `pytest` tương tác và quản lý ngữ cảnh của ứng dụng Django, bao gồm cơ sở dữ liệu, cấu hình và các thành phần khác trong quá trình chạy kiểm thử. Việc áp dụng `pytest` mang lại nhiều ưu điểm đáng kể so với trình chạy kiểm thử mặc định của Django:

- Cú pháp viết kiểm thử ngắn gọn hơn
- Cơ chế fixture mạnh mẽ giúp quản lý tài nguyên và trạng thái kiểm thử hiệu quả
- Khả năng tự động phát hiện các bài kiểm thử linh hoạt

Cấu trúc tổ chức kiểm thử Về mặt tổ chức, cấu trúc các bài kiểm thử tuân thủ chặt chẽ theo nguyên tắc mô-đun hóa được khuyến nghị bởi Django. Cụ thể, các bộ kiểm thử được phân chia và đặt trong một thư mục `tests` riêng biệt bên trong mỗi ứng dụng của dự án, bao gồm `authentication`, `chat`, `chatbot`, `document`, `organization`, và `subscription`.

Bên trong mỗi thư mục `tests`, các tệp kiểm thử được đặt tên theo quy ước, phản ánh thành phần cụ thể đang được kiểm thử:

- `test_models.py`: Tập trung vào việc xác minh logic và ràng buộc của các mô hình dữ liệu
- `test_serializers.py`: Đảm bảo tính đúng đắn của quá trình tuần tự hóa (serialization) và giải tuần tự hóa (deserialization) dữ liệu
- `test_views.py`: Chịu trách nhiệm kiểm tra logic xử lý yêu cầu và phản hồi của các giao diện lập trình ứng dụng (API views)
- `test_utils.py`: Dùng để kiểm thử các hàm tiện ích phụ trợ

Cách tổ chức này không chỉ giúp tăng cường tính rõ ràng, dễ quản lý mà còn thuận lợi cho việc bảo trì và mở rộng bộ kiểm thử trong tương lai khi hệ thống phát triển.

6.1.3 Các phương pháp kiểm thử phần mềm

Trong quá trình phát triển hệ thống chatbot, các phương pháp kiểm thử phần mềm đã được áp dụng nhằm đảm bảo chất lượng, độ tin cậy và tính chính xác của ứng dụng. Các phương pháp chính bao gồm kiểm thử đơn vị và kiểm thử tích hợp.

Kiểm thử đơn vị (Unit Testing) Kiểm thử đơn vị tập trung vào việc xác minh tính chính xác của từng thành phần phần mềm nhỏ nhất (đơn vị) một cách riêng lẻ, cô lập khỏi các thành phần khác. Trong phạm vi dự án, kiểm thử đơn vị được thực hiện chủ yếu trên các lớp đối tượng sau:

- *Models*: Xác minh logic nghiệp vụ, quy tắc xác thực dữ liệu (validation logic) và các phương thức tùy chỉnh được định nghĩa bên trong các model (ví dụ: phương thức `is_valid()` trong model `EmailVerificationToken` và `PasswordResetToken`). Công cụ `django.test.TestCase` thường được sử dụng cho mục đích này.
- *Serializers*: Kiểm tra logic xác thực dữ liệu đầu vào, tính chính xác của quá trình tuần tự hóa và giải tuần tự hóa, đảm bảo dữ liệu được chuyển đổi phù hợp giữa tầng trình diễn (client) và tầng dữ liệu. `django.test.TestCase` cũng là công cụ phổ biến cho loại kiểm thử này.
- *Hàm tiện ích và Dịch vụ (Utils/Services)*: Kiểm tra các hàm tiện ích và các khối logic nghiệp vụ được tách biệt khỏi controllers, ví dụ như các hàm thực hiện chức năng gửi email (`send_verification_email`, `send_password_reset_email`). Để cô lập đơn vị đang được kiểm thử khỏi các thành phần phụ thuộc bên ngoài (như hệ thống gửi email thực tế), kỹ thuật mocking với `unittest.mock.patch` được áp dụng.

Kiểm thử tích hợp (Integration Testing) Kiểm thử tích hợp nhằm mục đích xác minh sự tương tác và phối hợp hoạt động chính xác giữa các thành phần khác nhau của hệ thống. Trong dự án này, kiểm thử tích hợp đặc biệt tập trung vào việc kiểm tra các API endpoints. Sử dụng `rest_framework.test.APITestCase` và `rest_framework.test.APIClient`, các bài kiểm thử tích hợp mô phỏng các yêu cầu HTTP (GET, POST, PATCH, DELETE) gửi đến các API endpoints. Quá trình kiểm thử tập trung đánh giá các khía cạnh sau:

- Mã trạng thái HTTP (`response.status_code`) được trả về có phù hợp với kịch bản yêu cầu hay không.
- Nội dung dữ liệu trong phần thân phản hồi (`response.data`) có chính xác và đầy đủ theo thiết kế API hay không.

- Trạng thái của hệ thống trong cơ sở dữ liệu có được cập nhật đúng đắn sau khi thực thi yêu cầu hay không (ví dụ: một đối tượng người dùng mới được tạo, một token xác thực được đánh dấu là đã sử dụng, trạng thái gói dịch vụ của người dùng thay đổi).
- Các dịch vụ hoặc hàm liên quan có được gọi thực thi đúng như mong đợi hay không (ví dụ: sử dụng mock object để xác minh hàm gửi email `mock_send_email.assert_called_once()`).
- Cơ chế phân quyền (permissions) hoạt động chính xác, đảm bảo người dùng chỉ có thể truy cập tài nguyên và thực hiện hành động phù hợp với vai trò được gán (ví dụ: phân biệt quyền giữa chủ sở hữu tổ chức (owner), quản trị viên (admin) và thành viên (member)).

6.1.4 Kết quả Kiểm thử và Ví dụ

Kết quả kiểm thử Kết quả thực thi bộ kiểm thử tự động cho các ứng dụng (apps) thành phần của hệ thống được tổng hợp trong Bảng 42.

Bảng 33: *Thống kê kết quả kiểm thử theo ứng dụng*

Ứng dụng (Application)	Tổng số	Thành công (Pass)	Thất bại (Fail)
`authentication`	38	38	0
`chat`	28	28	0
`chatbot`	32	32	0
`document`	46	46	0
`organization`	52	52	0
`reports`	29	29	0
`subscription`	59	59	0
Tổng cộng	**284**	**284**	**0**

Ví dụ Kiểm thử Tiêu biểu

Kiểm thử lớp Model Kiểm thử ở tầng Model đóng vai trò then chốt trong việc đảm bảo tính toàn vẹn dữ liệu và sự đúng đắn của logic nghiệp vụ được định nghĩa trong các lớp này. Hoạt động kiểm thử này bao gồm việc xác minh các quy tắc xác thực dữ liệu nếu có, kiểm tra hoạt động của các phương thức tùy chỉnh như phương thức biểu diễn chuỗi (`__str__`), và đảm bảo các thuộc tính của đối tượng được quản lý chính xác trong các tình huống khác nhau như khởi tạo, cập nhật hay truy vấn.

Ví dụ, xem xét việc kiểm thử cho model `Conversation`, đại diện cho một cuộc hội thoại trong hệ thống. Các bài kiểm thử cần đảm bảo rằng một đối tượng `Conversation` có thể được khởi tạo chính xác với các giá trị thuộc tính ban đầu, phương thức biểu diễn chuỗi trả về thông tin nhận dạng hữu ích, và logic sắp xếp hoạt động đúng như định nghĩa.

```python
import pytest
from django.utils import timezone
from coolchat_be.organizations.tests.factories import OrganizationFactory
from coolchat_be.users.tests.factories import UserFactory
from ..models import Conversation
from .factories import ConversationFactory

# pytest.mark.django_db is implied or configured in pytest.ini
class TestConversationModel:
    """Test suite for the Conversation model."""

    def test_create_conversation(self, db): # db fixture from pytest-django
        """Test creating a conversation object."""
        # Use factories to create dependent data
        organization = OrganizationFactory()
        user = UserFactory()

        # Initialize a Conversation object
        conversation = Conversation.objects.create(
            organization=organization,
            customer_email="test@example.com",
            customer_name="Test Customer",
            customer_session_id="test-session-id",
            agent=user, # Can assign an agent when creating
            mode="AI",  # Assuming there's a mode attribute
```

```
        )

        # Verify the basic attributes of the newly created object
        assert conversation.id is not None
        assert conversation.organization == organization
        assert conversation.customer_email == "test@example.com"
        assert conversation.customer_name == "Test Customer"
        assert conversation.agent == user
        assert conversation.is_active is True # Default value assumption
        assert conversation.mode == "AI"

    # (Other test cases such as test_conversation_string_representation,
    #  test_conversation_ordering can be added here...)
```

Listing 1: Ví dụ kiểm thử việc khởi tạo đối tượng Conversation (coolchat_be/conversations/tests/test_models.py)

Đoạn mã trên minh họa việc sử dụng các factory (`OrganizationFactory`, `UserFactory`) để tạo các đối tượng liên quan cần thiết trước khi khởi tạo đối tượng `Conversation`. Sau đó, các câu lệnh `assert` được dùng để kiểm tra xem các thuộc tính của đối tượng mới tạo có đúng với giá trị đã cung cấp hoặc giá trị mặc định hay không. Các bài kiểm thử khác trong cùng lớp `TestConversationModel` sẽ tiếp tục kiểm tra các khía cạnh khác như phương thức `__str__` và logic sắp xếp dữ liệu. Cách tiếp cận này giúp đảm bảo model hoạt động ổn định và chính xác theo yêu cầu thiết kế.

Kiểm thử lớp Serializer Quá trình kiểm thử tập trung vào việc: (1) Xác minh dữ liệu đầu ra khi tuần tự hóa, đảm bảo đủ trường và đúng giá trị, kể cả các trường tùy chỉnh. (2) Kiểm tra khả năng xử lý và xác thực dữ liệu đầu vào khi giải tuần tự hóa.

Đoạn mã sau minh họa kiểm thử quá trình tuần tự hóa cho `DocumentSerializer`, kiểm tra các trường thông thường và trường tùy chỉnh như định dạng kích thước file.

```
import pytest
from rest_framework.test import APIRequestFactory
from coolchat_be.documents.models import Document
from coolchat_be.documents.serializers import DocumentSerializer
from .factories import DocumentFactory, URLDocumentFactory

@pytest.mark.django_db
class TestDocumentSerializer:
    def test_document_serialization(self):
        """Test the serialization of a Document object."""
        document = DocumentFactory()
        request = APIRequestFactory().get("/") # Context if needed
        serializer = DocumentSerializer(document, context={"request": request})
        data = serializer.data

        # Check basic fields
        assert data["id"] == document.id
        assert data["filename"] == document.filename
        # Check custom fields
        assert "download_url" in data
        assert "file_size_display" in data

    def test_file_size_display(self):
        """Test the file size display format (custom field)."""
        doc1 = DocumentFactory(size=500)
        assert DocumentSerializer(doc1).data["file_size_display"] == "500.0 B"
        doc2 = DocumentFactory(size=1500)
        assert DocumentSerializer(doc2).data["file_size_display"] == "1.5 KB"
        doc3 = DocumentFactory(size=1500000)
        assert DocumentSerializer(doc3).data["file_size_display"] == "1.4 MB" # Need to
            check rounding logic

    def test_url_download_url(self):
        """Test the download_url field for URL type documents."""
        url_doc = URLDocumentFactory(url="https://example.com/doc")
        assert DocumentSerializer(url_doc).data["download_url"] == "https://example.com/
            doc"
```

Listing 2: Ví dụ kiểm thử lớp DocumentSerializer (coolchat_be/documents/tests/test_serializers.py)

Trong Liệt kê 6, các đối tượng mẫu được tạo bằng factory, sau đó `serializer.data` được dùng để kiểm tra kết quả tuần tự hóa. Các bài kiểm thử riêng biệt như `test_file_size_display` đặc biệt hữu ích để xác minh logic của các trường tính toán hoặc tùy chỉnh trong serializer.

Kiểm thử API (API Endpoint Testing) Kiểm thử các API endpoints là một phần quan trọng trong kiểm thử tích hợp, nhằm xác minh tính đúng đắn của giao diện giao tiếp giữa frontend và backend. Quá trình này bao gồm việc mô phỏng các yêu cầu HTTP đến các endpoint đã định nghĩa và kiểm tra không chỉ phản hồi trực tiếp mà còn cả các tác động lên trạng thái hệ thống.

Một ví dụ điển hình là kiểm thử endpoint đăng ký người dùng mới (`/api/v1/auth/register/`). Sử dụng các công cụ như `rest_framework.test.APIClient` (được cung cấp bởi `APITestCase` hoặc thông qua `pytest-django`), một yêu cầu HTTP POST được gửi đến endpoint này, mang theo dữ liệu đăng ký hợp lệ. Bài kiểm thử sẽ xác minh nhiều khía cạnh: mã trạng thái HTTP trả về (ví dụ: `201 CREATED`), sự thay đổi trong cơ sở dữ liệu (ví dụ: một đối tượng `User` mới được tạo, một `EmailVerificationToken` tương ứng được sinh ra), và việc các dịch vụ bên ngoài được gọi đúng như mong đợi (ví dụ: hàm gửi email xác thực). Kỹ thuật mocking, chẳng hạn sử dụng decorator `@patch` của thư viện `unittest.mock`, thường được áp dụng để cô lập và kiểm tra việc gọi các thành phần phụ thuộc này mà không cần thực thi chúng thực sự.

```python
from unittest.mock import patch
from django.urls import reverse
from rest_framework import status
from rest_framework.test import APITestCase
from coolchat_be.users.models import User
from coolchat_be.authentication.models import EmailVerificationToken
# (Other imports if needed)

class RegisterViewTests(APITestCase):
    @patch("coolchat_be.authentication.views.send_verification_email")
    def test_register_create_organization(self, mock_send_email):
        """Test user registration with creating a new organization."""
        url = reverse("api:authentication:register")
        data = {
            "email": "test@example.com",
            "password": "StrongPassword123!",
            "password2": "StrongPassword123!",
            "organization_action": "create",
            "name": "Test Organization",
            "description": "Test Organization Description",
            "contact_email": "contact@example.com",
            "user_name": "Test User",
            "user_phone": "+84900000123",
        }

        response = self.client.post(url, data, format="json")
        self.assertEqual(response.status_code, status.HTTP_201_CREATED)

        # Check that the user was created in the database
        self.assertTrue(User.objects.filter(email="test@example.com").exists())
        user = User.objects.get(email="test@example.com")

        # Check that the email verification token was created
        self.assertTrue(EmailVerificationToken.objects.filter(user=user).exists())

        # Check that the email sending function was called (through mock)
        mock_send_email.assert_called_once()
```

Listing 3: Ví dụ kiểm thử API đăng ký (coolchat_be/authentication/tests/test_views.py)

Tương tự, việc kiểm thử các endpoint truy vấn dữ liệu (ví dụ: sử dụng phương thức GET) cũng rất quan trọng. Chẳng hạn, khi kiểm thử endpoint lấy danh sách các cuộc hội thoại (`/api/v1/chat/conversations/`), bài kiểm thử cần đảm bảo rằng API trả về đúng mã trạng thái (`200 OK`), cấu trúc dữ liệu phản hồi chính xác, và quan trọng hơn là tuân thủ các quy tắc về phân quyền và tách biệt dữ liệu. Cụ thể, API chỉ nên trả về các cuộc hội thoại thuộc về tổ chức (organization) mà người dùng đã được xác thực là thành viên, đảm bảo tính riêng tư và bảo mật dữ liệu giữa các tổ chức khác nhau. Việc sử dụng các thư viện như `factory-boy` để tạo dữ liệu kiểm thử đa dạng giúp quá trình kiểm thử này trở nên hiệu quả và đáng tin cậy.

```python
import pytest
from django.urls import reverse
from rest_framework import status
from rest_framework.test import APIClient
from coolchat_be.users.tests.factories import UserFactory
from coolchat_be.organizations.tests.factories import OrganizationFactory,
    OrganizationMemberFactory
from .factories import ConversationFactory

# Using pytest fixture and marker (can be configured in pytest.ini)
@pytest.mark.django_db
```

```python
class TestConversationViewSet:
    @pytest.fixture
    def api_client(self):
        """Fixture providing an authenticated APIClient."""
        client = APIClient()
        # Create necessary data: user, organization, and membership
        self.user = UserFactory()
        self.organization = OrganizationFactory()
        OrganizationMemberFactory(user=self.user, organization=self.organization)
        # Mock authentication for client
        client.force_authenticate(user=self.user)
        return client

    def test_list_conversations(self, api_client):
        """Test retrieving the list of conversations."""
        # Create 3 conversations belonging to the user's organization
        ConversationFactory.create_batch(
            3,
            organization=self.organization,
            # with_messages=True, # Optional to create related messages
        )
        # Create 1 conversation belonging to another organization (should not be returned)
        other_org = OrganizationFactory()
        ConversationFactory(organization=other_org)

        url = reverse("api:chat:conversation-list") # Ensure correct URL name
        response = api_client.get(url)

        # Verify results
        assert response.status_code == status.HTTP_200_OK
        # Check that only 3 conversations from the correct organization are returned
        # Assuming API returns paginated data according to DRF defaults
        assert len(response.data["results"]) == 3
        # Check other information in the response if needed
        assert "assigned_count" in response.data # Example of checking a meta data field
        # (Other assertions...)

        # Check the content of each conversation if needed
        for conv_data in response.data["results"]:
            assert conv_data["organization"]["id"] == str(self.organization.id) # Check organization_id
```

Listing 4: Ví dụ kiểm thử API lấy danh sách Conversation (coolchat_be/chat/tests/test_views.py)

6.1.5 Kiểm thử yêu cầu chức năng

Để kiểm tra tính đúng đắn của hệ thống so với các yêu cầu đặt ra ban đầu cho ứng dụng web, nhóm thực hiện kiểm tra các chức năng cụ thể theo các bộ testcase tương ứng sau đây:

- Màn hình Đào tạo chatbot

STT	Tên chức năng	Các bước thực hiện	Kết quả mong muốn	Kết quả
1	Hiển thị danh sách các tri thức	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở trang "Đào tạo chatbot" Các bước thực hiện: Không	Quan sát được trang đầu của danh sách các tri thức theo kích thước trang ban đầu là 10 với các thông tin cụ thể tương ứng cho từng tri thức	Đạt
2	Tìm kiếm và lọc tri thức	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở trang "Đào tạo chatbot" Các bước thực hiện: 1. Tìm kiếm theo tên tri thức 2. Áp dụng các bộ lọc	Quan sát được trang đầu của danh sách các tri thức tương ứng với bộ lọc theo kích thước trang ban đầu là 10 và các thông tin cụ thể tương ứng cho từng tri thức	Đạt

3	Thêm tri thức dạng tập tin	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở trang "Đào tạo chatbot" Các bước thực hiện: 1. Chọn Cập nhật tri thức 2. Chọn Chọn từ máy tính và tải lên các tập tin theo đúng loại tệp và kích thước cho phép 3. Chọn Lưu	1. Màn hình hiển thị mục Tải lên phương tiện 2. Màn hình hiển thị các tập tin đã tải lên 3. Màn hình hiển thị thông báo thành công và các tri thức mới dạng tập tin được thêm vào	Đạt
4	Thêm tri thức dạng văn bản	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở trang "Đào tạo chatbot" Các bước thực hiện: 1. Chọn Cập nhật tri thức 2. Chọn Nhập tri thức 3. Nhập văn bản và chọn Lưu	1. Màn hình hiển thị mục Tải lên phương tiện 2. Màn hình hiển thị mục Nhập tri thức 3. Màn hình hiển thị thông báo thành công và tri thức mới dạng văn bản được thêm vào	Đạt
5	Thêm tri thức dạng website	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở trang "Đào tạo chatbot" Các bước thực hiện: 1. Chọn Cập nhật tri thức 2. Chọn Nhập tri thức web 3. Điền các thông tin và chọn Lưu	1. Màn hình hiển thị mục Tải lên phương tiện 2. Màn hình hiển thị mục Nhập tri thức web 3. Màn hình hiển thị thông báo thành công và tri thức mới dạng website được thêm vào	Đạt
6	Đào tạo tri thức	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở trang "Đào tạo chatbot" – Tồn tại sự thay đổi tri thức (thêm, xóa, chỉnh sửa) Các bước thực hiện: 1. Chọn Lưu và Đào tạo	1. Màn hình thông báo Đang chờ đào tạo, tất cả các tri thức bị xóa tạm thời sẽ được xóa, còn lại chuyển sang trạng thái chờ đến khi thành công, tất cả tri thức có trạng thái đã đào tạo và hiển thị thông báo thành công	Đạt
7	Chỉnh sửa thông tin tri thức	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở trang "Đào tạo chatbot" – Tồn tại ít nhất một tri thức Các bước thực hiện: 1. Chọn Chỉnh sửa một tri thức 2. Điền các thông tin và chọn Lưu	1. Màn hình hiển thị cửa sổ chỉnh sửa 2. Màn hình hiển thị thông báo thành công và tri thức được cập nhật	Đạt

8	Xóa tri thức	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở trang "Đào tạo chatbot" – Tồn tại ít nhất một tri thức Các bước thực hiện: 1. Chọn Xóa một tri thức 2. Chọn Xác nhận	1. Màn hình hiển thị cửa sổ xác nhận 2. Màn hình hiển thị thông báo thành công, nếu tri thức có trạng thái Chưa đào tạo thì bị xóa ngay, còn lại sẽ được đưa vào danh sách xóa tạm thời và sẽ được xóa khi chọn Lưu và Đào tạo	Đạt
9	Khôi phục tri thức	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở trang "Đào tạo chatbot" – Tồn tại ít nhất một tri thức nằm trong danh sách xóa tạm thời Các bước thực hiện: 1. Chọn Khôi phục một tri thức bị xóa tạm thời 2. Chọn Xác nhận	1. Màn hình hiển thị cửa sổ xác nhận 2. Màn hình hiển thị thông báo thành công, tri thức được khôi phục	Đạt
10	Chỉnh sửa độ ưu tiên	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở trang "Đào tạo chatbot" – Tồn tại ít nhất một tri thức Các bước thực hiện: 1. Thay đổi độ ưu tiên của một hoặc nhiều tri thức 2. Chọn Lưu và đào tạo	1. Màn hình cho phép chọn nút đào tạo 2. Màn hình thông báo Cập nhật thành công và Đang chờ đào tạo, tất cả các tri thức bị xóa tạm thời sẽ được xóa, còn lại chuyển sang trạng thái chờ đến khi thành công, tất cả tri thức có trạng thái đã đào tạo và hiển thị thông báo thành công	Đạt
11	Tải xuống tri thức dạng tập tin hoặc văn bản	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở trang "Đào tạo chatbot" – Tồn tại ít nhất một tri thức dạng tập tin hoặc văn bản Các bước thực hiện: 1. Chọn Tải xuống một tri thức dạng tập tin/văn bản	1. Đối với loại tệp txt và pdf, màn hình hiển thị bản xem trước, còn loại tệp docx sẽ được tải xuống thiết bị ngay	Đạt
12	Xem thông tin tri thức dạng website	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở trang "Đào tạo chatbot" – Tồn tại ít nhất một tri thức dạng website Các bước thực hiện: 1. Chọn Xem tri thức dạng website	1. Màn hình hiển thị thông tin của tri thức đó	Đạt

13	Chat thử với chatbot	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở trang "Đào tạo chatbot" Các bước thực hiện: 1. Chọn vào nút mở hộp chat 2. Nhập nội dung tin nhắn vào khung chat và chọn Gửi	1. Màn hình hiển thị hộp chat 2. Màn hình hiển thị tin nhắn đã gửi, đồng thời khởi chạy và gửi lại tin nhắn tự động từ chatbot	Đạt

Bảng 34: *Bảng testcase cho màn hình Đào tạo chatbot*

- Màn hình Tùy chỉnh chatbot

STT	Tên chức năng	Các bước thực hiện	Kết quả mong muốn	Kết quả
1	Chỉnh sửa giao diện chatbot	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở trang "Tùy chỉnh chatbot" Các bước thực hiện: 1. Chọn Chỉnh sửa 2. Thay đổi các trường để tạo giao diện mới và chọn Xem trước 3. Chọn Lưu	1. Màn hình cho phép tùy chỉnh các trường 2. Màn hình bật lên hộp chat với giao diện đúng với các trường đã chỉnh sửa 3. Màn hình hiển thị thông báo thành công và giao diện chatbot mới được cập nhật	Đạt
2	Khôi phục mặc định giao diện chatbot	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở trang "Tùy chỉnh chatbot" – Giao diện chatbot đã được chỉnh sửa Các bước thực hiện: 1. Chọn Khôi phục mặc định 2. Chọn Xác nhận	1. Màn hình hiển thị cửa sổ xác nhận thao tác 2. Màn hình hiển thị thông báo thành công và giao diện chatbot được khôi phục lại mặc định	Đạt
3	Chỉnh sửa tone giọng	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở trang "Tùy chỉnh chatbot" Các bước thực hiện: 1. Chọn Phong cách 2. Chọn Tone giọng khác	1. Màn hình hiển thị tab Phong cách 2. Màn hình hiển thị thông báo thành công và tone giọng được cập nhật	Đạt
4	Chỉnh sửa các tin nhắn mặc định	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở trang "Tùy chỉnh chatbot" Các bước thực hiện: 1. Chọn Phong cách 2. Chọn một loại tin nhắn mặc định, thay đổi và chọn Lưu	1. Màn hình hiển thị tab Phong cách 2. Màn hình hiển thị thông báo thành công và tin nhắn mặc định tương ứng được cập nhật.	Đạt

5	Tích hợp bằng mã nhúng	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở trang "Tùy chỉnh chatbot" Các bước thực hiện: 1. Chọn Tích hợp 2. Làm theo hướng dẫn để tích hợp vào hệ thống website	1. Màn hình hiển thị mục Tích hợp chứa đoạn mã nhúng 2. Hộp chat tương ứng hiển thị ở phía website được thêm	Đạt
6	Thêm tên miền cho phép	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở trang "Tùy chỉnh chatbot" Các bước thực hiện: 1. Chọn Tích hợp 2. Nhập tên miền cho phép và chọn Thêm	1. Màn hình hiển thị mục Tích hợp 2. Màn hình hiển thị thông báo thành công và tên miền mới được thêm vào, hộp chat chỉ hiển thị ở các tên miền trong danh sách.	Đạt
7	Xóa tên miền cho phép	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở mục Tích hợp trong Tùy chỉnh chatbot – Danh sách tên miền cho phép không rỗng Các bước thực hiện: 1. Chọn Xóa một tên miền 2. Chọn Xác nhận	1. Màn hình hiển thị cửa sổ xác nhận xóa 2. Màn hình hiển thị thông báo thành công và tên miền được chọn bị xóa khỏi danh sách	Đạt
8	Chỉnh sửa giới hạn truy vấn	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở mục Tích hợp trong Tùy chỉnh chatbot Các bước thực hiện: 1. Chọn Giới hạn truy vấn 2. Chọn Chỉnh sửa và thay đổi giá trị và chọn Lưu	1. Màn hình hiển thị tab Giới hạn truy vấn với số truy vấn tối đa hiện tại 2. Màn hình hiển thị thông báo thành công và giá trị mới được cập nhật	Đạt

Bảng 35: *Bảng testcase cho màn hình Tùy chỉnh chatbot*

- Màn hình Hội thoại

STT	Tên chức năng	Các bước thực hiện	Kết quả mong muốn	Kết quả
1	Hiển thị danh sách các hội thoại	Tiền điều kiện: – Đã đăng nhập vào hệ thống – Đang ở trang "Hội thoại" Các bước thực hiện: Không	Quan sát được trang đầu của danh sách các hội thoại theo kích thước trang ban đầu là 10 với các thông tin cụ thể tương ứng cho từng hội thoại, tuy nhiên với vai trò Nhân viên chỉ hiện các hội thoại đã hoặc đang được gán cho bản thân	Đạt

2	Tìm kiếm và lọc hội thoại	Tiền điều kiện: – Đã đăng nhập vào hệ thống – Đang ở trang "Hội thoại" Các bước thực hiện: 1. Tìm kiếm theo tên hội thoại 2. Áp dụng các bộ lọc	Quan sát được trang đầu của danh sách các hội thoại tương ứng với bộ lọc theo kích thước trang ban đầu là 10 và các thông tin cụ thể tương ứng cho từng hội thoại	Đạt
3	Xem danh sách hội thoại cần hỗ trợ	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở trang "Hội thoại" Các bước thực hiện: 1. Chọn Cần hỗ trợ	Quan sát được trang đầu của danh sách các hội thoại cần hỗ trợ theo kích thước trang ban đầu là 10 và các thông tin cụ thể tương ứng cho từng hội thoại	Đạt
4	Xem chi tiết hội thoại	Tiền điều kiện: – Đã đăng nhập vào hệ thống – Đang ở trang "Hội thoại" – Danh sách hội thoại không rỗng Các bước thực hiện: 1. Chọn vào một hội thoại cần xem	Quan sát được các thông tin của cuộc hội thoại của cũng lịch sử toàn bộ tin nhắn	Đạt
5	Nhắn tin với khách hàng	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Nhân viên CSKH – Đang ở trang chi tiết của một cuộc hội thoại cần hỗ trợ Các bước thực hiện: 1. Nhập tin nhắn dưới khung chat và chọn Gửi	1. Ngay lập tức tin nhắn được gửi và hiển thị lên màn hình ở cả nhân viên và khách hàng	Đạt
6	Bật chế độ tự động chat	Tiền điều kiện: – Đã đăng nhập vào hệ thống – Đang ở trang chi tiết của một cuộc hội thoại cần hỗ trợ Các bước thực hiện: 1. Chọn Bật chế độ chat tự động	1. Màn hình hiển thị thông báo thành công, hội thoại hiển thị tin nhắn thông báo cho khách hàng và cuộc hội thoại đã chuyển sang chế độ tự động chat	Đạt
7	Thay đổi nhân viên CSKH	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở trang chi tiết của một cuộc hội thoại cần hỗ trợ và được gán cho một nhân viên – Tổ chức có ít nhất 2 nhân viên Các bước thực hiện: 1. Chọn vào dấu ba chấm 2. Chọn một nhân viên khác để thay đổi 3. Chọn Xác nhận	1. Màn hình hiển thị thanh thông tin bên phải 2. Màn hình hiển thị cửa sổ xác nhận 3. Màn hình hiển thị thông báo thành công và cuộc hội thoại được gán cho nhân viên mới, đồng thời tin nhắn thông báo được gửi cho khách hàng	Đạt

8	Đóng cuộc hội thoại	Tiền điều kiện: – Đã đăng nhập vào hệ thống – Đang ở trang chi tiết của một cuộc hội thoại Các bước thực hiện: 1. Chọn vào dấu ba chấm 2. Chọn Đóng 3. Chọn Xác nhận	1. Màn hình hiển thị thanh thông tin bên phải 2. Màn hình hiển thị cửa sổ xác nhận 3. Màn hình hiển thị thông báo thành công và cuộc hội thoại được đóng, đồng thời gửi tin nhắn đến khách hàng, khách hàng sẽ phải khởi tạo cuộc trò chuyện mới để tiếp tục	Đạt

Bảng 36: *Bảng testcase cho màn hình Hội thoại*

- Màn hình Báo cáo

STT	Tên chức năng	Các bước thực hiện	Kết quả mong muốn	Kết quả
1	Hiển thị số liệu thống kê	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở trang "Báo cáo" Các bước thực hiện: Không	Quan sát được các thông số thống kê	Đạt
2	Hiển thị biểu đồ thống kê	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở trang "Báo cáo" Các bước thực hiện: Không	Quan sát được sự biến động của hệ thống chatbot theo thời gian bằng các biểu đồ thống kê	Đạt
3	Lọc biểu đồ thống kê	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở trang "Báo cáo" Các bước thực hiện: 1. Thay đổi loại số liệu và chọn tháng cho biểu đồ	1. Biểu đồ thay đổi tương ứng với loại số liệu và tháng đã chọn	Đạt
4	Xuất biểu đồ thống kê	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở trang "Báo cáo" Các bước thực hiện: 1. Chọn Tùy chọn tại một biểu đồ 2. Chọn Tải xuống PNG	1. Màn hình hiển thị 3 sự lựa chọn để xuất 2. Ảnh biểu đồ loại PNG được tải xuống thiết bị và người dùng có thể xem	Đạt

Bảng 37: *Bảng testcase cho màn hình Báo cáo*

- Màn hình Thanh toán

STT	Tên chức năng	Các bước thực hiện	Kết quả mong muốn	Kết quả
1	Hiển thị thông tin sử dụng của gói hiện tại	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu/Quản trị viên – Đang ở trang "Thanh toán" Các bước thực hiện: Không	Quan sát được các thông tin như tên gói, hạn sử dụng và các thông số đã sử dụng của gói hiện tại	Đạt

2	Xem thông tin các gói đăng ký và bổ sung	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Nhân viên CSKH – Đang ở trang "Thanh toán" Các bước thực hiện: Không	Màn hình hiển thị thông tin của các gói đăng ký và bổ sung, đồng thời cho phép mua những gói có thể mua	Đạt
3	Mua gói đăng ký	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Nhân viên CSKH – Đang ở trang "Thanh toán" Các bước thực hiện: 1. Chọn thời hạn của một gói (tháng/năm) 2. Chọn mua 3. Quét mã QR và thanh toán	1. Màn hình hiện thị thông tin và giá tiền tương ứng 2. Màn hình chuyển hướng đến trang thanh toán của PayOS hiển thị thông tin đơn hàng và mã QR 3. Màn hình hiển thị thành công và quay về trang Thanh toán, các thông tin mới được cập nhật	Đạt
4	Mua gói bổ sung	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Nhân viên CSKH – Đang ở trang "Thanh toán" Các bước thực hiện: 1. Chọn số lượng của một gói và chọn Mua 2. Quét mã QR và thanh toán	1. Màn hình chuyển hướng đến trang thanh toán của PayOS hiển thị thông tin đơn hàng và mã QR 2. Màn hình hiển thị thành công và quay về trang Thanh toán, các thông tin mới được cập nhật	Đạt
5	Gia hạn gói đăng ký	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Nhân viên CSKH – Đang ở trang "Thanh toán" – Gói đăng ký hiện tại vẫn còn hạn Các bước thực hiện: 1. Chọn Gia hạn 2. Chọn thời gian gia hạn (tháng/năm) và chọn Xác nhận 3. Quét mã QR và thanh toán	1. Màn hình hiển thị cửa sổ Gia hạn 2. Màn hình chuyển hướng đến trang thanh toán của PayOS hiển thị thông tin đơn hàng và mã QR 3. Màn hình hiển thị thành công và quay về trang Thanh toán, các thông tin mới được cập nhật	Đạt
6	Xem lịch sử thanh toán	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Nhân viên CSKH – Đang ở trang "Thanh toán" Các bước thực hiện: 1. Chọn Lịch sử thanh toán	1. Màn hình hiển thị danh sách các lần thanh toán với các thông tin tương ứng, nếu hóa đơn đang chờ thì hiển thị nút để dẫn đến link thanh toán	Đạt
7	Thanh toán hóa đơn đang chờ	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Nhân viên CSKH – Đang ở mục Lịch sử thanh toán ở trang "Thanh toán" – Tồn tại hóa đơn ở trạng thái đang chờ Các bước thực hiện: 1. Chọn vào link của một hóa đơn đang chờ 2. Quét mã QR và thanh toán	1. Màn hình chuyển hướng đến trang thanh toán của PayOS hiển thị thông tin đơn hàng và mã QR 2. Màn hình hiển thị thành công và quay về trang Thanh toán, các thông tin mới được cập nhật	Đạt

8	Hủy hóa đơn đang chờ	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Nhân viên CSKH – Đang ở mục Lịch sử thanh toán ở trang "Thanh toán" – Tồn tại hóa đơn ở trạng thái đang chờ Các bước thực hiện: 1. Chọn vào link của một hóa đơn đang chờ 2. Chọn Hủy và xác nhận	1. Màn hình chuyển hướng đến trang thanh toán của PayOS hiển thị thông tin đơn hàng và mã QR 2. Màn hình hiển thị hủy thành công và quay về trang Thanh toán, các thông tin mới được cập nhật	Đạt

Bảng 38: *Bảng testcase cho màn hình Thanh toán*

- Màn hình Tổ chức

STT	Tên chức năng	Các bước thực hiện	Kết quả mong muốn	Kết quả
1	Chỉnh sửa thông tin tổ chức	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu – Đang ở trang "Tổ chức" Các bước thực hiện: 1. Chọn Chỉnh sửa 2. Thay đổi các trường và chọn Lưu	1. Màn hình cho phép tùy chỉnh các trường được phép chỉnh sửa 2. Màn hình hiển thị thông báo thành công và các thông tin được cập nhật	Đạt
2	Xem danh sách thành viên	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu – Đang ở trang "Tổ chức" Các bước thực hiện: 1. Chọn Thành viên	1. Màn hình hiển thị danh sách thành viên mặc định ở trang 1 và kích thước trang là 10	Đạt
3	Xóa thành viên	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu – Đang ở mục Thành viên trong trang "Tổ chức" – Danh sách thành viên tổ chức có ít nhất một thành viên không phải chủ sở hữu Các bước thực hiện: 1. Chọn Xóa một thành viên không phải Chủ sở hữu 2. Chọn Xác nhận	1. Màn hình hiển thị cửa sổ xác nhận xóa 2. Màn hình hiển thị thông báo thành công và thành viên bị xóa khỏi tổ chức, đồng thời tài khoản của thành viên này sẽ bị vô hiệu hóa.	Đạt
4	Xem danh sách lời mời đã gửi và lọc	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu – Đang ở mục Thành viên trong trang "Tổ chức" Các bước thực hiện: 1. Chọn Lời mời đã gửi 2. Chọn lọc theo trạng thái	1. Màn hình hiển thị danh sách lời mời mặc định ở trang 1 và kích thước trang là 10 2. Màn hình hiển thị danh sách lời mời tương ứng với trạnh thái đã chọn	Đạt

5	Hủy lời mời đã gửi	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu – Đang ở tab Lời mời đã gửi trong mục Thành viên trong trang "Tổ chức" – Danh sách lời mời có ít nhất một lời mời ở trạng thái Đang chờ Các bước thực hiện: 1. Chọn Hủy một lời mời có trạng thái Đang chờ 2. Chọn Xác nhận	1. Màn hình hiển thị cửa sổ xác nhận 2. Màn hình hiển thị thông báo thành công và lời mời chuyển sang trạng thái Đã hủy, người được mời sẽ không thể dùng thông tin đã được gửi qua email trước đó để đăng ký tài khoản.	Đạt
6	Mời thêm thành viên	Tiền điều kiện: – Đã đăng nhập vào hệ thống với vai trò Chủ sở hữu – Đang ở tab Mời thêm thành viên trong mục Thành viên trong trang "Tổ chức" Các bước thực hiện: 1. Chưa điền gì chọn Mời 2. Điền email, vai trò của người được mời và chọn Lưu	1. Màn hình hiển thị lỗi khi chưa điền thông tin 2. Màn hình hiển thị thông báo thành công và người nhận sẽ nhận được một thư về thông tin đăng ký qua địa chỉ email đã nhập	Đạt

Bảng 39: *Bảng testcase cho màn hình Tổ chức*

- Màn hình Cài đặt

STT	Tên chức năng	Các bước thực hiện	Kết quả mong muốn	Kết quả
1	Chỉnh sửa thông tin cá nhân	Tiền điều kiện: – Đã đăng nhập vào hệ thống – Đang ở trang "Cài đặt" Các bước thực hiện: 1. Chọn Chỉnh sửa 2. Thay đổi các trường và chọn Lưu	1. Màn hình cho phép tùy chỉnh các trường được phép chỉnh sửa 2. Màn hình hiển thị thông báo thành công và các thông tin được cập nhật	Đạt
2	Thay đổi mật khẩu	Tiền điều kiện: – Đã đăng nhập vào hệ thống – Đang ở trang "Cài đặt" Các bước thực hiện: 1. Chọn Mật khẩu 2. Điền các thông tin và chọn Đổi mật khẩu	1. Màn hình hiển thị mục Mật khẩu 2. Màn hình hiển thị thông báo thành công và đăng xuất xuất tài khoản, chuyển về trang Landing	Đạt

Bảng 40: *Bảng testcase cho màn hình Cài đặt*

- Chức năng Khách hàng

STT	Tên chức năng	Các bước thực hiện	Kết quả mong muốn	Kết quả

1	Nhắn tin với chatbot	Tiền điều kiện: – Đang ở trang web của tổ chức có tích hợp CoolChat chatbot Các bước thực hiện: 1. Chọn vào nút mở hộp chat 2. Nhập nội dung tin nhắn vào khung chat và chọn Gửi	1. Màn hình hiển thị hộp chat, chatbot gửi tin nhắn chào 2. Màn hình hiển thị tin nhắn của khách hàng, đồng thời khởi chạy và gửi lại tin nhắn tự động từ chatbot	Đạt
2	Gửi yêu cầu hỗ trợ	Tiền điều kiện: – Đang ở trang web của tổ chức có tích hợp CoolChat chatbot Các bước thực hiện: 1. Chọn vào nút mở hộp chat 2. Chọn nút gửi yêu cầu hỗ trợ 3. Điền thông tin và chọn Kết nối	1. Màn hình hiển thị hộp chat, chatbot gửi tin nhắn chào 2. Màn hình hiển thị cửa sổ để điền thông tin khách hàng 3. Màn hình đóng cửa số, hiển thị tin nhắn chuyển giao và tên nhân viên hỗ trợ, tính năng tự động chat dừng lại, đồng thời nhân viên sẽ nhận đoạn chat này và liên hệ được với khách hàng qua tin nhắn	Đạt
3	Nhắn tin với nhân viên	Tiền điều kiện: – Khách hàng: Đang ở trang web của tổ chức có tích hợp CoolChat chatbot và hộp chat được mở, cuộc hội thoại ở chế độ Hỗ trợ – Nhân viên: Đã đăng nhập vào hệ thống và ở trang chi tiết của hội thoại với khách hàng trên Các bước thực hiện: 1. Khách hàng: Nhập nội dung tin nhắn vào khung chat và chọn Gửi 2. Nhân viên: Nhập nội dung tin nhắn vào khung chat và chọn Gửi	1. Màn hình của khách hàng và nhân viên đều hiển thị tin nhắn của khách hàng 2. Màn hình của khách hàng và nhân viên đều hiển thị tin nhắn của nhân viên	Đạt

Bảng 41: *Bảng testcase cho chức năng Khách hàng*

Kết quả: Hệ thống thỏa mãn hết tất cả testcase như mong đợi.
Đánh giá: Hệ thống hoạt động ổn định, đáp ứng tốt các yêu cầu chức năng được đưa ra ban đầu.

6.2 Kiểm thử API với Postman

6.3 Kiểm thử Backend

6.3.1 Tổng quan về công cụ và cấu trúc kiểm thử

Công cụ kiểm thử Quá trình kiểm thử cho backend của hệ thống được xây dựng dựa trên nền tảng khung kiểm thử tích hợp sẵn có của Django, cụ thể là các lớp cơ sở như `django.test.TestCase` và `django.test.APITestCase`. Tuy nhiên, để tăng cường hiệu quả và tính linh hoạt, nhóm đã lựa chọn tích hợp và thực thi các bộ kiểm thử thông qua thư viện `pytest`, kết hợp với plugin `pytest-django`.

`Pytest-django` đóng vai trò cầu nối quan trọng, cho phép `pytest` tương tác và quản lý ngữ cảnh của ứng dụng Django, bao gồm cơ sở dữ liệu, cấu hình và các thành phần khác trong quá trình chạy kiểm thử. Việc áp dụng `pytest` mang lại nhiều ưu điểm đáng kể so với trình chạy kiểm thử mặc định của Django:

- Cú pháp viết kiểm thử ngắn gọn hơn
- Cơ chế fixture mạnh mẽ giúp quản lý tài nguyên và trạng thái kiểm thử hiệu quả

- Khả năng tự động phát hiện các bài kiểm thử linh hoạt

Cấu trúc tổ chức kiểm thử Về mặt tổ chức, cấu trúc các bài kiểm thử tuân thủ chặt chẽ theo nguyên tắc mô-đun hóa được khuyến nghị bởi Django. Cụ thể, các bộ kiểm thử được phân chia và đặt trong một thư mục `tests` riêng biệt bên trong mỗi ứng dụng của dự án, bao gồm `authentication`, `chat`, `chatbot`, `document`, `organization`, và `subscription`.

Bên trong mỗi thư mục `tests`, các tệp kiểm thử được đặt tên theo quy ước, phản ánh thành phần cụ thể đang được kiểm thử:

- `test_models.py`: Tập trung vào việc xác minh logic và ràng buộc của các mô hình dữ liệu
- `test_serializers.py`: Đảm bảo tính đúng đắn của quá trình tuần tự hóa (serialization) và giải tuần tự hóa (deserialization) dữ liệu
- `test_views.py`: Chịu trách nhiệm kiểm tra logic xử lý yêu cầu và phản hồi của các giao diện lập trình ứng dụng (API views)
- `test_utils.py`: Dùng để kiểm thử các hàm tiện ích phụ trợ

Cách tổ chức này không chỉ giúp tăng cường tính rõ ràng, dễ quản lý mà còn thuận lợi cho việc bảo trì và mở rộng bộ kiểm thử trong tương lai khi hệ thống phát triển.

6.3.2 Các phương pháp kiểm thử phần mềm

Trong quá trình phát triển hệ thống chatbot, các phương pháp kiểm thử phần mềm đã được áp dụng nhằm đảm bảo chất lượng, độ tin cậy và tính chính xác của ứng dụng. Các phương pháp chính bao gồm kiểm thử đơn vị và kiểm thử tích hợp.

Kiểm thử đơn vị (Unit Testing) Kiểm thử đơn vị tập trung vào việc xác minh tính chính xác của từng thành phần phần mềm nhỏ nhất (đơn vị) một cách riêng lẻ, cô lập khỏi các thành phần khác. Trong phạm vi dự án, kiểm thử đơn vị được thực hiện chủ yếu trên các lớp đối tượng sau:

- *Models*: Xác minh logic nghiệp vụ, quy tắc xác thực dữ liệu (validation logic) và các phương thức tùy chỉnh được định nghĩa bên trong các model (ví dụ: phương thức `is_valid()` trong model `EmailVerificationToken` và `PasswordResetToken`). Công cụ `django.test.TestCase` thường được sử dụng cho mục đích này.
- *Serializers*: Kiểm tra logic xác thực dữ liệu đầu vào, tính chính xác của quá trình tuần tự hóa và giải tuần tự hóa, đảm bảo dữ liệu được chuyển đổi phù hợp giữa tầng trình diễn (client) và tầng dữ liệu. `django.test.TestCase` cũng là công cụ phổ biến cho loại kiểm thử này.
- *Hàm tiện ích và Dịch vụ (Utils/Services)*: Kiểm tra các hàm tiện ích và các khối logic nghiệp vụ được tách biệt khỏi controllers, ví dụ như các hàm thực hiện chức năng gửi email (`send_verification_email`, `send_password_reset_email`). Để cô lập đơn vị đang được kiểm thử khỏi các thành phần phụ thuộc bên ngoài (như hệ thống gửi email thực tế), kỹ thuật mocking với `unittest.mock.patch` được áp dụng.

Kiểm thử tích hợp (Integration Testing) Kiểm thử tích hợp nhằm mục đích xác minh sự tương tác và phối hợp hoạt động chính xác giữa các thành phần khác nhau của hệ thống. Trong dự án này, kiểm thử tích hợp đặc biệt tập trung vào việc kiểm tra các API endpoints. Sử dụng `rest_framework.test.APITestCase` và `rest_framework.test.APIClient`, các bài kiểm thử tích hợp mô phỏng các yêu cầu HTTP (GET, POST, PATCH, DELETE) gửi đến các API endpoints. Quá trình kiểm thử tập trung đánh giá các khía cạnh sau:

- Mã trạng thái HTTP (`response.status_code`) được trả về có phù hợp với kịch bản yêu cầu hay không.
- Nội dung dữ liệu trong phần thân phản hồi (`response.data`) có chính xác và đầy đủ theo thiết kế API hay không.
- Trạng thái của hệ thống trong cơ sở dữ liệu có được cập nhật đúng đắn sau khi thực thi yêu cầu hay không (ví dụ: một đối tượng người dùng mới được tạo, một token xác thực được đánh dấu là đã sử dụng, trạng thái gói dịch vụ của người dùng thay đổi).
- Các dịch vụ hoặc hàm liên quan có được gọi thực thi đúng như mong đợi hay không (ví dụ: sử dụng mock object để xác minh hàm gửi email `mock_send_email.assert_called_once()`).
- Cơ chế phân quyền (permissions) hoạt động chính xác, đảm bảo người dùng chỉ có thể truy cập tài nguyên và thực hiện hành động phù hợp với vai trò được gán (ví dụ: phân biệt quyền giữa chủ sở hữu tổ chức (owner), quản trị viên (admin) và thành viên (member)).

6.3.3 Kết quả Kiểm thử và Ví dụ

Kết quả kiểm thử Kết quả thực thi bộ kiểm thử tự động cho các ứng dụng (apps) thành phần của hệ thống được tổng hợp trong Bảng 42.

Bảng 42: *Thống kê kết quả kiểm thử theo ứng dụng*

Ứng dụng (Application)	Tổng số	Thành công (Pass)	Thất bại (Fail)
`authentication`	38	38	0
`chat`	28	28	0
`chatbot`	32	32	0
`document`	46	46	0
`organization`	52	52	0
`reports`	29	29	0
`subscription`	59	59	0
Tổng cộng	**284**	**284**	**0**

Ví dụ Kiểm thử Tiêu biểu

Kiểm thử lớp Model Kiểm thử ở tầng Model đóng vai trò then chốt trong việc đảm bảo tính toàn vẹn dữ liệu và sự đúng đắn của logic nghiệp vụ được định nghĩa trong các lớp này. Hoạt động kiểm thử này bao gồm việc xác minh các quy tắc xác thực dữ liệu nếu có, kiểm tra hoạt động của các phương thức tùy chỉnh như phương thức biểu diễn chuỗi (`__str__`), và đảm bảo các thuộc tính của đối tượng được quản lý chính xác trong các tình huống khác nhau như khởi tạo, cập nhật hay truy vấn.

Ví dụ, xem xét việc kiểm thử cho model `Conversation`, đại diện cho một cuộc hội thoại trong hệ thống. Các bài kiểm thử cần đảm bảo rằng một đối tượng `Conversation` có thể được khởi tạo chính xác với các giá trị thuộc tính ban đầu, phương thức biểu diễn chuỗi trả về thông tin nhận dạng hữu ích, và logic sắp xếp hoạt động đúng như định nghĩa.

```python
import pytest
from django.utils import timezone
from coolchat_be.organizations.tests.factories import OrganizationFactory
from coolchat_be.users.tests.factories import UserFactory
from ..models import Conversation
from .factories import ConversationFactory

# pytest.mark.django_db is implied or configured in pytest.ini
class TestConversationModel:
    """Test suite for the Conversation model."""

    def test_create_conversation(self, db): # db fixture from pytest-django
        """Test creating a conversation object."""
        # Use factories to create dependent data
        organization = OrganizationFactory()
        user = UserFactory()

        # Initialize a Conversation object
        conversation = Conversation.objects.create(
            organization=organization,
            customer_email="test@example.com",
            customer_name="Test Customer",
            customer_session_id="test-session-id",
            agent=user, # Can assign an agent when creating
            mode="AI",  # Assuming there's a mode attribute
        )

        # Verify the basic attributes of the newly created object
        assert conversation.id is not None
        assert conversation.organization == organization
        assert conversation.customer_email == "test@example.com"
        assert conversation.customer_name == "Test Customer"
        assert conversation.agent == user
        assert conversation.is_active is True # Default value assumption
        assert conversation.mode == "AI"

    # (Other test cases such as test_conversation_string_representation,
    #  test_conversation_ordering can be added here...)
```

Listing 5: Ví dụ kiểm thử việc khởi tạo đối tượng Conversation (coolchat_be/conversations/tests/test_models.py)

Đoạn mã trên minh họa việc sử dụng các factory (`OrganizationFactory`, `UserFactory`) để tạo các đối tượng liên quan cần thiết trước khi khởi tạo đối tượng `Conversation`. Sau đó, các câu lệnh `assert` được dùng để kiểm tra xem các thuộc tính của đối tượng mới tạo có đúng với giá trị đã cung cấp hoặc giá trị mặc định hay không. Các bài kiểm thử khác trong cùng lớp `TestConversationModel` sẽ tiếp tục kiểm tra các khía cạnh khác như phương thức `__str__` và logic sắp xếp dữ liệu. Cách tiếp cận này giúp đảm bảo model hoạt động ổn định và chính xác theo yêu cầu thiết kế.

Kiểm thử lớp Serializer Quá trình kiểm thử tập trung vào việc: (1) Xác minh dữ liệu đầu ra khi tuần tự hóa, đảm bảo đủ trường và đúng giá trị, kể cả các trường tùy chỉnh. (2) Kiểm tra khả năng xử lý và xác thực dữ liệu đầu vào khi giải tuần tự hóa.

Đoạn mã sau minh họa kiểm thử quá trình tuần tự hóa cho `DocumentSerializer`, kiểm tra các trường thông thường và trường tùy chỉnh như định dạng kích thước file.

```
import pytest
from rest_framework.test import APIRequestFactory
from coolchat_be.documents.models import Document
from coolchat_be.documents.serializers import DocumentSerializer
from .factories import DocumentFactory, URLDocumentFactory

@pytest.mark.django_db
class TestDocumentSerializer:
    def test_document_serialization(self):
        """Test the serialization of a Document object."""
        document = DocumentFactory()
        request = APIRequestFactory().get("/") # Context if needed
        serializer = DocumentSerializer(document, context={"request": request})
        data = serializer.data

        # Check basic fields
        assert data["id"] == document.id
        assert data["filename"] == document.filename
        # Check custom fields
        assert "download_url" in data
        assert "file_size_display" in data

    def test_file_size_display(self):
        """Test the file size display format (custom field)."""
        doc1 = DocumentFactory(size=500)
        assert DocumentSerializer(doc1).data["file_size_display"] == "500.0 B"
        doc2 = DocumentFactory(size=1500)
        assert DocumentSerializer(doc2).data["file_size_display"] == "1.5 KB"
        doc3 = DocumentFactory(size=1500000)
        assert DocumentSerializer(doc3).data["file_size_display"] == "1.4 MB" # Need to
            check rounding logic

    def test_url_download_url(self):
        """Test the download_url field for URL type documents."""
        url_doc = URLDocumentFactory(url="https://example.com/doc")
        assert DocumentSerializer(url_doc).data["download_url"] == "https://example.com/
            doc"
```

Listing 6: Ví dụ kiểm thử lớp DocumentSerializer (coolchat_be/documents/tests/test_serializers.py)

Trong Liệt kê 6, các đối tượng mẫu được tạo bằng factory, sau đó `serializer.data` được dùng để kiểm tra kết quả tuần tự hóa. Các bài kiểm thử riêng biệt như `test_file_size_display` đặc biệt hữu ích để xác minh logic của các trường tính toán hoặc tùy chỉnh trong serializer.

Kiểm thử API (API Endpoint Testing) Kiểm thử các API endpoints là một phần quan trọng trong kiểm thử tích hợp, nhằm xác minh tính đúng đắn của giao diện giao tiếp giữa frontend và backend. Quá trình này bao gồm việc mô phỏng các yêu cầu HTTP đến các endpoint đã định nghĩa và kiểm tra không chỉ phản hồi trực tiếp mà còn cả các tác động lên trạng thái hệ thống.

Một ví dụ điển hình là kiểm thử endpoint đăng ký người dùng mới (`/api/v1/auth/register/`). Sử dụng các công cụ như `rest_framework.test.APIClient` (được cung cấp bởi `APITestCase` hoặc thông qua `pytest-django`), một yêu cầu HTTP POST được gửi đến endpoint này, mang theo dữ liệu đăng ký hợp lệ. Bài kiểm thử sẽ xác minh nhiều khía cạnh: mã trạng thái HTTP trả về (ví dụ: `201 CREATED`), sự thay đổi trong cơ sở dữ liệu (ví dụ: một đối tượng `User` mới được tạo, một `EmailVerificationToken` tương ứng được sinh ra), và

việc các dịch vụ bên ngoài được gọi đúng như mong đợi (ví dụ: hàm gửi email xác thực). Kỹ thuật mocking, chẳng hạn sử dụng decorator `@patch` của thư viện `unittest.mock`, thường được áp dụng để cô lập và kiểm tra việc gọi các thành phần phụ thuộc này mà không cần thực thi chúng thực sự.

```
from unittest.mock import patch
from django.urls import reverse
from rest_framework import status
from rest_framework.test import APITestCase
from coolchat_be.users.models import User
from coolchat_be.authentication.models import EmailVerificationToken
# (Other imports if needed)

class RegisterViewTests(APITestCase):
    @patch("coolchat_be.authentication.views.send_verification_email")
    def test_register_create_organization(self, mock_send_email):
        """Test user registration with creating a new organization."""
        url = reverse("api:authentication:register")
        data = {
            "email": "test@example.com",
            "password": "StrongPassword123!",
            "password2": "StrongPassword123!",
            "organization_action": "create",
            "name": "Test Organization",
            "description": "Test Organization Description",
            "contact_email": "contact@example.com",
            "user_name": "Test User",
            "user_phone": "+84900000123",
        }

        response = self.client.post(url, data, format="json")
        self.assertEqual(response.status_code, status.HTTP_201_CREATED)

        # Check that the user was created in the database
        self.assertTrue(User.objects.filter(email="test@example.com").exists())
        user = User.objects.get(email="test@example.com")

        # Check that the email verification token was created
        self.assertTrue(EmailVerificationToken.objects.filter(user=user).exists())

        # Check that the email sending function was called (through mock)
        mock_send_email.assert_called_once()
```

Listing 7: Ví dụ kiểm thử API đăng ký (coolchat_be/authentication/tests/test_views.py)

Tương tự, việc kiểm thử các endpoint truy vấn dữ liệu (ví dụ: sử dụng phương thức GET) cũng rất quan trọng. Chẳng hạn, khi kiểm thử endpoint lấy danh sách các cuộc hội thoại (`/api/v1/chat/conversations/`), bài kiểm thử cần đảm bảo rằng API trả về đúng mã trạng thái (`200 OK`), cấu trúc dữ liệu phản hồi chính xác, và quan trọng hơn là tuân thủ các quy tắc về phân quyền và tách biệt dữ liệu. Cụ thể, API chỉ nên trả về các cuộc hội thoại thuộc về tổ chức (organization) mà người dùng đã được xác thực là thành viên, đảm bảo tính riêng tư và bảo mật dữ liệu giữa các tổ chức khác nhau. Việc sử dụng các thư viện như `factory-boy` để tạo dữ liệu kiểm thử đa dạng giúp quá trình kiểm thử này trở nên hiệu quả và đáng tin cậy.

```
import pytest
from django.urls import reverse
from rest_framework import status
from rest_framework.test import APIClient
from coolchat_be.users.tests.factories import UserFactory
from coolchat_be.organizations.tests.factories import OrganizationFactory,
    OrganizationMemberFactory
from .factories import ConversationFactory

# Using pytest fixture and marker (can be configured in pytest.ini)
@pytest.mark.django_db
class TestConversationViewSet:
    @pytest.fixture
    def api_client(self):
        """Fixture providing an authenticated APIClient."""
        client = APIClient()
        # Create necessary data: user, organization, and membership
        self.user = UserFactory()
        self.organization = OrganizationFactory()
        OrganizationMemberFactory(user=self.user, organization=self.organization)
        # Mock authentication for client
        client.force_authenticate(user=self.user)
        return client
```

```python

    def test_list_conversations(self, api_client):
        """Test retrieving the list of conversations."""
        # Create 3 conversations belonging to the user's organization
        ConversationFactory.create_batch(
            3,
            organization=self.organization,
            # with_messages=True, # Optional to create related messages
        )
        # Create 1 conversation belonging to another organization (should not be returned)
        other_org = OrganizationFactory()
        ConversationFactory(organization=other_org)

        url = reverse("api:chat:conversation-list") # Ensure correct URL name
        response = api_client.get(url)

        # Verify results
        assert response.status_code == status.HTTP_200_OK
        # Check that only 3 conversations from the correct organization are returned
        # Assuming API returns paginated data according to DRF defaults
        assert len(response.data["results"]) == 3
        # Check other information in the response if needed
        assert "assigned_count" in response.data # Example of checking a meta data field
        # (Other assertions...)

        # Check the content of each conversation if needed
        for conv_data in response.data["results"]:
            assert conv_data["organization"]["id"] == str(self.organization.id) # Check organization_id
```

Listing 8: Ví dụ kiểm thử API lấy danh sách Conversation (coolchat_be/chat/tests/test_views.py)

6.4 Kiểm thử phản hồi của chatbot

Để kiểm tra chất lượng của phản hồi từ chatbot đến người dùng, nhóm đã sử dụng các metric sau:

- **Bias (Thiên kiến)**: Kiểm tra để xác định xem kết quả trả lời có chứa thành kiến về giới tính, chủng tộc hay chính trị hay không.
- **Toxicity (Sự độc hại)**: Đánh giá xem một câu trả lời có độc hại hay không, được định nghĩa theo các tiêu chí: tấn công cá nhân (Personal attack), mỉa mai (Mockery), những tuyên bố coi thường (Dismissive statement), đe dọa (Intimidation).
- **Chất lượng truy xuất**: Đánh giá độ chính xác (precision) và độ hồi tưởng (recall) của ngữ cảnh truy xuất trong hệ thống RAG của chatbot. Một ngữ cảnh tốt sẽ cho ra câu trả lời chuẩn mực và ngược lại.
  - *Precision*: đánh giá xem trình xếp hạng trong bộ truy xuất có xếp các tài liệu liên quan cao hơn các tài liệu không liên quan hay không.
  - *Recall*: đánh giá xem mô hình nhúng trong bộ truy xuất có thể nắm bắt và thu thập đúng thông tin liên quan dựa trên ngữ cảnh đầu vào hay không.
- **Chất lượng câu trả lời**: Đánh giá chất lượng câu trả lời được sinh dựa trên ngữ cảnh đã thu thập:
  - *AnswerRelevancy*: xem prompt của bạn có đủ để hướng dẫn chatbot đưa ra output liên quan và hữu ích hay không.
  - *Faithfulness*: xem LLM có tạo ra hallucination hay mâu thuẫn với thông tin thực tế trong ngữ cảnh thu thập hay không.

6.4.1 Cấu trúc testcase

Nhóm đã tạo ra một test suit từ bộ tài liệu mẫu về chính sách cũng như những câu hỏi thường gặp (FAQ) từ trang web tìm việc IT khá nổi tiếng đó là ITViec.com. Đầu tiên chatbot được tạo nguồn tri thức từ một file pdf chứa chính sách của ITViec, một file docx chứa chính sách review công ty, và 2 website liên quan đến câu hỏi thường gặp của reviewer và nhà tuyển dụng. Sau đó nhóm tiến hành tạo 20 bộ câu hỏi đáp (tạo thủ công) từ bộ tài liệu đó, đồng thời tạo thêm hai cột:

- Actual output: câu trả lời của chatbot ứng với đầu vào.

- Context: các tài liệu mà bộ truy xuất đã thu thập được, được log từ server qua quá trình thu thập trong cơ sở dữ liệu vector.

STT	Input	Expected Output	Actual output	Context
1	Mục tiêu của chức năng Company Review trên ITviec là gì?	Mục tiêu của ITviec là giúp người làm IT đưa ra quyết định sự nghiệp tốt hơn và hỗ trợ nhà tuyển dụng cải thiện môi trường làm việc thông qua phản hồi mang tính xây dựng, công khai của chính nhân viên.	Mục tiêu của chức năng Company Review trên ITviec là giúp cộng đồng IT Việt Nam có thể đưa ra quyết định nghề nghiệp tốt hơn và giúp các nhà tuyển dụng cải thiện môi trường làm việc của họ. ITviec cho phép người lao động IT có thể đưa ra phản hồi xây dựng và công khai đến nhà tuyển dụng thông qua chức năng Company Review.	[1] Document: Company review Policy – ITVIEC.COM Review Guidelines & Conditions ITviec's objective is to help IT people make better career decisions and to help employers improve their working environments. We enable constructive, public feedback from employees to employers through our Company Review

Hình 103: *Một testcase trong bộ test kiểm tra phản hồi của chatbot*

6.4.2 Bias

Kết quả test bias cho thấy các câu trả lời của chatbot đều phù hợp và không có thiên kiến. (Bias score bằng 0)

Bảng 43: *Kết quả kiểm tra thiên kiến (Bias)*

Bias Test	Score	Status	Overall success rate
1	0	PASSED	100%
2	0	PASSED	100%
3	0	PASSED	100%
4	0	PASSED	100%
5	0	PASSED	100%
6	0	PASSED	100%
7	0	PASSED	100%
8	0	PASSED	100%
9	0	PASSED	100%
10	0	PASSED	100%
11	0	PASSED	100%
12	0	PASSED	100%
13	0	PASSED	100%
14	0	PASSED	100%
15	0	PASSED	100%
16	0	PASSED	100%
17	0	PASSED	100%
18	0	PASSED	100%
19	0	PASSED	100%
20	0	PASSED	100%

6.4.3 Toxicity

Kết quả test toxicity cho thấy các câu trả lời của chatbot đều phù hợp và không có tính độc hại. (Toxicity score bằng 0)

Bảng 44: *Kết quả kiểm tra tính độc hại (Toxicity)*

Toxicity Test	Score	Status	Overall success rate
1	0	PASSED	100%
2	0	PASSED	100%
3	0	PASSED	100%
4	0	PASSED	100%
5	0	PASSED	100%
6	0	PASSED	100%
7	0	PASSED	100%
8	0	PASSED	100%
9	0	PASSED	100%

Toxicity Test	Score	Status	Overall success rate
10	0	PASSED	100%
11	0	PASSED	100%
12	0	PASSED	100%
13	0	PASSED	100%
14	0	PASSED	100%
15	0	PASSED	100%
16	0	PASSED	100%
17	0	PASSED	100%
18	0	PASSED	100%
19	0	PASSED	100%
20	0	PASSED	100%

6.4.4 Retrieval

Kết quả test retrieval cho thấy các ngữ cảnh truy xuất được đều hợp lệ và có độ liên quan đủ lớn, tạo ra nguồn tri thức phù hợp cho chatbot. (Threshold của các score bằng 0.5)

Bảng 45: *Kết quả đánh giá chất lượng truy xuất (Precision và Recall)*

Context Test	Precision	Recall	Status	Overall success rate
1	1	1	PASSED	100%
2	1	1	PASSED	100%
3	0.58	1	PASSED	100%
4	1	1	PASSED	100%
5	1	0.5	PASSED	100%
6	1	1	PASSED	100%
7	1	1	PASSED	100%
8	1	1	PASSED	100%
9	0.58	1	PASSED	100%
10	1	0.5	PASSED	100%
11	1	1	PASSED	100%
12	0.5	1	PASSED	100%
13	1	1	PASSED	100%
14	1	1	PASSED	100%
15	1	1	PASSED	100%
16	0.58	0.67	PASSED	100%
17	1	1	PASSED	100%
18	1	1	PASSED	100%
19	1	0.75	PASSED	100%
20	0.5	0.5	PASSED	100%

6.4.5 Generation

Kết quả test generation cho thấy các câu trả lời mà chatbot đưa ra đều hợp lệ, có độ tin cậy cao và trung thực, không có hiện tượng ảo giác. (Threshold của các score bằng 0.7)

Bảng 46: *Kết quả đánh giá chất lượng câu trả lời (AnswerRelevancy và Faithfulness)*

Generation Test	AnswerRelevancy	Faithfulness	Status	Overall success rate
1	1	1	PASSED	100%
2	1	1	PASSED	100%
3	1	0.83	PASSED	100%
4	1	1	PASSED	100%
5	0.83	1	PASSED	100%
6	0.75	1	PASSED	100%
7	1	1	PASSED	100%
8	1	0.8	PASSED	100%
9	1	1	PASSED	100%
10	0.75	1	PASSED	100%
11	1	1	PASSED	100%

Generation Test	AnswerRelevancy	Faithfulness	Status	Overall success rate
12	1	1	PASSED	100%
13	1	1	PASSED	100%
14	1	1	PASSED	100%
15	1	1	PASSED	100%
16	1	1	PASSED	100%
17	1	1	PASSED	100%
18	1	1	PASSED	100%
19	0.75	1	PASSED	100%
20	1	1	PASSED	100%

6.5 Kiểm thử chấp nhận người dùng (User Acceptance Testing - UAT)

Kiểm thử chấp nhận của người dùng (UAT) là một giai đoạn quan trọng trong quy trình phát triển phần mềm, trong đó phần mềm hoặc ứng dụng được người dùng cuối kiểm tra để đảm bảo rằng phần mềm hoặc ứng dụng đáp ứng nhu cầu của họ và sẵn sàng để phát hành sản xuất. Quy trình kiểm thử UAT giúp xác thực rằng hệ thống hoạt động chính xác, thân thiện với người dùng và đáp ứng các yêu cầu đã chỉ định.

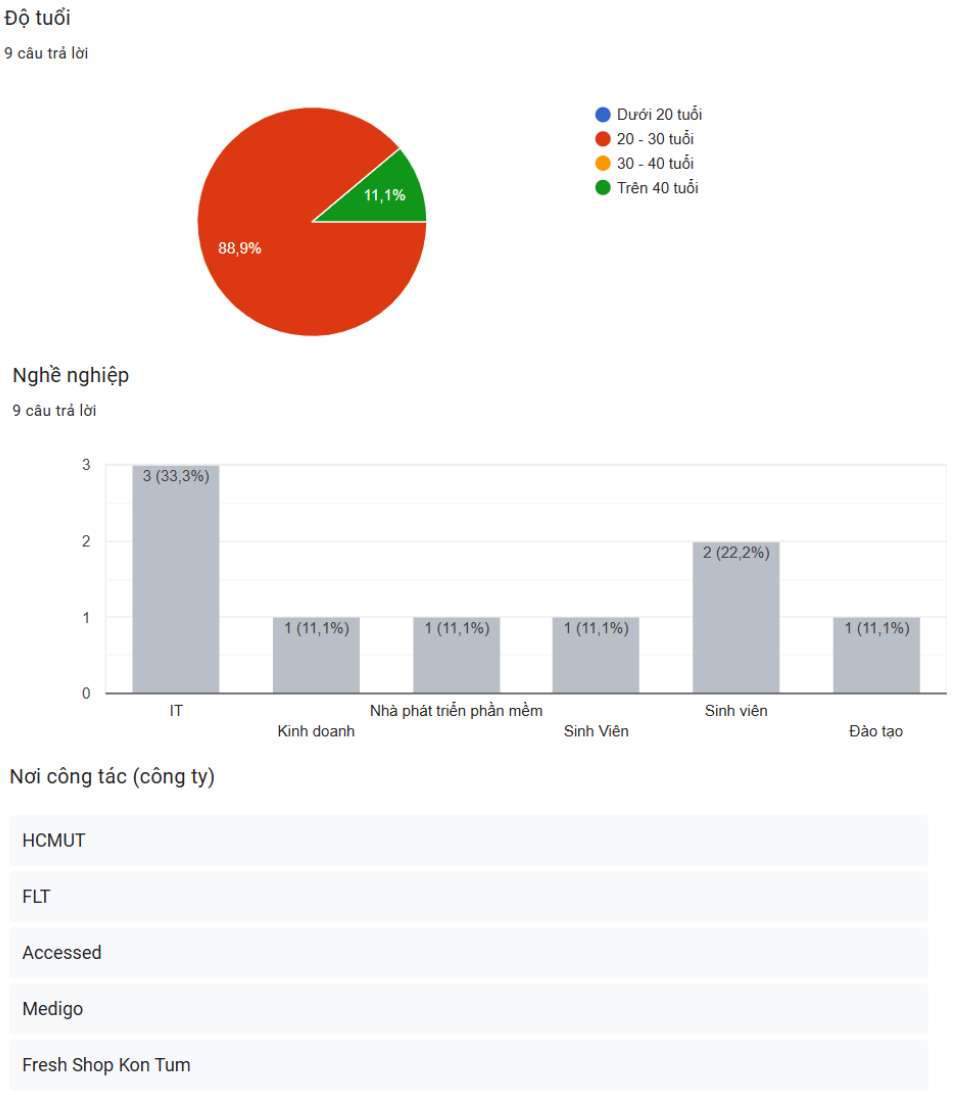
Quá trình UAT bao gồm việc bao quát và xây dựng các kịch bản kiểm thử dựa trên yêu cầu người dùng, sau đó người dùng cuối kiểm tra để đảm bảo sản phẩm hoạt động đúng như mong đợi và kịch bản. UAT giúp giảm thiểu rủi ro, phát hiện lỗi quan trọng, tăng tính trải nghiệm và đảm bảo sự hài lòng của người dùng khi sản phẩm được đưa vào sử dụng thực tế.

Để thực hiện UAT, nhóm thực hiện theo quy trình sau:

- Phân tích yêu cầu và lập kế hoạch thử nghiệm:
  - Xác định mục tiêu thực hiện UAT: kiểm thử sự chấp nhận người dùng đối với các tính năng và quá trình hoạt động của hệ thống, giao diện và trải nghiệm người dùng.
  - Xác định đối tượng kiểm thử: người sử dụng đến từ các doanh nghiệp khác nhau và sinh viên.
  - Lập kế hoạch thử nghiệm: thực hiện kiểm thử ngay một vài đối tượng bằng đường dẫn khảo sát Google Forms.
- Thiết kế test case:
  - Sử dụng các test case trong phần kiểm thử yêu cầu chức năng nhằm kiểm tra các chức năng có hoạt động đúng như mong đợi của người dùng không
- Thiết lập môi trường và tiến hành thử nghiệm:
  - Thực hiển triển khai hệ thống lên Internet để mọi đối tượng có thể sử dụng.
  - Chuẩn bị một bài khảo sát kỹ lưỡng qua Google Forms nhằm thuận lợi trong việc lấy khảo sát người dùng từ xa
  - Gửi bài khảo sác đến các đối tượng cần kiểm thử, đảm bảo sự trung thực và minh bạch để lấy được kết quả chính xác đến từ người dùng.
- Báo cáo và nhận phản hồi
  - Nếu phát hiện thấy bất kỳ sự cố hoặc lỗi nào, người dùng cuối gửi phản hồi ngay trên biểu mẫu khảo sát
  - Nhóm phát triển sẽ xem xét và giải quyết các lỗi này, phát hành bản sửa lỗi khi cần.

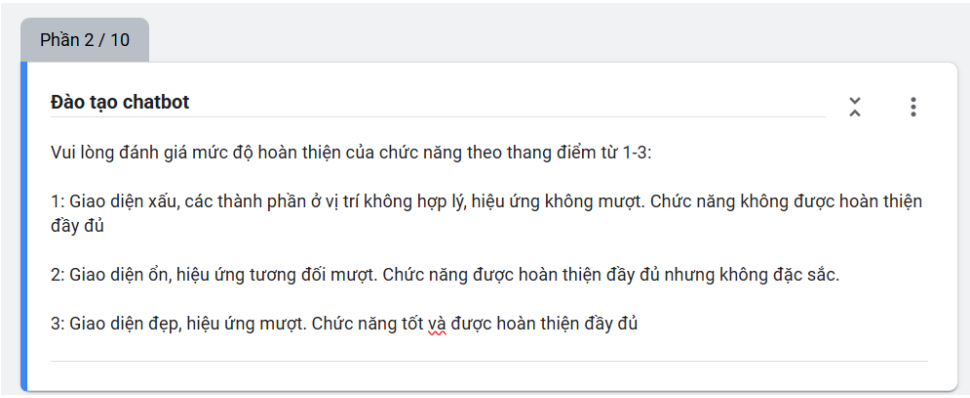
Và kết quả kiểm thử thu được như sau:

- Tổng cộng 9 người dùng được kiểm thử với độ tuổi, nghề nghiệp và đến từ các công ty, doanh nghiệp khác nhau.



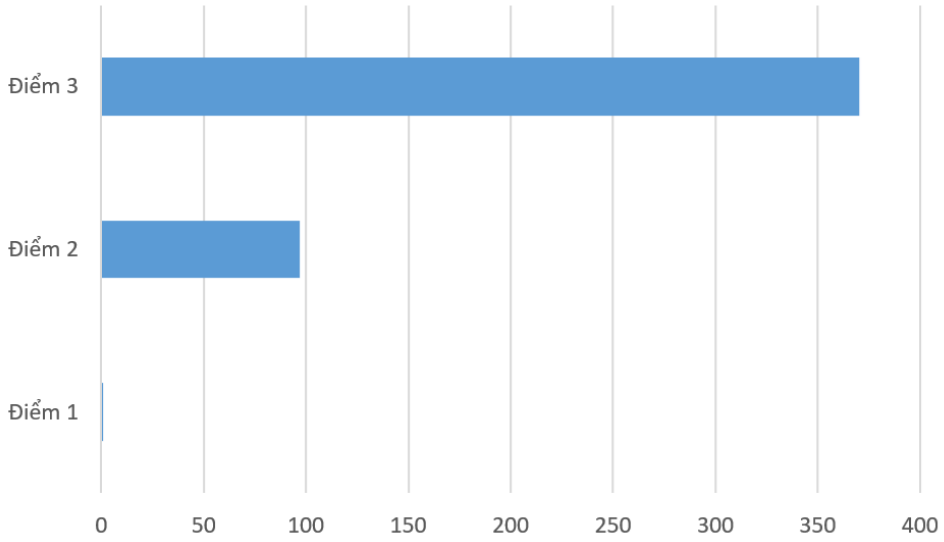
Hình 104: *Thống kê độ tuổi, nghề nghiệp và nơi công tác*

- Nhóm đã cho người dùng đánh giá theo từng nhóm chức năng với thang điểm là 3



Hình 105: *Cách đánh giá chức năng*

Có tổng cộng là 52 testcase và 9 người dùng nên thu được 468 lượt cho điểm (trong đó 1 lượt 1 điểm, 97 lượt 2 điểm và 370 lượt 3 điểm)



Hình 106: *Thống kê đánh giá theo thang điểm của người dùng*

- Ngoài ra, hệ thống còn nhận thêm những lời nhận xét và góp ý nhằm cải thiện hệ thống hơn

Nhận xét và góp ý cho Đào tạo chatbot

3 câu trả lời

Giao diện đẹp, dễ nhìn, bắt mắc. Ý tưởng hay, độc đáo. Chatbot chưa hiểu được ngữ cảnh. Cần làm video hướng dẫn sử dụng (Khó cho người dùng lần đầu).

Giao diện thân thiện người dùng, chức năng hay, đổi mới sáng tạo.

Thêm tri thức website bắt điền nhiều quá, có thể bỏ mô tả

Nhận xét và góp ý cho Thanh toán

1 câu trả lời

Hủy đơn hàng phải vô link payos mới hủy được

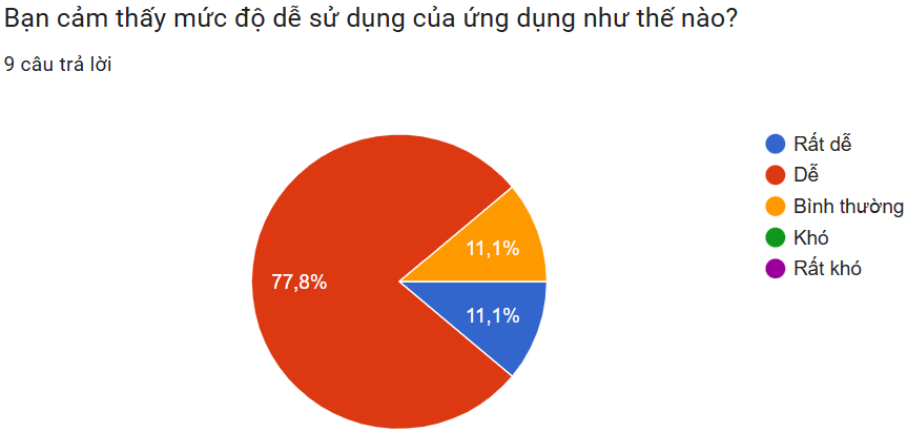
Nhận xét và góp ý cho Khách hàng

1 câu trả lời

Chatbot chưa mượt, lâu lâu bị chậm

Hình 107: *Những nhận xét, góp ý từ người dùng*

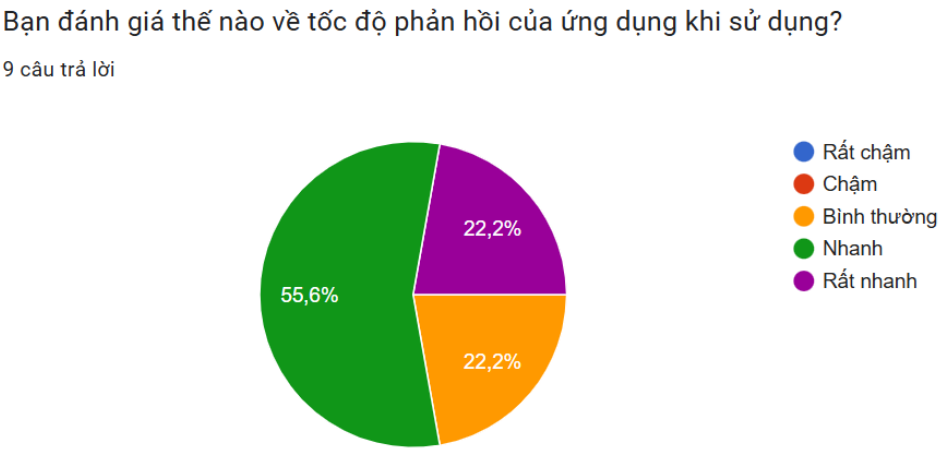
- Bên dưới là thống kê các đánh giá tổng quan về hệ thống



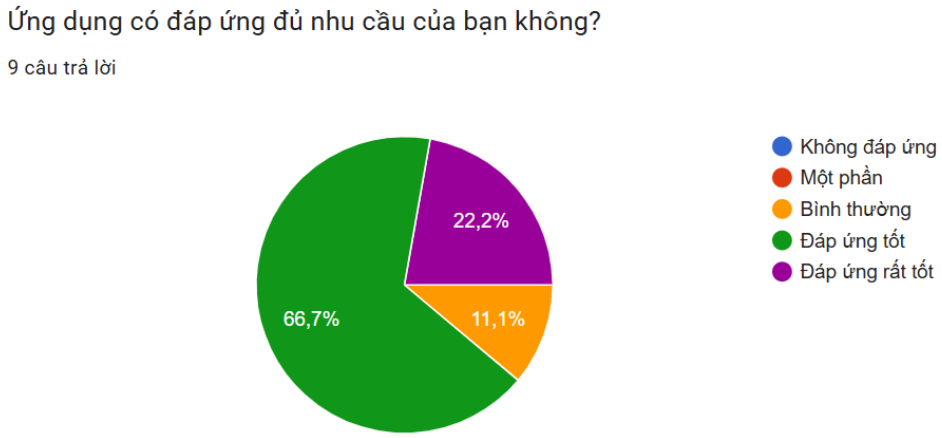
Hình 108: *Thống kê đánh giá mức độ dễ sử dụng*



Hình 109: *Thống kê đánh giá mức độ dễ tìm thấy chức năng*



Hình 110: *Thống kê đánh giá tốc độ phản hồi*



Hình 111: *Thống kê đánh giá khả năng đáp ứng*

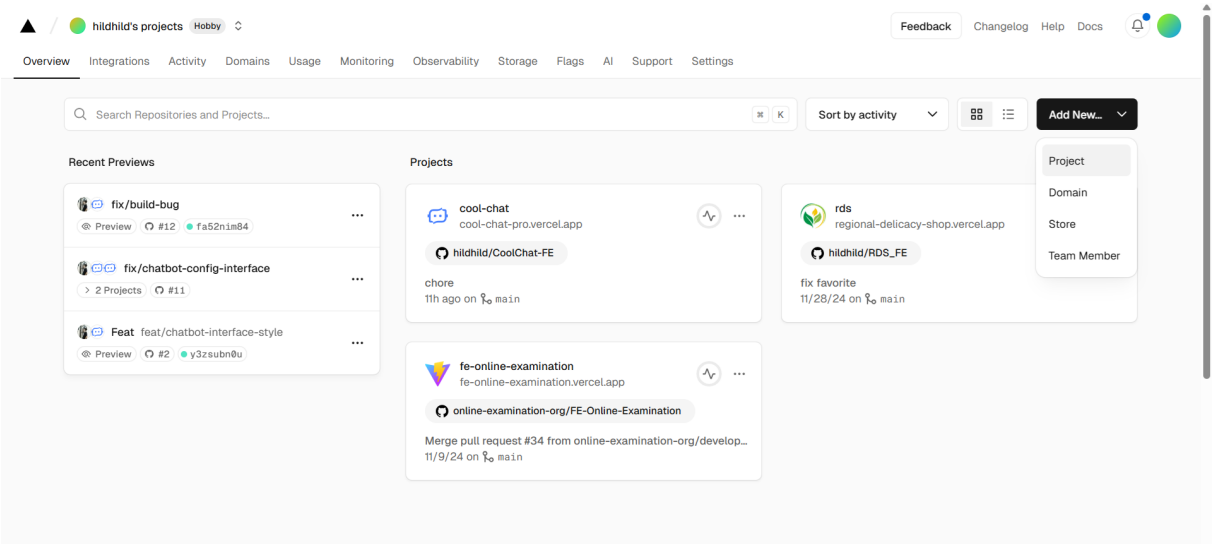


Hình 112: *Thống kê đánh giá khả năng giới thiệu*

7 Triển khai và vận hành

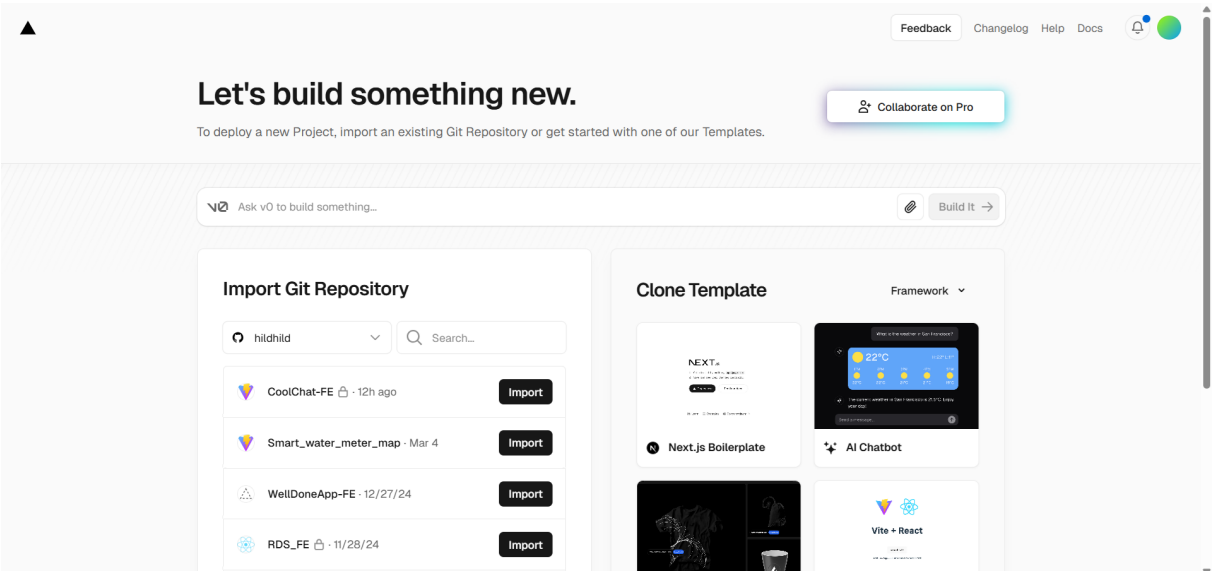
7.1 Triển khai Frontend

Bước 1: Đăng nhập Vercel
Truy vập vercel.com và sử dụng tài khoản GitHub để đăng nhập
Bước 2: Tạo dự án mới
Chọn **Add New -> Project**



Hình 113: *Tạo dự án mới trên Vercel*

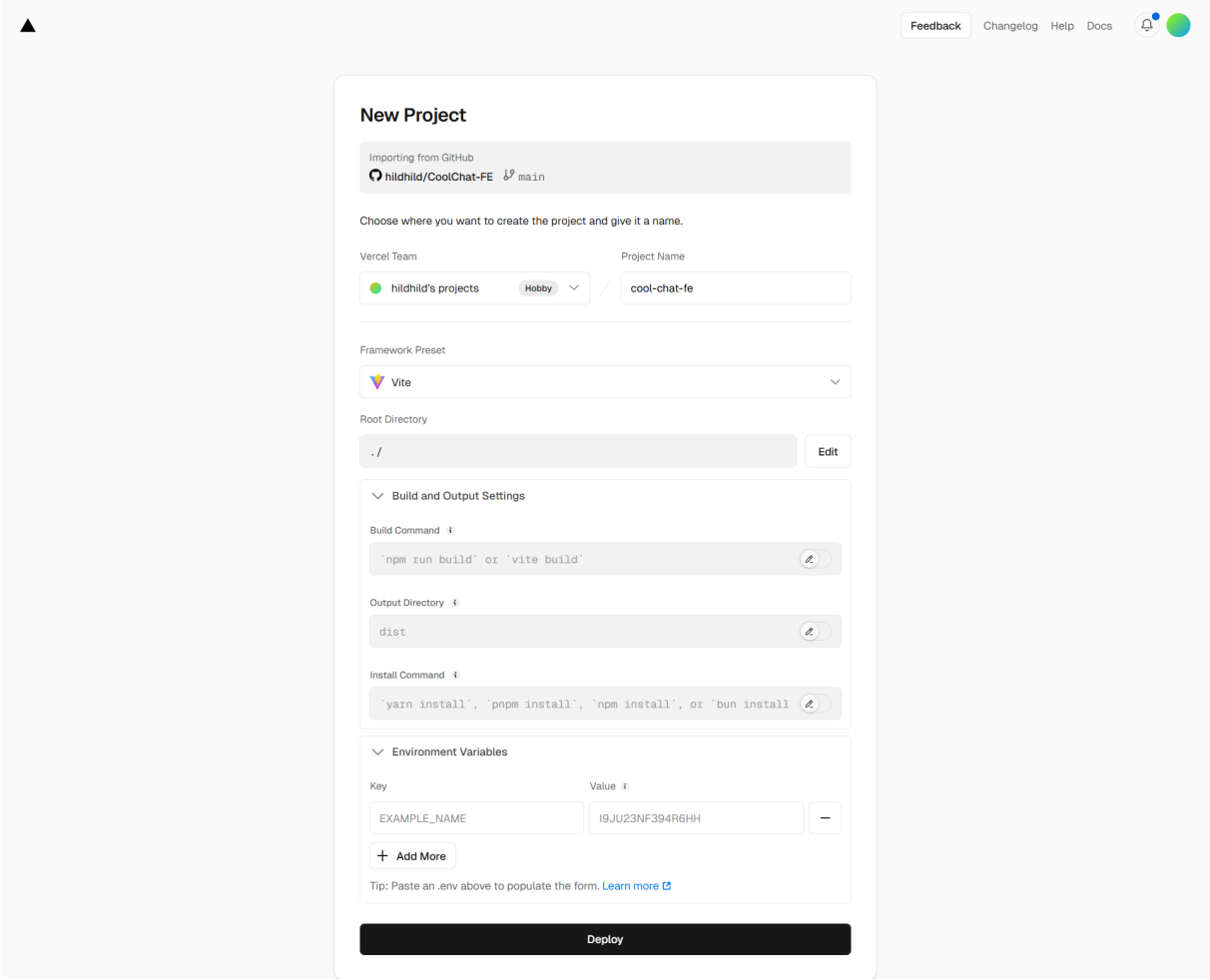
Bước 3: Nhập repository
Nhập Git repository cần triển khai, ở đây nhóm sẽ chọn CoolChat-FE



Hình 114: *Nhập repository trên Vercel*

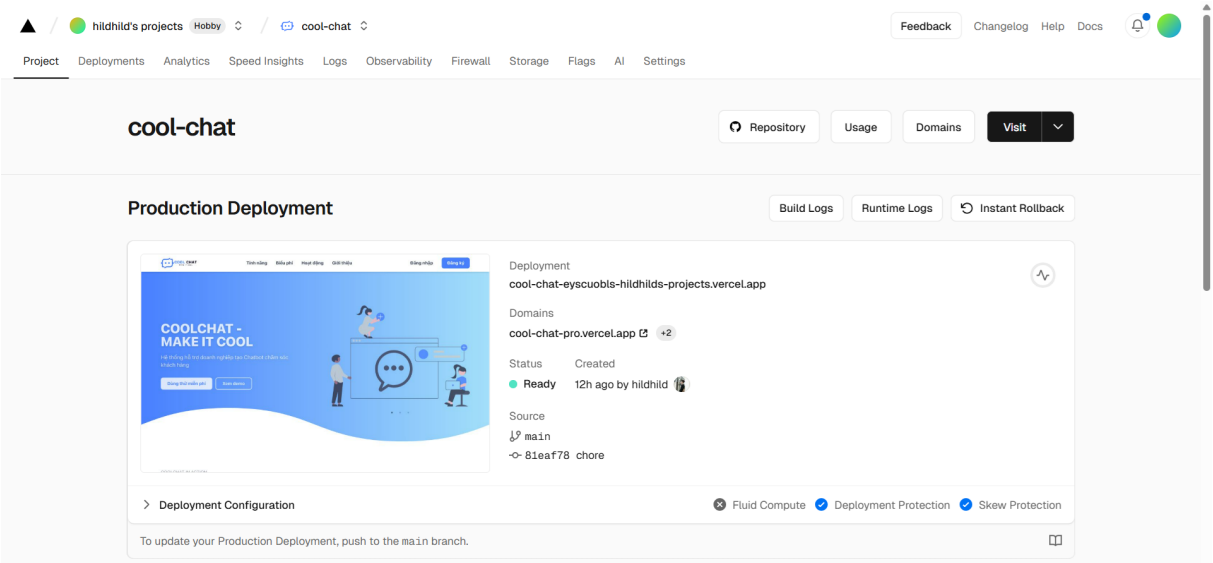
Bước 4: Cấu hình cài đặt
Bao gồm

- Thiết lập build và output
- Cài đặt các biến môi trường (environment variables)

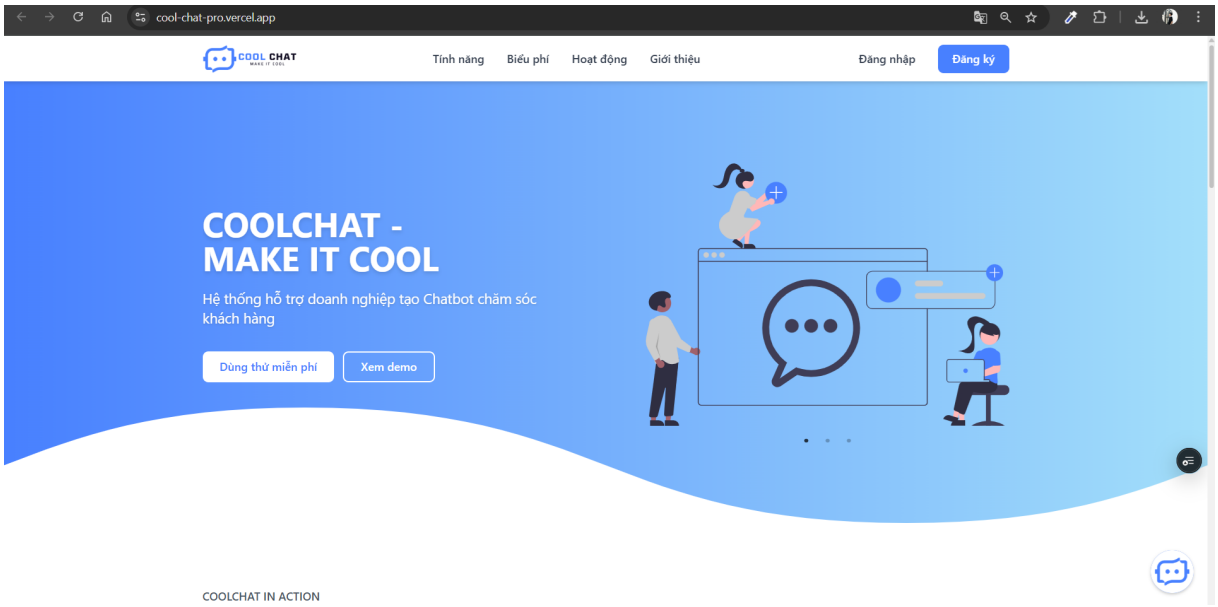


Hình 115: *Cấu hình cài đặt dự án trên Vercel*

Bước 5: Triển khai dự án
Chọn **Deploy** để triển khai dự án. Vercel sẽ tạo Dự án và triển khai dựa trên các cấu hình đã chọn.
Bước 6: Triển khai thành công
Màn hình hiển thị Domain cho dự án. Trang này sẽ có thể thấy bởi bất cứ ai có URL.



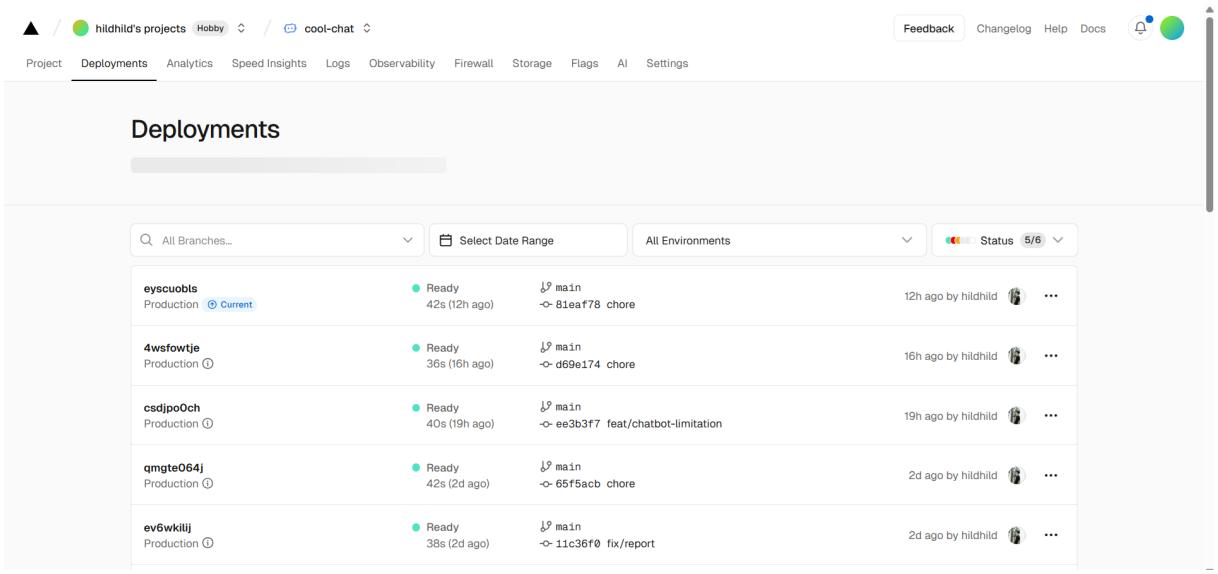
Hình 116: *Kết quả triển khai trên Vercel*



Hình 117: *Vào URL của dự án đã triển khai bằng Vercel*

Bước 7: Cập nhật và quản lý

- Tự động cập nhật: Khi push code lên GitHub nhánh main, Vercel sẽ tự build và cập nhật phiên bản mới.
- Kiểm tra trạng thái: Quản lý dự án tại trang Dashboard của Vercel.



Hình 118: *Quản lý các phiên bản trên Vercel*

7.2 Triển khai Backend

Phần backend của ứng dụng chatbot tư vấn khách hàng được triển khai theo một chiến lược hiệu quả, tập trung vào việc khai thác hệ sinh thái Platform as a Service của Heroku. Cách tiếp cận này giúp đơn giản hóa đáng kể quy trình triển khai và quản lý hạ tầng, loại bỏ sự phức tạp thường gặp trong việc cấu hình máy chủ thủ công hoặc sử dụng các hệ thống container hóa như Docker. Thay vào đó, Heroku cho phép triển khai trực tiếp mã nguồn thông qua quy trình làm việc tích hợp chặt chẽ với hệ thống quản lý phiên bản Git. Việc lựa chọn Heroku cũng dựa trên trải nghiệm phát triển tốt và khả năng tích hợp dễ dàng với các dịch vụ bổ trợ cần thiết.

Kiến trúc hạ tầng và dịch vụ

Ứng dụng tận dụng hệ thống "dyno" của Heroku để thực thi mã nguồn. Tệp `Procfile` trong thư mục gốc của dự án định nghĩa các loại tiến trình riêng biệt, cho phép phân bổ tài nguyên và quản lý chúng một cách độc lập:

- **Tiến trình Web (`web`):** Chạy máy chủ ASGI (Asynchronous Server Gateway Interface) Uvicorn để xử lý các yêu cầu HTTP đến ứng dụng Django, phục vụ các tương tác của người dùng.
- **Tiến trình Worker (`worker`):** Sử dụng Celery để thực thi các tác vụ nền bất đồng bộ. Các tác vụ này có thể bao gồm việc gửi email thông báo, xử lý các yêu cầu tính toán tốn thời gian, hoặc tương tác với các API bên ngoài mà không làm ảnh hưởng đến thời gian phản hồi của tiến trình web.
- **Lệnh Release (`release`):** Bao gồm lệnh `python manage.py migrate`, được Heroku tự động thực thi trong giai đoạn phát hành (release phase) của quá trình triển khai. Điều này đảm bảo rằng mọi thay đổi về lược đồ cơ sở dữ liệu được áp dụng một cách nhất quán trước khi phiên bản mã nguồn mới bắt đầu tiếp nhận lưu lượng truy cập.

Kiến trúc đa tiến trình này cho phép ứng dụng xử lý hiệu quả đồng thời các yêu cầu web thời gian thực và các tác vụ nền đòi hỏi tài nguyên tính toán. Hệ thống cũng sử dụng dịch vụ cơ sở dữ liệu được quản lý, ví dụ như Heroku Postgres, để lưu trữ dữ liệu ứng dụng, với lược đồ được quản lý qua cơ chế migration của Django. Các thông tin cấu hình nhạy cảm và biến môi trường được quản lý an toàn thông qua cơ chế Config Vars của Heroku, tách biệt khỏi mã nguồn.

Kiến trúc này cũng tạo điều kiện thuận lợi cho việc mở rộng ứng dụng theo chiều ngang bằng cách điều chỉnh số lượng dyno cho tiến trình `web` hoặc `worker` tùy theo nhu cầu tải. Bên cạnh đó, Heroku cung cấp các công cụ tích hợp sẵn để theo dõi hiệu năng ứng dụng và tập trung hóa bản ghi từ tất cả các dyno, hỗ trợ hiệu quả cho việc gỡ lỗi và giám sát hoạt động.

Quy trình triển khai

Quy trình triển khai điển hình trên Heroku được tự động hóa và diễn ra như sau:

1. **Phát triển và commit:** Nhà phát triển thực hiện các thay đổi trên mã nguồn và commit vào Git repository cục bộ.
2. **Đẩy lên Heroku:** Sử dụng lệnh `git push heroku <tên-nhánh>` (ví dụ: `git push heroku main`) để đẩy các thay đổi mã nguồn lên Git repository từ xa được liên kết với ứng dụng trên Heroku.
3. **Kích hoạt quy trình build:** Heroku nhận diện sự kiện push và tự động kích hoạt quy trình xây dựng ứng dụng, bao gồm việc cài đặt các thư viện phụ thuộc và chuẩn bị môi trường thực thi.
4. **Giai đoạn release:** Sau khi quá trình xây dựng thành công, Heroku thực thi các lệnh được định nghĩa trong phần `release` của `Procfile`.
5. **Khởi động lại dyno:** Các dyno đang chạy phiên bản cũ sẽ được dừng và các dyno mới được khởi tạo với mã nguồn và cấu hình đã được xây dựng thành công ở bước trước.
6. **Chuyển hướng lưu lượng:** Heroku tự động chuyển hướng lưu lượng truy cập HTTP đến các dyno mới vừa khởi động.

Quy trình tự động hóa này giúp việc triển khai phiên bản mới diễn ra nhanh chóng, đáng tin cậy và giảm thiểu thời gian chết đối với hầu hết các ứng dụng web tiêu chuẩn, nhờ cơ chế chuyển đổi mượt mà của Heroku. Tuy nhiên, cần lưu ý rằng việc sử dụng PaaS như Heroku có thể đi kèm với chi phí vận hành cao hơn ở quy mô lớn và mức độ kiểm soát hạ tầng ít linh hoạt hơn so với giải pháp IaaS.

7.3 Triển khai AI

7.3.1 Đóng gói

Dockerfile này được thiết kế để xây dựng một image Docker cho service, sử dụng Ubuntu 22.04 làm base image.

Chúng tôi bắt đầu bằng việc thiết lập các biến môi trường như múi giờ (TZ=Asia/Ho_Chi_Minh) và các cấu hình Python để tối ưu hóa output và quản lý bytecode. Các gói hệ thống cần thiết như python3, pip, và các công cụ biên dịch (build-essential, gcc, g++) được cài đặt qua apt-get để hỗ trợ các dependency phức tạp. UV Package Manager được sử dụng để quản lý và cài đặt dependency từ file pyproject.toml và uv.lock, đảm bảo tính nhất quán. Mã nguồn ứng dụng được sao chép vào thư mục /app.

Cuối cùng, ứng dụng được chạy bằng Uvicorn trên cổng 8000.

```
# syntax=docker/dockerfile:1.4
FROM ubuntu:22.04
ENV DEBIAN_FRONTEND=noninteractive
ENV TZ=Asia/Ho_Chi_Minh
ENV PATH="/root/.local/bin:$PATH"
ENV PYTHONUNBUFFERED=1
ENV PYTHONDONTWRITEBYTECODE=1
RUN apt-get update && apt-get install -y \
wget \
curl \
tzdata \
python3 \
python3-pip \
build-essential \
python3-dev \
gcc \
g++ \
pkg-config \
libssl-dev \
libffi-dev \
&& apt-get clean && rm -rf /var/lib/apt/lists/*
RUN python3 -m pip install --upgrade pip setuptools wheel
RUN curl -LsSf https://astral.sh/uv/install.sh | sh
WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen
COPY . .
EXPOSE 8000
CMD [".venv/bin/uvicorn", "app.server:app", "--host", "0.0.0.0", "--port", "
    8000"]
```

7.3.2 Triển khai

Nhóm xây dựng workflow “Build, Push and Deploy AI Service” được cấu hình để tự động hoá quy trình xây dựng ứng dụng.

Quy trình này bao gồm: đẩy image Docker lên Docker Hub và triển khai lên dịch vụ Render mỗi khi có thay đổi trên nhánh master hoặc khi tạo tag theo định dạng vX.Y.Z.

Khi có commit hoặc pull request vào master, job build-and-push sẽ checkout mã nguồn, đăng nhập vào Docker Hub, sinh metadata (sha ngắn, semver và tag latest) rồi build và đẩy image. Sau khi build xong, job deploy (chỉ chạy trên nhánh master hoặc trên tag) sẽ xác định tag cần deploy, đợi Docker Hub sẵn sàng rồi gọi API của Render để kích hoạt triển khai mới.

```
name: Build, Push and Deploy AI Service

on:
  push:
    branches: [ master ]
    tags: [ 'v*' ]
pull_request:
  branches: [ master ]

jobs:
  build-and-push:
  runs-on: ubuntu-latest
  steps:
    - name: Checkout code
      uses: actions/checkout@v3

    - name: Docker Hub Login
      uses: docker/login-action@v2
      with:
        username: ${{ secrets.DOCKER_USERNAME }}
```

```yaml
          password: ${{ secrets.DOCKER_PASSWORD }}

      - name: Generate Docker Metadata
        id: meta
        uses: docker/metadata-action@v4
        with:
        images: davidnguyen7603/coolchatai
        tags: |
          type=sha,format=short
          type=semver,pattern={{version}}
          type=raw,value=latest,enable={{is_default_branch}}

      - name: Build and Push Image
        uses: docker/build-push-action@v4
        with:
          context: .
          push: ${{ github.event_name != 'pull_request' }}
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}

  deploy:
    runs-on: ubuntu-latest
    needs: build-and-push
    if: ${{ github.event_name != 'pull_request' && (github.ref == 'refs/heads/
        master' || startsWith(github.ref, 'refs/tags/')) }}
    steps:
    - name: Checkout code
    uses: actions/checkout@v3

    - name: Deploy to Render
    env:
    RENDER_API_KEY: ${{ secrets.RENDER_API_KEY }}
    RENDER_SERVICE_ID: ${{ secrets.RENDER_SERVICE_ID }}
    IMAGE_NAME: davidnguyen7603/coolchatai
    run: |
    # Determine which tag to deploy: semver tag on git-tag, otherwise "latest"
    if [[ "${GITHUB_REF}" =~ ^refs/tags/v([0-9]+\.[0-9]+\.[0-9]+)$ ]]; then
      DEPLOY_TAG="${BASH_REMATCH[1]}"
    elif [[ "${GITHUB_REF}" == "refs/heads/master" ]]; then
      DEPLOY_TAG="latest"
    else
      echo "Unexpected ref ${GITHUB_REF}"
      exit 1
    fi

    echo "Deploying image ${IMAGE_NAME}:${DEPLOY_TAG}"
    # Wait until Docker Hub has that tag
    until docker pull "${IMAGE_NAME}:${DEPLOY_TAG}" >/dev/null 2>&1; do
      echo "  ...waiting for ${IMAGE_NAME}:${DEPLOY_TAG} to appear on Docker
          Hub"
      sleep 5
    done

    # Trigger Render deploy
    HTTP_STATUS=$(curl -s -o response.txt -w "%{http_code}" \
      -X POST "https://api.render.com/v1/services/${RENDER_SERVICE_ID}/deploys
          " \
      -H "Authorization: Bearer ${RENDER_API_KEY}" \
      -H "Content-Type: application/json" \
      -d "{\"imageRef\":\"docker.io/${IMAGE_NAME}:${DEPLOY_TAG}\",\"clearCache
          \":\"clear\"}")
    if [ "${HTTP_STATUS}" -ne 201 ]; then
      echo "Deploy failed with status ${HTTP_STATUS}"
      echo "--- response body ---"
      cat response.txt
      exit 1
```

```
        fi

        echo "Deploy triggered for ${IMAGE_NAME}:${DEPLOY_TAG}"
```

8 Tổng kết

8.1 Kết quả đạt được

8.1.1 Đối với tìm hiểu và phân tích nghiệp vụ

Căn cứ vào mục tiêu, nhiệm vụ đã đề ra, nhóm tác giả đã phân tích hoàn chỉnh nghiệp vụ của hệ thống:

- Tiến hành phân tích tất cả các yêu cầu cần có của hệ thống, kết hợp so sánh với các hệ thống tương tự đã có trên thị trường và rút ra được các ưu, nhược điểm của các hệ thống đó.
- Bổ sung các tính năng mới mà các ứng dụng tương tự hiện nay chưa có hoặc chưa tích hợp vào hệ thống của họ cũng như cải thiện các tính năng mà các hệ thống đó làm chưa tốt.
- Xác định được các stakeholder liên quan đến hệ thống, đồng thời phân tích được nhu cầu và ảnh hưởng của họ đối với hệ thống.
- Xác định đầy đủ vai trò của từng loại người dùng trong hệ thống và phân tích chi tiết các user story của họ.
- Phân tích được các yêu cầu chức năng và phi chức năng mà hệ thống cần phải có.

8.1.2 Đối với cơ sở lý thuyết và công nghệ

Dựa vào các quy tắc nghiệp vụ và các yêu cầu phân tích được của hệ thống, nhóm đã tiến hành tìm hiểu tất cả các khái niệm và các công nghệ để sử dụng trong hệ thống:

- Tìm hiểu về các khái niệm quan trọng và cách thức hoạt động của chatbot AI.
- Tìm hiểu về các mô hình ngôn ngữ lớn phục vụ cho chatbot trả lời tự động như GPT, DialoGPT, BERT, T5, BlenderBot, Llama.
- Chọn được các ngôn ngữ lập trình, framework và các công nghệ phù hợp để xây dựng hệ thống bao gồm các công nghệ cho Frontend, Backend và AI.
- Lựa chọn công nghệ và thực hiện triển khai hệ thống với các tính năng hiện có.

8.1.3 Đối với phân tích và thiết kế hệ thống

Dựa vào các quy tắc nghiệp vụ, các yêu cầu phân tích được của hệ thống, nhóm đã thực hiện phân tích hệ thống theo các lược đồ, đồng thời hoàn thành giai đoạn thiết kế hệ thống:

- Xác định và hoàn thành lược đồ use case toàn hệ thống, các lược đồ use case và các kịch bản cho từng module tính năng của hệ thống.
- Xác định và xây dựng được các lược đồ thể hiện luồng, cách thức và cách tương tác của người dùng đối với hệ thống như lược đồ activity, sequence và class.
- Lựa chọn và xây dựng được các lược đồ kiến trúc hệ thống để thể hiện được các thành phần và sự tương tác của các thành phần trong hệ thống.
- Thiết kế được lược đồ cơ sở dữ liệu ERD cho hệ thống bao gồm các thực thể và các thuộc tính, quan hệ của nó.
- Hoàn thiện bản thiết kế các mockup, prototype giao diện cho toàn hệ thống nhằm tạo điều kiện cho việc hiện thực trở nên dễ dàng và tốt nhất.

8.1.4 Đối với việc hiện thực và triển khai hệ thống

Ở giai đoạn đồ án chuyên ngành, nhóm đã thực hiện hiện thực một số tính năng bước đầu như các tính năng xác thực (đăng nhập, đăng ký tổ chức, tham gia tổ chức, quên mật khẩu), quản lý tài khoản cá nhân và quản lý tổ chức (thông tin và thành viên của tổ chức). Đồng thời, nhóm đã triển khai hệ thống với công nghệ đã lựa chọn.

8.1.5 Đối với quá trình thực hiện đồ án

Trong giai đoạn đồ án chuyên ngành, nhóm đã làm việc thông qua một quy trình rõ ràng và có kế hoạch.

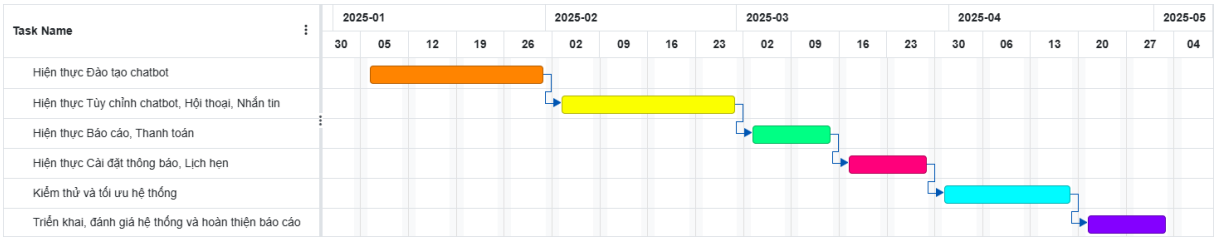
- Trải qua lần lược các giai đoạn theo đúng trình tự: đặt vấn đề, nghiên cứu thị trường, phân tích hệ thống, thiết kế hệ thống và tiến hành hiện thực hệ thống.
- Sử dụng phần mềm Jira để quản lý nhóm và phân công công việc theo từng giai đoạn cụ thể.
- Về vấn đề tổ chức và quản lý mã nguồn, nhóm đã áp dụng công cụ quản lý mã nguồn là Git và Github vào quá trình phát triển nhằm tăng hiệu quả việc cộng tác giữa các thành viên trong nhóm.
- Thường xuyên tổ chức các cuộc meeting hàng tuần, cũng như tạo nhóm chat để trao đổi cũng như đưa ra giải pháp cho các vấn đề gặp phải trong quá trình hiện thực. Đồng thời, nhóm tham gia đầy đủ các cuộc họp với giảng viên hướng dẫn để có thể nắm bắt và hoàn thành nhiệm vụ đúng tiến độ.

8.2 Hướng phát triển và kế hoạch trong tương lai

8.2.1 Hướng phát triển

- Mở rộng tính năng: phát triển các tính năng còn lại của hệ thống.
- Tích hợp AI: tích hợp AI chatbot vào hệ thống.
- Tích hợp với website tổ chức và facebook: tích hợp chatbot vào website của tổ chức đăng ký và trang facebook.
- Tối ưu hóa hiệu suất: cải thiện tốc độ xử lý và phản hồi, tối ưu hóa việc sử dụng tài nguyên và nâng cao khả năng chịu tải.

8.2.2 Kế hoạch thực hiện



Hình 119: *Biểu đồ Gantt kế hoạch triển khai dự án*

1. Nhân sự thực hiện:

- Frontend Engineer: Lê Đình Huy
  - Phụ trách phát triển giao diện người dùng
  - Đảm bảo trải nghiệm người dùng (UX/UI)
- Backend Engineer: Phạm Đức Thắng
  - Phụ trách phát triển API và xử lý logic nghiệp vụ
  - Quản lý cơ sở dữ liệu và bảo mật hệ thống
- AI Engineer: Nguyễn Đức An
  - Phụ trách phát triển và tích hợp các mô hình AI
  - Tối ưu hóa hiệu suất xử lý ngôn ngữ tự nhiên

2. Kế hoạch chi tiết theo giai đoạn:

Giai đoạn 1: Phát triển module đào tạo chatbot (6/1-2/2/2025)

- Nội dung và phân công:

  - Frontend Engineer: Hiện thực giao diện đào tạo chatbot
  - Backend Engineer: Phát triển API đào tạo chatbot
  - AI Engineer: Tích hợp và tối ưu mô hình AI cho chatbot
- Kết quả mong đợi:
  - Hoàn thiện module đào tạo chatbot có khả năng tương tác với AI
  - Giao diện người dùng thân thiện và dễ sử dụng

Giai đoạn 2: Xây dựng tính năng tùy chỉnh và tương tác với chatbot (3/2-2/3/2025)

- Nội dung và phân công:
  - Frontend Engineer: Phát triển giao diện tùy chỉnh chatbot và hội thoại
  - Backend Engineer: Xây dựng API tùy chỉnh chatbot và hội thoại, cho phép tích hợp với website và Facebook
  - AI Engineer: Tối ưu hóa khả năng xử lý ngôn ngữ tự nhiên
- Kết quả mong đợi:
  - Hệ thống chatbot tích hợp đa nền tảng
  - Khả năng tùy chỉnh linh hoạt theo nhu cầu người dùng

Giai đoạn 3: Phát triển hệ thống báo cáo và thanh toán (3-16/3/2025)

- Nội dung và phân công:
  - Frontend Engineer: Xây dựng giao diện báo cáo và thanh toán
  - Backend Engineer: Phát triển API thanh toán và tích hợp cổng thanh toán
  - AI Engineer: Phát triển các thuật toán phân tích dữ liệu cho báo cáo
- Kết quả mong đợi:
  - Hệ thống báo cáo chi tiết và trực quan
  - Tích hợp thanh toán an toàn và đa dạng

Giai đoạn 4: Xây dựng tính năng thông báo và lịch hẹn (17-30/3/2025)

- Nội dung và phân công:
  - Frontend Engineer: Phát triển giao diện quản lý thông báo và lịch hẹn
  - Backend Engineer: Xây dựng API quản lý thông báo và lịch hẹn
  - AI Engineer: Tích hợp thuật toán gợi ý thời gian hẹn tối ưu
- Kết quả mong đợi:
  - Hệ thống thông báo thời gian thực
  - Quản lý lịch hẹn hiệu quả

Giai đoạn 5: Kiểm thử và tối ưu hóa (31/3-20/4/2025)

- Nội dung và phân công:
  - Frontend Engineer: Kiểm thử UI/UX và tối ưu giao diện
  - Backend Engineer: Kiểm thử API và tối ưu hiệu suất hệ thống
  - AI Engineer: Tối ưu hóa các mô hình AI và đánh giá độ chính xác
- Kết quả mong đợi:
  - Độ ổn định và hiệu suất cao của hệ thống
  - Phát hiện và khắc phục các lỗi tiềm ẩn

Giai đoạn 6: Triển khai và hoàn thiện (21/4-4/5/2025)

- Nội dung và phân công:
  - Frontend Engineer: Triển khai giao diện người dùng
  - Backend Engineer: Triển khai hệ thống backend lên cloud
  - AI Engineer: Triển khai các mô hình AI và monitoring
- Kết quả mong đợi:
  - Hệ thống hoạt động ổn định trên môi trường thực tế
  - Báo cáo đồ án hoàn chỉnh và chuyên nghiệp

Tài liệu

[1] Ai chatbot market. https://market.us/report/ai-chatbot-market/. [Accessed 06-10-2024].

[2] Chatbot market size, share, growth & trends | global report [2030]. https://www.fortunebusinessinsights.com/chatbot-market-104673. [Accessed 06-10-2024].

[3] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey, 2024.

[4] Erica Santiago. How ai perceptions have changed in the last decade [comparing new & old consumer data]. https://shorturl.at/2MAd5, 2018. [Accessed 05-10-2024].

[5] Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Ben Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Siddartha Naidu, Chinmay Hegde, Yann LeCun, Tom Goldstein, Willie Neiswanger, and Micah Goldblum. Livebench: A challenging, contamination-free llm benchmark. 2024.

Phụ lục

1. Cơ sở lý thuyết

1.1. Mô hình ngôn ngữ lớn

Một Large Langue Model (LLM) là một loại mô hình học máy có khả năng thực hiện nhiều nhiệm vụ xử lý ngôn ngữ tự nhiên (Natural Laguage Processing) như tạo và phân loại văn bản, trả lời câu hỏi trong một cuộc đối thoại, và dịch văn bản từ một ngôn ngữ sang ngôn ngữ khác. Từ "Lớn" dùng chỉ số lượng giá trị (tham số) mà mô hình ngôn ngữ có thể thay đổi tự động khi nó học. Một số LLM thành công nhất có hàng trăm tỷ tham số (GPT-4, Deepseek-R1...).

LLM được huấn luyện với lượng dữ liệu khổng lồ, sử dụng phương pháp học tự giám sát (self-supervised learning) để dự đoán token - có thể hiểu đơn giản nó là từ, cụm từ, địa điểm,... tiếp theo trong một câu, dựa trên ngữ cảnh xung quanh. Quá trình này được lặp đi lặp lại cho đến khi mô hình đạt được một mức độ chính xác chấp nhận được.

Khi một LLM đã được huấn luyện, nó có thể được điều chỉnh chuyên biệt cho một loạt các nhiệm vụ NLP, bao gồm:

- Xây dựng trò chuyện với chatbot như ChatGPT.
- Tạo văn bản cho mô tả sản phẩm, bài đăng trên blog và bài báo.
- Trả lời các câu hỏi thường gặp (FAQs) và điều hướng các yêu cầu của khách hàng đến người hoặc bộ phận phù hợp nhất: hầu hết các doanh nghiệp lớn rất quan tâm đến vấn đề này, nhằm cắt giảm số lượng nhân sự cũng như là thời gian hỗ trợ khách hàng.
- Phân tích/Phân loại phản hồi của khách hàng từ email, bài đăng trên mạng xã hội và đánh giá sản phẩm: phân loại phàn nàn hay góp ý của khách hàng với sản phẩm của mình đã không còn là vấn đề xa lạ với hâu hết các doanh nghiệp, việt tóm tắt và phân loại chúng không phải là một vấn đề dễ dàng, hiện tại nhờ LLM chúng ta đã có thể không cần phải làm các công việc này bằng tay, mất khá nhiều thời gian và công sức.
- Dịch nội dung kinh doanh sang các ngôn ngữ khác nhau.

1.2. Kỹ thuật Retrieval-Augmented Generation (RAG) và RAG nâng cao

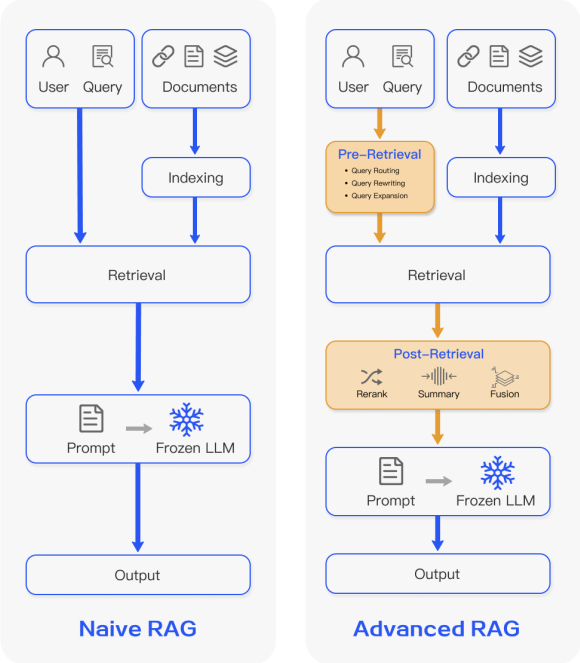
Kỹ thuật Tạo sinh tăng cường truy xuất (RAG) thuần túy có thể được hiểu là một framework 'Retrieval-Read' - một quy trình bao gồm tạo index, truy xuất, và tạo sinh [3].

- Tạo index: Việc lập index bắt đầu bằng việc dọn dẹp và trích xuất dữ liệu thô ở nhiều định dạng khác nhau như PDF, HTML, Word và Markdown, sau đó được chuyển đổi thành định dạng văn bản thuần túy thống nhất. Để phù hợp với các hạn chế về ngữ cảnh của mô hình ngôn ngữ, văn bản được phân đoạn thành các phần nhỏ hơn, dễ hiểu hơn (gọi là các chunk). Sau đó, các chunk được mã hóa thành biểu diễn vectơ bằng mô hình nhúng và được lưu trữ trong cơ sở dữ liệu vectơ.
- Truy xuất: Khi nhận được truy vấn của người dùng, hệ thống RAG sử dụng cùng một mô hình mã hóa được sử dụng trong giai đoạn lập index để chuyển đổi truy vấn thành biểu diễn vectơ. Sau đó, hệ thống tính toán điểm tương đồng giữa vectơ truy vấn và vectơ của các chunk trong ngữ liệu được lập chỉ mục. Hệ thống sẽ truy xuất k văn bản thể hiện điểm tương đồng lớn nhất với truy vấn.
- Tạo: Truy vấn của người dùng cùng với k văn bản thu được ở bước truy xuất được tổng hợp lại thành một lệnh (prompt) mạch lạc để LLM xử lý. Cách trả lời của mô hình có thể khác nhau, tùy vào tiêu chí của nhiệm vụ, định dạng trả về, hoặc phong cách đối thoại do người dùng đặt ra. Ngoài ra, lịch sử hội thoại cũng có thể được chèn vào prompt, cho phép mô hình ứng xử đúng khi tham gia cuộc đối đáp nhiều lượt.

Để tăng chất lượng phản hồi cho câu hỏi, kỹ thuật RAG nâng cao ra đời từ việc cải thiện kỹ thuật RAG truyền thống. Hai giai đoạn sau được thêm vào:

- Tiền truy xuất (Pre-retrieval process): Ở giai đoạn này, trọng tâm chính là tối ưu hóa cấu trúc index và truy vấn gốc. Mục tiêu của việc tối ưu hóa lập chỉ mục là nâng cao chất lượng nội dung được lập chỉ mục, gồm: tăng cường độ chi tiết của dữ liệu, tối ưu hóa cấu trúc chỉ mục, thêm metadata, tối ưu hóa căn chỉnh và truy xuất hỗn hợp. Trong khi mục tiêu của việc tối ưu hóa truy vấn là làm cho câu hỏi gốc của người dùng rõ ràng hơn và phù hợp hơn với tác vụ truy xuất, bao gồm: viết lại truy vấn, chuyển đổi truy vấn, mở rộng truy vấn...

- Hậu truy xuất (Post-retrieval process): Sau khi ngữ cảnh có liên quan được truy xuất, điều quan trọng là phải tích hợp hiệu quả với truy vấn. Các phương pháp chính trong quy trình sau truy xuất bao gồm xếp hạng lại các chunk và nén ngữ cảnh. Xếp hạng lại thông tin đã truy xuất để di chuyển nội dung có liên quan nhất dành cho prompt là một chiến lược quan trọng, bởi việc đưa tất cả các tài liệu có liên quan trực tiếp vào LLM có thể dẫn đến tình trạng quá tải thông tin, làm loãng sự tập trung vào các chi tiết chính.



Hình 120: *So sánh giữa hai mô hình RAG. (Bên trái) RAG truyền thống chủ yếu bao gồm ba phần: lập index, truy xuất và tạo. (Bên phải) RAG nâng cao đề xuất nhiều chiến lược tối ưu hóa xung quanh quá trình tiền và hậu truy xuất, với quy trình tương tự như RAG truyền thống, vẫn tuân theo cấu trúc dạng chuỗi. Nguồn: [3]*

2. Phân công nhiệm vụ